packt

2ND EDITION

Mastering Transformers

The Journey from BERT to Large Language Models and Stable Diffusion

SAVAŞ YILDIRIM | MEYSAM ASGARI-CHENAGHLU

Mastering Transformers

The Journey from BERT to Large Language Models and Stable Diffusion

Savaş Yıldırım

Meysam Asgari-Chenaghlu

packt

Mastering Transformers

Copyright © 2024 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Niranjan Naikwadi

Publishing Product Manager: Sanjana Gupta

Book Project Manager: Aparna Nair

Senior Editor: Gowri Rekha

Technical Editor: Rahul Limbachiya

Copy Editor: Safis Editing

Proofreader: Gowri Rekha

Indexer: Subalakshmi Govindhan

Production Designer: Prafulla Nikalje

DevRel Marketing Coordinator: Vinishka Kalra

First published: September 2021

Second edition: May 2024

Production reference: 1230524

Published by Packt Publishing Ltd.

Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK

ISBN 978-1-83763-378-4

www.packtpub.com

First of all, I would like to thank my wife, Aylin Oktay Yıldırım, for her continuous support, patience, and encouragement throughout the long process of writing this book. I would also like to thank my colleagues at Toronto Metropolitan University and Istanbul Bilgi University for their support.

– Savaş Yıldırım

I extend my deepest gratitude to my wife, Narjes, whose unwavering support, love, understanding, and encouragement have been my guiding lights throughout this journey. I also want to express my heartfelt appreciation to my father. Though he is no longer with us, his memory remains a source of inspiration, guiding me through both the triumphs and challenges of life.

– Meysam Asgari-Chenaghlu

Contributors

About the authors

Savaş Yıldırım graduated from Istanbul Technical University's Department of Computer Engineering and holds a Ph.D. degree in NLP from Trakya University. Currently, he is an associate professor at Istanbul Bilgi University, Turkey, and a visiting researcher at Ryerson University, Canada. He is a proactive lecturer and researcher with more than 20 years of teaching experience in machine learning, deep learning, and **natural language processing** (**NLP**). He has made significant contributions to the Turkish NLP community by developing many open source software and resources. He also provides comprehensive consultancy to AI companies on their R&D projects. In his spare time, he writes and directs short films and enjoys practicing yoga.

Meysam Asgari-Chenaghlu earned his Ph.D. from the University of Tabriz and currently serves as a staff AI researcher at ultimate.ai. With a focus on deep learning, AI, and NLP, he has successfully completed numerous projects ranging from search engines to generative AI and large language models. He has authored several research articles within the field, making significant contributions to the NLP community through his published work.

About the reviewers

Shivani Modi is a data scientist with expertise in machine learning, deep learning, and NLP, holding a master's degree from Columbia University. Her five years of professional experience span IBM, SAP, and C3 AI, where she has excelled in deploying scalable AI models across various sectors. At Konko AI, Shivani spearheaded the development of tools for optimizing LLM selection and deployment. Shivani's dedication to mentoring and talent development, coupled with her hands-on experience in leading complex projects, underscores her status as a thought leader in AI innovation. Her upcoming project aims to revolutionize how developers utilize LLMs, ensuring their secure and efficient implementation.

Ved Upadhyay, a seasoned data science and AI professional, brings over 7 years of hands-on experience in addressing enterprise-level challenges in deep learning. His expertise spans diverse industries, including retail, e-commerce, pharmaceuticals, agrotech, and socio-tech, where he has successfully implemented AI solutions. Currently a senior data scientist at Walmart, Ved leads multiple initiatives focused on generative AI, customer targeting, customer propensity, and responsible AI. He earned his master's degree in data science from the University of Illinois at Urbana-Champaign and also worked as a deep learning researcher at IIIT Hyderabad in the past.

Table of Contents

Preface xv

Part 1: Recent Developments in the Field, Installations, and Hello World Applications

1

From Bag-of-Words to the Transformers 3

Evolution of NLP approaches 4
Recalling traditional NLP approaches 7
Language modeling and generation 9
Leveraging DL 9
Considering the word order with RNN models 10
LSTMs and gated recurrent units 12
Contextual word embeddings and TL 13
Overview of the Transformer architecture 14
Attention mechanism 14
Multi-head attention mechanisms 17
Using TL with Transformers 22
Multimodal learning 24
Summary 26
References 27

2

A Hands-On Introduction to the Subject 29

Technical requirements 30
Installing transformer with Anaconda 30
Installation on Linux 31
Installation on Windows 32
Installation on macOS 34
Installing TensorFlow, PyTorch, and Transformer 35
Installing and using Google Colab 36
Working with language models and tokenizers 37

Working with community-provided models 39
Working with multimodal transformers 42
Working with benchmarks and datasets 44
Important benchmarks 44
GLUE benchmark 45
SuperGLUE benchmark 46
XTREME benchmark 46
XGLUE benchmark 46
SQuAD benchmark 47
Accessing the datasets with an application programming interface 47
Data manipulation using the datasets library 51
Sorting, indexing, and shuffling 51
Caching and reusability 52
Dataset filter and map function 53
Processing data with the map function 53
Working with local files 54
Preparing a dataset for model training 55
Benchmarking for speed and memory 56
Summary 61

Part 2: Transformer Models: From Autoencoders to Autoregressive Models

3

Autoencoding Language Models 65

Technical requirements 65
BERT – one of the autoencoding language models 66
BERT language model pretraining tasks 66
A deeper look into the BERT language model 67
Autoencoding language model training for any language 70
Sharing models with the community 80
Other autoencoding models 81
Introducing ALBERT 81
RoBERTa 85
ELECTRA 87
DeBERTa 87
Working with tokenization algorithms 88
BPE 90
WordPiece tokenization 91
Sentence piece tokenization 91
The tokenizers library 92
Summary 98

4

From Generative Models to Large Language Models 99

Technical requirements 99
An introduction to GLMs 100
Working with GLMs 102
GPT model family 102
Transformer-XL 105
XLNet 105
Working with text-to-text models 106
Multi-task learning with T5 106
Zero-Shot Text Generalization with T0 111
Another Denoising-Based Seq2Seq Model – BART 112
GLM training 115
NLG using AR models 120
Summary 122
References 123

5

Fine-Tuning Language Models for Text Classification 125

Technical requirements 126
Introduction to text classification 126
Fine-tuning a BERT model for single-sentence binary classification 127
Training a classification model with native PyTorch 134
Fine-tuning BERT for multi-class classification with custom datasets 139
Fine-tuning the BERT model for sentence-pair regression 145
Multilabel text classification 150
Utilizing run_glue.py to fine-tune the models 156
Summary 157
References 157

6

Fine-Tuning Language Models for Token Classification 159

Technical requirements 159
Introduction to token classification 160
Understanding NER 161
Understanding POS tagging 162
Understanding QA 163
Fine-tuning language models for NER 164
Question answering using token classification 172
Question answering for many tasks 180
Summary 181

7

Text Representation 183

Technical requirements 184
Introduction to sentence embeddings 184
Cross-encoder versus bi-encoder 185
Benchmarking sentence similarity models 187
Using BART for zero-shot learning 190
Semantic similarity experiment with FLAIR 193
Average word embeddings 196
RNN-based document embeddings 197
Transformer-based BERT embeddings 197
SBERT embeddings 197
Text clustering with Sentence-BERT 200
Topic modeling with BERTopic 203
Semantic search with SBERT 205
Instruction fine-tuned embedding models 209
Summary 211
Further reading 212

8

Boosting Model Performance 213

Technical requirements 213
Improving performance with data augmentation 214
Character-level augmentation 214
Word-level augmentation 215
Sentence-level augmentation 216
Boosting IMDB text classification with augmentation 216
Adapting the model to the domain 221
Optimizing the parameters with HPO 228
Summary 232

9

Parameter Efficient Fine-Tuning 233

Technical requirements 233
Introduction to PEFT 234
Understanding Types of PEFT 235
Additive methods 235
Selective methods 236
Low-rank fine-tuning 236
Hands-on PEFT experiments 236
Fine-tuning a BERT checkpoint with adapter tuning 237
Efficiently fine-tune FLAN-T5 for an NLI task with Lora 241
Tuning with QLoRA 251
Summary 252
References 253

Part 3: Advanced Topics

10

Large Language Models 257

Technical requirements 257
Why large language models? 258
Importance of reward function 259
The instruction-following ability of LLMs 262
Fine-tuning large language models 263
Summary 269

11

Explainable AI (XAI) in NLP 271

Technical requirements 272
Interpreting attention heads 272
Visualizing attention heads with exBERT 273
Multiscale visualization of attention heads with BertViz 278
Understanding the inner parts of BERT with probing classifiers 287
Explain the model decision 288
Interpret Transformers' decision with LIME 289
Interpret Transformers' decision with SHAP 293
Summary 295

12

Working with Efficient Transformers 297

Technical requirements 298
Introduction to efficient, light, and fast transformers 298
Implementation for model size reduction 300
Working with DistilBERT for knowledge distillation 300
Pruning transformers 302
Quantization 305
Working with efficient self-attention 306
Sparse attention with fixed patterns 307
Learnable patterns 321
Low-rank factorization, kernel methods, and other approaches 326
Easier quantization using bitsandbytes 326
Summary 327
References 327

13

Cross-Lingual and Multilingual Language Modeling 329

Technical requirements 330
Translation language modeling and cross-lingual knowledge sharing 330
XLM and mBERT 332
mBERT 332
XLM 333
Cross-lingual similarity tasks 337
Cross-lingual text similarity 337
Visualizing cross-lingual textual similarity 340
Cross-lingual classification 344
Cross-lingual zero-shot learning 349
Massive multilingual translation 352
Fine-tuning the performance of multilingual models 354
Summary 357
References 357

14

Serving Transformer Models 359

Technical requirements 360
FastAPI Transformer model serving 360
Dockerizing APIs 364
Faster Transformer model serving using TFX 365
Load testing using Locust 368
Faster inference using ONNX 371
SageMaker inference 371
Summary 373
Further reading 374

15

Model Tracking and Monitoring 375

Technical requirements 375
Tracking model metrics 376
Tracking model training with TensorBoard 376
Tracking model training live with W&B 379
Summary 384
Further reading 384

Part 4: Transformers beyond NLP

16

Vision Transformers 387

Technical requirements 387
Vision transformers 388
Image classification using transformers 392
Semantic segmentation and object detection using transformers 393
Visual prompt models 398
Summary 401

17

Multimodal Generative Transformers 403

Technical requirements 403
Multimodal learning 404
Generative multimodal AI 405
Stable Diffusion for text-to-image generation 405
Stable Diffusion in action 407
Music generation using MusicGen 409
Text-to-speech generation using transformers 410
Summary 411

18

Revisiting Transformers Architecture for Time Series 413

Technical requirements 413
Understanding time series concepts 414
Transformers and time series modeling 415
Summary 421

Index 423

Other Books You May Enjoy 436

Preface

Transformer-based language models have emerged as a cornerstone in the field of **Natural Language Processing** (**NLP**), representing a paradigm shift. Their superior fine-tuning and zero-shot abilities have proven to be faster and more precise, surpassing the performance of traditional machine learning methods in various complex natural language tasks. This practical guide to NLP is a valuable resource for developers, familiarizing them with the Transformers architecture.

This book offers clear, step-by-step explanations of crucial concepts, supplemented with practical examples. We start with an easy-to-understand overview of the revolution in NLP. This includes a basic understanding of relevant deep learning concepts and technologies, along with comprehensive guidance on managing various NLP tasks.

This book is also highly beneficial for developers looking to broaden their understanding of multimodal models and generative AI. Transformers are not only used for NLP tasks but are also increasingly employed in computer vision tasks, signal processing, and many other areas. Besides NLP, there's a fast-growing area of multimodal learning and generative AI that's been showing some exciting progress. We're talking about things such as **GPT-4, Gemini, Claude, DALL-E**, and Stable Diffusion-based models here. If you're a developer, it's worth keeping an eye on these technologies to see how you can best utilize them for your specific needs.

Who this book is for

This book is for generative AI and deep learning researchers; hands-on practitioners; and machine learning/NLP teachers, educators, and their students who have a good command of programming topics and knowledge in the fields of machine learning, multimodal learning, and artificial intelligence and want to develop applications in the cutting-edge field of NLP.

You will have to know at least Python or another programming language, be familiar with machine learning literature, and have a basic understanding of computer science, as this book covers the practical aspects of NLP and generative AI.

What this book covers

Chapter 1, *From Bag-of-Words to Transformers*, will present a gentle and easy-to-understand introduction to Transformers. Additionally, it will compare older methods, including non-deep learning and deep learning models, such as CNNs, RNNs, and LSTMs for text, with Transformers.

Chapter 2, *A Hands-On Introduction to the Subject*, provides the first hands-on experiment for setting up your environment. In this chapter, you will learn about all aspects of Transformers at an introductory level. Installing `tensorflow`, `pytorch`, `conda`, `transformers`, and `sentenceTransformers` libraries will be explained. Another section of this chapter will be dedicated to installing GPU versions of related libraries. We will provide some hands-on code examples for you to start learning through examples. You will write your first `hello-world` program with Transformers by loading community-provided pre-trained language models.

Chapter 3, *Autoencoding Language Models*, covers how to use the encoder part of Transformer. You will learn about the architecture of the BERT model and how to train/test autoencoding language models such as BERT and ALBERT. You will understand the difference between the models and the objective functions of the architectures. You will also learn how to share the models with the community and also how to fine-tune other pre-trained language models shared by the community.

Chapter 4, *From Generative Models to Large Language Models*, covers a generative solution with decoder-only Transformers and encoder-decoder models. In this chapter, you will see the details of **Generative Language Models** (**GLMs**) and **Large Language Models** (**LLMs**). You will learn how to pre-train any language model, such as **Generated Pre-trained Transformer 2** (**GPT-2**), on your own text and use it in various tasks, such as **Natural Language Generation** (**NLG**). You will understand the basics of a **Text-to-Text Transfer Transformer** (**T5**) model, how to conduct a hands-on multitask learning experiment with T5, and how to train a **Multilingual T5** (**mT5**) model on your own **Machine Translation** (**MT**) data. After finishing this chapter, you will have an understanding of GLMs and their various use cases in text2text applications, such as summarization, paraphrasing, multitask learning, zero-shot learning, and MT.

Chapter 5, *Fine-Tuning Language Model for Text Classification*, teaches you how to load a pre-trained model for text classification and fine-tune a base model for any downstream text classification task, such as sentiment analysis. You will work with well-known datasets, such as GLUE, as well as your own datasets. You will also take advantage of the Trainer class of the transformers library, which handles much of the complexity of the training and fine-tuning processes.

Chapter 6, *Fine-Tuning Language Models for Token Classification*, covers how to solve token-wise classification tasks such as named-entity recognition, part-of-speech tasks, and span prediction by fine-tuning Transformer models. Token classification, one of the challenging topics in NLP, will be implemented by fine-tuning (or creating from scratch) using the `Trainer` class of the Python transformers library. NER, POS, and QA could be counted as token classification problems. GLUE and community-provided datasets are used for quick prototyping. We will use shared models and datasets to fine-tune the models for token classification problems. We will also discuss how to benchmark token classification for evaluation.

Chapter 7, *Text Representation*, will look at how to represent text with a Transformer-based architecture for a variety of NLP tasks. Text representation is another trivial task in modern NLP. Representing sentences using various models such as the universal sentence encoder and siamese BERT (sentence BERT) with additional libraries such as sentenceTransformers will be explained here. Datasets such as MNLI and XNLI will be described, as well as benchmarking sentence representation models with `STSBenchmark` will be explored. Few-/zero-shot learning using efficient sentence representation will also be covered. Unsupervised tasks such as clustering using transfer learning will be presented.

Chapter 8, *Boosting Model Performance*, discusses many techniques for increasing vanilla fine-tuning performance. In this chapter, we'll explore how to boost the Transformers model beyond vanilla training. Data augmentation, domain adaptation, and hyperparameter optimization are the main topics for improving model performance.

Chapter 9, *Parameter Efficient Fine-Tuning*, instead of fine-tuning all parameters of an LLM, we can find a cheaper and more efficient way to fine-tune with LoRa-like advanced techniques. Although fine-tuning pre-trained Transformer models is a very useful way to solve NLP tasks, vanilla fine-tuning can be parameter inefficient in many ways. We can use an adapter such as LoRa to reduce the number of parameters to be fine-tuned.

Chapter 10, *Large Language Models*, LLMs such as T5 and LLaMA are discussed in this chapter, with fine-tuning and efficient inference as the main focus. The field of LLMs has made significant progress in recent years with the development of models such as **GPT-3** (**175B**), **PaLM** (**540B**), **BLOOM** (**175B**), **LlaMA** (**65B**), **Falcon** (**180B**), and **Mistral** (**7B**). These models have shown impressive abilities in various natural language tasks. It is challenging to cover such an important topic in a single chapter; however, we will have already covered many aspects of the topic, particularly in *Chapter 4*. Moreover, throughout the book, we will discuss the paradigm of neural language models and their training process.

Chapter 11, *Explainable AI (XAI) for NLP*, we discuss potential approaches with hands-on experiments. It is very difficult for us to make sense of the decisions made by deep learning models during inference. So, we need different ways to make sense of the decisions made by these black-box models. In this chapter, we will see how to apply **Explainable AI** (**XAI**) approaches to NLP.

Chapter 12, *Working with Efficient Transformers*, as it is getting difficult to run large models under limited computational capacity, it is important now to pre-train a smaller general-purpose language model such as DistilBERT, which can then be fine-tuned with good performances on a wide variety of problems like its non-distilled counterparts. Also, Transformer-based architectures face complexity bottlenecks due to the quadratic complexity of the attention dot product in transformers and long-context NLP tasks. Character-based language models, speech processing, and long documents are among the long-context problems. In recent years, we have seen much progress in making self-attention more efficient, such as Reformer, Performer, and Bird as solutions to its complexity.

Chapter 13, *Cross-Lingual and Multilingual Language Modeling*, in this chapter, we discuss how to work with more than one language. Architecture and models such as XLM will be described in this chapter. Moving from uni-lingual to multilingual and cross-lingual language modeling will be explained. The concept of knowledge sharing between languages will be presented, as well as the impact of byte-pair encoding on tokenization in order to achieve better input. Cross-lingual sentence similarity using a cross-lingual corpus (XNLI) will be detailed. Tasks such as cross-lingual classification and utilization of cross-lingual sentence representation for training on one language and testing on another one will be presented with concrete examples of real-life problems in NLP, such as multilingual intent classification.

Chapter 14, *Serving Transformer Models*, how to go to production is the key topic of this chapter. Like any other real-life and modern solution, NLP-based solutions must be able to be served in a production environment. However, metrics such as response time should be in mind while developing such solutions. This chapter will detail how to serve a Transformer-based NLP solution in environments where CPU/GPU is available. **TensorFlow Extended** (**TFX**) for machine learning deployment as a solution will be described here. Also, other solutions for serving Transformers as APIs, such as FastAPI, will be illustrated.

Chapter 15, *Model Tracking and Monitoring*, in this chapter, we need to monitor our model during training to know where to stop and check whether there is a problem such as overfitting. We will track our experiments by logging and monitoring using TensorBoard. It allows us to efficiently host, track, and share our experiment results, such as tracking loss and accuracy metrics or projecting the learned embeddings to a lower dimensional space.

Chapter 16, *Vision Transformers*, other than NLP, we can also work with images. Transformers achieved great goals in the field of NLP, and as you will have seen in previous chapters, they can accomplish many different tasks. In this chapter, **Vision Transformer** (**ViT**) is explained. Just like NLP, different models have been created for the vision as well, and each single one of them created a new view of the computer vision perspective. By reading this chapter, you will know how you can use models such as ViT for computer vision tasks, how pretrained computer vision models based on Transformers work, and how you can fine-tune them for specific tasks.

Chapter 17, *Multimodal Generative Transformers*, we can also work with all modalities in one model; Text, Image, Audio, etc. Models such as CLIP show promising results in multimodal search (text-image). Other models and approaches, such as Stable Diffusion, perform very well in generating images from textual prompts. In this chapter, we will look into these models and first create a semantic search engine using CLIP. Next, you will learn how it is possible to create multimodal classifiers using CLIP and cross-modal pretrained models. You will learn how to use Stable Diffusion and other related pretrained prompt-based models, such as Midjourney. After that, you will learn how to create object detection solutions with no training data using zero-shot prompt-based multimodal pretrained models.

Chapter 18, *Revisiting Transformers Architecture for Time Series*, covers how to restructure a transformer for time series tasks to leverage the benefits of Transformers. Transformers are well known for NLP-specific tasks. Their main power comes from their capability to model time series data. This data can be textual or non-textual. In this chapter, you will learn how to use transformers for time series data modeling and predicting.

To get the most out of this book

Software/hardware covered in the book	Operating system requirements
Programming language: Python	Windows, macOS, or Linux
Libraries: `transformers`, `PyTorch`, `peft`, `trl`	
Jupyter Notebook	

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

> **Note**
> *This book contains a few screenshots which have been captured to show the overview of workflows and also the UI. Due to this, the content in this images may appear small at 100% zoom. Please check out the PDF copy provided with the book to zoom in for clearer images.*

Download the example code files

You can download the example code files for this book from GitHub at `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

Code in text: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: "`Mount the downloaded WebStorm-10*.dmg disk image file as another disk in your system.`"

A block of code is set as follows:

```
from transformers import pipeline
classifier = pipeline("zero-shot-classification",
    model="facebook/bart-large-mnli")
sequence_to_classify = "I am going to france."
candidate_labels = ['travel', 'cooking', 'dancing']
classifier(sequence_to_classify, candidate_labels)
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
tokenized_sentence = \
    tokenizer.encode("Oh it works just fine")
tokenized_sentence.tokens
['[CLS]', 'oh', 'it', 'works', 'just', 'fine','[SEP]']
```

Any command-line input or output is written as follows:

```
conda create -n Transformer
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: "Select **System info** from the **Administration** panel."

> **Tips or important notes**
> Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at `customercare@packtpub.com` and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit `www.packtpub.com/support/errata` and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

Share Your Thoughts

Once you've read *Mastering Transformers*, we'd love to hear your thoughts! Please click here to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below

`https://packt.link/free-ebook/9781837633784`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1: Recent Developments in the Field, Installations, and Hello World Applications

An introduction to the basics of Transformers is presented in this part of the book. Assistance will be provided in developing your first 'Hello World' program using Transformers, installing the necessary packages, utilizing pre-trained language models, and so forth. The corresponding code can be executed with a CPU or a GPU. By delving into the practical work, this part sets the stage for tackling the intricate challenges ahead.

This part has the following chapters:

- *Chapter 1, From Bag-of-Words to Transformers*
- *Chapter 2, A Hands-On Introduction to the Subject*

1

From Bag-of-Words to the Transformers

Over the past two decades, there have been significant advancements in the field of **natural language processing** (**NLP**). We have gone through various paradigms and have now arrived at the era of the *Transformer* architecture. These advancements have helped us represent words or sentences more effectively in order to solve NLP tasks. On the other hand, different use cases of merging textual inputs to other modalities, such as images, have emerged as well. Conversational **artificial intelligence** (**AI**) has seen the dawn of a new era. **Chatbots** were developed that act like humans by answering questions, describing concepts, and even solving mathematical equations step by step. All of these advancements happened in a very short period. One of the enablers of this huge advancement, without a doubt, was Transformer models.

Finding a cross-semantic understanding of different natural languages, natural languages and images, natural languages, and programming languages, and even in a broader sense, natural languages and almost any other modality, has opened a new gate for us to be able to use natural language as our primary input to perform many complex tasks in the field of AI. The easiest imaginable way is to just describe what we are looking for in a picture so the model will give us what we want (`https://huggingface.co/spaces/CVPR/regionclip-demo`):

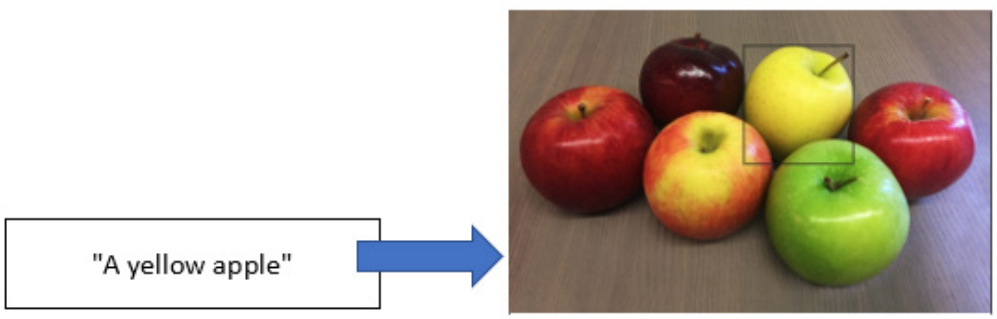


Figure 1.1 – Zero-shot object detection with the prompt "A yellow apple"

The models have developed this skill through a process of ongoing learning and improvement. At first, distributional semantics and n-gram language models were traditionally utilized to understand the meanings of words and documents for years. It has been seen that these approaches had several limitations. On the other hand, with the rise of newer approaches for diffusing different modalities, modern approaches for training language models, especially **large language models** (**LLMs**), enabled many different use cases to come to life.

Classical **deep learning** (**DL**) architectures have significantly enhanced the performance of NLP tasks and have overcome the limitations of traditional approaches. **Recurrent neural networks** (**RNNs**), **feed-forward neural networks** (**FFNNs**), and **convolutional neural networks** (**CNNs**) are some of the widely used DL architectures for the solution. However, these models have also faced their own challenges. Recently, the Transformer model became standard, eliminating all the shortcomings of other models. It differed not only in solving a single monolingual task but also in the performance of multilingual, multitasking tasks. These contributions have made **transfer learning** (**TL**) more viable in NLP, which aims to make models reusable for different tasks or languages.

In this chapter, we will begin by examining the attention mechanism and provide a brief overview of the Transformer architecture. We will also highlight the distinctions between Transformer models and previous NLP models.

In this chapter, we will cover the following topics:

- Evolution of NLP approaches
- Recalling traditional NLP approaches
- Leveraging DL
- Overview of the Transformer architecture
- Using TL with Transformers
- Multimodal learning

Evolution of NLP approaches

Let us discuss how NLP has evolved over the past two decades as significant advancements have occurred in the field. Recently, the evolution has been primarily characterized by the transformative Transformer architecture. This architecture did not emerge out of thin air but, rather, evolved from various neural-based NLP approaches into an attention-based encoder-decoder architecture, which continues to evolve. In the past decade, the Transformer architecture and its variants have gained popularity due to the following developments:

- Contextual word embeddings thanks to self-attention
- Attention mechanisms, which overcome the problem of forcing input sentences to encode all information into one context vector

- Better subword tokenization algorithms for handling unseen words or rare words
- Injecting additional memory tokens into sentences, such as the paragraph ID in **Doc2vec** or a **Classification** (**[CLS]**) token in **Bidirectional Encoder Representations from Transformers** (**BERT**)
- Parallelizable architectures that make for faster training and fine-tuning
- Model compression (distillation, quantization, and so on)
- TL capabilities: DL models can be easily adapted to new tasks or languages
- Cross-lingual, multilingual, and multitasking learning capabilities
- Multimodal training

For several years, traditional NLP approaches such as **n-gram language models**, **TF-IDF-based models**, **BM25 information retrieval models**, and **one-hot encoded document-term matrices** have been utilized to solve a range of NLP tasks, including sequence classification, language generation, and machine translation. However, these traditional approaches have limitations, such as difficulty in handling sparsity, rare words, and long-term dependencies. To address these challenges, we have developed DL-based approaches such as RNN, CNN, and FFNN, as well as several variants.

A document vector created by the TF-IDF model would have a size of more than 30,000 features. In 2013, **word2vec**, a two-layer FFNN word-encoder model, sorted out the dimensionality curse by generating compact and dense representations of words called **word embeddings**. This early model was able to create fast and efficient static word embeddings by converting unsupervised textual data into supervised data (*self-supervised learning*) through the prediction of target words using nearby neighboring words. Meanwhile, **GloVe**, another widely used and popular model, argued that count-based models can be better than neural models. It leverages both global and local statistics of a corpus to learn embeddings based on word-word co-occurrence statistics.

Word embeddings have proven effective for some syntactic and semantic tasks, as demonstrated in the following figure, which illustrates how the offsets between terms in the embeddings allow for vector-oriented reasoning. For example, we can discern the generalization of gender relations, a semantic relation, from the offset between the terms *Man* and *Woman* (*Man -> Woman*). Then, we can arithmetically estimate the vector of *Actress* by adding the vector of the term *Actor* and the offset calculated before. Likewise, we can learn the syntactic relationships, such as word plural forms. For instance, if the vectors of *Actor*, *Actors*, and *Actress* are given, we can estimate the vector of *Actresses*:

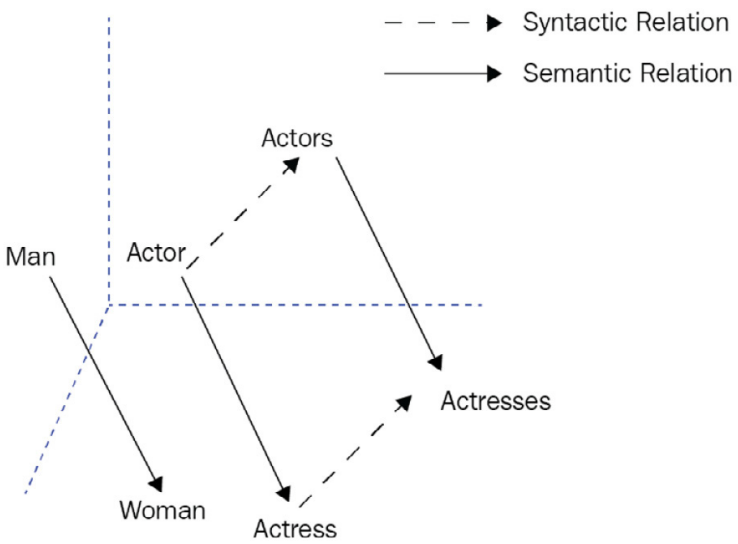


Figure 1.2 – Word embeddings offset for relationship extraction

The representativeness of word embeddings has become a fundamental component for DL models such as RNNs or CNNs. The recurrent and convolutional architectures, respectively, started to be used as encoders and decoders in **sequence-to-sequence** (**seq2seq**) problems where each token is represented with an embedding. The main challenge with these early models was polysemous words (words with more than one meaning). The senses of the words are ignored since a single fixed representation is assigned to each word, which is an especially severe problem for sentence semantics.

Pioneering neural network models such as **Universal Language Model Fine-tuning** (**ULMFiT**) and **Embeddings from Language Models** (**ELMo**) were able to encode sentence-level information and alleviate polysemy issues, unlike static word embeddings. In this way, we have a new concept called **contextual word embeddings**.

The ULMFiT and ELMo approaches were based on **LSTM** (**Long Short-Term Memory**) networks, a variant of RNN. They also exploited the concept of pre-training and fine-tuning, which allows for TL by using pre-trained models trained on a general task with general textual datasets and fine-tuning them on a target task with supervision. This was a significant development because TL, which had previously been successful in image processing, was being applied for the first time in the field of NLP.

In the meantime, the idea of an attention mechanism (*Neural Machine Translation by Jointly Learning to Align and Translate. Bahdanau et al., 2015*) made a strong impression in the NLP field and achieved significant success, especially in **seq2seq** problems. Earlier methods would pass the last state (known as a **context vector** or **thought vector**) obtained from the entire input sequence to the output sequence without linking or elimination. Thanks to the attention mechanism, certain parts of the input can be associated with certain parts of the output.

In 2017, the Transformer-based encoder-decoder model was introduced and found to be successful due to its innovative use of the attention mechanism. The design is based on an FFNN by discarding RNN recurrency and using only attention mechanisms (*Vaswani et al., Attention is All You Need, 2017*). It overcame many difficulties that other approaches faced and has become a new paradigm. Throughout this book, you will be exploring and understanding how Transformer-based models work.

Recalling traditional NLP approaches

Although traditional models will soon become obsolete, they can always shed light on innovative designs. The most important of these is the distributional method, which is still used. **Distributional semantics** is a theory that explains the meaning of a word by analyzing its distributional evidence rather than relying on predefined dictionary definitions or other static resources. This approach suggests that words that frequently occur in similar contexts tend to have similar meanings. For example, words such as *dog* and *cat* often occur in similar contexts, suggesting that they may have related meanings. The idea was first proposed by Zellig S. Harris in his work, *Distributional Structure of Words*, in 1954. One of the benefits of using a distributional approach is that it allows researchers to track the semantic evolution of words over time or across different domains or senses of words, which is a task that is not possible using dictionary definitions alone.

For many years, traditional approaches to NLP have relied on **Bag-of-Words** (**BoW**) and n-gram language models to understand words and sentences. BoW approaches, also known as **vector space models** (**VSMs**), represent words and documents using one-hot encoding, a sparse representation method. These one-hot encoding techniques have been used to solve a variety of NLP tasks, such as text classification, word similarity, semantic relation extraction, and word-sense disambiguation. On the other hand, n-gram language models assign probabilities to sequences of words, which can be used to calculate the likelihood that a given sequence belongs to a corpus or to generate random sequences based on a given corpus.

Along with these models, in order to assign importance to a term, the **Term Frequency** (**TF**) and the **Inverse Document Frequency** (**IDF**) metrics are often used. IDF helps to reduce the weight of high-frequency words, such as stop words and functional words, which have little discriminatory power in understanding the content of a document. The discriminatory power of a term also depends on the domain – for example, a list of articles about DL is likely to have the word *network* in almost every document. It would not be a surprise to see the term *network* in the domain data. The **Document Frequency** (**DF**) of a word is calculated by counting the number of documents in which it appears, and this can be used to scale down the weights of all terms. The TF is simply the raw count of a term in a document.

Some of the advantages and disadvantages of a TF-IDF-based BoW model are listed as follows:

Advantages	**Disadvantages**
• Easy to implement • Human-interpretable results • Domain adaptation	• Dimensionality curse • No solution for unseen words • Hardly captures semantic relations (is-a, has-a, synonym) • Ignores word order • Slow for large vocabulary

Table 1.1 – Advantages and disadvantages of a TF-IDF BoW model

Using a BoW approach to represent a small sentence can be impractical because it involves representing each word in the dictionary as a separate dimension in a vector, regardless of whether it appears in the sentence or not. This can result in a high-dimensional vector with many zero cells, making it difficult to work with and requiring a large amount of memory to store.

Latent semantic analysis (**LSA**) has been widely used to overcome the dimensionality problem of the BoW model. It is a linear method that captures pairwise correlations between terms. LSA-based probabilistic methods can still be considered as a single layer of hidden topic variables. However, current DL models include multiple hidden layers, with billions of parameters. In addition to that, Transformer-based models showed that they can discover latent representations much better than such traditional models.

The traditional pipeline for NLP tasks begins with a series of preparation steps, such as tokenization, stemming, noun phrase detection, chunking, and stop-word elimination. After these steps are completed, a document-term matrix is constructed using a weighting schema, with TF-IDF being the most popular choice. This matrix can then be used as input for various **machine learning** (**ML**) pipelines, including sentiment analysis, document similarity, document clustering, and measuring the relevancy score between a query and a document. Similarly, terms can be represented as a matrix and used as input for token classification tasks, such as named entity recognition and semantic relation extraction. The classification phase typically involves the application of supervised ML algorithms, such as **support vector machines** (**SVMs**), random forests, logistic regression, naive Bayes, and multiple learners (boosting or bagging).

Language modeling and generation

Traditional approaches to language generation tasks often rely on n-gram language models, also known as Markov processes. These are stochastic models that estimate the probability of a word (event) based on a subset of previous words. There are three main types of n-gram models: unigram, bigram, and n-gram (generalized). Let us look at these in more detail:

- **Unigram** models assume that all words are independent and do not form a chain. The probability of a word in a vocabulary is simply calculated by its frequency in the total word count.
- **Bigram** models, also known as first-order Markov processes, estimate the probability of a word based on the previous word. This probability is calculated by the ratio of the joint probability of two consecutive words to the probability of the first word.
- **N-gram** models, also known as N-order Markov processes, estimate the probability of a word based on the previous *n-1* words.

We have already discussed the paradigms underlying traditional NLP models in the *Recalling traditional NLP approaches* subsection and provided a brief introduction. We will now move on to discuss how neural language models have impacted the field of NLP and how they have addressed the limitations of traditional models.

Leveraging DL

For decades, we have witnessed successful architectures, especially in word and sentence representation. Neural network-based language models have been effective in addressing feature representation and language modeling problems because they allow for the training of more advanced neural architectures on large datasets, which enables the learning of compact, high-quality representations of language.

In 2013, the word2vec model introduced a simple and effective architecture for learning continuous word representations that outperformed other models on a variety of syntactic and semantic language tasks, such as sentiment analysis, paraphrase detection, and relation extraction. The low computational complexity of word2vec has also contributed to its popularity. Due to the success of embedding methods, word embedding representation has gained attraction and is now widely used in modern NLP models.

Word2vec and similar models learn word embeddings through the use of a prediction-based neural architecture based on nearby word predictions. This approach differs from traditional BoW methods that rely on count-based techniques for capturing distributional semantics. The authors of the GloVe model have addressed the question of whether count-based or prediction-based methods are better for distributional word representations and claimed that the two approaches are not significantly different. **FastText** is another widely used model that incorporates subword information by representing each word as a bag of character n-grams, with each n-gram represented by a constant vector. Words are then represented as the sum of their sub-vectors, an idea first introduced by H. Schütze in 1993. This allows

FastText to compute word representations even for unseen words (or rare words) and learn the internal structure of words, such as suffixes and affixes, which is particularly useful for morphologically rich languages. Likewise, modern Transformer architectures exploit incorporating subword information and use various subword tokenization methods, such as **WordPiece**, **SentencePiece**, or **Byte-Pair Encoding** (**BPE**).

Let us quickly discuss popular RNN models.

Considering the word order with RNN models

Traditional BoW models do not consider word order because they treat all words as individual units and place them in a basket. RNN models, on the other hand, learn the representation of each token (word) by cumulatively incorporating the information of previous tokens, and the final token ultimately provides the representation of the entire sentence.

Like other neural network models, RNN models process tokens produced by a tokenization algorithm that breaks down raw text into atomic units, known as tokens. These tokens are then associated with numeric vectors, called token embeddings, which are learned during training. Alternatively, we can use well-known word-embedding algorithms, such as word2vec or FastText, to generate these token embeddings in advance.

Here is a simple illustration of an RNN architecture for the sentence `The cat is sad.`, where `x0` is the vector embeddings of `the`, `x1` is the vector embeddings of `cat`, and so forth. *Figure 1.3* illustrates an RNN being unfolded into a full **deep neural network** (**DNN**). Unfolding means that we associate a layer to each word. For the `The cat is sad.` sequence, we take care of a sequence of five tokens. The hidden state in each layer acts as the memory of the network and is shared between the steps. It encodes information about what happened in all previous timesteps and in the current timestep. This is represented in the following diagram:

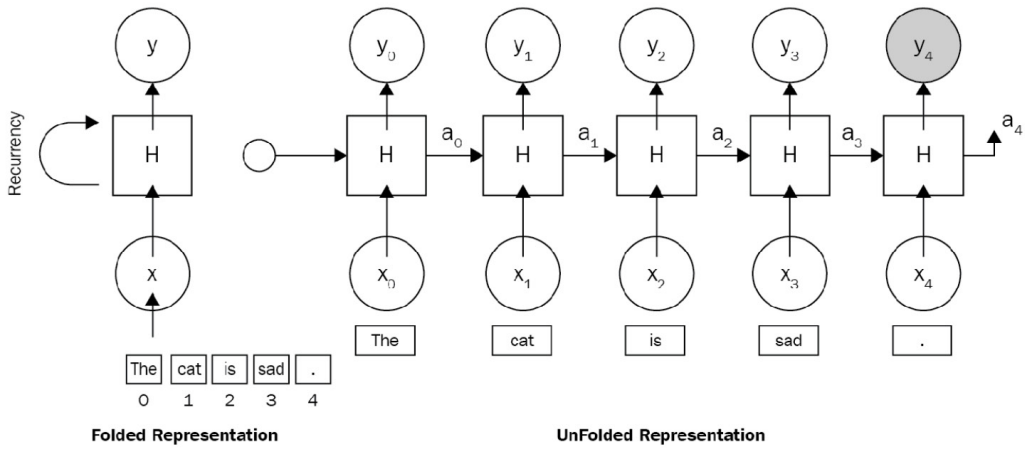


Figure 1.3 – An RNN architecture

The following are some advantages of an RNN architecture:

- **Variable-length input**: The capacity to work on variable-length input, no matter the size of the sentence being input. We can feed the network with sentences of 3 or 300 words without changing the parameter.
- **Caring about word order**: It processes the sequence word by word in order, caring about the word position.
- **Suitable for working in various modes**: We can train a machine translation model or sentiment analysis using the same recurrency paradigm. Both architectures would be based on an RNN:
  - **One-to-many mode**: RNN can be redesigned in a one-to-many model for language generation or music generation. (e.g., a word -> its definition in a sentence)
  - **Many-to-one mode**: It can be used for text classification or sentiment analysis (e.g., a sentence as a list of words -> sentiment score)
  - **Many-to-many mode**: The use of many-to-many models is to solve encoder-decoder problems such as machine translation, question answering, and text summarization or **Named Entity Recognition** (**NER**)-like sequence labeling problems

The disadvantages of an RNN architecture are listed here:

- **Long-term dependency problem**: When we process a very long document and try to link the terms that are far from each other, we need to care about and encode all irrelevant other terms between these terms.
- **Prone to exploding or vanishing gradient problems**: When working on long documents, updating the weights of the very first words is a big deal, which makes a model untrainable due to a vanishing gradient problem.
- **Hard to apply parallelizable training**: Parallelization breaks the main problem down into a smaller problem and executes the solutions at the same time, but RNN follows a classic sequential approach. Each layer strongly depends on previous layers, which makes parallelization impossible.
- **The computation is slow as the sequence is long**: An RNN could be very efficient for short text problems. It processes longer documents very slowly, besides the long-term dependency problem.

Although an RNN can theoretically attend to the information many timesteps before, in the real world, problems such as long documents and long-term dependencies are impossible to discover. Long sequences are represented within many deep layers. These problems have been addressed by many studies, some of which are outlined here:

- Hochreiter and Schmidhuber. *Long Short-term Memory*. 1997.
- Bengio et al. *Learning long-term dependencies with gradient descent is difficult*. 1993.
- K. Cho et al. *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. 2014.

LSTMs and gated recurrent units

LSTM networks (*Schmidhuber, 1997*) and **gated recurrent units** (**GRUs**) (*Cho, 2014*) are types of RNNs that are designed to address the problem of long-term dependencies. One key feature of LSTMs is the use of a cell state, which is a horizontal sequence line located above the LSTM unit and controlled by specialized gates that handle forget, insert, or update operations. The complex structure of an LSTM is shown in the following diagram:

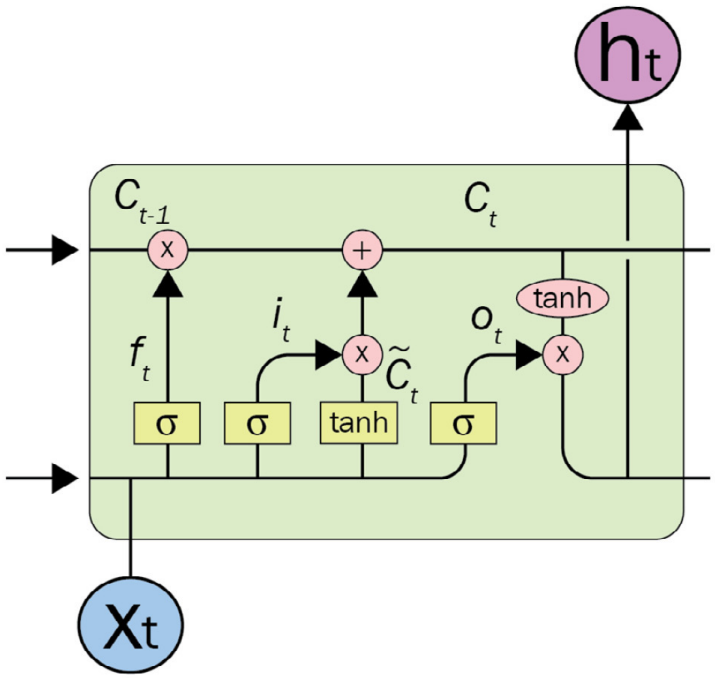


Figure 1.4 – An LSTM unit

The design is able to decide the following:

- What kind of information we will store in the cell state
- Which information will be forgotten or deleted

In the original RNN, in order to learn the state of any token, it recurrently processes the entire state of previous tokens. Carrying entire information from earlier timesteps leads to vanishing gradient problems, which makes the model untrainable. You can think of it this way: the difficulty of carrying information increases exponentially with each step. For example, let's say that carrying information with the previous token is 2 units of difficulty; with the 2 previous tokens, it would be 4 units, and with 10 previous tokens, it becomes 1,024 units of difficulty.

The gate mechanism in LSTM allows the architecture to skip some unrelated tokens at a certain timestep or remember long-range states to learn the current token state. The GRU is similar to an LSTM in many ways, the main difference being that a GRU does not use the cell state. Rather, the architecture is simplified by transferring the functionality of the cell state to the hidden state, and it only includes two gates: an **update gate** and a **reset gate**. The update gate determines how much information from the previous and current timesteps will be pushed forward. This feature helps the model keep relevant information from the past, which minimizes the risk of a vanishing gradient problem as well. The reset gate detects the irrelevant data and makes the model forget it.

In a seq2seq pipeline with traditional RNNs, it was necessary to compress all the information into a single representation in the input data. However, this can make it difficult for the output tokens to focus on specific parts of the input tokens. *Bahdanau's attention mechanism* was first applied to RNNs and has shown to be effective in addressing this issue and has become a key concept in the Transformer architecture. We will examine how the Transformer architecture uses attention in more detail later in the book.

Contextual word embeddings and TL

Traditional word embeddings such as word2vec were important components of many neural language understanding models. However, using these embeddings in a language modeling task would result in the same representation for the word *bank* regardless of the context in which it appears. For example, the word *bank* would have the same embedding whether it refers to a *financial institution* or *side of a river* with traditional vectors. At this point, context is important for humans as well as machines to understand the meaning of a word.

In this line, ELMo and ULMFiT were the pioneering models that achieved contextual word embeddings and efficient TL in NLP. They got good results on a variety of language understanding tasks, such as question answering, NER, relation extraction, and so forth. These contextual embeddings capture both the meaning of a word and the context in which it appears. Unlike traditional word embeddings, which use a static representation for each word, they use bi-directional LSTM to encode a word by considering the entire sentence in which it appears.

They demonstrated that pre-trained word embeddings can be reused for other NLP tasks once the pre-training has been completed. ULMFiT, in particular, was successful in applying TL to NLP tasks. At that time, TL was already commonly used in computer vision, but existing NLP approaches still required task-specific modifications and training from scratch. ULMFiT introduced an effective TL method that can be applied to any NLP task and also demonstrated techniques for fine-tuning a language model. The ULMFiT process consists of three stages. The first stage is the pre-training of a general domain language model on a large corpus to capture general language features at different layers. The second stage involves fine-tuning the pre-trained language model on a target task dataset using discriminative fine-tuning to learn task-specific features. The final stage involves fine-tuning the classifier on the target task with gradual unfreezing. This approach maintains low-level representations while adapting to high-level ones.

Now, we finally come to our main topic—Transformers!

Overview of the Transformer architecture

Transformer models have received immense interest because of their effectiveness in an enormous range of NLP problems, from text classification to text generation. The attention mechanism is an important part of these models and plays a very crucial role. Before Transformer models, the attention mechanism was proposed as a helper for improving conventional DL models such as RNNs. To understand Transformers and their impact on NLP, we will first study the attention mechanism.

Attention mechanism

The attention mechanism allowed for the creation of a more advanced model by connecting specific tokens in the input sequence to specific tokens in the output sequence. For instance, suppose you have the keyword phrase `Canadian Government` in the input sentence for an English-to-Turkish translation task. In the output sentence, the `Kanada Hükümeti` tokens make strong connections with the input phrase and establish a weaker connection with the remaining words in the input, as illustrated in the following figure:

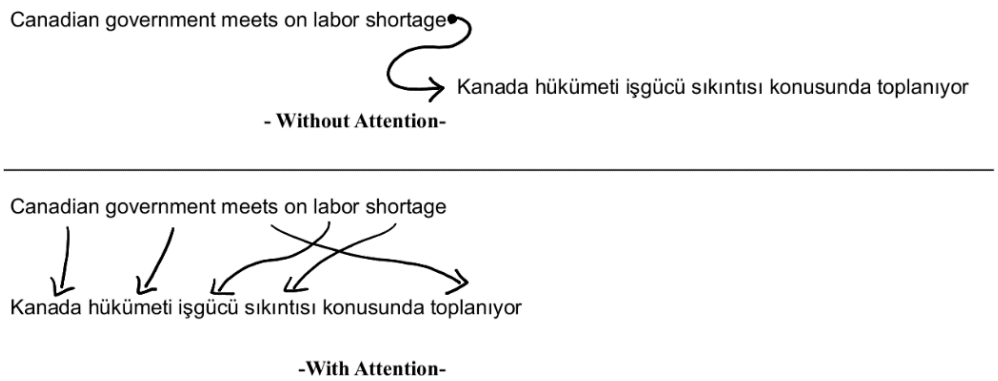


Figure 1.5 – Sketchy visualization of an attention mechanism

This mechanism makes models more successful in seq2seq tasks such as translation, question answering, and text summarization.

One of the first variations of the attention mechanism was proposed by Bahdanau et al. (2015). This mechanism is based on the fact that RNN-based models such as GRUs or LSTMs have an information bottleneck on tasks such as **neural machine translation** (**NMT**). These encoder-decoder-based models get the input in the form of a token ID and process it in a **recurrent fashion** (**encoder**). Afterward, the processed intermediate representation is fed into another **recurrent unit** (**decoder**) to extract the

results. This avalanche-like information is like a rolling ball that consumes all the information, and rolling it out is hard for the decoder part because it does not see all the dependencies and only gets the intermediate representation (context vector) as input.

To align this mechanism, Bahdanau proposed an attention mechanism to use weights on intermediate hidden values. These weights align the amount of attention that a model must pay to the input in each decoding step. Such wonderful guidance assists models in specific tasks such as NMT, which is a many-to-many task. Different attention mechanisms have been proposed with different improvements. *Additive*, *multiplicative*, *general*, and *dot-product* attention appear within the family of these mechanisms. The latter, which is a modified version with a scaling parameter, is noted as *scaled dot-product* attention. This specific attention type is the foundation of Transformers models and is called a **multi-head attention mechanism**. Additive attention is also what was introduced earlier as a notable change in NMT tasks. You can see an overview of the different types of attention mechanisms here:

Name	**Attention score function**
Content-based attention	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \text{cosine}[\boldsymbol{s}_t, \boldsymbol{h}_i]$
Additive	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\boldsymbol{s}_t; \boldsymbol{h}_i])$
Location-based	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a \boldsymbol{s}_t)$
General	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \mathbf{W}_a \boldsymbol{h}_i$
Dot-product	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \boldsymbol{h}_i$
Scaled dot-product	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \frac{\boldsymbol{s}_t^\top \boldsymbol{h}_i}{\sqrt{n}}$

Table 1.2 – Types of attention mechanisms

In *Table 1.2*, *h* and *s* represent the hidden state and state itself, respectively, while *W* denotes the weights specific to the attention mechanism.

Since attention mechanisms are not specific to NLP; they are also used in different use cases in various fields, from computer vision to speech recognition. The following figure shows a visualization of a multimodal approach trained for neural image captioning (K Xu et al., *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention*, 2015):

Figure 1.6 – Attention in Computer Vision

Another example is as follows:

Figure 1.7 – Another example for Attention mechanism in computer vision

Next, let's understand multi-head attention mechanisms.

Multi-head attention mechanisms

The multi-head attention mechanism that is shown in the following diagram is an essential part of the Transformer architecture:

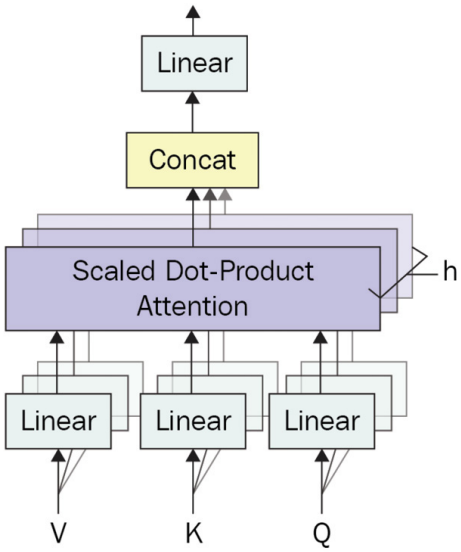


Figure 1.8 – Multi-head attention mechanism

Before jumping into scaled dot-product attention mechanisms, it's better to get a good understanding of self-attention. **Self-attention**, as shown in *Figure 1.8*, is a basic form of a scaled self-attention mechanism. This mechanism uses an input matrix and produces an attention score between various items

Q in *Figure 1.9* is known as the **query**, *K* is known as the **key**, and *V* is noted as the **value**. Three types of matrices shown as *theta*, *phi*, and *g* are multiplied by *X* before producing *Q*, *K*, and *V*. The multiplied result between the query (*Q*) and key (*K*) yields an attention score matrix. This can also be seen as a database where we use the query and keys in order to find out how various items are related in terms of numeric evaluation. Multiplication of the attention score and the *V* matrix produces the final result of this type of attention mechanism. The main reason for it being called *self-attention* is because of its unified input *X* (*Q*, *K*, and *V* are computed from *X*). You can see all this depicted in the following diagram:

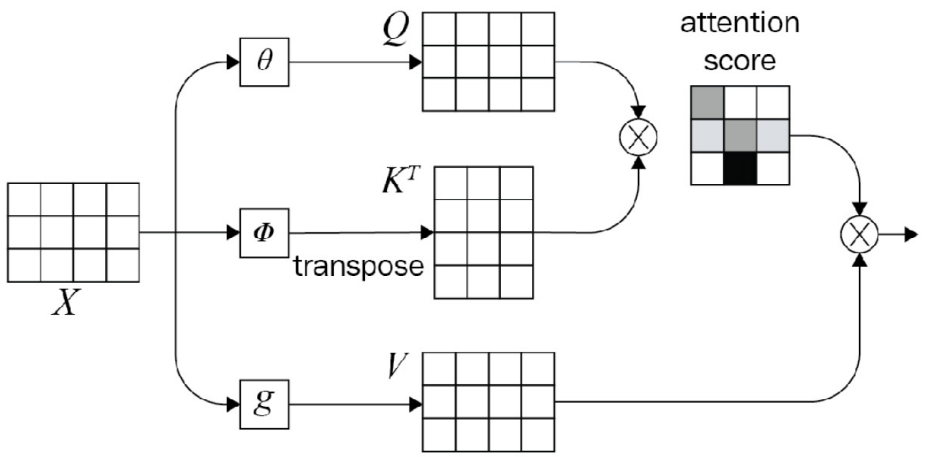


Figure 1.9 – Mathematical representation for the attention mechanism (diagram inspiration from https://blogs.oracle.com/datascience/multi-head-self-attention-in-nlp)

A scaled dot-product attention mechanism is very similar to a self-attention (dot-product) mechanism except it uses a scaling factor. The multi-head part, on the other hand, ensures the model can look at various aspects of input at all levels. Transformer models attend to encoder annotations and the hidden values from past layers. The architecture of the Transformer model does not have a recurrent step-by-step flow; instead, it uses positional encoding to receive information about the position of each token in the input sequence. The concatenated values of the embeddings (randomly initialized) and the fixed values of positional encoding are the input fed into the layers in the first encoder part and are propagated through the architecture, as illustrated in the following diagram:

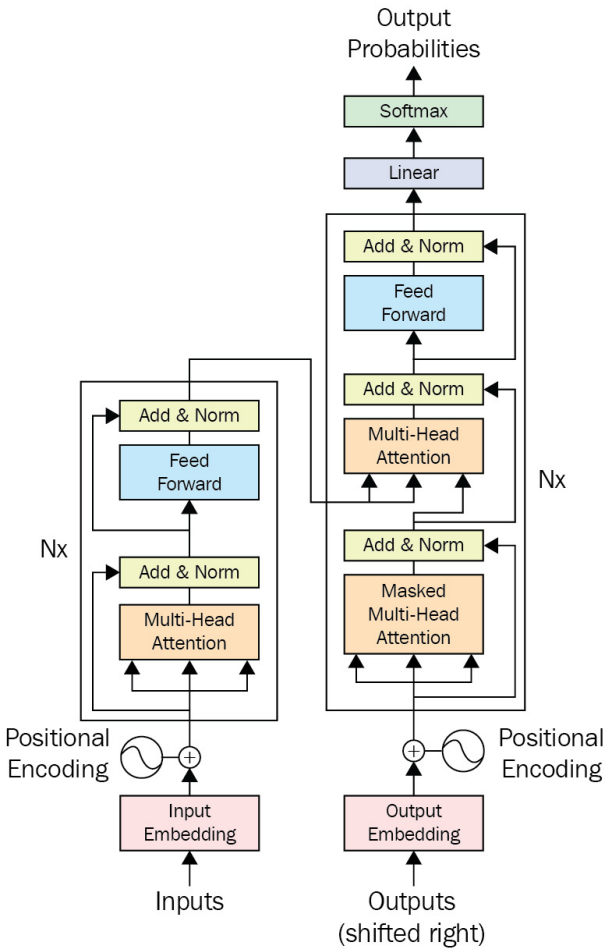


Figure 1.10 – A Transformer architecture

The positional information is obtained by evaluating sine and cosine waves at different frequencies. An example of positional encoding is visualized in the following figure:

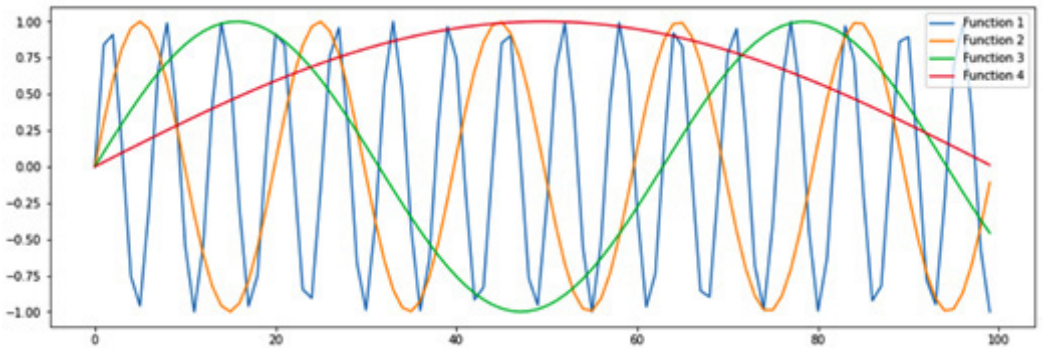


Figure 1.11 – Positional encoding

A good example of performance for the Transformer architecture and the scaled dot-product attention mechanism is given in the following figure:

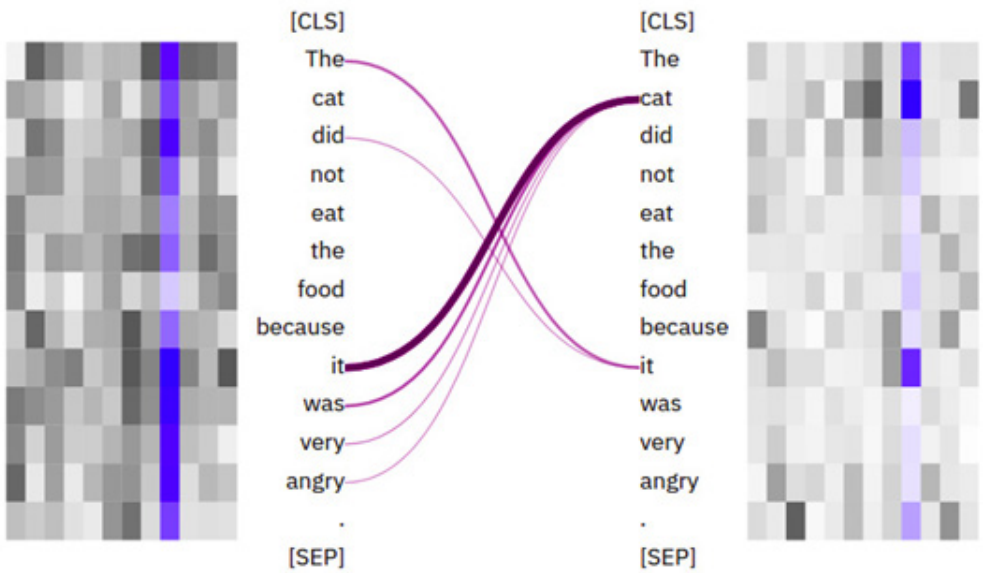


Figure 1.12 – Attention mapping for Transformers

The word `it` refers to different entities in different contexts. As seen in *Figure 1.12*, it refers to `cat`. If we changed `angry` to `stale`, it would refer to `food`. Another improvement made by using a Transformer architecture is in parallelism. Conventional sequential recurrent models such as LSTMs and GRUs do not have such capabilities because they process the input token by token. Feed-forward layers, on the other hand, speed up a bit more because single matrix multiplication is far faster than a recurrent unit. Stacks of multi-head attention layers gain a better understanding of complex sentences.

On the decoder side of the attention mechanism, a very similar approach to the encoder is utilized with small modifications. A multi-head attention mechanism is the same, but the output of the encoder stack is also used. This encoding is given to each decoder stack in the second multi-head attention layer. This little modification introduces the output of the encoder stack while decoding. This modification lets the model be aware of the encoder output while decoding and, at the same time, helps it during training to have a better gradient flow over various layers. The final softmax layer at the end of the decoder layer is used to provide outputs for various use cases such as NMT, for which the original Transformer architecture was introduced.

This architecture has two inputs, noted as inputs and outputs (shifted right). One is always present (inputs) in both training and inference, while the other is just present in training and inference, which is produced by the model. The reason we do not use model predictions in inference is to stop the model from going too wrong by itself. But what does it mean? Imagine a neural translation model trying to translate a sentence from English to French—at each step, it makes a prediction for a word, and it uses that predicted word to predict the next one. But if it goes wrong at some step, all the subsequent predictions will be wrong, too. To stop the model from going wrong like this, we provide the correct words as a shifted-right version.

A visual example of a Transformer model is given in the following diagram. It shows a Transformer model with two encoders and two decoder layers. The **Add & Normalize** layer from this diagram adds and normalizes the input it takes from the **Feed Forward** layer:

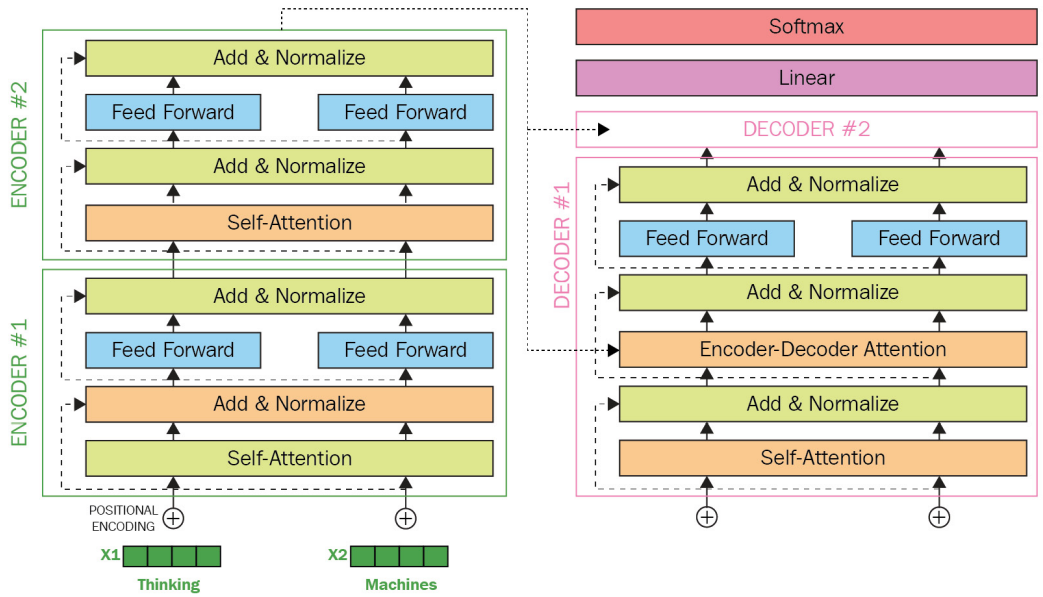


Figure 1.13 – Transformer model (inspiration from http://jalammar.github.io/illustrated-Transformer/)

Another major improvement that is used by a Transformer-based architecture is based on a simple universal text compression scheme to prevent unseen tokens on the input side. This approach, which takes place by using different methods such as byte-pair encoding and sentence-piece encoding, improves a Transformer's performance in dealing with unseen tokens. It also guides the model when the model encounters morphologically close tokens. Such tokens were unseen in the past and are rarely used in the training, and yet, an inference might be seen. In some cases, chunks of it are seen in training; the latter happens in the case of morphologically rich languages such as Turkish, German, Czech, and Latvian. For example, a model might see the word *training* but not *trainings*. In such cases, it can tokenize *trainings* as *training+s*. These two are commonly seen when we look at them as two parts.

Transformer-based models have quite common characteristics—for example, they are all based on this original architecture with differences in the steps they use and don't use. In some cases, minor differences are made—for example, improvements to the multi-head attention mechanism taking place.

Now, we will discuss how to apply TL within Transformers in the following section.

Using TL with Transformers

TL is a field of AI that aims to make models reusable for different tasks—for example, a model trained on a given task such as *A* is reusable (fine-tuning) on a different task such as *B*. In an NLP field, this is achievable by using Transformer-like architectures that can capture the understanding of language itself by language modeling. Such models are called **language models**—they provide a model for the language they have been trained on. TL is not a new technique, and it has been used in various fields such as computer vision. ResNet, Inception, **Visual Geometry Group** (**VGG**), and EfficientNet are examples of such models that can be used as pre-trained models able to be fine-tuned on different computer vision tasks.

Shallow TL using models such as Word2vec, GloVe, and Doc2vec is also possible in NLP. It is called *shallow* because there is no model to be transferred behind this kind of TL and instead, the pre-trained vectors for words/tokens are transferred. You can use these token- or document-embedding models followed by a classifier or use them combined with other models such as RNNs instead of using random embeddings.

TL in NLP using Transformer models is also possible because these models can learn a language itself without any labeled data. Language modeling is a task used to train transferable weights for various problems. Masked language modeling is one of the methods used to learn a language itself. As with Word2vec's window-based model for predicting center tokens, in masked language modeling, a similar approach takes place but with key differences. Given a probability, each word is masked and replaced with a special token such as [*MASK]*. The language model (a Transformer-based model, in our case) must predict the masked words. Instead of using a window, unlike with Word2vec, a whole sentence is given, and the output of the model must be the same sentence with masked words filled.

One of the first models that used the Transformer architecture for language modeling is **BERT**, which is based on the encoder part of the Transformer architecture. Masked language modeling is accomplished by BERT using the same method described before and after training a language model. BERT is a transferable language model for different NLP tasks such as token classification, sequence classification, or even question answering.

Each of these tasks is a fine-tuning task for BERT once a language model is trained. BERT is best known for its key characteristics on the base Transformer encoder model, and by altering these characteristics, different versions of it—small, tiny, base, large, and extra-large—are proposed. Contextual embedding enables a model to have the correct meaning of each word based on the context in which it is given—for example, the word *cold* can have different meanings in two different sentences: *cold-hearted killer* and *cold weather*. The number of layers at the encoder part, the input dimension, the output embedding dimension, and the number of multi-head attention mechanisms are key characteristics, as illustrated in the following figure:

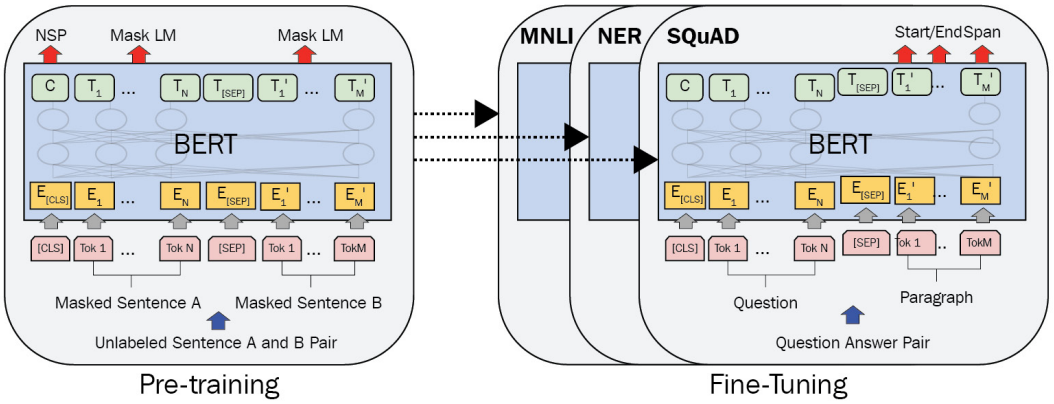


Figure 1.14 – Pre-training and fine-tuning procedures for BERT (image inspiration from *J. Devlin et al., Bert: Pre-training of deep bidirectional Transformers for language understanding, 2018*)

As you can see in *Figure 1.14*, the pre-training phase also consists of another objective known as *next-sentence prediction*. As we know, each document is composed of sentences followed by each other, and another important part of training a model to grasp the language is to understand the relationships of sentences to each other—in other words, whether they are related or not. To achieve these tasks, BERT introduced special tokens such as `[CLS]` and `[SEP]`. A `[CLS]` token is an initially meaningless token used as the starting token for all tasks, and it contains all information about the sentence. In sequence-classification tasks such as **next sentence prediction** (**NSP**), a classifier on top of the output of this token (output position of *0*) is used. It is also useful in evaluating the sense of a sentence or capturing its semantics—for example, when using a Siamese BERT model, comparing these two `[CLS]` tokens for different sentences by a metric such as cosine similarity is very helpful. On the other hand, `[SEP]` is used to distinguish between two sentences, and it is only used to separate two sentences. After pre-training, if someone aims to fine-tune BERT on a sequence classification

task such as sentiment analysis, they will use a classifier on top of the output embedding of `[CLS]`. It is also notable that all TL models can be frozen during fine-tuning or freed; **frozen** means seeing all weights and biases inside the model as constants and stopping training on them. In the example of sentiment analysis, just the classifier will be trained, not the model if it is frozen.

In the next section, you will learn about multimodal learning. You will also get familiar with different architectures that use this learning paradigm with respect to Transformers.

Multimodal learning

Multimodal learning is a general topic in AI that refers to solutions where the associated data is not in a single modality (only image, only text, etc.) but instead, more than one modality is involved. As an example, consider a problem where both an image and text are involved as input or output. Another example can be a cross-modality problem where the input and output modalities are not the same.

Before jumping into multimodal learning using Transformers, it is useful to describe how they can be used for images as well. Transformers get the input in the form of a sequence but, unlike text, images are not 1D sequences. One of the approaches in this field tries to convert the image into patches. Each patch is linearly projected into a vector shape and positional encoding is applied.

Figure 1.15 shows the architecture of the **Vision Transformer** (**ViT**) and how it works:

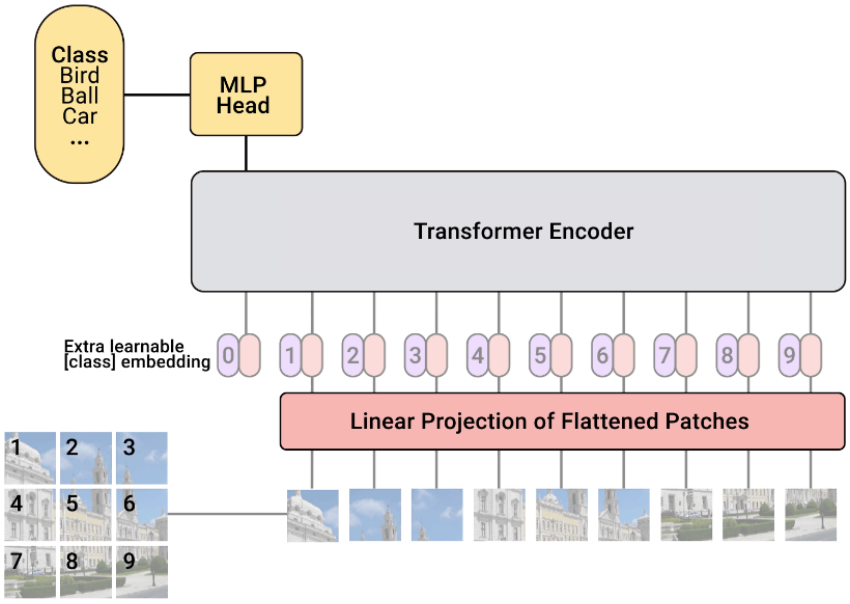


Figure 1.15 – Vision Transformer (https://ai.googleblog.com/2020/12/transformers-for-image-recognition-at.html)

Like architectures such as BERT, a classification head can be applied for tasks such as image classification. However, other use cases and applications can be drawn from this approach as well.

Using a Transformer for images or text separately can create a nice model that can understand text or images. But if we want to have a model that can understand both at the same time and create a link between text and images, it would require training both with constraints. **Contrastive Language–Image Pre-training** (**CLIP**) is one of the models that can understand images and text. It can be used for semantic search where the input can be a text/image and the output is a text/image.

The next figure shows how the CLIP model is trained by using a dual encoder:

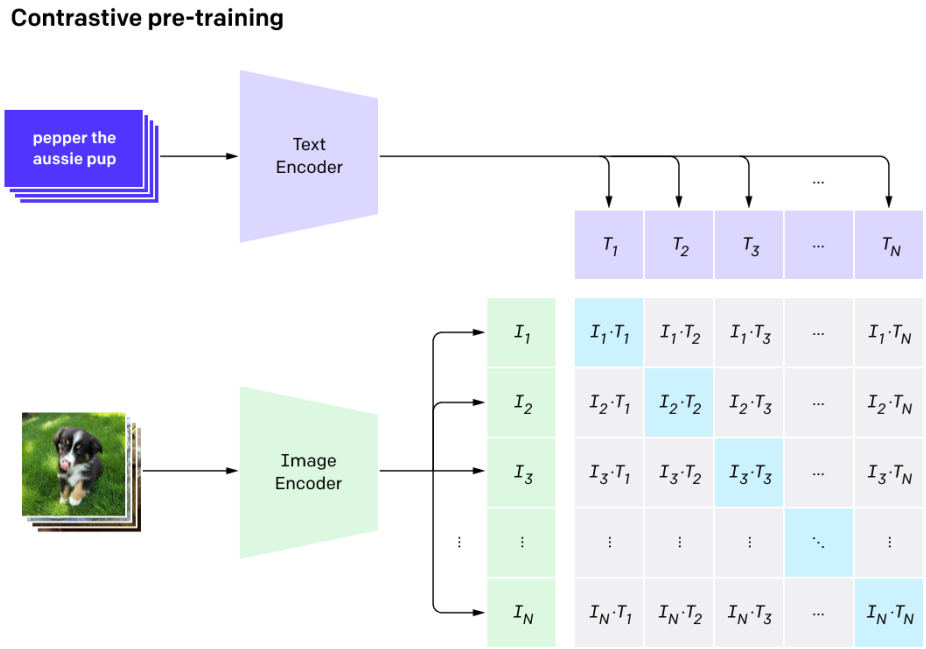


Figure 1.16 – CLIP model contrastive pre-training (`https://openai.com/blog/clip/`)

As it is clear from the CLIP architecture, it will be very useful for zero-shot prediction for text and image modalities. DALL-E and diffusion-based models such as Stable Diffusion are in this category.

The **Stable Diffusion pipeline** is shown in the next figure:

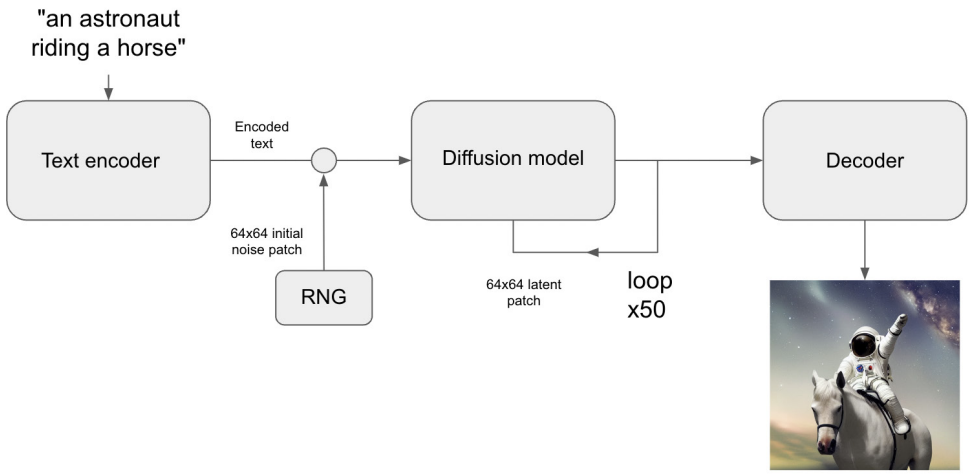


Figure 1.17 – Stable Diffusion pipeline

The preceding diagram can also be viewed at `https://www.tensorflow.org/tutorials/generative/generate_images_with_stable_diffusion`, and the license is as follows: `https://creativecommons.org/licenses/by/4.0/`.

For example, Stable Diffusion uses a text encoder to convert text into dense vectors, and, accordingly, a diffusion model tries to create a vector representation of the respective image. The decoder tries to decode this vector representation, and finally, an image with semantic similarity to the text input is produced.

Multimodal learning not only helps us use different modalities for tasks that are always related to image-text but it can also be used in many different modalities combined with text, such as speech, numerical data, and graphs.

Summary

With this, we have reached the end of the chapter. You should now understand the evolution of NLP methods and approaches, from BoW to Transformers. We looked at how to implement BoW, RNN, and CNN-based approaches and understood what Word2vec is and how it helps improve the conventional DL-based methods using shallow TL. We also investigated the foundation of the Transformer architecture, with BERT as an example. We learned about TL and how it is utilized by BERT. We also described the general idea behind multimodal learning and provided a quick introduction to ViT. Models such as CLIP and Stable Diffusion were also described.

At this point, we have learned the basic information that is necessary to continue to the next chapters. We understand the main idea behind Transformer-based architectures and how TL can be applied using this architecture.

In the next chapter, we will see how it is possible to run a simple Transformer example from scratch. The related information about the installation steps will be given, and working with datasets and benchmarks will also be investigated in detail.

References

- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). *Efficient estimation of word representations in vector space.* arXiv preprint arXiv:1301.3781.
- Bahdanau, D., Cho, K., and Bengio, Y. (2014). *Neural machine translation by jointly learning to align and translate.* arXiv preprint arXiv:1409.0473.
- Pennington, J., Socher, R., and Manning, C. D. (2014, October). *GloVe: Global vectors for word representation.* In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP) (pp. 1532-1543).
- Hochreiter, S. and Schmidhuber, J. (1997). *Long short-term memory. Neural computation,* 9(8), 1735-1780.
- Bengio, Y., Simard, P, and Frasconi, P. (1994). *Learning long-term dependencies with gradient descent is difficult.* IEEE transactions on neural networks, 5(2), 157-166.
- Cho, K., et al. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation.* arXiv preprint arXiv:1406.1078.
- Kim, Y. (2014). *Convolutional neural networks for sentence classification.* CoRR abs/1408.5882 (2014). arXiv preprint arXiv:1408.5882.
- Vaswani, A., et al. (2017). *Attention is all you need*. arXiv preprint arXiv:1706.03762.
- Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. (2018). *Bert: Pre-training of deep bidirectional Transformers for language understanding.* arXiv preprint arXiv:1810.04805.
- Alammar (2022). *The Illustrated stable diffusion.* `https://jalammar.github.io/illustrated-stable-diffusion/`
- Zhong, Y., et al. (2022). *Regionclip: Region-based language-image pretraining.* In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 16793-16803).
- Dosovitskiy, A., et al. (2020). *An image is worth 16x16 words: Transformers for image recognition at scale.* arXiv preprint arXiv:2010.11929.
- Radford, A., et al. (2021, July). *Learning transferable visual models from natural language supervision.* In International Conference on Machine Learning (pp. 8748-8763). PMLR.

2

A Hands-On Introduction to the Subject

So far, we have had an overall look at the evolution of **natural language processing** (**NLP**) using **deep learning** (**DL**)-based methods. We have learned some basic information about transformers and their respective architecture. In this chapter, we are going to have a deeper look into how a transformer model can be used. Tokenizers and models, such as **bidirectional encoder representations from transformer** (**BERT**), will be described in more technical detail with hands-on examples, including how to load a tokenizer/model and use community-provided pretrained models. However, before using any specific model, we will understand the installation steps required to provide the necessary environment by using Anaconda. In the installation steps, installing libraries and programs on various operating systems such as Linux, Windows, and macOS will be covered. The installation of **PyTorch** and **TensorFlow**, with two versions of a **central processing unit** (**CPU**) and a **graphics processing unit** (**GPU**), is also shown. A quick jump into a **Google Colaboratory** (**Google Colab**) installation of the transformer library is provided. There is also a section dedicated to using models in the PyTorch and TensorFlow frameworks.

The Hugging Face models repository is also another important part of this chapter, in which finding different models and steps to use various pipelines are discussed—for example, models such as **bidirectional and auto-regressive transformer** (**BART**), BERT, and **TAble PArSing** (**TAPAS**) are detailed, with a glance at **generative pretrained transformer 2** (**GPT-2**) text generation. However, this is purely an overview, and this part of the chapter relates to getting the environment ready and using pretrained models. No model training is discussed here, as this is given greater significance in upcoming chapters.

We first discuss how to use the `Transformer` library for inference by community-provided models; then, we describe the `datasets` library and how to load various datasets and benchmarks and use metrics. Cross-lingual datasets and how to use local files with the `datasets` library are also considered here. With this chapter, we get the system ready for the rest of the book to develop your own NLP pipeline with transformers.

We will cover the following topics in this chapter:

- Installing transformer with Anaconda
- Working with language models and tokenizers
- Working with community-provided models
- Working with multimodal transformers
- Working with benchmarks and datasets
- Benchmarking for speed and memory

Technical requirements

You will need to install the libraries and software listed next. Although having the latest version is a plus, it is mandatory to install versions that are compatible with each other. For more information about the latest version installation for the Hugging Face transformer, take a look at their official web page at `https://huggingface.co/docs/transformers/installation`:

- `Anaconda`
- `transformers 4.25.1`
- `pytorch 1.13.1`
- `tensorflow 2.9.2`
- `datasets 1.4.1`

Finally, all the code shown in this chapter is available in this book's GitHub repository at `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

Installing transformer with Anaconda

Anaconda is a distribution of the Python and R programming languages that makes package distribution and deployment easy for scientific computation. In this section, we will describe the installation of the `Transformer` library. However, it is also possible to install this library without the aid of Anaconda. The main motivation for using Anaconda is to explain the process more easily and moderate the packages used.

To start installing the related libraries, the installation of Anaconda is a mandatory step. The official guidelines provided by the Anaconda documentation offer simple steps to install it for common operating systems (macOS, Windows, and Linux).

Let's see how we can install it on different platforms in the next sub-sections.

Installation on Linux

Many Linux distributions are available for users to enjoy, but **Ubuntu** is one of the preferred ones. In this section, the steps to install Anaconda are covered for Linux. Proceed as follows:

1. Download the Anaconda installer for Linux from `https://www.anaconda.com/products/individual#Downloads` and go to the Linux section, as illustrated in the following screenshot:

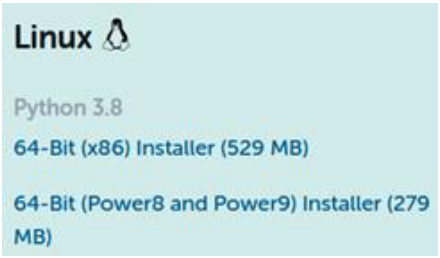


Figure 2.1 – Anaconda download link for Linux

2. Run a `bash` command to install it and complete the following steps:
3. Open the Terminal and run the following command:

```
bash Terminal./FilePath/For/Anaconda.sh
```

4. Press *Enter* to see the license agreement and press *Q* if you do not want to read it all, and then do the following:
5. Click **Yes** to agree.
6. Click **Yes** for the installer to always initialize the `conda` root environment.
7. After running a `python` command from the Terminal, you should see an Anaconda prompt after the Python version information.
8. You can access Anaconda Navigator by running an `anaconda-navigator` command from the Terminal. As a result, you will see the Anaconda **graphical user interface** (**GUI**) start loading the related modules, as shown in the following screenshot:

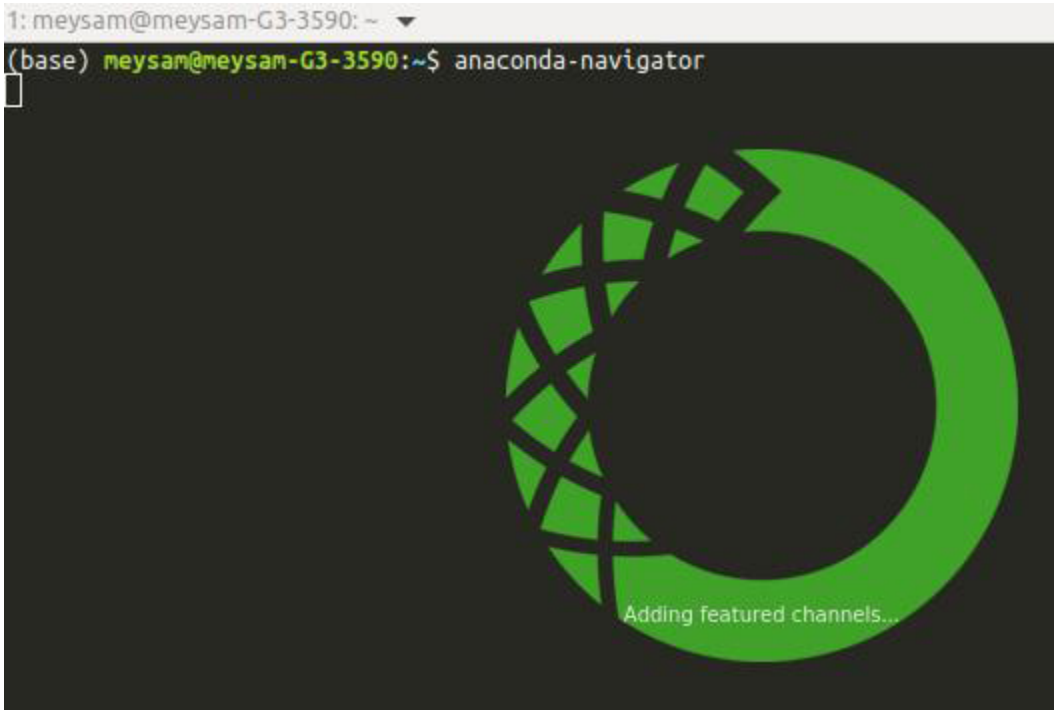


Figure 2.2 – Anaconda Navigator

Let's move on to the next section!

Installation on Windows

The following steps describe how you can install Anaconda on Windows operating systems:

1. Download the installer from `https://www.anaconda.com/products/individual#Downloads` and go to the Windows section, as illustrated in the following screenshot:

Figure 2.3 – Anaconda download link for Windows

2. Open the installer and follow the guide by clicking the **I Agree** button.
3. Select the location for installation, as illustrated in the following screenshot:

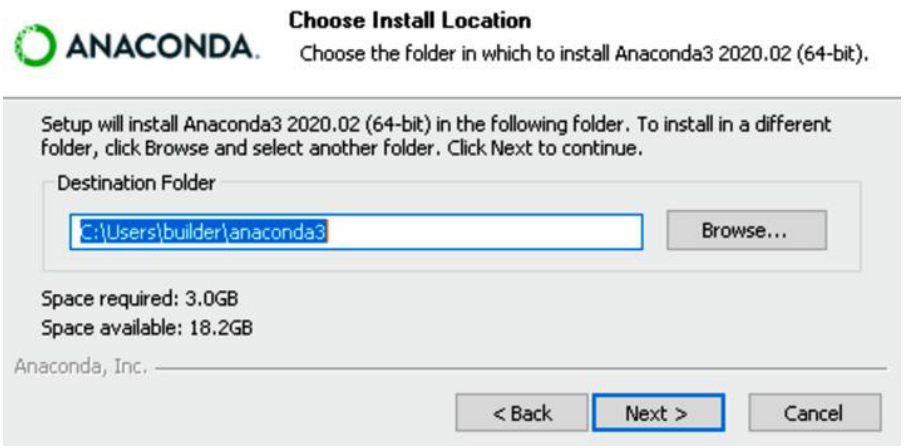


Figure 2.4 – Anaconda installer for Windows

Don't forget to check the **Add anaconda3 to my PATH environment variable** checkbox, as illustrated in the following screenshot. If you do not check this box, the Anaconda version of Python will not be added to the Windows environment variables, and you will not be able to directly run it with a `python` command from the Windows shell or the Windows command line:

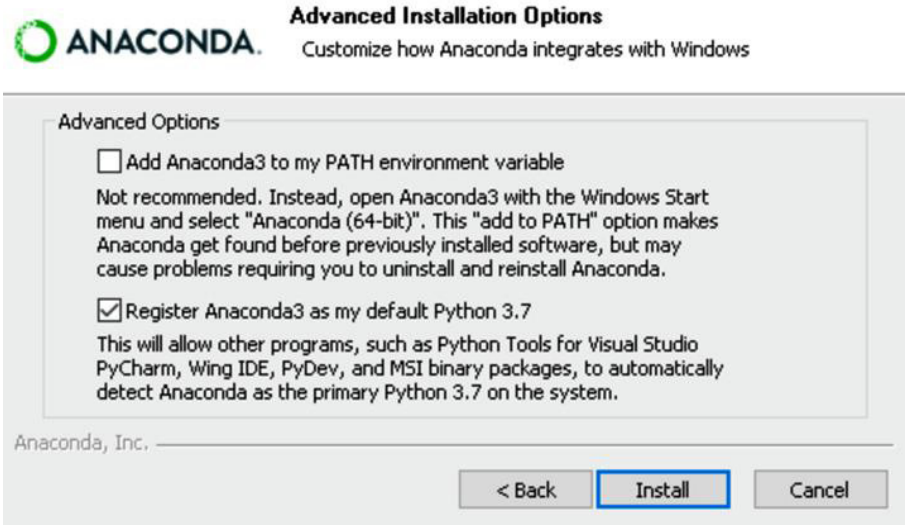


Figure 2.5 – Anaconda installer advanced options

4. Follow the rest of the installation instructions and finish the installation.
5. You should now be able to start Anaconda Navigator from the **Start** menu.

Now that we know how to download and install it on Linux, let's see how we can do it on macOS.

Installation on macOS

The following steps must be followed to install Anaconda on macOS:

1. Download the installer from `https://www.anaconda.com/products/individual#Downloads` and go to the macOS section, as illustrated in the following screenshot:

Figure 2.6 – Anaconda download link for macOS

2. Open the installer.
3. Follow the instructions and click the **Install** button to install macOS in a predefined location, as illustrated in the following screenshot. You can change the default directory, but this is not recommended:

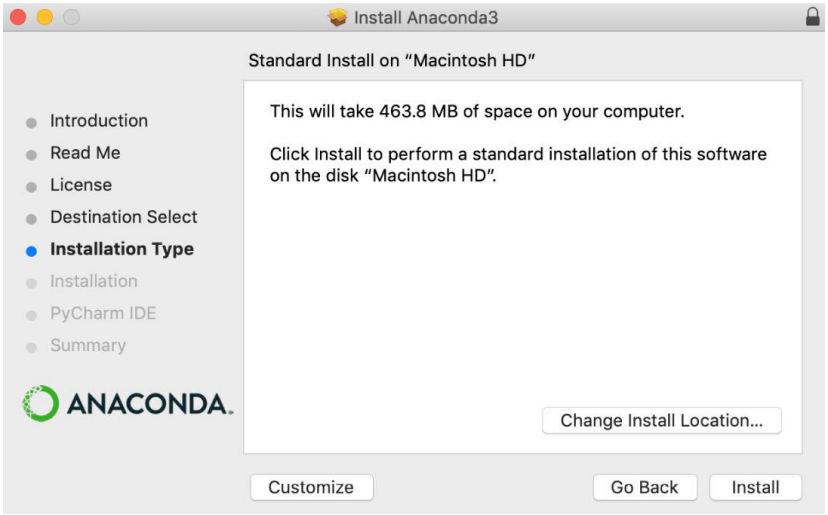


Figure 2.7 – Anaconda installer for macOS

Once you finish the installation, you should be able to access Anaconda Navigator.

Installing TensorFlow, PyTorch, and Transformer

The installation of TensorFlow and PyTorch as two major libraries that are used for DL can be made through `pip` or `conda` itself. `conda` provides a **command-line interface** (**CLI**) for the easier installation of these libraries.

For a clean installation and to avoid interrupting other environments, it is better to create a `conda` environment for the `huggingface` library. You can do this by running the following code:

```
conda create -n Transformer
```

This command will create an empty environment for installing other libraries. Once created, we will need to activate it as follows:

```
conda activate Transformer
```

The installation of the `Transformer` library is easily done by running the following commands:

```
conda install -c conda-forge tensorflow
conda install -c conda-forge pytorch
conda install -c conda-forge transformers
```

The `-c` argument in the `conda install` command lets Anaconda use additional channels to search for libraries.

Note that it is a requirement to have TensorFlow and PyTorch installed because the `Transformer` library uses both of these libraries. An additional note is the easy handling of CPU and GPU versions of TensorFlow by Conda. If you simply put `–gpu` after `tensorflow`, it will install the GPU version automatically. For the installation of PyTorch through the `cuda` library (GPU version), you are required to have related libraries such as `cuda`, but `conda` handles this automatically, and no further manual setup or installation is required. The following screenshot shows how `conda` automatically takes care of installing the PyTorch GPU version by installing the related `cudatoolkit` and `cudnn` libraries:

```
1: meysam@meysam-G3-3590: ~
(base) meysam@meysam-G3-3590:~$ conda install pytorch
Collecting package metadata (current_repodata.json): done
Solving environment: /
The environment is inconsistent, please check the package plan carefully
The following packages are causing the inconsistency:
  - defaults/linux-64::anaconda-navigator==1.9.12=py38_0
  - defaults/linux-64::ipykernel==5.3.4=py38h5ca1d4c_0
  - defaults/noarch::conda-verify==3.4.2=py_1
  - defaults/linux-64::notebook==6.1.4=py38_0
  - defaults/linux-64::nb_conda_kernels==2.2.3=py38_0
  - defaults/noarch::nbclient==0.5.1=py_0
  - conda-forge/linux-64::conda==4.9.2=py38h578d9bd_0
  - defaults/noarch::jupyter_client==6.1.7=py_0
  - defaults/linux-64::terminado==0.9.1=py38_0
  - defaults/linux-64::nbconvert==6.0.7=py38_0
  - defaults/linux-64::conda-build==3.18.11=py38_0
  - conda-forge/linux-64::widgetsnbextension==3.5.1=py38h32f6830_1
  - conda-forge/noarch::ipywidgets==7.5.1=pyh9f0ad1d_1
  - defaults/linux-64::_ipyw_jlab_nb_ext_conf==0.1.0=py38_0
  - defaults/linux-64::conda-package-handling==1.7.2=py38h03888b9_0
done

## Package Plan ##

  environment location: /home/meysam/anaconda3

  added / updated specs:
    - pytorch

The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    _openmp_mutex-4.5          |            1_gnu          22 KB
    _pytorch_select-0.1        |            cpu_0           3 KB
    anyio-2.2.0                |   py38h06a4308_1         125 KB
    babel-2.9.1                |     pyhd3eb1b0_0         5.5 MB
    ca-certificates-2021.7.5   |       h06a4308_1         113 KB
    certifi-2021.5.30          |   py38h06a4308_0         138 KB
```

Figure 2.8 – Conda installing PyTorch and related cuda libraries

Note that all of these installations can also be done without `conda`, but the reason behind using Anaconda is its ease of use. In terms of using environments or installing GPU versions of TensorFlow or PyTorch, Anaconda works like magic and is a good time saver.

Installing and using Google Colab

Even if the utilization of Anaconda saves time and is useful, in most cases, not everyone has such good and reasonable computation resources available. **Google Colab** is a good alternative in such cases. The installation of the `Transformer` library in Colab is carried out with the following command:

```
!pip install transformers
```

An exclamation mark before the statement makes the code run in a Colab shell, which is the equivalent to running the code in the Terminal instead of running it using a Python interpreter. This will automatically install the `Transformer` library.

We will see how to work with a BERT model and tokenizer.

Working with language models and tokenizers

In this section, we will look at using the `Transformer` library with language models, along with their related **tokenizers**. In order to use any specified language model, we first need to import it. We will start with the BERT model provided by Google and use its pretrained version, as follows:

```
from transformers import BertTokenizer
tokenizer = \
    BertTokenizer.from_pretrained('bert-base-uncased')
```

The first line of the preceding code snippet imports the BERT tokenizer, and the second line downloads a pretrained tokenizer for the BERT base version. Note that the uncased version is trained with uncased letters, so it does not matter whether the letters appear in upper- or lowercase. To test and see the output, you must run the following line of code:

```
text = "Using Transformers is easy!"
tokenizer(text)
```

This will be the output:

```
{'input_ids': [101, 2478, 19081, 2003, 3733, 999, 102], 'token_type_
ids': [0, 0, 0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1, 1, 1]}
```

`input_ids` shows the token ID for each token, and `token_type_ids` shows the type of each token that separates the first and second sequence, as shown in the following screenshot:

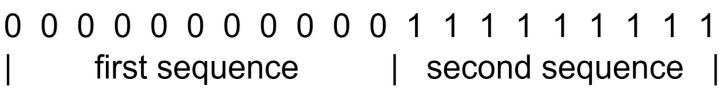


Figure 2.9 – Sequence separation for BERT

`attention_mask` is a mask of 0s and 1s that is used to show the start and end of a sequence for the transformer model in order to prevent unnecessary computations. Each tokenizer has its own way of adding special tokens to the original sequence. In the case of the BERT tokenizer, it adds a `[CLS]` token to the beginning and an `[SEP]` token to the end of the sequence, which can be seen by 101 and 102. These numbers come from the token IDs of the pretrained tokenizer.

A tokenizer can be used for both PyTorch- and TensorFlow-based `Transformer` models. In order to have an output for each one, the `pt` and `tf` keywords must be used in `return_tensors`. For example, you can use a tokenizer by simply running the following command:

```
encoded_input = tokenizer(text, return_tensors="pt")
```

`encoded_input` has the tokenized text to be used by the PyTorch model. In order to run the model—for example, the BERT base model—the following code can be used to download the model from the `huggingface` model repository:

```
from transformer import BertModel
model = BertModel.from_pretrained("BERT-base-uncased")
```

The output of the tokenizer can be passed to the downloaded model with the following line of code:

```
output = model(**encoded_input)
```

This will give you the output of the model in the form of embeddings and cross-attention outputs.

When loading and importing models, you can specify which version of a model you are trying to use. If you simply put `TF` before the name of a model, the `Transformer` library will load the TensorFlow version of it. The following code shows how to load and use the TensorFlow version of BERT base:

```
from transformers import BertTokenizer, TFBertModel
tokenizer = \
    BertTokenizer.from_pretrained('BERT-base-uncased')
model = TFBertModel.from_pretrained("BERT-base-uncased")
text = " Using Transformer is easy!"
encoded_input = tokenizer(text, return_tensors='tf')
output = model(**encoded_input)
```

For specific tasks, such as filling masks using language models, there are pipelines designed by Hugging Face that are ready to use. For example, the task of filling a mask can be seen in the following code snippet:

```
from transformers import pipeline
unmasker = \
    pipeline('fill-mask', model='BERT-base-uncased')
unmasker("The man worked as a [MASK].")
```

This code will produce the following output, which shows the scores and possible tokens to be placed in the `[MASK]` token:

```
[{'score': 0.09747539460659027,  'sequence': 'the man worked
as a carpenter.',  'token': 10533,  'token_str': 'carpenter'},
{'score': 0.052383217960596085,  'sequence': 'the man worked
as a waiter.',  'token': 15610,  'token_str': 'waiter'},
{'score': 0.049627091735601425,  'sequence': 'the man worked
```

```
as a barber.',  'token': 13362,  'token_str': 'barber'},
{'score': 0.03788605332374573,  'sequence': 'the man worked
as a mechanic.',  'token': 15893,  'token_str': 'mechanic'},
{'score': 0.03768084570765495,  'sequence': 'the man worked as a
salesman.',  'token': 18968,  'token_str': 'salesman'}]
```

To get a neat view using pandas, run the following code:

```
pd.DataFrame(unmasker("The man worked as a [MASK]."))
```

The result can be seen in the following screenshot:

	score	sequence	token	token_str
0	0.097475	the man worked as a carpenter.	10533	carpenter
1	0.052383	the man worked as a waiter.	15610	waiter
2	0.049627	the man worked as a barber.	13362	barber
3	0.037886	the man worked as a mechanic.	15893	mechanic
4	0.037681	the man worked as a salesman.	18968	salesman

Figure 2.10 – Output of the BERT mask filling

So far, we have learned how to load and use a pretrained BERT model and have understood the basics of tokenizers, as well as the difference between PyTorch and TensorFlow versions of the models. In the next section, we will learn to work with community-provided models by loading different models, reading the related information provided by the model authors, and using different pipelines such as text-generation or **question-answering** (**QA**) pipelines.

Working with community-provided models

Hugging Face has tons of community models provided by collaborators from large **artificial intelligence** (**AI**) and **information technology** (**IT**) companies such as Google and Facebook. Individuals and universities also provide many interesting models. Accessing and using them is also very easy. To start, you should visit the Transformer models directory available on their website (`https://huggingface.co/models`), as shown in the following screenshot:

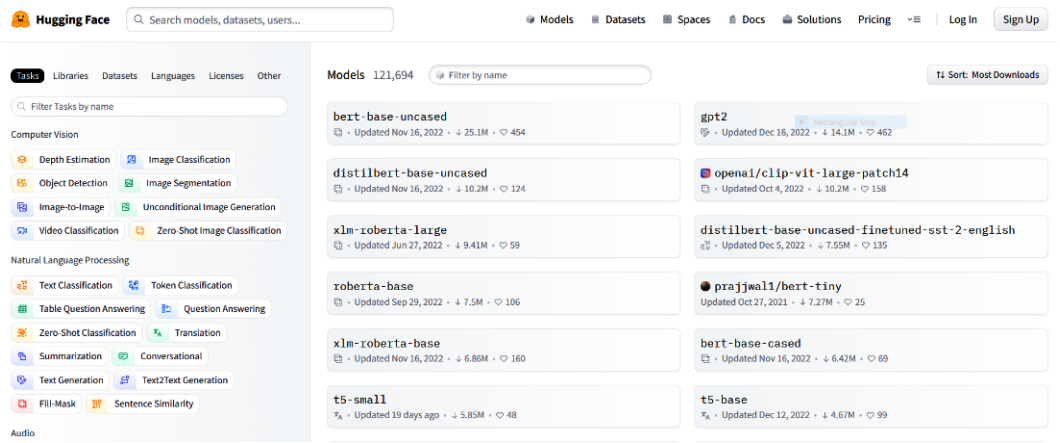


Figure 2.11 – An overview of Hugging Face models repository

Apart from these models, there are also many good and useful datasets available for NLP tasks. To start using some of these models, you can explore them by keyword searches, or just specify your major NLP task and pipeline.

For example, we are looking for a table QA model. After finding a model that we are interested in, a page such as the following one will be available from the Hugging Face website (`https://huggingface.co/google/tapas-base-finetuned-wtq`):

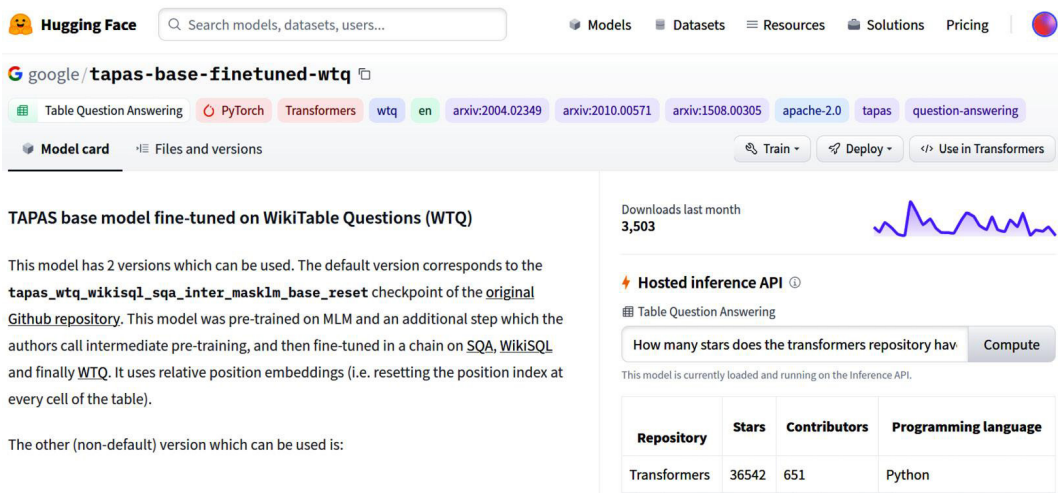


Figure 2.12 – TAPAS model page

On the right side, there is a panel where you can test this model. Note that this is a table QA model that can answer questions about a table provided to the model. If you ask a question, it will reply by highlighting the answer. The following screenshot shows how it gets input and provides an answer for a specific table:

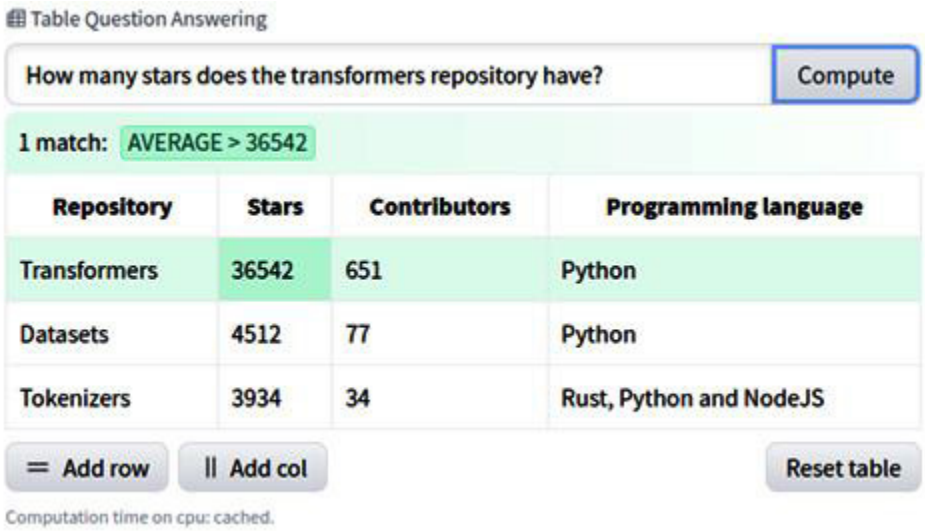


Figure 2.13 – Table QA using TAPAS

Each model has a page provided by the model authors that is also known as a **model card**. You can use the model by the examples provided on the model page. For example, you can visit the GPT-2 `huggingface` repository page and take a look at the example provided by the authors (`https://huggingface.co/gpt2`), as shown in the following screenshot:

How to use

You can use this model directly with a pipeline for text generation. Since the generation relies on some randomness, we set a seed for reproducibility:

```
>>> from transformers import pipeline, set_seed
>>> generator = pipeline('text-generation', model='gpt2')
>>> set_seed(42)
>>> generator("Hello, I'm a language model,", max_length=30, num_return_sequence

[{'generated_text': "Hello, I'm a language model, a language for thinking, a lan
 {'generated_text': "Hello, I'm a language model, a compiler, a compiler library
 {'generated_text': "Hello, I'm a language model, and also have more than a few
 {'generated_text': "Hello, I'm a language model, a system model. I want to know
 {'generated_text': 'Hello, I\'m a language model, not a language model"\n\nThe
```

Figure 2.14 – Text-generation code example from the Hugging Face GPT-2 page

Using pipelines is recommended because all the dirty work is taken care of by the `Transformer` library. As another example, let's assume you need an out-of-the-box zero-shot classifier. The following code snippet shows how easy it is to implement and use such a pretrained model:

```
from transformers import pipeline
classifier = pipeline("zero-shot-classification",
    model="facebook/bart-large-mnli")
sequence_to_classify = "I am going to france."
candidate_labels = ['travel', 'cooking', 'dancing']
classifier(sequence_to_classify, candidate_labels)
```

The preceding code will provide the following result:

```
{'labels': ['travel', 'dancing', 'cooking'], 'scores':
[0.9866883158683777, 0.007197578903287649, 0.006114077754318714],
'sequence': 'I am going to france.'}
```

We are done with the installation and the `hello-world` application part. So far, we have introduced the installation process, completed the environment settings, and experienced the first transformer pipeline. In the next section, we will introduce the `datasets` library, which will be an essential utility in the upcoming experimental chapters.

Working with multimodal transformers

Multimodal transformers such as CLIP are very useful where more than one modality is involved. In order to provide a use-case for this kind of model, a very simple and straight forward example is given.

Zero-shot image classification can be very useful when only class names or phrases related to the classes are available. CLIP, which is a multimodal model, is able to represent images and texts in the same semantic space and can be very handy in this situation. To have a zero-shot image classifier with no prior knowledge about what classes can be or might be, imagine a case where class names are given with no examples for each of them. The only knowledge that is available at the moment is these names or maybe phrases and a set of images, which were not seen before.

1. The first thing that is needed to perform this experiment is to have a few images or download them from the internet:

   ```
   from PIL import Image
   import requests
   url = "http://images.cocodataset.org/test-
   stuff2017/000000000448.jpg"
   image = Image.open(requests.get(url, stream=True).raw)
   ```

2. Now that you have downloaded the image (a test-related image from the COCO dataset), you can easily see if you are using Jupyter Notebook or Colab by just running the following code:

   ```
   image
   ```

Figure 2.15 – A sample for testing CLIP (http://images.cocodataset.org/test-stuff2017/000000000448.jpg)

3. The next step is to just create a prompt to be added for all the classes and then create inputs for the text part:

```
prompt = "a photo of a "
class_names = ["fighting", "meeting"]
inputs = [prompt + class_name for class_name in class_names]
```

4. Up to this point, the inputs for text and image are ready, but we need to load the model:

```
from transformers import CLIPProcessor, CLIPModel
model = (CLIPModel.from_pretrained("openai/clip-vit-large-
patch14"))
processor = \
    (CLIPProcessor.from_pretrained("openai/clip-vit-large-
patch14"))
```

5. This processor is a wrapper that is very useful for tokenization. At the final step, you can hand the pre-processed and tokenized data to the model and finally get the output:

```
inputs = processor(text=inputs, images=image,
    return_tensors="pt", padding=True)
outputs = model(**inputs)
logits = outputs.logits_per_image
probs = logits_per_image.softmax(dim=1)
```

Logits have scores related to each combination (first prompt and image and second prompt and image) *10.9* and *18.5*. However, to get the final class probabilities, a final SoftMax is applied to it and will give the results, which are probabilities in our case (*0.99* for "meeting").

Now that we know the basics of multimodal transformers, let's see how we can use benchmarks and datasets in the next section.

Working with benchmarks and datasets

Thanks to the transformer architecture and transformer library, we are able to transfer what we have learned from a particular task to any other task, which is called **transfer learning** (**TL**). By transferring representations between related problems, we are able to train general-purpose models that share common linguistic knowledge across tasks. We can apply it since current deep learning approaches allow us to solve many tasks at the same time, also known as **multitask learning** (**MTL**), or in order, also known as **sequential transfer learning** (**STL**). Benchmarking mechanisms test the extent to which these capabilities are possessed by the LM.

Before introducing the `datasets` library, it is worth talking about important benchmarks such as **General Language Understanding Evaluation** (**GLUE**), **Cross-lingual TRansfer Evaluation of Multilingual Encoders** (**XTREME**), and **Stanford Question Answering Dataset** (**SquAD**). Benchmarking is especially critical for evaluating transfer learnings within multitask and multilingual environments. In ML, we mostly tend to focus on a particular metric, which is a performance score on a certain task or dataset. With these benchmarks, we can measure the transfer learning capacity of the language models when we try to solve many tasks at the same time.

Let's start with the benchmarks.

Important benchmarks

In the following subsections, we will introduce the important benchmarks that are widely used by transformer-based architectures. These benchmarks exclusively contribute a lot to MTL and to multilingual and zero-shot learning, including many challenging tasks. We will look at the following benchmarks:

- **GLUE**
- **SuperGLUE**
- **XTREME**
- **XGLUE**
- **SQuAD**

In order to use fewer pages, we only detail the tasks for the GLUE benchmark.

GLUE benchmark

Recent studies addressed the fact that multitask training approaches can achieve better results than single-task learning when using a particular model for a task. In this direction, the **GLUE** benchmark has been introduced for MTL, which is a collection of tools and datasets for evaluating the performance of MTL models across a list of tasks. It offers a public leaderboard for monitoring submission performance using the benchmark, along with a single-number metric summarizing 11 tasks. This benchmark includes many sentence-understanding tasks covering various datasets of differing sizes, text types, and difficulty levels. The tasks are categorized under three types, outlined as follows:

- **Single-sentence tasks**:
  - The **Corpus of Linguistic Acceptability** (**CoLA**)dataset. This task consists of English acceptability judgments drawn from articles on linguistic theory.
  - The **Stanford Sentiment Treebank** (**SST-2**) dataset. This task includes sentences from movie reviews and human annotations of their sentiment with `pos/neg` labels.
- **Similarity and paraphrase tasks**:
  - The **Microsoft Research Paraphrase Corpus** (**MRPC**) dataset. This task looks at whether the sentences in a pair are semantically equivalent.
  - The **Quora Question Pairs** (**QQP**) dataset. It contains a pair of questions, and the task is to check if they are semantically identical.
  - The **Semantic Textual Similarity Benchmark** (**STS-B**) dataset. The task involves a set of sentence pairs taken from news headlines, each with a similarity score ranging from 1 to 5.
- **Inference tasks**:
  - The **Multi-Genre Natural Language Inference** (**MNLI**) corpus. It is a collection of sentence pairs, and the task is to predict the relation between premise and hypothesis (entailment, contradiction, or neither).
  - The **Question Natural Language Inference** (**QNLI**) dataset. This is a converted version of SquAD. The task is to check whether a sentence contains the answer to a question.
  - The **Recognizing Textual Entailment** (**RTE**) dataset. This is a task of textual entailment challenges to combine data from various sources. This dataset is similar to the previous QNLI dataset, where the task is to check whether a first text entails a second one.
  - The **Winograd Natural Language Inference** (**WNLI**) schema challenge. This is originally a pronoun resolution task linking a pronoun and a phrase in a sentence. GLUE converted the problem into sentence-pair classification, as detailed next.

SuperGLUE benchmark

Like Glue, **SuperGLUE** is a new benchmark styled with a new set of more difficult language-understanding tasks and currently offers a public leaderboard of around eight language tasks associated with a single-number performance metric similar to that of GLUE. The motivation behind it is that as of writing this book, the current state-of-the-art GLUE score (it is now *91.3* and was *90.8* at the time of the first edition of the book) surpasses human performance (*87.1*). Thus, SuperGLUE provides a more challenging and diverse task toward general-purpose, language-understanding technologies. However, it seems that now the models outperform SuperGLUE as well. While the human performance of SuperGLUE is *89.8*, the current best model performance is *91.3*.

You can access both GLUE and SuperGLUE benchmarks at `gluebenchmark.com`.

XTREME benchmark

In recent years, NLP researchers have increasingly focused on learning general-purpose representations rather than a single task that can be applied to many related tasks. Another way of building a general-purpose language model is by using multilingual tasks. It has been observed that recent multilingual models, such as **Multilingual BERT** (**mBERT**) and **XLM-R**, pre-trained on massive amounts of multilingual corpora, have performed better when transferring them to other languages. Thus, the main advantage here is that cross-lingual generalization enables us to build successful NLP applications in resource-poor languages through zero-shot cross-lingual transfer.

In this direction, the **XTREME** benchmark has been designed. It currently includes around 40 different languages belonging to 12 language families and includes nine different tasks that require reasoning for various levels of syntax or semantics. However, it is still challenging to scale up a model to cover over 7,000 world languages, and there exists a trade-off between language coverage and model capability. Please check out the following link for more details on this: `https://sites.research.google/xtreme`.

XGLUE benchmark

XGLUE is another cross-lingual benchmark to evaluate and improve the performance of cross-lingual pretrained models for **natural language understanding** (**NLU**) and **natural language generation** (**NLG**). It originally consisted of 11 tasks over 19 languages. The main difference from XTREME is that the training data are only available in English for each task. This forces the language models to learn only from the textual data in English and transfer this knowledge to other languages, which is called zero-shot cross-lingual transfer capability. The second difference is that it has tasks for NLU and NLG at the same time. Please check out the following link for more details on this: `https://microsoft.github.io/XGLUE/`.

SQuAD benchmark

SQuAD is a widely used QA dataset in the NLP field. It provides a set of QA pairs to benchmark the reading comprehension capabilities of NLP models. It consists of a list of questions, a reading passage, and an answer annotated by crowdworkers on a set of Wikipedia articles. The answer to the question is a span of text from the reading passage. The initial version, **SQuAD1.1**, doesn't have an unanswerable option where the datasets are collected, so each question has an answer to be found somewhere in the reading passage. The NLP model is forced to answer the question even if this appears impossible. **SQuAD2.0** is an improved version whereby the NLP models must not only answer questions when possible but should also abstain from answering when it is impossible to answer. SQuAD2.0 contains 50,000 unanswerable questions written adversarially by crowdworkers to look similar to answerable ones. Additionally, it also has 100,000 questions taken from SQuAD1.1.

Accessing the datasets with an application programming interface

The `datasets` library provides a very efficient utility to load, process, and share datasets with the community through the Hugging Face hub. Like TensorFlow datasets, the `datasets` library makes it easier to download, cache, and dynamically load the sets directly from the original dataset host upon request. The library also provides evaluation metrics along with the data. Indeed, the hub does not hold or distribute the datasets. Instead, it keeps all information about the dataset, including the owner, preprocessing script, description, and download link. To see other features, please check the `dataset_infos.json` and `DataSet-Name.py` files of the corresponding dataset under the GitHub repository of the book (`https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`).

Let's start by installing the `dataset` library as follows:

```
pip install datasets
```

The following code automatically loads the `cola` dataset using the Hugging Face hub. The `datasets.load_dataset()` function downloads the loading script from the actual path if the data is not cached already:

```
from datasets import load_dataset
cola = load_dataset('glue', 'cola')
cola['train'][25:28]
```

Important note

Reusability of the datasets: As you re-run the code, the datasets library starts caching your loading and manipulation request. It first stores the dataset and starts caching your operations on the dataset, such as splitting, selection, and sorting. You will see a warning message such as "reusing dataset xtreme (/home/savas/.cache/huggingface/dataset...)" or "loading cached sorted…."

In the preceding example, we downloaded the `cola` dataset from the GLUE benchmark and selected a few examples from the `train` split of it.

Currently, there are 19,298 datasets and 116 metrics (it was 661 NLP datasets and 21 metrics in the first edition of the book) for diverse tasks, as the following code snippet shows:

```
from pprint import pprint
from datasets import list_datasets, list_metrics
all_d = list_datasets()
metrics = list_metrics()
print(f"{len(all_d)} datasets and {len(metrics)} metrics \
    exist in the hub\n")
pprint(all_d[:20], compact=True)
pprint(metrics, compact=True)
```

This is the output:

```
19298 datasets and 116 metrics exist in the hub.
['acronym_identification', 'ade_corpus_v2', 'adversarial_qa', 'aeslc',
'afrikaans_ner_corpus', 'ag_news', 'ai2_arc', 'air_dialogue', 'ajgt_
twitter_ar', 'allegro_reviews', 'allocine', 'alt', 'amazon_polarity',
'amazon_reviews_multi', 'amazon_us_reviews', 'ambig_qa', 'amttl',
'anli', 'app_reviews', 'aqua_rat']
['accuracy', 'BERTscore', 'bleu', 'bleurt', 'comet', 'coval', 'f1',
'gleu', 'glue', 'indic_glue', 'meteor', 'precision', 'recall',
'rouge', 'sacrebleu', 'sari', 'seqeval', 'squad', 'squad_v2', 'wer',
'xnli']
```

A dataset might have several configurations. For instance, GLUE, as an aggregated benchmark, has many subsets, such as CoLA, SST-2, and MRPC, as we mentioned before. To access each GLUE benchmark dataset, we pass two arguments, where the first is `glue` and the second is a particular dataset of its example dataset (`cola` or `sst2`) that can be chosen. Likewise, the Wikipedia dataset has several configurations provided for several languages.

A dataset comes with the `DatasetDict` object, including several `Dataset` instances. When the split selection `(split='...')` is used, we get `Dataset` instances. For example, the `CoLA` dataset comes with `DatasetDict`, where we have three splits: **train**, **validation**, and **test**. While the train and validation datasets include two labels (`1` for acceptable and `0` for unacceptable), the label value of the test split is `-1`, which means no label.

Let's see the structure of the `CoLA` dataset object as follows:

```
cola = load_dataset('glue', 'cola')
cola
DatasetDict({
train: Dataset({
features: ['sentence', 'label', 'idx'],
```

```
        num_rows: 8551 })
validation: Dataset({
features: ['sentence', 'label', 'idx'],
        num_rows: 1043 })
test: Dataset({
      features: ['sentence', 'label', 'idx'],
       num_rows: 1063  })
})
cola['train'][12]
{'idx': 12, 'label':1,'sentence':'Bill rolled out of the room.'}
cola['validation'][68]
{'idx': 68, 'label': 0, 'sentence': 'Which report that John was
incompetent did he submit?'}
cola['test'][20]
{'idx': 20, 'label': -1, 'sentence': 'Has John seen Mary?'}
```

The dataset object has some additional metadata information that might be helpful for us: `split`, `description`, `citation`, `homepage`, `license`, and `info`. Let's run the following code:

```
print("1#",cola["train"].description)
print("2#",cola["train"].citation)
print("3#",cola["train"].homepage)


1# GLUE, the General Language Understanding Evaluation
benchmark(https://gluebenchmark.com/) is a collection of
resources for training,evaluating, and analyzing natural language
understanding systems.2# @article{warstadt2018neural,  title={Neural
Network Acceptability Judgments},  author={Warstadt,
Alex and Singh, Amanpreet and Bowman, Samuel
R},  journal={arXiv preprint arXiv:1805.12471},  year={2018}}@
inproceedings{wang2019glue,  title={{GLUE}: A Multi-Task Benchmark and
Analysis Platform for Natural Language Understanding},  author={Wang,
Alex and Singh, Amanpreet and Michael, Julian and Hill, Felix and
Levy, Omer and Bowman, Samuel R.},  note={In the Proceedings of
ICLR.},  year={2019}}3# https://nyu-mll.github.io/CoLA/
```

The GLUE benchmark provides many datasets, as mentioned previously. Let's download the MRPC dataset as follows:

```
mrpc = load_dataset('glue', 'mrpc')
```

Likewise, to access other GLUE tasks, we will change the second parameter as follows:

```
load_dataset('glue', 'XYZ')
```

In order to apply a sanity check of data availability, run the following piece of code:

```
glue=['cola', 'sst2', 'mrpc', 'qqp', 'stsb', 'mnli',
    'mnli_mismatched', 'mnli_matched', 'qnli', 'rte',
```

```
    'wnli', 'ax']
for g in glue:
    _=load_dataset('glue', g)
```

XTREME (working with a cross-lingual dataset) is another popular cross-lingual dataset that we already discussed. Let's pick the `MLQA` example from the XTREME set. MLQA is a subset of the XTREME benchmark, which is designed to assess the performance of cross-lingual QA models. It includes about 5,000 extractive QA instances in the SQuAD format across seven languages, which are English, German, Arabic, Hindi, Vietnamese, Spanish, and Simplified Chinese.

For example, `MLQA.en.de` is an English-German QA example dataset and can be loaded as follows:

```
en_de = load_dataset('xtreme', 'MLQA.en.de')
en_de
    DatasetDict({
test: Dataset({features: ['id', 'title', 'context', 'question',
'answers'], num_rows: 4517
}) validation: Dataset({ features: ['id', 'title', 'context',
'question', 'answers'], num_rows: 512})})
```

It could be more convenient to view it within a pandas DataFrame:

```
import pandas as pd
pd.DataFrame(en_de['test'][0:4])
```

Here is the output of the preceding code:

	answers	context	id	question	title
0	{'answer_start': [31], 'text': ['cell']}	An established or immortalized cell line has a...	037e8929e7e4d2f949ffbabd10f0f860499ff7c9	Woraus besteht die Linie?	Cell culture
1	{'answer_start': [232], 'text': ['1885']}	The 19th-century English physiologist Sydney R...	4b36724f3cbde7c287bde512ff09194cbba7f932	Wann hat Roux etwas von seiner Medullarplatte ...	Cell culture
2	{'answer_start': [131], 'text': ['TRIPS']}	After the Uruguay round, the GATT became the b...	13e58403df16d88b0e2c665953e89575704942d4	Was muss ratifiziert werden, wenn ein Land ger...	TRIPS Agreement

Figure 2.16 – English-German cross-lingual QA dataset

Now, we are ready to manipulate the dataset instances!

Data manipulation using the datasets library

Datasets come with many dictionaries of subsets, where the `split` parameter is used to decide which subset(s) or portion of the subset is to be loaded. If this is `none` by default, it will return a dataset dictionary of all subsets (`train`, `test`, `validation`, or any other combination). If the `split` parameter is specified, it will return a single dataset rather than a dictionary. For the following example, we retrieve a `train` split of the `cola` dataset only:

```
cola_train = load_dataset('glue', 'cola', split ='train')
```

We can get a mixture of the `train` and `validation` subsets as follows:

```
cola_sel = load_dataset('glue', 'cola',
    split = 'train[:300]+validation[-30:]')
```

The `split` expression means that the first 300 examples of `train` and the last 30 examples of `validation` are obtained as `cola_sel`.

We can apply different combinations, as shown in the following split examples:

- The first 100 examples from `train` and `validation`, as shown here:

```
split='train[:100]+validation[:100]'
```

- 50% of `train` and the last 30% of `validation`, as shown here:

```
split='train[:50%]+validation[-30%:]'
```

- The first 20% of `train` and the examples in the slice [*30:50*] from `validation`, as shown here:

```
split='train[:20%]+validation[30:50]'
```

Now, we will see how to sort and shuffle the dataset!

Sorting, indexing, and shuffling

The following execution calls the `sort()` function of the `cola_sel` object. We see the first 15 and the last 15 labels:

```
cola_sel.sort('label')['label'][:15]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
cola_sel.sort('label')['label'][-15:]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

We are already familiar with Python slicing notation. Likewise, we can also access several rows using similar slice notation or with a list of indices as follows:

```
cola_sel[6,19,44]
{'idx': [6, 19, 44],
'label': [1, 1, 1],
 'sentence':['Fred watered the plants flat.',
  'The professor talked us into a stupor.',
  'The trolley rumbled through the tunnel.']}
```

We shuffle the dataset as follows:

```
cola_sel.shuffle(seed=42)[2:5]
{'idx': [159, 1022, 46],
'label': [1, 0, 1],
'sentence': ['Mary gets depressed if she listens to the Grateful
Dead.',
'It was believed to be illegal by them to do that.',
'The bullets whistled past the house.']}
```

> **Important note**
>
> Seed value: When shuffling, we need to pass a seed value to control the randomness and achieve a consistent output between the author and the reader.

Caching and reusability

Using cached files allows us to load large datasets by means of memory mapping (if datasets fit on the drive) by using a fast backend. Such smart caching helps to save and reuse the results of operations executed on the drive. To see cache logs with regard to the dataset, run the following code:

```
cola_sel.cache_files
[{'filename': '/home/savas/.cache/huggingface...,'skip': 0,  'take':
300}, {'filename': '/home/savas/.cache/huggingface...','skip':
1013,  'take': 30}]
```

The output indicated that the data are already cached!

Dataset filter and map function

We might want to work with a specific selection of a dataset. For instance, we can retrieve sentences only, including the term `kick` in the `cola` dataset, as shown in the following execution. The `datasets.Dataset.filter()` function returns sentences, including `kick`, where an anonymous function and a `lambda` keyword are applied:

```
cola_sel = load_dataset('glue', 'cola',
    split='train[:100%]+validation[-30%:]')
cola_sel.filter(lambda s: "kick" in s['sentence'])["sentence"][:3]
['Jill kicked the ball from home plate to third base.', 'Fred kicked
the ball under the porch.', 'Fred kicked the ball behind the tree.']
```

The following filtering is used to get positive (acceptable) examples from the set:

```
cola_sel.filter(lambda s: s['label']== 1 )["sentence"][:3]
["Our friends won't buy this analysis, let alone the next one we
propose.",
"One more pseudo generalization and I'm giving up.",
"One more pseudo generalization or I'm giving up."]
```

In some cases, we might not know the integer code of a class label. Suppose we have many classes, and the code of the `culture` class is hard to remember out of 10 classes. Instead of giving the integer code `1`, as with our preceding example, which is the code for `acceptable`, we can pass an `acceptable` label to the `str2int()` function as follows:

```
cola_sel.filter(lambda s: s['label']== \
    cola_sel.features['label'].str2int('acceptable')
    )["sentence"][:3]
```

This produces the same output as for the previous execution.

Processing data with the map function

The `datasets.Dataset.map()` function iterates over the dataset, applying a processing function to each example in the set, and it modifies the content of the examples. The following execution shows a new `'len'` feature being added that denotes the length of a sentence:

```
cola_new=cola_sel.map(lambda e:{'len': \
    len(e['sentence'])})
pd.DataFrame(cola_new[0:3])
```

This is the output of the preceding code snippet:

	idx	label	len	sentence
0	0	1	71	Our friends won't buy this analysis, let alone...
1	1	1	49	One more pseudo generalization and I'm giving up.
2	2	1	48	One more pseudo generalization or I'm giving up.

Figure 2.17 – Cola dataset with an additional column

As another example, the following piece of code cut the sentence after 20 characters. We do not create a new feature but, instead, update the content of the sentence feature as follows:

```
cola_cut=cola_new.map(lambda e: {'sentence': e['sentence'][:20]+ '_'})
```

The output is shown in the following screenshot:

	idx	label	len	sentence
0	0	1	71	Our friends won't bu_
1	1	1	49	One more pseudo gene_
2	2	1	48	One more pseudo gene_

Figure 2.18 – Cola dataset with an update

This is enough to work with an already prepared dataset. Now, it is time to work with our real datasets.

Working with local files

To load a dataset from local files in a **comma-separated values** (**CSV**), **text** (**TXT**), or **JavaScript object notation** (**JSON**) format, we pass the file type (`csv`, `text`, or `json`) to the generic `load_dataset()` loading script, as shown in the following code snippet. Under the `../data/` folder, there are three CSV files (`a.csv`, `b.csv`, and `c.csv`), which are randomly selected toy examples from the SST-2 dataset. We can load a single file, as shown in the `data1` object, merge many files, as in the `data2` object, or make dataset splits, as in `data3`:

```
from datasets import load_dataset
data1 = load_dataset('csv',
data_files='../data/a.csv',
delimiter="\t")
data2 = load_dataset('csv',
data_files=['../data/a.csv',
    '../data/b.csv',
    '../data/c.csv'],
```

```
delimiter="\t")
data3 = load_dataset('csv',
data_files={
    'train':['../data/a.csv','../data/b.csv'],
    'test':['../data/c.csv']}, delimiter="\t")
```

In order to get the files in other formats, we pass `json` or `text` instead of `csv`, as follows:

```
data_json = load_dataset('json', data_files='a.json')
data_text = load_dataset('text', data_files='a.txt')
```

So far, we have discussed how to load, handle, and manipulate datasets that are either already hosted in the hub or are on our local drive. Now, we will study how to prepare datasets for transformer model training.

Preparing a dataset for model training

Let's start with the tokenization process. Each model has its own tokenization model that is trained before the actual language model. We will discuss it in detail in the next chapter. To use a tokenizer, we should install the `Transformer` library. The following example loads the tokenizer model from the pretrained `distilBERT-base-uncased` model. We use `map` and an anonymous function with `lambda` to apply a tokenizer to each split in `data3`. If `batched` is selected as `True` in the `map` function, it provides a batch of examples to the `tokenizer` function. The `batch_size` value is `1000` by default, which is the number of examples per batch passed to the function. If not selected, the whole dataset is passed as a single batch. The code can be seen here:

```
from Transformer import DistilBERTTokenizer
tokenizer = (DistilBERTTokenizer.from_pretrained(
    'distilBERT-base-uncased'))
encoded_data3 = data3.map(lambda e: tokenizer(
    e['sentence'], padding=True, truncation=True,
    max_length=12), batched=True, batch_size=1000)
```

As shown in the following output, we see the difference between `data3` and `encoded_data3`, where two additional features—`attention_mask` and `input_ids`—are added to the datasets accordingly. We already introduced these two features in the previous part in this chapter. Put simply, `input_ids` are the indices corresponding to each token in the sentence. They are expected features needed by the `Trainer` class of Transformer, which we will discuss in the next fine-tuning chapters.

We mostly pass several sentences at once (called a **batch**) to the tokenizer and further pass the tokenized batch to the model. To do so, we pad each sentence to the maximum sentence length in the batch or a particular maximum length specified by the `max_length` parameter—`12` in this toy example. We also truncate longer sentences to fit that maximum length. The code can be seen in the following snippet:

```
data3

DatasetDict({
train: Dataset({
   features: ['sentence','label'], num_rows: 199 })
test: Dataset({
   features: ['sentence','label'], num_rows: 100 })})
encoded_data3
DatasetDict({
train: Dataset({
  features: ['attention_mask', 'input_ids', 'label',   'sentence'],
   num_rows: 199 })
test: Dataset({
features: ['attention_mask', 'input_ids', 'label', 'sentence'],
num_rows: 100 })})

print(encoded_data3['test'][12])
{'attention_mask': [1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0], 'input_ids':
[101, 2019, 5186, 16010, 2143, 1012, 102, 0, 0, 0, 0, 0], 'label': 0,
'sentence': 'an extremely unpleasant film . '}
```

We are done with the `datasets` library. Up to this point, we have evaluated all aspects of the datasets. We have covered GLUE-like benchmarking, where classification metrics are taken into consideration. In the next section, we will focus on how to benchmark computational performance for speed and memory rather than classification.

Benchmarking for speed and memory

Just comparing the classification performance of large models on a specific task or a benchmark might turn out to be no longer sufficient. We may need faster inference and memory performance. Thus, we must also consider the computational cost of a particular model for a given environment (**random-access memory** (**RAM**), CPU, and GPU) in terms of memory usage and speed. The computational cost of training and deploying to production for inference are the two main values to be measured. Two classes of the `Transformer` library, `PyTorchBenchmark` and `TensorFlowBenchmark`, make it possible to benchmark models for both TensorFlow and PyTorch.

Before we start our experiment, we need to check our GPU capabilities with the following execution:

```
import torch
print(f"The GPU total memory is \
```

```
    {torch.cuda.get_device_properties(0).total_memory/(1024**3)} GB")
The GPU total memory is 2.89 GB
```

The output is obtained from NVIDIA GeForce GTX 1050 (**3 gigabytes** (**GB**)). The `Transformer` library currently only supports single-device benchmarking. When we conduct benchmarking on a GPU, we are expected to indicate which GPU device the Python code will run on, which is done by setting the `CUDA_VISIBLE_DEVICES` environment variable. For example, `export CUDA_VISIBLE_DEVICES=0` indicates that the first `cuda` device will be used.

In the following code example, we compare four randomly selected pretrained BERT models, as listed in the `models` array. We try different sequence lengths with the help of the `sequence_lengths` list parameter. We keep the batch size at `4`. If you have a better GPU capacity, you can extend the parameter search space with batch values in the range 4–64 and with other parameters:

```
from transformers import PyTorchBenchmark, PyTorchBenchmarkArguments
models= ["BERT-base-uncased",
    "distilBERT-base-uncased","distilroBERTa-base",
    "distilBERT-base-german-cased"]
batch_sizes=[4]
sequence_lengths=[32,64, 128, 256,512]
args = PyTorchBenchmarkArguments(models=models,
    batch_sizes=batch_sizes,
    sequence_lengths=sequence_lengths,
    multi_process=False)
benchmark = PyTorchBenchmark(args)
```

Important note

Benchmarking for TensorFlow: The code examples are for PyTorch benchmarking in this part. For TensorFlow benchmarking, we simply use the `TensorFlowBenchmarkArguments` and `TensorFlowBenchmark` counterpart classes instead.

We are ready to conduct the benchmarking experiment by running the following code:

```
results = benchmark.run()
```

This may take some time, depending on your CPU/GPU capacity and argument selection. If you face an out-of-memory problem for this, you should take the following actions to overcome it:

- Restart your kernel or your operating system.
- Delete all unnecessary objects in the memory before starting.
- Set a lower batch size, such as *2*, or even *1*.

The following output indicates the inference speed performance. Since our search space has four different models and five different sequence lengths, we see 20 rows in the results:

```
====================       INFERENCE - SPEED - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length     Time in s
--------------------------------------------------------------------------------
      bert-base-uncased              4               32            0.021
      bert-base-uncased              4               64            0.031
      bert-base-uncased              4              128            0.057
      bert-base-uncased              4              256             0.12
      bert-base-uncased              4              512            0.269
   distilbert-base-uncased           4               32            0.007
   distilbert-base-uncased           4               64            0.011
   distilbert-base-uncased           4              128            0.021
   distilbert-base-uncased           4              256            0.044
   distilbert-base-uncased           4              512            0.095
      distilroberta-base             4               32            0.009
      distilroberta-base             4               64            0.014
      distilroberta-base             4              128            0.025
      distilroberta-base             4              256            0.053
      distilroberta-base             4              512            0.118
 distilbert-base-german-cased        4               32            0.007
 distilbert-base-german-cased        4               64            0.012
 distilbert-base-german-cased        4              128            0.021
 distilbert-base-german-cased        4              256            0.044
 distilbert-base-german-cased        4              512            0.095
--------------------------------------------------------------------------------
```

Figure 2.19 – Inference speed performance

Likewise, we see the inference memory usage for 20 different scenarios as follows:

```
====================       INFERENCE - MEMORY - RESULT      ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length    Memory in MB
--------------------------------------------------------------------------------
      bert-base-uncased              4               32            1453
      bert-base-uncased              4               64            1487
      bert-base-uncased              4              128            1547
      bert-base-uncased              4              256            1661
      bert-base-uncased              4              512            1901
   distilbert-base-uncased           4               32            1908
   distilbert-base-uncased           4               64            1900
   distilbert-base-uncased           4              128            1900
   distilbert-base-uncased           4              256            1900
   distilbert-base-uncased           4              512            1900
      distilroberta-base             4               32            1907
      distilroberta-base             4               64            1900
      distilroberta-base             4              128            1900
      distilroberta-base             4              256            2098
      distilroberta-base             4              512            2492
 distilbert-base-german-cased        4               32            2499
 distilbert-base-german-cased        4               64            2492
 distilbert-base-german-cased        4              128            2492
 distilbert-base-german-cased        4              256            2491
 distilbert-base-german-cased        4              512            2491
--------------------------------------------------------------------------------
```

Figure 2.20 – Inference memory usage

To observe memory usage across the parameters, we will plot them using the `results` object, which stores the statistics. The following execution will plot the time inference performance across models and sequence lengths:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,8))
t=sequence_lengths
models_perf=[list(
    results.time_inference_result[m]['result'][batch_sizes[0]].
values()
    ) for m in models]
plt.xlabel('Seq Length')
plt.ylabel('Time in Second')
plt.title('Inference Speed Result')
plt.plot(t, models_perf[0], 'rs--',
    t, models_perf[1], 'g--.',
    t, models_perf[2], 'b--^',
    t, models_perf[3], 'c--o')
plt.legend(models)
plt.show()
```

As shown in the following screenshot, two DistilBERT models showed close results and performed better than other two models. The `BERT-based-uncased` model performs poorly compared to the others, especially as the sequence length increases:

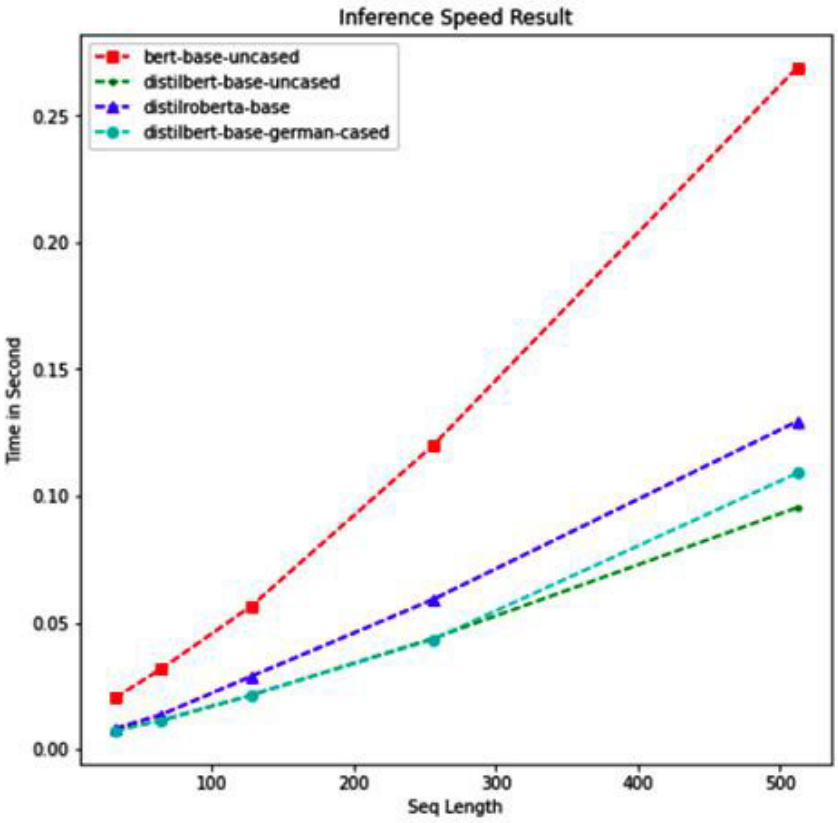


Figure 2.21 – Inference speed result

To plot the memory performance, you need to use the `memory_inference_result` result of the `results` object instead of `time_inference_result`, as shown in the preceding code.

For more interesting benchmarking examples, please check out the following links:

- `https://huggingface.co/transformers/benchmarks.html`
- `https://github.com/huggingface/transformers/tree/master/notebooks`

Now that we are done with this section, we have successfully completed this chapter. Congratulations on achieving the installation, running your first `hello-world` transformer program, working with the `datasets` library, and benchmarking!

Summary

In this chapter, we have covered a variety of introductory topics, and we also got our hands dirty with the `hello-world` transformer application. On the other hand, this chapter plays a crucial role in terms of applying what has been learned so far to the upcoming chapters. So, what has been learned so far? We took the first small step by setting up the environment and system installation. In this context, the `anaconda` package manager helped us to install the necessary modules for the main operating systems. We also went through language models, community-provided models, and tokenization processes. We have tested a very simple approach for zero-shot image classification using multimodal models and a very simple prompt. Additionally, we introduced multitask (**GLUE**) and cross-lingual benchmarking (**XTREME**), which enables these language models to become stronger and more accurate. The `datasets` library was introduced, which facilitates efficient access to NLP datasets provided by the community. Finally, we learned how to evaluate the computational cost of a particular model in terms of memory usage and speed. Transformers frameworks make it possible to benchmark models for both TensorFlow and PyTorch.

The models used in this section were already trained and shared with us by the community. Now, it is our turn to train a language model and disseminate it to the community. In the next chapter, we will learn how to train a BERT language model, as well as a tokenizer, and how to share them with the community.

Part 2: Transformer Models: From Autoencoders to Autoregressive Models

In this part, we'll delve into the fascinating world of encoder-based and generative language models. We'll explore how these models are pre-trained and fine-tuned, providing you with valuable insights into everyday topics such as fine-tuning, inference, and parameter optimization. But we're not stopping there. We'll also tackle advanced subjects such as enhancing model performance and effective fine-tuning strategies. The knowledge gained here will pave the way for the more advanced topics to come.

This part has the following chapters:

- *Chapter 3, Autoencoding Language Models*
- *Chapter 4, From Generative Models to Large Language Models*
- *Chapter 5, Fine-Tuning Language Model for Text Classification*
- *Chapter 6, Fine-Tuning Language Model for Token Classification*
- *Chapter 7, Text Representation*
- *Chapter 8, Boosting Model Performance*
- *Chapter 9, Parameter Efficient Fine-Tuning*

3
Autoencoding Language Models

In the previous chapter, we looked at and studied how a typical transformer model can be used by Hugging Face's Transformers. All of the topics in this book so far have included instructions on how to use pre-trained or pre-built models and less information has been given about specific models and their training.

In this chapter, we will learn how we can train **autoencoding language models** on any given language from scratch. This training will include pretraining and task-specific training of the models. First, we will start with learning about the BERT model and how it works. Then, we will train the language model using a simple and small corpus. Afterward, we will look at how the model can be used inside any Keras model.

For an overview of what will be learned in this chapter, we will cover the following topics:

- **Bidirectional Encoder Representations from Transformers** (**BERT**) – one of the autoencoding language models
- Autoencoding language model training for any language
- Sharing models with the community
- Understanding other autoencoding models
- Working with tokenization algorithms

Technical requirements

The technical requirements for this chapter are as follows:

- `Anaconda`
- `transformers >= 4.0.0`

- `pytorch >= 1.0.2`
- `tensorflow >= 2.4.0`
- `datasets >= 1.4.1`
- `tokenizers`

Please also check the corresponding GitHub code of *Chapter 3* using `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

BERT – one of the autoencoding language models

BERT was one of the first autoencoding language models to utilize the encoder Transformer stack with slight modifications for language modeling.

The BERT architecture is a multilayer Transformer encoder based on the original Transformer implementation. The Transformer model itself was originally made for machine translation tasks, but the main improvement made by BERT is the utilization of this part of the architecture to provide better language modeling. This language model, after pretraining, is able to provide a global understanding of the language it is trained on.

In the next subsections, you will learn about autoencoding models such as BERT. You will also learn how to pretrain a model and share it with a community. In the next section, BERT is explained in more detail.

BERT language model pretraining tasks

To have a clear understanding of the MLM used by BERT, let's define it in more detail. MLM is the task of training a model on input (a sentence with some masked tokens) and obtaining the output as the whole sentence with the masked tokens filled. But how and why does it help a model to obtain better results on downstream tasks such as classification? The answer is simple: if the model can do a **cloze test** (a linguistic test for evaluating language understanding by filling in blanks), then it has a general understanding of the language itself. For other tasks, it has been pretrained (by language modeling) and will perform better.

Here's an example of a cloze test: *George Washington was the first President of the ___ States.*

It is expected that "United" should fill in the blank. For a masked language model, the same task is applied, and it is required to fill in the masked tokens. However, masked tokens are selected randomly from a sentence.

Another task that BERT is trained on is **Next Sentence Prediction** (**NSP**). This pretraining task ensures that BERT learns not only the relations of all tokens to each other in predicting masked ones but also helps it understand the relation between two sentences. A pair of sentences is selected and given to BERT with a `[SEP]` splitter token in between. It is also known from the dataset whether the second sentence comes after the first one or not.

The following is an example of NSP: *It is required that the reader fill in the blanks. Bitcoin prices are way over too high compared to other altcoins.*

In this example, the model is required to predict it as negative (the sentences are not related to each other).

These two pretraining tasks enable BERT to have an understanding of the language itself. BERT token embeddings provide a contextual embedding for each token. Contextual embedding means each token has an embedding that is completely related to the surrounding tokens. Unlike word2vec and other such models, BERT provides better information for each token embedding. NSP tasks, on the other hand, enable BERT to have better embeddings for `[CLS]` tokens. This token, as was discussed in the first chapter, provides information about the whole input. `[CLS]` is used for classification tasks and, in the pretraining part, learns the overall embedding of the whole input. The following figure shows an overview of the BERT model and the respective input and output of the BERT model:

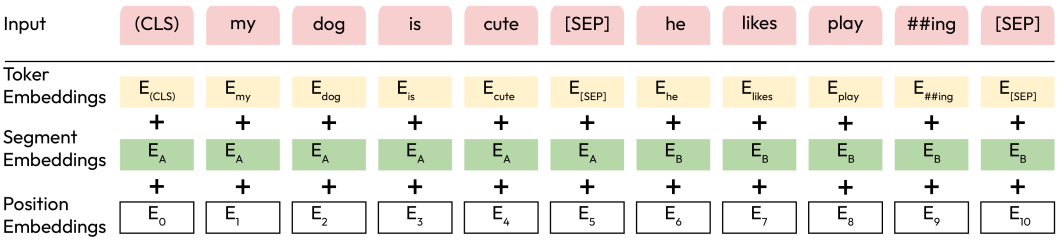


Figure 3.1 – The BERT model

Let's move on to the next section!

A deeper look into the BERT language model

Tokenizers are one of the most important parts of many NLP applications in their respective pipelines. For BERT, **WordPiece** tokenization is used. Generally, WordPiece, **SentencePiece**, and **byte pair encoding** (**BPE**) are the three most widely known tokenizers, used by different Transformer-based architectures, which are also covered in the next sections. The primary differences lie in their merging strategies, unit representations (bytes, characters, or subword units), and their flexibility in tokenization schemes and segmentation modes. Each of these algorithms has its advantages and can be chosen based on the specific requirements of the task at hand. The main reason that BERT or any other Transformer-based architecture uses subword tokenization is the ability of such tokenizers to deal with unknown tokens.

BERT also uses positional encoding to ensure the position of the tokens is given to the model. If you recall from *Chapter 1*, BERT and similar models use non-sequential operations such as dense neural layers. Traditional models, such as LSTM- and RNN-based models, care the natural position In order to provide this extra information to BERT, positional encoding comes in handy.

The pretraining of BERT, such as autoencoding models, provides language-wise information for the model, but in practice, when dealing with different problems such as sequence classification, token classification, or question answering, different parts of the model output are used.

For example, in the case of a sequence classification task, such as sentiment analysis or sentence classification, it is proposed by the original BERT article that CLS embedding from the last layer must be used. However, other research performs classification using different techniques using BERT (using average token embedding from all tokens, deploying an LSTM over the last layer, or even using a CNN on top of the last layer). The last `[CLS]` embedding for sequence classification can be used by any classifier, but the proposed, and the most common one, is a dense layer with an input size equal to the final token embedding size and an output size equal to the number of classes with a softmax activation function. Using sigmoid is also another alternative when the output could be multilabel and the problem itself is a multilabel classification problem.

To give you more detailed information about how BERT actually works, the following diagram shows an example of an NSP task. Note that the tokenization is simplified here for better understanding:

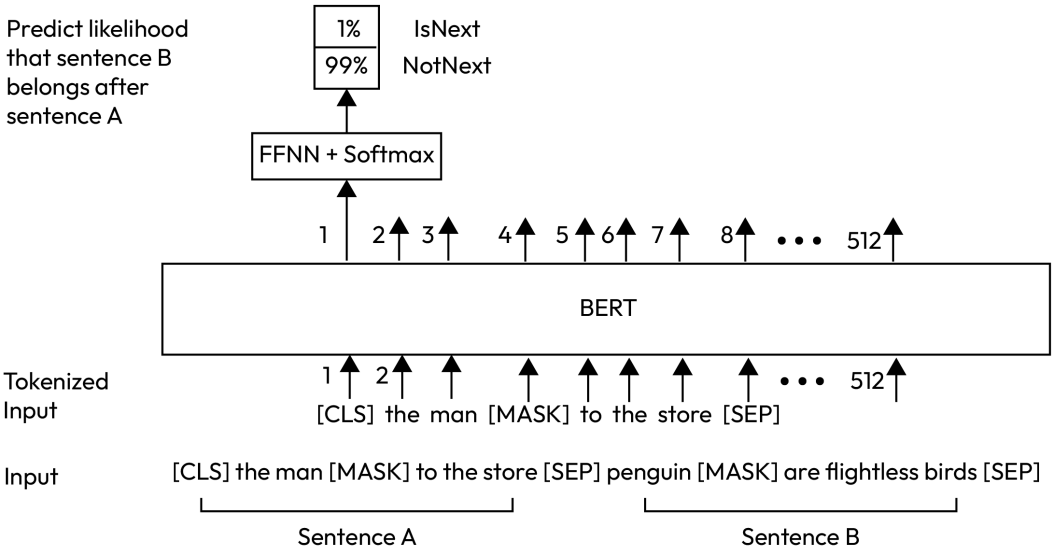


Figure 3.2 – BERT example for an NSP task

The BERT model has different variations with different settings. For example, the size of the input is variable. In the preceding example, it is set to 512, and the maximum sequence size that the model can get as input is 512. However, this size includes special tokens, `[CLS]` and `[SEP]`, so it will be reduced to 510. On the other hand, using WordPiece as a tokenizer yields subword tokens and the sequence size, before we can have fewer words, and, after tokenization, the size will increase because the tokenizer breaks words into subwords if they are not commonly seen in the pretrained corpus.

The following screenshot shows an illustration of BERT for different tasks. For an NER task, the output of each token is used instead of `[CLS]`. In the case of question answering, the question and the answer are concatenated using the `[SEP]` delimiter token, and the answer is annotated using "*Start/End*" and the span output from the last layer. In this case, Paragraph is the context that Question is asking about it:

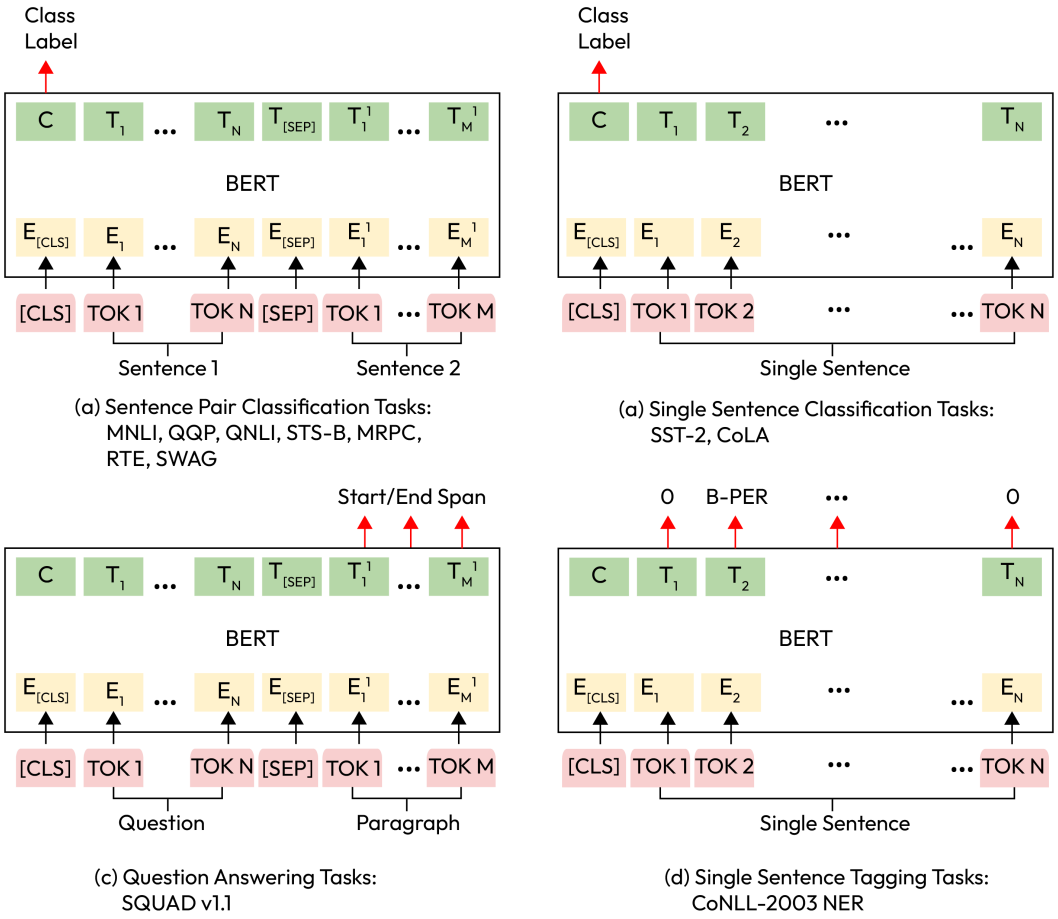


Figure 3.3 – BERT model for various NLP tasks

Regardless of all of these tasks, the most important of BERT's abilities is its contextual representation of text. The reason it is successful in various tasks is because of the Transformer encoder architecture that represents input in the form of dense vectors. These vectors can be easily transformed into output by very simple classifiers.

Positional encoding is essential in maintaining word order. It adds very small amounts of values to the word embeddings to ensure that they are semantically close to what they mean, but they also have a specific order.

Up to this point, you have learned about BERT and how it works. You have learned detailed information on various tasks that BERT can be used for and the important points of this architecture.

In the next section, you will learn how you can pretrain BERT and use it after training.

Autoencoding language model training for any language

We have discussed how BERT works and that it is possible to use the pretrained version of it provided by the Hugging Face repository. In this section, you will learn how to use the Hugging Face library to train your own BERT.

Before we start, it is essential to have good training data, because it will be used for the language modeling. This data is called the **corpus**, which is normally a huge pile of data (sometimes it is preprocessed and cleaned). This unlabeled corpus must be appropriate for the use case you wish to have your language model trained on; for example, if you are trying to have a special BERT for, let's say, the English language. Although there are tons of huge, good datasets, such as **Common Crawl** (`https://commoncrawl.org/`), we would prefer a small one for faster training.

The IMDB dataset of 50 K movie reviews (available at `https://www.kaggle.com/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews`) is a large dataset for sentiment analysis, but small if you use it as a corpus for training your language model:

1. You can easily download and save it in the `.txt` format for language model and tokenizer training by using the following code:

```
import pandas as pd
imdb_df = pd.read_csv("IMDB Dataset.csv")
reviews = imdb_df.review.to_string(index=None)
with open("corpus.txt", "w") as f:
    f.writelines(reviews)
```

2. After preparing the corpus, the tokenizer must be trained. The `tokenizers` library provides fast and easy training for the WordPiece tokenizer. In order to train it on your corpus, you must run the following code:

```
from tokenizers import BertWordPieceTokenizer
bert_wordpiece_tokenizer =BertWordPieceTokenizer()
bert_wordpiece_tokenizer.train("corpus.txt")
```

3. This will train the tokenizer. You can access the trained vocabulary by using the `get_vocab()` function of the trained tokenizer object. You can get the vocabulary by using the following code:

```
bert_wordpiece_tokenizer.get_vocab()
```

The following is the output:

```
{'almod': 9111, 'events': 3710, 'bogart': 7647, 'slapstick':
9541, 'terrorist': 16811, 'patter': 9269, '183': 16482, '##cul':
14292, 'sophie': 13109, 'thinki': 10265, 'tarnish': 16310,
'##outh': 14729, 'peckinpah': 17156, 'gw': 6157, '##cat':
14290, '##eing': 14256, 'successfully': 12747, 'roomm': 7363,
'stalwart': 13347,...}
```

4. It is essential to save the tokenizer to be used afterward. Using the `save_model()` function of the object and providing the directory will save the tokenizer vocabulary for further usage:

```
bert_wordpiece_tokenizer.save_model("tokenizer")
```

5. You can reload it by using the `from_file()` function:

```
tokenizer = \
    BertWordPieceTokenizer.from_file("tokenizer/vocab.txt")
```

6. You can use the tokenizer by following this example:

```
tokenized_sentence = \
    tokenizer.encode("Oh it works just fine")
tokenized_sentence.tokens
['[CLS]', 'oh', 'it', 'works', 'just', 'fine','[SEP]']
```

The special `[CLS]` and `[SEP]` tokens will be automatically added to the list of tokens because BERT needs them for processing input.

7. Let's try another sentence using our tokenizer:

```
tokenized_sentence = \
    tokenizer.encode("ohoh i thougt it might be workingg well")
['[CLS]', 'oh', '##o', '##h', 'i', 'thoug', '##t', 'it',
'might', 'be', 'working', '##g', 'well', '[SEP]']
```

8. It seems like a good tokenizer for noisy and misspelled text. Now that you have your tokenizer ready and saved, you can train your own BERT. The first step is to use `BertTokenizerFast` from the Transformers library. You are required to load the trained tokenizer from the previous step by using the following command:

```
from transformers import BertTokenizerFast
tokenizer = BertTokenizerFast.from_pretrained("tokenizer")
```

We have used `BertTokenizerFast` because it is suggested by the Hugging Face documentation. There is also `BertTokenizer`, which, according to the definition from the library documentation, is not implemented as fast as the fast version. In most of the pretrained models' documentation and cards, it is highly recommended to use the `BertTokenizerFast` version.

9. The next step is preparing the corpus for faster training by using the following command:

```
from transformers import LineByLineTextDataset
dataset = \
    LineByLineTextDataset(tokenizer=tokenizer,
        file_path="corpus.txt",
        block_size=128)
```

10. You must provide a data collator for MLM:

```
from transformers import DataCollatorForLanguageModeling
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=True,
    mlm_probability=0.15)
```

The data collator gets the data and prepares it for the training. For example, the preceding data collator takes data and prepares it for MLM with a probability of `0.15`. The purpose of using such a mechanism is to do the preprocessing on the fly, which makes it possible to use fewer resources. On the other hand, it slows down the training process because each sample has to be preprocessed on the fly at training time.

11. The training arguments also provide information for the trainer in the training phase, which can be set by using the following command:

```
from transformers import TrainingArguments
training_args = TrainingArguments(
    output_dir="BERT",
    overwrite_output_dir=True,
    num_train_epochs=1,
    per_device_train_batch_size=128)
```

12. We'll now make the BERT model itself, which we are going to use with the default configuration (the number of attention heads, Transformer encoder layers, and so on):

```
from transformers import BertConfig, BertForMaskedLM
bert = BertForMaskedLM(BertConfig())
```

13. The final step is to make a Trainer object:

```
from transformers import Trainer
trainer = Trainer(model=bert,
    args=training_args,
    data_collator=data_collator,
    train_dataset=dataset)
```

14. Finally, you can train your language model using the following command:

```
trainer.train()
```

It will show you a progress bar indicating the progress made in training:

Figure 3.4 – BERT model training progress

During the model training, a log directory called `runs` will be used to store the checkpoint in steps:

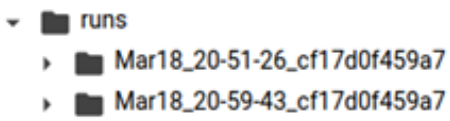


Figure 3.5 – BERT model checkpoints

15. After the training is finished, you can easily save the model using the following command:

```
trainer.save_model("MyBERT")
```

Up to this point, you have learned how you can train BERT from scratch for any specific language that you desire. You've learned how to train the tokenizer and BERT model using the corpus you have prepared.

16. The default configuration that you provided for BERT is the most essential part of this training process because it defines the architecture of BERT and its hyperparameters. You can take a peek at these parameters by using the following code:

```
from transformers import BertConfig
BertConfig()
```

The following is the output:

```
BertConfig {
  "attention_probs_dropout_prob": 0.1,
  "gradient_checkpointing": false,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 768,
  "initializer_range": 0.02,
  "intermediate_size": 3072,
  "layer_norm_eps": 1e-12,
  "max_position_embeddings": 512,
  "model_type": "bert",
  "num_attention_heads": 12,
  "num_hidden_layers": 12,
  "pad_token_id": 0,
  "position_embedding_type": "absolute",
  "transformers_version": "4.4.2",
  "type_vocab_size": 2,
  "use_cache": true,
  "vocab_size": 30522
}
```

Figure 3.6 – BERT model configuration

If you wish to replicate `tiny`, `mini`, `small`, `base`, and `relative` models from the original BERT configurations (`https://github.com/google-research/bert`), you can change these settings:

	H=128	H=256	H=512	H=768
L=2	2/128 (BERT-Tiny)	2/256	2/512	2/768
L=4	4/128	4/256 (BERT-Mini)	4/512 (BERT-Small)	4/768
L=6	6/128	6/256	6/512	6/768
L=8	8/128	8/256	8/512 (BERT-Medium)	8/768
L=10	10/128	10/256	10/512	10/768
L=12	12/128	12/256	12/512	12/768 (BERT-Base)

Figure 3.7 – BERT model configurations

Note that changing these parameters, especially `max_position_embedding`, `num_attention_heads`, `num_hidden_layers`, `intermediate_size`, and `hidden_size`, directly affects the training time. Increasing them dramatically increases the training time for a large corpus.

17. For example, you can easily make a new configuration for a tiny version of BERT for faster training using the following code:

```
tiny_bert_config = \
    BertConfig(max_position_embeddings=512,
        hidden_size=128,
        num_attention_heads=2,
        num_hidden_layers=2,
        intermediate_size=512)
tiny_bert_config
```

The following is the result of the code:


```
BertConfig {
  "attention_probs_dropout_prob": 0.1,
  "gradient_checkpointing": false,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 128,
  "initializer_range": 0.02,
  "intermediate_size": 512,
  "layer_norm_eps": 1e-12,
  "max_position_embeddings": 512,
  "model_type": "bert",
  "num_attention_heads": 2,
  "num_hidden_layers": 2,
  "pad_token_id": 0,
  "position_embedding_type": "absolute",
  "transformers_version": "4.4.2",
  "type_vocab_size": 2,
  "use_cache": true,
  "vocab_size": 30522
}
```


Figure 3.8 – Tiny BERT model configuration

18. By using the same method, we can make a tiny BERT model using this configuration:

```
tiny_bert = BertForMaskedLM(tiny_bert_config)
```

19. Using the same parameters for training, you can train this tiny new BERT:

```
trainer = Trainer(model=tiny_bert, args=training_args,
    data_collator=data_collator,
    train_dataset=dataset)
trainer.train()
```

The following is the output:

Figure 3.9 – Tiny BERT model configuration

It is clear that the training time has dramatically decreased, but you should be aware that this is a tiny version of BERT with fewer layers and parameters, which is not as good as the BERT base model.

Up to this point, you have learned how to train your own model from scratch, but it is essential to note that using the `Datasets` library is a better choice when dealing with datasets for training language models or leveraging it to perform task-specific training.

20. The BERT language model can also be used as an embedding layer combined with any deep learning model. For example, you can load any pretrained BERT model or your own version that has been trained in *Step 19*. The following code shows how you must load it to be used in a Keras model:

```
from transformers import (
    TFBertModel, BertTokenizerFast)
bert = TFBertModel.from_pretrained("bert-base-uncased")
tokenizer = \
    BertTokenizerFast.from_pretrained("bert-base-uncased")
```

21. You do not need the whole model; instead, you can access the layers by using the following code:

```
bert.layers
[<Transformers.models.bert.modeling_tf_bert.TFBertMainLayer at
0x7f72459b1110>]
```

22. As you can see, there is just a single layer from `TFBertMainLayer`, which you can access within your Keras model. However, before using it, it is nice to test it and see what kind of output it provides:

```
tokenized_text = tokenizer.batch_encode_plus(
    ["hello how is it going with you",
    "lets test it"],
    return_tensors="tf",
    max_length=256,
    truncation=True,
    pad_to_max_length=True)
bert(tokenized_text)
```

The output is as follows:

```
TFBaseModelOutputWithPooling([('last_hidden_state',
                               <tf.Tensor: shape=(2, 256, 768), dtype=float32, numpy=
                               array([[[ 1.00471362e-01,  6.77026287e-02, -8.33595246e-02, ...,
                                        -4.93304580e-01,  1.16539136e-01,  2.26647347e-01],
                                       [ 3.23623657e-01,  3.70719165e-01,  6.14685774e-01, ...,
                                        -6.27267540e-01,  3.79083097e-01,  7.05310702e-02],
                                       [ 1.99532971e-01, -8.75509441e-01, -6.47868365e-02, ...,
                                        -1.28077380e-02,  3.07651043e-01, -2.07325034e-02],
                                       ...,
                                       [-6.53299838e-02,  1.19046383e-01,  5.76846600e-01, ...,
                                        -2.95460820e-01,  2.49744654e-02,  1.13964394e-01],
                                       [-2.64715493e-01, -7.86386207e-02,  5.47280848e-01, ...,
                                        -1.37515247e-01, -5.94691373e-02, -5.17928638e-02],
                                       [-2.44958848e-01, -1.14799395e-01,  5.92173815e-01, ...,
                                        -1.56882048e-01, -3.39757390e-02, -8.46135616e-02]],

                                      [[ 2.94558890e-02,  2.30868042e-01,  2.92651534e-01, ...,
                                        -1.30421281e-01,  1.89659461e-01,  4.68427837e-01],
                                       [ 1.70523107e+00,  6.91360056e-01,  7.31509984e-01, ...,
                                         2.89302200e-01,  5.36758840e-01, -1.54553086e-01],
                                       [ 1.04596823e-01,  9.63676572e-02,  6.99661374e-02, ...,
                                        -4.15922999e-01, -1.18989825e-01, -6.72240376e-01],
                                       ...,
                                       [ 8.00909758e-01,  2.38983199e-01,  4.15492684e-01, ...,
                                         3.90530713e-02,  2.34373003e-01,  1.22278236e-01],
                                       [ 2.60862708e-01,  4.43267114e-02,  3.63648295e-01, ...,
                                        -7.53704458e-04,  3.84620279e-02, -2.14213312e-01],
                                       [-2.30111778e-01, -4.98388559e-01, -1.26496106e-02, ...,
                                         4.49867934e-01,  6.16019145e-02, -2.61357218e-01]]],
                                     dtype=float32)>),
                              ('pooler_output',
                               <tf.Tensor: shape=(2, 768), dtype=float32, numpy=
                               array([[-0.9204854 , -0.37138987, -0.6051259 , ..., -0.4473697 ,
                                       -0.64347583,  0.9423271 ],
                                      [-0.8854158 , -0.26547667,  0.21015054, ...,  0.17237163,
                                       -0.6402989 ,  0.8888342 ]], dtype=float32)>)])
```

Figure 3.10 – BERT model output

As can be seen from the result, there are two outputs: one for the last hidden state and one for the pooler output. The last hidden state provides all token embeddings from BERT with additional `[CLS]` and `[SEP]` tokens at the start and end, respectively.

23. Now that you have learned more about the TensorFlow version of BERT, you can make a Keras model using this new embedding:

```
from tensorflow import keras
import tensorflow as tf
max_length = 256
tokens = keras.layers.Input(shape=(max_length,),
    dtype=tf.dtypes.int32)
masks = keras.layers.Input(shape=(max_length,),
    dtype=tf.dtypes.int32)
embedding_layer = bert.layers[0]([tokens,masks])[0][:,0,:]
```

```
dense = tf.keras.layers.Dense(units=2,
    activation="softmax")(embedding_layer)
model = keras.Model([tokens,masks],dense)
```

24. The `model` object, which is a Keras model, has two inputs: one for tokens and one for masks. Tokens have `inputs_ids` from the tokenizer output and the masks will have `attention_mask`. Let's try it and see what happens:

```
tokenized = tokenizer.batch_encode_plus(
    ["hello how is it going with you",
    "hello how is it going with you"],
    return_tensors="tf",
    max_length= max_length,
    truncation=True,
    pad_to_max_length=True)
```

25. It is important to use `max_length`, truncation, and `pad_to_max_length` when using a tokenizer. These parameters make sure that you have the output in a usable shape by padding it to the maximum length of `256`, which was defined before. Now, you can run the model using this sample:

```
model([tokenized["input_ids"],tokenized["attention_mask"]])
```

The following is the output:

```
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[0.45928752, 0.5407125 ],
       [0.45928752, 0.5407125 ]], dtype=float32)>
```

Figure 3.11 – BERT model classification output

26. When training the model, you need to compile it using the `compile` function:

```
model.compile(optimizer="Adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"])
model.summary()
```

The output is as follows:

```
Layer (type)                 Output Shape              Param #     Connected to
==================================================================================================
input_tokens (InputLayer)    [(None, 256)]             0
__________________________________________________________________________________________________
input_masks (InputLayer)     [(None, 256)]             0
__________________________________________________________________________________________________
bert (TFBertMainLayer)       multiple                  109482240   input_tokens[0][0]
                                                                   input_masks[0][0]
__________________________________________________________________________________________________
tf.__operators__.getitem_3 (Sli (None, 768)            0           bert[3][0]
__________________________________________________________________________________________________
output_layer (Dense)         (None, 2)                 1538        tf.__operators__.getitem_3[0][0]
==================================================================================================
Total params: 109,483,778
Trainable params: 109,483,778
Non-trainable params: 0
__________________________________________________________________________________________________
```

Figure 3.12 – BERT model summary

27. From the model summary, you can see that the model has *109,483,778* trainable parameters, including BERT. However, if you have your BERT model pretrained and you want to freeze it in task-specific training, you can do so with the following command:

    ```
    model.layers[2].trainable = False
    ```

    As far as we know, the layer index of the embedding layer is `2`, so we can simply freeze it. If you rerun the `summary` function, you will see that the trainable parameters are reduced to *1,538* which is the number of parameters of the last layer:

```
Layer (type)                 Output Shape              Param #     Connected to
==================================================================================================
input_tokens (InputLayer)    [(None, 256)]             0
__________________________________________________________________________________________________
input_masks (InputLayer)     [(None, 256)]             0
__________________________________________________________________________________________________
bert (TFBertMainLayer)       multiple                  109482240   input_tokens[0][0]
                                                                   input_masks[0][0]
__________________________________________________________________________________________________
tf.__operators__.getitem_3 (Sli (None, 768)            0           bert[3][0]
__________________________________________________________________________________________________
output_layer (Dense)         (None, 2)                 1538        tf.__operators__.getitem_3[0][0]
==================================================================================================
Total params: 109,483,778
Trainable params: 1,538
Non-trainable params: 109,482,240
__________________________________________________________________________________________________
```

Figure 3.13 – BERT model summary with fewer trainable parameters

28. As you recall, we used the IMDB sentiment analysis dataset for training the language model. Now, you can use it for training the Keras-based model for sentiment analysis. But first, you need to prepare the input and output:

```
import pandas as pd
imdb_df = pd.read_csv("IMDB Dataset.csv")
reviews = list(imdb_df.review)
tokenized_reviews = tokenizer.batch_encode_plus(
        reviews, return_tensors="tf",
        max_length=max_length,
        truncation=True,
        pad_to_max_length=True)
import numpy as np
train_split = int(0.8 *
    len(tokenized_reviews["attention_mask"]))
train_tokens = tokenized_reviews["input_ids"][:train_split]
test_tokens = tokenized_reviews["input_ids"][train_split:]
train_masks = tokenized_reviews["attention_mask"][:train_split]
test_masks = tokenized_reviews["attention_mask"][train_split:]
sentiments = list(imdb_df.sentiment)
labels = np.array([[0,1] if sentiment == "positive" else\
    [1,0] for sentiment in sentiments])
train_labels = labels[:train_split]
test_labels = labels[train_split:]
```

29. Finally, your data is ready, and you can fit your model:

```
model.fit([train_tokens,train_masks], train_labels, epochs=5)
```

After fitting the model, your model is ready to be used. Up to this point, you have learned how to perform model training for a classification task. You have learned how to save it, and in the next section, you will learn how it is possible to share the trained model with the community.

Sharing models with the community

Trained models can be easily shared with the community. This provides more visibility for your work. Hugging Face provides a very easy-to-use model-sharing mechanism:

1. You can simply use the following `cli` tool to log in:

```
Transformers-cli login
```

2. After you've logged in using your own credentials, you can create a repository:

```
Transformers-cli repo create a-fancy-model-name
```

3. You can put any model name for the `a-fancy-model-name` parameter, and then it is essential to make sure you have `git-lfs`:

   ```
   git lfs install
   ```

   Git LFS is a Git extension used for handling large files. Hugging Face pretrained models are usually large files that require extra libraries such as LFS to be handled by Git.

4. Then, you can clone the repository you have just created:

   ```
   git clone https://huggingface.co/username/a-fancy-model-name
   ```

5. Afterward, you can add and remove from the repository as you like, and then, just like Git usage, you have to run the following command:

   ```
   git add . && git commit -m "Update from $USER"
   git push
   ```

Autoencoding models rely on the left encoder side of the original Transformer and are highly efficient at solving classification problems. Even though BERT is a typical example of an autoencoding model, there are many alternatives discussed in the literature. Let's take a look at these important alternatives.

Other autoencoding models

In this part, we will review autoencoding model alternatives that slightly modify the original BERT. These alternative re-implementations aim to get better downstream tasks by exploiting many sources: optimizing the pretraining process and the number of layers or heads, improving data quality, designing better objective functions, and so forth. The source of improvements roughly falls into two parts: better architectural design choice and pretraining control.

Many effective alternatives have been shared lately, so it is impossible to understand and explain them all here. We can take a look at some of the most cited models in the literature and the most used ones on NLP benchmarks. Let's start with **A Lite BERT** (**ALBERT**) as a re-implementation of BERT that focuses especially on architectural design choice.

Introducing ALBERT

The performance of language models is considered to improve as their size gets bigger. However, training such models is getting more challenging due to both memory limitations and longer training times. To address these issues, the Google team proposed the ALBERT model, which is indeed a reimplementation of the BERT architecture by utilizing several new techniques that reduce memory consumption and increase the training speed. The new design led to the language models scaling much better than the original BERT. Along with 18 times fewer parameters, ALBERT trains 1.7 times faster than the original BERT large model.

The ALBERT model mainly consists of three modifications of the original BERT:

- Factorized embedding parameterization
- Cross-layer parameter sharing
- Inter-sentence coherence loss

The first two modifications are parameter-reduction methods that are related to the issue of model size and memory consumption in the original BERT. The third corresponds to a new objective function: **Sentence-Order Prediction** (**SOP**), replacing the NSP task of the original BERT, which led to a much thinner model and improved performance.

Factorized embedding parameterization is used to decompose the large vocabulary-embedding matrix into two smaller matrices. These matrices separate the size of the hidden layers from the size of the vocabulary. This decomposition reduces the embedding parameters from `O(V × H)` to `O(V × E + E × H)` where `V` is vocabulary, `H` is hidden layer size, and `E` is embeddings, which leads to more efficient usage of the total model parameters if `H E` is satisfied.

For instance, let's consider the following example: In the ALBERT large model, the value of `H` is `1026`, and the size of `V` is `30000`. In this case, assuming that the `E` value is `128`, which satisfies the `H>>E` condition. The `VxH` value is approximately 30 M (1026 x 30,000). However, (30,000 × 128 + 128 x 1026) is around 4 M. This leads to a model that is ten times more efficient.

Cross-layer parameter sharing prevents the total number of parameters from increasing as the network gets deeper. The technique is considered another way to improve parameter efficiency since we can keep the parameter size smaller by sharing or copying. In the original paper, they experimented with many ways to share parameters, such as either sharing FF-only parameters across layers or sharing attention-only parameters or entire parameters.

The other modification of ALBERT is inter-sentence coherence loss. As we already discussed, the BERT architecture takes advantage of two loss calculations: the MLM loss and NSP. NSP comes with binary cross-entropy loss for predicting whether or not two segments appear in a row in the original text. The negative examples are obtained by selecting two segments from different documents. However, the ALBERT team criticized NSP for being a topic detection problem, which is considered a relatively easy problem. Therefore, the team proposed a loss based primarily on coherence rather than topic prediction.

The SOP loss was utilized in this case. This loss function prioritizes the modeling of coherence between sentences rather than focusing on topic classification. It employs a similar technique to BERT for positive examples, using two consecutive segments from the same document. For negative examples, it uses the same two consecutive segments but with their order swapped. This approach enables the model to learn more nuanced distinctions between coherence properties at the discourse level.

Now, let's dive into more detail and see how it works in practice:

1. Let's compare the original BERT and ALBERT configuration with the `Transformers` library. The following piece of code shows how you can configure a BERT-Base initial model. As you see in the output, the number of parameters is around 110 M:

```
#BERT-BASE (L=12, H=768, A=12, Total Parameters=110M)
from transformers import BertConfig, BertModel
bert_base= BertConfig()
model = BertModel(bert_base)
print(f"{model.num_parameters() /(10**6)} million parameters")
109.48224 million parameters
```

2. The following piece of code shows how you can define the ALBERT model with two classes, `AlbertConfig` and `AlbertModel`, from the `Transformers` library:

```
# Albert-base Configuration
from transformers import AlbertConfig, AlbertModel
albert_base = AlbertConfig(hidden_size=768,
    num_attention_heads=12,
    intermediate_size=3072,)
model = AlbertModel(albert_base)
print(f"{model.num_parameters() /(10**6)}\
    million parameters")'
11.683584 million parameters
```

   As a result, the default ALBERT configuration points to `Albert-xxlarge`. We need to set the hidden size, the number of attention heads, and the intermediate size to fit the ALBERT base model. The code shows the ALBERT base model as 11 M, which is 10 times smaller than the BERT base model. The original paper on ALBERT reported benchmarking as in the following table:

Model		Parameters	SQuAD1.1	SQuAD2.0	MNLI	SST-2	RACE	Avg	Speedup
BERT	base	108M	90.4/83.2	80.4/77.6	84.5	92.8	68.2	82.3	4.7x
	large	334M	92.2/85.5	85.0/82.2	86.6	93.0	73.9	85.2	1.0
ALBERT	base	12M	89.3/82.3	80.0/77.1	81.6	90.3	64.0	80.1	5.6x
	large	18M	90.6/83.9	82.3/79.4	83.5	91.7	68.5	82.4	1.7x
	xlarge	60M	92.5/86.1	86.1/83.1	86.4	92.4	74.8	85.5	0.6x
	xxlarge	235M	**94.1/88.3**	**88.1/85.1**	**88.0**	**95.2**	**82.3**	**88.7**	0.3x

Figure 3.14 – ALBERT model benchmarking

3. From this point on, in order to train an ALBERT language model from scratch, we need to go through similar phases to those we already illustrated in BERT training in the previous section, titled *A deeper look into the BERT language model*, by using the uniform Transformers API. There's no need to explain the same steps here! Instead, let's load an already-trained ALBERT language model as follows:

```
from transformers import AlbertTokenizer, AlbertModel
tokenizer = \
    AlbertTokenizer.from_pretrained("albert-base-v2")
model = AlbertModel.from_pretrained("albert-base-v2")
text = "The cat is so sad ."
encoded_input = tokenizer(text, return_tensors='pt')
output = model(**encoded_input)
```

4. The preceding pieces of code download the ALBERT model weights and their configuration from the Hugging Face hub or our local cache directory if already cached, which means you've already called the `AlbertTokenizer.from_pretrained()` function before. Since the `model` object is a pre-trained language model, the things we can do with this model are limited for now. We need to train it on a downstream task to be able to use it for inference, which will be the main subject of further chapters. Instead, we can take advantage of its masked language model objective as follows:

```
from transformers import pipeline
fillmask= pipeline('fill-mask',
    model='albert-base-v2')
pd.DataFrame(fillmask("The cat is so [MASK] ."))
```

The following is the output:

sequence	score	token	token_str
[CLS] the cat is so cute.[SEP]	0.281025	10901	_cute
[CLS] the cat is so adorable.[SEP]	0.094893	26354	_adorable
[CLS] the cat is so happy.[SEP]	0.042963	1700	_happy
[CLS] the cat is so funny.[SEP]	0.040976	5066	_funny
[CLS] the cat is so affectionate.[SEP]	0.024233	28803	_affectionate

Figure 3.15 – The fill-mask output results for albert-base-v2

The fill-mask pipeline computes the scores for each vocabulary token with the `softmax()` function and sorts the most probable tokens, where `cute` is the winner with a probability score of `0.281`. You may notice that entries in the `token_str` column start with the _ character, which is due to the metaspace component of the tokenizer of ALBERT.

Let's take a look at the next alternative, **Robustly Optimized BERT Pretraining Approach** (**RoBERTa**), which mostly focuses on the pretraining phase.

RoBERTa

RoBERTa is another popular BERT reimplementation. It has provided many more improvements in training strategy than architectural design. It outperformed BERT in many tasks on GLUE. **Dynamic masking** is one of its original design choices. Although **static masking** is better for some tasks, the RoBERTa team showed that dynamic masking can perform well for overall performance. Let's discuss the changes made to BERT to get RoBERTa and summarize all of its features.

The changes in architecture are as follows:

- Removing the NSP training objective
- Dynamically changing the masking patterns instead of static masking, which is done by generating masking patterns whenever they feed a sequence to the model
- BPE sub-word tokenizer

The changes in training are as follows:

- **Controlling the training data**: More data is used, such as 160 GB instead of the 16 GB originally used in BERT. Not only the size of the data but the quality and diversity were taken into consideration in the RoBERTa Model.
- Longer iterations of up to 500 K pretraining steps.
- A longer batch size.
- Longer sequences, which lead to less padding.
- A large 50 K BPE vocabulary instead of a 30 K BPE vocabulary.

Thanks to the Transformers uniform API, as in the preceding ALBERT model pipeline, we initialize the RoBERTa model as follows:

```
from transformers import RobertaConfig, RobertaModel
conf= RobertaConfig()
model = RobertaModel(conf)
print(f"{model.num_parameters() /(10**6)} million parameters")
109.48224 million parameters
```

In order to load the pretrained model, we execute the following pieces of code:

```
from transformers import RobertaTokenizer, RobertaModel
tokenizer = \
    RobertaTokenizer.from_pretrained('roberta-base')
```

```
model = RobertaModel.from_pretrained('roberta-base')
text = "The cat is so sad ."
encoded_input = tokenizer(text, return_tensors='pt')
output = model(**encoded_input)
```

These lines illustrate how the model processes a given text. The output representation at the last layer is not useful at the moment. As we've mentioned several times, we need to fine-tune the main language models. The following execution applies the fill-mask function using the `roberta-base` model:

```
from transformers import pipeline
fillmask= pipeline("fill-mask ",model="roberta-base",
    tokenizer=tokenizer)
pd.DataFrame(fillmask("The cat is so <mask> ."))
```

The following is the output:

sequence	score	token	token_str
<s>The cat is so cute.</s>	0.191843	11962	Ġcute
<s>The cat is so sweet.</s>	0.051524	4045	Ġsweet
<s>The cat is so funny.</s>	0.033595	6269	Ġfunny
<s>The cat is so handsome.</s>	0.032893	19222	Ġhandsome
<s>The cat is so beautiful.</s>	0.032314	2721	Ġbeautiful

Figure 3.16 – The fill-mask task results for roberta-base

Like the previous ALBERT fill-mask model, this pipeline ranks the suitable candidate words. Please ignore the Ġ prefix in the tokens – that is an encoded space character produced by the byte-level BPE tokenizer, which we will discuss later. You should have noticed that we used the `[MASK]` and `<mask>` tokens in the ALBERT and RoBERTa pipeline in order to hold the place for a masked token. This is because of the configuration of the tokenizer. To learn which token expression will be used, you can check `tokenizer.mask_token`. Please see the following execution:

```
tokenizer = AlbertTokenizer.from_pretrained('albert-base-v2')
print(tokenizer.mask_token)
[MASK]
tokenizer = RobertaTokenizer.from_pretrained('roberta-base')
print(tokenizer.mask_token)
<mask>
```

To ensure proper mask token use, we can add the `fillmask.tokenizer.mask_token` expression in the pipeline as follows:

```
fillmask(f"The cat is very {fillmask.tokenizer.mask_token}.")
```

In the next section, you will learn more about **ELECTRA** and how it works.

ELECTRA

The ELECTRA model (proposed by *Kevin Clark et al.* in 2020) focuses on a new masked language model utilizing the replaced token detection training objective. During pre-training the model is forced to learn to distinguish real input tokens from synthetically generated replacements where the synthetic negative example is sampled from plausible tokens rather than randomly sampled tokens. The ALBERT model criticized the NSP objective of BERT for being a topic detection problem and using low-quality negative examples. ELECTRA trains two neural networks, a generator and a discriminator, so that the former produces high-quality negative examples, whereas the latter distinguishes the original token from the replaced token. We know GAN networks from the field of computer vision, in which the G generator produces fake images and tries to fool the D discriminator, and the discriminator network tries to avoid being fooled. The ELECTRA model applies almost the same generator-discriminator approach to replace original tokens with high-quality negative examples that are plausible replacements but synthetically generated.

In order to not repeat the same code with other examples, we only provide a simple fill-mask example for the ELECTRA generator as follows:

```
fillmask = \
    pipeline("fill-mask", model="google/electra-small-generator")
fillmask(f"The cat is very \{fillmask.tokenizer.mask_token} .")
```

You can see the entire list of models at `https://huggingface.co/Transformers/model_summary.html`.

The model checkpoints can be found at `https://huggingface.co/models`.

DeBERTa

When we look at the GLUE and **SuperGLUE** NLP benchmarks, we can say that the **Decoding-Enhanced BERT with Disentangled Attention** (**DeBERTa**) model has received great attention. It has also given successful results in many academic and industrial projects recently. The following two techniques of DeBERTa have an effect on performance:

- **Disentangled attention mechanism**: This is based on the assumption that the importance of a word pair is determined by both its content and its relative position. The dependency between words becomes much stronger when they are used side by side rather than when they are used in different sentences.

- **Enhanced mask decoder**: In the `softmax` layer, the vanilla BERT model decodes masked words based on aggregated contextual vectors of word contents and position. DeBERTa includes absolute word position embeddings just before the softmax layer.

DeBERTa is supported by Hugging Face's `transformers` library, as shown in the following initialization code:

```
from transformers import DebertaConfig, DebertaModel
configuration = DebertaConfig()
model = DebertaModel(configuration)
configuration = model.config
```

Well done! We've finally completed the autoencoding model part. Now, we'll move on to tokenization algorithms, which have an important effect on the success of Transformers.

Working with tokenization algorithms

In the opening part of the chapter, we trained the BERT model using a specific tokenizer, namely `BertWordPieceTokenizer`. Now, it is worth discussing the tokenization process in detail here. Tokenization is a way of splitting textual input into tokens and assigning an identifier to each token before feeding the neural network architecture. The most intuitive way to do this is to split the sequence into smaller chunks in terms of space. However, such approaches do not meet the requirements of some languages, such as Japanese, and also may lead to huge vocabulary problems. Almost all Transformer models leverage subword tokenization not only for reducing dimensionality but also for encoding rare (or unknown) words not seen in training. The tokenization relies on the idea that every word, including rare words or unknown words, can be decomposed into meaningful smaller chunks that are widely seen symbols in the training corpus.

Some traditional tokenizers developed within Moses and the `nltk` library apply advanced rule-based techniques. However, the tokenization algorithms that are used with Transformers are based on self-supervised learning and extract the rules from the corpus. Simple intuitive solutions for rule-based tokenization are based on using characters, punctuation, or whitespace. Character-based tokenization causes language models to lose the input meaning. Even though it can reduce the vocabulary size, which is good, it makes it hard for the model to capture the meaning of "cat" by means of the encodings of the characters "c," "a," and "t." Moreover, the dimension of the input sequence becomes very large. Likewise, punctuation-based models cannot treat some expressions, such as haven't or ain't, properly.

Recently, several advanced subword tokenization algorithms, such as BPE, have become an integral part of Transformer architectures. These modern tokenization procedures consist of two phases: The pre-tokenization phase, which simply splits the input into tokens either using space or language-dependent rules, and the tokenization training phase, which trains the tokenizer and builds a base vocabulary of a

reasonable size based on tokens. Before training our own tokenizers, let's load a pre-trained tokenizer. The following code loads a Turkish tokenizer, which is of the `BertTokenizerFast` type, from the `Transformers` library with a vocabulary size of 32 K:

```
from transformers import AutoModel, AutoTokenizer
tokenizerTUR = AutoTokenizer.from_pretrained(
    "dbmdz/bert-base-turkish-uncased")
print(f"VOC size is: {tokenizerTUR.vocab_size}")
print(f"The model is: {type(tokenizerTUR)}")
VOC size is: 32000
The model is: Transformers.models.bert.tokenization_bert_fast.
BertTokenizerFast
```

The following code loads an English BERT tokenizer for the `bert-base-uncased` model:

```
from transformers import AutoModel, AutoTokenizer
tokenizerEN = \
    AutoTokenizer.from_pretrained("bert-base-uncased")
print(f"VOC size is: {tokenizerEN.vocab_size}")
print(f"The model is {type(tokenizerEN)}")
VOC size is: 30522
The model is ... BertTokenizerFast
```

Let's see how they work! We tokenize the word "telecommunication" with these two tokenizers:

```
word_en="telecommunication"
print(f"is in Turkish Model ? {word_en in tokenizerTUR.vocab}")
print(f"is in English Model ? {word_en in tokenizerEN.vocab}")
is in Turkish Model ? False
is in English Model ? True
```

The `word_en` token is already in the vocabulary of the English tokenizer but not in that of the Turkish one. So, let's see what happens with the Turkish tokenizer:

```
tokens=tokenizerTUR.tokenize(word_en)
tokens
['tel', '##eco', '##mm', '##un', '##ica', '##tion']
```

Since the Turkish tokenizer model has no such a word in its vocabulary, it needs to break the word into parts that make sense to it. All these split tokens are already stored in the model vocabulary. Please notice the output of the following execution:

```
[t in tokenizerTUR.vocab for t in tokens]
[True, True, True, True, True, True]
```

Let's tokenize the same word with the English tokenizer that we already loaded:

```
tokenizerEN.tokenize(word_en)
['telecommunication']
```

Since the English model has the word "telecommunication" in its base vocabulary, it does not need to break it into parts but rather takes it as a whole. By learning from the corpus, the tokenizers are capable of transforming a word into mostly grammatically logical subcomponents. Let's take a difficult example from Turkish. As an agglutinative language, Turkish allows us to add many suffixes to a word stem to construct very long words. Here is one of the longest words in the Turkish language used in a text (`https://en.wikipedia.org/wiki/Longest_word_in_Turkish`):

Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizdenmişsinizcesine

It means, "As though you happen to have been from among those whom we will not be able to make a maker of unsuccessful ones easily/quickly." The Turkish BERT tokenizer may not have seen this word in training, but it has seen its pieces; *muvaffak* (successful) as the stem, *iyet* (successfulness), *siz* (unsuccessfulness), *leş* (become unsuccessful), and so forth. The Turkish tokenizer extracts components that seem to be grammatically logical for the Turkish language when comparing the results with a Wikipedia article:

```
 print(tokenizerTUR.tokenize(long_word_tur))
['muvaffak', '##iyet', '##siz', '##les', '##tir', '##ici', '##les',
'##tir', '##iver', '##emeye', '##bilecekleri', '##mi', '##z', '##den',
'##mis', '##siniz', '##cesine']
```

The Turkish tokenizer is an example of the `WordPiece` algorithm since it works with a BERT model. Almost all language models, including BERT, DistilBERT, and ELECTRA, require a `WordPiece` tokenizer.

Now, we are ready to take a look at the tokenization approaches used with Transformers. First, we'll discuss the widely used tokenization of `BPE`, `WordPiece`, and `SentencePiece` a bit, and then train them with Hugging Face's fast tokenizers library.

BPE

BPE is a data compression technique. It scans the data sequence and iteratively replaces the most frequent pair of bytes with a single symbol. It was first adapted and proposed in *Neural Machine Translation of Rare Words with Subword Units* (*Sennrich et al.* 2015), to solve the problem of unknown words and rare words for machine translation. Currently, it is successfully being used within GPT-2 and many other state-of-the-art models. Many modern tokenization algorithms are based on such compression techniques.

It represents text as a sequence of character n-grams, which are also called character-level subwords. The training starts initially with a vocabulary of all Unicode characters (or symbols) seen in the corpus. This can be small for English but can be large for character-rich languages such as Japanese. Then, it iteratively computes character bigrams and replaces the most frequent ones with special new symbols. For example, *t* and *h* are frequently occurring symbols. We replace consecutive symbols with the *th* symbol. This process is kept iteratively running until the vocabulary has attained the desired vocabulary size. The most common vocabulary size is around 30 K.

BPE is particularly effective at representing unknown words. However, it may not guarantee the handling of rare words and/or words including rare subwords. In such cases, it associates rare characters with a special symbol, `<UNK>`, which may lead to losing meaning in words a bit. As a potential solution, **byte-level BPE** (**BBPE**) has been proposed, which uses a 256-byte set of vocabulary instead of Unicode characters to ensure that every base character is included in the vocabulary.

WordPiece tokenization

WordPiece is another popular word segmentation algorithm widely used with BERT, **DistilBERT**, and ELECTRA. It was proposed by *Schuster and Nakajima* to solve the Japanese and Korean voice problem in 2012. The motivation behind this work was that, although not a big issue for the English language, word segmentation is important preprocessing for many Asian languages, because in these languages spaces are rarely used. Therefore, we come across word segmentation approaches in NLP studies in Asian languages more often. Similar to BPE, WordPiece uses a large corpus to learn vocabulary and merging rules. While BPE and BBPE learn the merging rules based on co-occurrence statistics, the WordPiece algorithm uses maximum likelihood estimation to extract the merging rules from a corpus. It first initializes the vocabulary with Unicode characters, which are also called vocabulary symbols. It treats each word in the training corpus as a list of symbols (initially Unicode characters), and then it iteratively produces a new symbol merging two symbols out of all the possible candidate symbol pairs based on the likelihood maximization rather than frequency. This production pipeline continues until the desired vocabulary size is reached.

Sentence piece tokenization

Previous tokenization algorithms treat text as a space-separated word list. This space-based splitting does not work in some languages. In the German language, compound nouns are written without spaces, for example, "*Menschenrechte*" (human rights). The solution is to use language-specific pre-tokenizers. In German, an NLP pipeline leverages a compound-splitter module to check whether a word can be subdivided into smaller words. However, East Asian languages (for example, Chinese, Japanese, Korean, and Thai) do not use spaces between words. The **SentencePiece algorithm** is designed to overcome this space limitation, which is a simple and language-independent tokenizer proposed by *Kudo et al.* in 2018. It treats the input as a raw input stream where space is part of the character set. The tokenizer using SentencePiece produces `the _ character`, which is also why we saw _ in the output of the ALBERT model example earlier. Other popular language models that use `SentencePiece` are XLNet, Marian, and T5.

So far, we have discussed subword tokenization approaches. It is time to start conducting experiments for training with the `tokenizers` library.

The tokenizers library

You may have noticed that the already-trained tokenizers for Turkish and English are part of the Transformers library in the previous code examples. On the other hand, the Hugging Face team provided the `tokenizers` library independently from the Transformers library to be fast and give us more freedom. The library was originally written in Rust, which makes multi-core parallel computations possible, and is wrapped in Python (`https://github.com/huggingface/tokenizers`).

To install the `tokenizers` library, we use this:

```
$ pip install tokenizers
```

The `tokenizers` library provides several components so that we can build an end-to-end tokenizer from preprocessing the raw text to decoding tokenized unit IDs:

`Normalizer→ PreTokenizer → Modeling → Post-Processor → Decoding`

The following diagram depicts the tokenization pipeline:

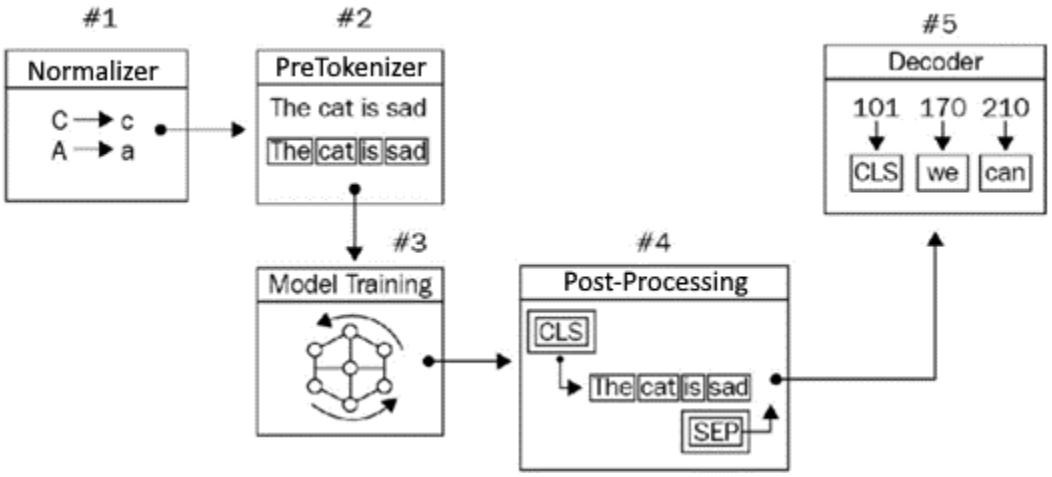


Figure 3.17 – Tokenization pipeline

This pipeline can be explained as follows:

- The normalizer allows us to apply primitive text processing such as lowercasing, stripping, Unicode normalization, and removing accents.
- The PreTokenizer prepares the corpus for the next training phase. It splits the input into tokens depending on the rules, such as whitespace.

- Model training is a subword tokenization algorithm such as BPE, BBPE, and WordPiece, which we've discussed already. It discovers subwords/vocabulary and learns generation rules.
- Post-processing provides advanced class construction that is compatible with Transformers models such as *BertProcessors*. We mostly add special tokens such as `[CLS]` and `[SEP]` to the tokenized input just before feeding the architecture.
- The decoder is in charge of converting token IDs back to the original string. It is just for inspecting what is going on.

Let's see how you can train a BPE-based tokenizer.

Training BPE

Let's train a BPE tokenizer using Shakespeare's plays:

1. It is loaded as follows:

```
import nltk
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
plays=['shakespeare-macbeth.txt','shakespeare-hamlet.txt',
    'shakespeare-caesar.txt']
shakespeare=[" ".join(s) for ply in plays \
    for s in gutenberg.sents(ply)]
```

   We will need a post-processor (`TemplateProcessing`) for all the tokenization algorithms ahead. We need to customize the post-processor to make the input convenient for a particular language model. For example, the following template will be suitable for the BERT model since it needs the `[CLS]` token at the beginning of the input and `[SEP]` tokens both at the end and in the middle.

2. We define the template as follows:

```
from tokenizers.processors import TemplateProcessing
special_tokens=["[UNK]","[CLS]","[SEP]","[PAD]","[MASK]"]
temp_proc= TemplateProcessing(
    single="[CLS] $A [SEP]",
    pair="[CLS] $A [SEP] $B:1 [SEP]:1",
    special_tokens=[
        ("[CLS]", special_tokens.index("[CLS]")),
        ("[SEP]", special_tokens.index("[SEP]")),
    ],
)
```

3. We import the necessary components to build an end-to-end tokenization pipeline:

```
from tokenizers import Tokenizer
from tokenizers.normalizers import (
    Sequence,Lowercase, NFD, StripAccents)
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.models import BPE
from tokenizers.decoders import BPEDecoder
```

4. We start by instantiating BPE as follows:

```
tokenizer = Tokenizer(BPE())
```

5. The preprocessing part has two components: normalizer and `pre-tokenizer`. We may have more than one normalizer. So, we compose a sequence of normalizer components that includes multiple normalizers where `NFD()` is a Unicode normalizer and `StripAccents()` removes accents. For pre-tokenization, `Whitespace()` gently breaks the text based on space. Since the decoder component must be compatible with the model, `BPEDecoder` is selected for the BPE model:

```
tokenizer.normalizer = Sequence(
    [NFD(),Lowercase(),StripAccents()])
tokenizer.pre_tokenizer = Whitespace()
tokenizer.decoder = BPEDecoder()
tokenizer.post_processor=temp_proc
```

6. We are now ready to train the tokenizer on the data. The following execution instantiates `BpeTrainer()`, which helps us to organize the entire training process by setting hyperparameters. We set the vocabulary size parameter to `5000` since our Shakespeare corpus is relatively small. For a large-scale project, we use a bigger corpus and normally set the vocabulary size to around 30 K:

```
from tokenizers.trainers import BpeTrainer
trainer = BpeTrainer(vocab_size=5000,
    special_tokens= special_tokens)
tokenizer.train_from_iterator(shakespeare, trainer=trainer)
print(f"Trained vocab size:{tokenizer.get_vocab_size()}" )
Trained vocab size: 5000
```

We have completed the training!

Note

Training from the filesystem: To start the training process, we passed an in-memory Shakespeare object as a list of strings to `tokenizer.train_from_iterator()`. For a large-scale project with a large corpus, we need to design a Python generator that yields string lines mostly by consuming the files from the filesystem rather than in-memory storage. You should also check `tokenizer.train()` to train from the filesystem storage as applied in the preceding BERT training section.

7. Let's grab a random sentence from *Macbeth*, name it `sen`, and tokenize it with our fresh tokenizer:

```
sen= "Is this a dagger which I see before me, \
    the handle toward my hand?"
sen_enc=tokenizer.encode(sen)
print(f"Output: {format(sen_enc.tokens)}")
Output: ['[CLS]', 'is', 'this', 'a', 'dagger', 'which', 'i',
'see', 'before', 'me', ',', 'the', 'hand', 'le', 'toward', 'my',
'hand', '?', '[SEP]']
```

8. Thanks to the preceding post-processor function, we see additional `[CLS]` and `[SEP]` tokens in the proper position. There is only one split word, handle (`hand`, `le`), since we passed to the model a sentence from *Macbeth* that the model already knew. Besides, we used a small corpus, and the tokenizer is not forced to use compression. Let's pass a challenging phrase, "Hugging Face," that the tokenizer might not know:

```
sen_enc2=tokenizer.encode("Macbeth and Hugging Face")
print(f"Output: {format(sen_enc2.tokens)}")
Output: ['[CLS]', 'macbeth', 'and', 'hu', 'gg', 'ing', 'face',
'[SEP]']
```

9. The term "Hugging" is lower case and split into three pieces `hu`, `gg`, and `ing`, since the model vocabulary contains all other tokens but `Hugging`. Let's pass two sentences now:

```
two_enc=tokenizer.encode("I like Hugging Face!",
    "He likes Macbeth!")
print(f"Output: {format(two_enc.tokens)}")
Output: ['[CLS]', 'i', 'like', 'hu', 'gg', 'ing', 'face', '!',
'[SEP]', 'he', 'li', 'kes', 'macbeth', '!', '[SEP]']
```

   Notice that the post-processor injected the `[SEP]` token as an indicator.

10. It is time to save the model. We can either save the sub-word tokenization model or the entire tokenization pipeline. First, let's save the BPE model only:

```
tokenizer.model.save('.')
['./vocab.json', './merges.txt']
```

11. The model saved two files regarding vocabulary and merging rules. The `merge.txt` file is composed of 4,948 merging rules:

```
$ wc -l ./merges.txt
4948 ./merges.txt
```

12. The top five rules ranked are as shown in the following where we see that *[t, h]* is the first ranked rule due to that being the most frequent pair. For testing, the model scans the textual input and tries to merge these two symbols first if applicable:

```
$ head -3 ./merges.txt
#version: 0.2 - Trained by `huggingface/tokenizers`
t h
o u
a n
th e
r e
```

13. The BPE algorithm ranks the rules based on frequency. When you manually calculate character bigrams in the Shakespeare corpus, you will find *[t, h]* the most frequent pair.

14. Let's now save and load the entire tokenization pipeline:

```
tokenizer.save("MyBPETokenizer.json")
tokenizerFromFile =\
    Tokenizer.from_file("MyBPETokenizer.json")
sen_enc3 = \
    tokenizerFromFile.encode("I like Hugging Face and Macbeth")
print(f"Output: {format(sen_enc3.tokens)}")
Output: ['[CLS]', 'i', 'like', 'hu', 'gg', 'ing', 'face', 'and',
'macbeth', '[SEP]']
```

We successfully reloaded the tokenizer!

Training the WordPiece model

In this section, we will train the WordPiece model:

1. We start by importing the necessary modules:

```
from tokenizers.models import WordPiece
from tokenizers.decoders import WordPiece \
as WordPieceDecoder
from tokenizers.normalizers import BertNormalizer
```

2. The following lines instantiate an empty WordPiece tokenizer and prepare it for training. `BertNormalizer` is a pre-defined normalizer sequence that includes the processes of cleaning the text, transforming accents, handling Chinese characters, and lowercasing:

```
tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer=BertNormalizer()
tokenizer.pre_tokenizer = Whitespace()
tokenizer.decoder= WordPieceDecoder()
```

3. Now, we instantiate a proper trainer, `WordPieceTrainer()` for `WordPiece()`, to organize the training process:

```
from tokenizers.trainers import WordPieceTrainer
trainer = WordPieceTrainer(vocab_size=5000,\
    special_tokens=["[UNK]","[CLS]","[SEP]","[PAD]","[MASK]"])
tokenizer.train_from_iterator(shakespeare, trainer=trainer)
output = tokenizer.encode(sen)
print(output.tokens)
['is', 'this', 'a', 'dagger', 'which', 'i', 'see', 'before',
'me', ',', 'the', 'hand', '##le', 'toward', 'my', 'hand', '?']
```

4. Let's use `WordPieceDecoder()` to treat the sentences properly:

```
tokenizer.decode(output.ids)
'is this a dagger which i see before me, the handle toward my
hand?'
```

5. We have not come across any `[UNK]` tokens in the output since the tokenizer somehow knows or splits the input for encoding. Let's force the model to produce `[UNK]` tokens as in the following code. Let's pass a Turkish sentence to our tokenizer:

```
tokenizer.encode("Kralsın aslansın Macbeth!").tokens
'[UNK]', '[UNK]', 'macbeth', '!']
```

Well done! We have a couple of unknown tokens since the tokenizer does not find a way to decompose the given word from the merging rules and the base vocabulary.

So far, we have designed our tokenization pipeline all the way from the normalizer component to the decoder component. On the other hand, the `tokenizers` library provides us with an already made (not trained) empty tokenization pipeline with proper components to build quick prototypes for production. Here are some pre-made tokenizers:

- `CharBPETokenizer`: The original BPE
- `ByteLevelBPETokenizer`: The byte-level version of the BPE

- `SentencePieceBPETokenizer`: A BPE implementation compatible with the one used by `SentencePiece`
- `BertWordPieceTokenizer`: The famous BERT tokenizer, using WordPiece

The following code imports these pipelines:

```
from tokenizers import (ByteLevelBPETokenizer,
    CharBPETokenizer,
    SentencePieceBPETokenizer,
    BertWordPieceTokenizer)
```

All these pipelines are already designed for us. The rest of the process (such as training, saving the model, and using the tokenizer) is the same as our previous BPE and WordPiece training procedures.

Well done! We have made great progress and trained our first Transformer model as well as its tokenizer.

Summary

In this chapter, we have experienced autoencoding models both theoretically and practically. Starting with learning the basics of BERT, we trained it as well as a corresponding tokenizer from scratch. We also discussed how to work inside other frameworks, such as Keras. Besides BERT, we also reviewed other autoencoding models such as ALBERT, RoBERTa, ELECTRA, and DeBERTa. To avoid excessive code repetition, we did not provide the full implementation for training other models. During the BERT training, we trained the WordPiece tokenization algorithm. In the last part, we examined other tokenization algorithms since it is worth discussing and understanding all of them as different Transformer architectures utilize different tokenization algorithms.

Autoencoding models use the left decoder side of the original Transformer and are mostly fine-tuned for classification problems. In the next chapter, we will discuss and learn about the right decoder part of Transformers to implement language generation models.

4

From Generative Models to Large Language Models

In this chapter, you will learn about **Generative Language Models** (**GLMs**) and **Large Language Model** (**LLMs**). After this, you will learn how to pre-train any language model, such as **Generated Pre-trained Transformer 2** (**GPT-2**), on your own text and use it for tasks such as **Natural Language Generation** (**NLG**). You will learn the basics of **Text-to-Text Transfer Transformer** (**T5**) models, conduct a hands-on multitask learning experiment with T5, and train a **Multilingual T5** (**mT5**) model on your own **Machine Translation** (**MT**) data. After finishing this chapter, you will have an overview of GLMs and their various use cases in text2text applications, such as summarization, paraphrasing, multitask learning, zero-shot learning, and MT.

The following topics will be covered in this chapter:

- Working with GLMs
- Working with **text-to-text** models
- AR language model training
- GLM training
- NLG using AR models

Technical requirements

The following libraries/packages are required to successfully complete this chapter:

- `Anaconda`
- `transformers 4.0.0`
- `pytorch 1.0.2`
- `tensorflow 2.4.0`

- `SimpleT5 0.1.4`
- `datasets 2.10.0`
- `tokenizers`

All notebooks with coding exercises will be available at the Book GitHub page at `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

An introduction to GLMs

While **Autoencoder** (**AE**) language models, such as **Bidirectional Encoder Representations from Transformers** (**BERT**), are purely based on the encoder part of transformers and are well-suited for classification problems, generative models are either encoder-decoder or decoder-only models. GLMs were originally intended for language generation tasks such as MT or text summarization, but we can see different successful use cases used by GLMs.

AE models are particularly effective for classification tasks, such as text classification and sentiment analysis, as well as token-level tasks such as **named entity recognition** (**NER**) or **part-of-speech** (**POS**) tagging. Here, AE models simply perform classification by mapping either the special CLS token at the beginning of the sentence or the individual tokens at any position to the predefined class labels. On the other hand, although GLMs have been widely used for language generation tasks, they have also been successfully used for classification tasks recently.

GLMs are pre-trained using one of two objectives: either a causal language modeling objective, which allows the model to predict the next word in a sequence (for example, GPT-3 or PaLM), or a denoising objective, where the model learns to restore the original sentence from a corrupted one (such as T5 and BART). While the former purely relies on the decoder part of the transformers, also called **Auto Regressive** (**AR**) model, the latter uses both encoder-decoder parts of the transformers, also called `text-to-text` or `seq-to-seq` model.

These models are commonly used in tasks such as MT, dialog systems, and text summarization. An interesting feature that emerges here is that it is possible to use generated text for classification purposes mostly solved by Encoder-only models. However, the reverse is not true. We could not use Encoder-only models for generation. For instance, it is nearly impossible to translate a text with BERT.

Controlling the textual output of the generative models is crucial for preventing issues such as toxic text generation and model hallucinations during problem-solving. By controlling the output, we can easily use GLMs for classification tasks. The critical trick is to express tasks as instructions. Here are some examples:

Task	Input Template	Possible Textual Output
Sentiment Analysis	input text + "what is the sentiment?"	"it is positive" "it is negative"
Text Classification	input text + "what is the text about?"	"it is about the economy" "it is about health".
Text Regression	Input text + "Can you rate the sentence quality on a scale of 1-5?"	"it is 3.0" "it is 1.0"
NER	"... John … Paris …" + prompt	"Paris is a city entity" "John is a person entity"

Table 4.1 – Prompting for tasks

A piece of text called a **prompt** is added to the input so that the model is directed toward the desired target.

Thanks to the generation capacity of LLMs and prompting (we will discuss prompting in detail in later chapters), generative models approach any task as a text-to-text problem, which leads to successful solutions. This approach provides the opportunity to apply multi-task learning very easily since we can handle diverse tasks in the same batch. We can map many tasks into a single structure, which is text-to-text. Major language models such as **GPT-3**, **PaLM**, and **ChatGPT**—which are currently popular—take advantage of this generative feature. We will also discuss LLMs in *Chapter 10*.

A notable example of using GLMs and instruction-based multi-task learning is the FLAN framework. The framework was able to model 1,800 different instruction fine-tuning tasks simultaneously without changing the model's architecture. By scaling (the initial proposal used **LaMDA 137B**) the model to the 540-billion-parameter language model (PaLM) and scaling (the initial task size was 62) to 1,800 tasks, the model can be successful even in tasks that are unaware of its existence during training, which is called **zero-shot learning**. Thanks to such generative models, which allow scaling up in this direction, it is not surprising that we encounter fascinating applications every day.

It is also important to note that prompt engineering as we see it today will vanish in time. We have seen many research articles that aim to find the most effective prompts. However, with advancements in instruction fine-tuning, we are getting closer to having more and more natural instructions. ChatGPT is a fantastic example of these advancements. Instruction capability of ChatGPT (the GPT-3.5 and davinci-003 models) have shown that a better understanding of instructions and a greater ability to follow them by the model can lead to many new features. You might have seen many people working with these fine-tuned models and creating great tools such as conversational question-answering systems.

But (at the time this book was being written) some problems still exist. LLMs still suffer from hallucination, and much research is being conducted to solve this issue. Sometimes, even if you provide a nice and well-written context and ask a question about it (**Open Book Question Answering** or **OBQA**), you will be welcomed by a hallucinated answer. It is very important to use such tools, but it is more important to know their problems as well.

Working with GLMs

The Transformer architecture was originally intended to be effective for text-to-text tasks such as MT and summarization, but it has been used in diverse NLP problems, ranging from token classification to coreference resolution. Subsequent works began to use the left and right parts of the architecture separately and more creatively. Other than next-word prediction, some denoising objectives are also utilized to fully recover the original input from corrupted or truncated input.

While text-to-text models such as T5 use both encoder and decoder parts, decoder-only models such as GPT uses only decoder part. Decoder-only AR prevents the model from accessing words to the right of the current word in a forward direction (or to the left of the current word in a backward direction), which is called **unidirectionality**.

GPT and its successors (GPT-2, GPT-3, **InstructGPT** (a.k.a GPT-3.5), and ChatGPT), **Transformer-XL**, and **XLNet** are some of the popular decoder-only AR models in the literature. Even though XLNet is based on autoregression, it somehow managed to make use of both contextual sides of the word in a bidirectional fashion, with the help of the permutation-based language objective. Now, we start introducing them and show how to train the models with a variety of experiments. Let's look at GPTs first.

GPT model family

AR models are made up of multiple decoder blocks of transformers. Each block contains a masked multi-head self-attention layer along with a pointwise feed-forward layer. The activation in the final transformer block is fed into a SoftMax function that produces the word probability distributions over an entire vocabulary of words to predict the next word. Notable examples are mostly from Open AI's GPT model family.

In the original GPT paper *Improving Language Understanding by Generative Pre-Training* (2018), the authors addressed several bottlenecks that traditional ML-based NLP pipelines are subject to. For example, firstly, traditional pipelines require both a massive amount of task-specific data and task-specific architecture. Secondly, it is hard to apply task-aware input transformations with minimal changes to the architecture of the pre-trained model. The original GPT and its successors (GPT-2 and GPT-3), designed by the OpenAI team, have focused on these problems and their solutions to alleviate these bottlenecks. The major contribution of the original GPT study is that the pre-trained model achieved satisfactory results, not only for a single task but diverse tasks, which is called multi-task learning. Having learned the generative model from unlabeled data, which is self-supervised pre-training, the model is simply fine-tuned to a downstream task (or multiple tasks) with a relatively small amount of task-specific data, which is called **supervised fine-tuning** (**SFT**). This two-stage scheme is widely used in other transformer models as well, where self-supervised pre-training is followed by supervised fine-tuning.

GPT models showed us that GLMs can solve zero-shot task generalization problems by relying specifically on multi-task learning. To make multi-task learning feasible, the GPT architecture is kept as generic as possible: only the inputs are transformed in a task-specific manner, while the entire architecture is kept exactly the same. This approach converts the textual input into a suitable format for the task so that the pre-trained model can understand the task from it. The left part of *Figure 4.1* (from the original paper) illustrates the Transformers architecture and training objectives used in the original GPT work. The right side shows how to transform input for fine-tuning on several tasks.

To put it simply, for a single-sequence task such as text classification, the input is passed through the network as-is, and the linear layer takes the last activations to make a decision. For sentence-pair tasks such as textual entailment, the input that is made up of two sequences is marked with a delimiter, shown as the second example in *Figure 4.1*. In both scenarios, the architecture sees uniform token sequences being processed by the pre-trained model. The delimiter used in this transformation helps the pre-trained model to know which part is the premise and the hypothesis in the case of textual entailment. Thanks to input transformation, we do not have to make substantial changes in the architecture across the tasks.

You can see a representation of input transformation here:

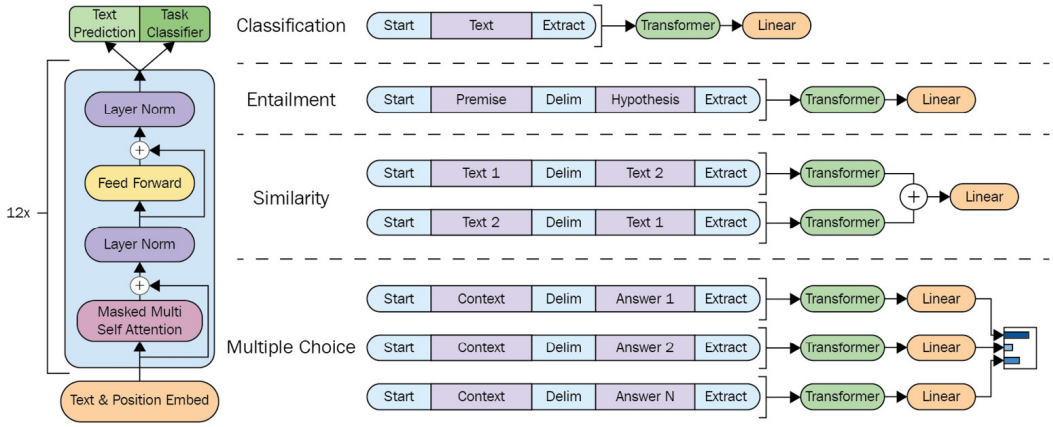


Figure 4.1 – Input transformation (from the original paper)

The GPT and its successors have mainly focused on achieving a particular architectural design that eliminates the need for a fine-tuning phase. This is based on the idea that a model can learn a great deal about a language in the pre-training phase, leaving little work to be done during fine-tuning. During inference, the model is given a text as well as a few examples of tasks as input. In some cases, such as zero-shot learning, no examples are provided at all.

Successors of the original GPTs

The popular ChatGPT, as the names suggest, is based on GPT, which was developed by the OpenAI team. Along the way, the team designed several other models, including GPT2, GPT3, InstructGPT, and ChatGPT.

GPT-2 (see the paper *Language Models are Unsupervised Multitask Learners*, 2019), a successor to the original GPT called **WebText**, is a larger model than GPT and was trained on much more training data than the original one. GPT-2 achieved better results on seven out of the eight tasks in a zero-shot setting in which there was no fine-tuning applied but had limited success in some other tasks. The idea behind the GPT-2 is that language models mostly need self-supervised pre-training. Although the ideas behind the model were very valuable, the performance was not very good. However, it worked surprisingly well in some cases. Researchers pursued this approach because they realized that this zero-shot/few-shot learning idea could be applied by scaling some aspects of training process. They saw that a task-agnostic model could be trained on a huge and diverse dataset.

Since they had seen that the zero and few-shot capabilities of language models substantially improved by scaling up the model size, the dataset size, and the task size in multi-tasking, the OpenAI team trained the GPT-3 model (see the paper *Language models are few-shot learners*, 2020) with 175 billion parameters, which is 100 times bigger than GPT-2. The study showed that scaling up models substantially improves task-agnostic and few-shot learning performance.

The architectures of GPT-2 are GPT-3 are similar, with the main differences usually being in the model size, diverse tasks, and the dataset quantity/quality. Due to the massive amount of data in the dataset and the large number of parameters it is trained on, it achieved better results on many downstream tasks in zero-shot, one-shot, and few-shot (such as *K=32 shots*) settings without any gradient-based fine-tuning. The team showed that the model performance increased as the parameter size and the number of examples increased for many tasks, including translation, question answering, and masked-token tasks.

One of the main problems of GLMs is the potential for generating toxic information. In response to concerns about toxic language and misinformation, the team introduced a new approach called InstructGPT as another extension of the GPT model family. To enhance the safety and usefulness of the model, the team employed a technique called **reinforcement learning from human feedback** (**RLHF**) to refine GPT-3. In comparison to GPT-3, the InstructGPT model demonstrates an improved ability to comprehend and execute instructions expressed in natural language. Furthermore, it exhibits a reduced likelihood of generating false or harmful information.

The OpenAI team has developed a three-step process for InstructGPT. The initial step involves fine-tuning the model, followed by the creation of a **reward model** (**RM**). The third and final step is to take the supervised fine-tuning model and refine it further using reinforcement learning. One advantage of InstructGPT is its strong alignment with human intention. This is achieved through the utilization of a reinforcement learning framework, which enables it to learn from human feedback. It is said that Popular ChatGPT is based on InstructionGPT, but we have not seen any documentation about how ChatGPT has been trained yet.

Transformer-XL

Another important generative model is Transformer-XL. The author of the model stated that Transformer models suffer from the fixed-length context due to a lack of recurrence in the initial design and context fragmentation, although they are capable of learning long-term dependencies. Most transformers break down documents into a list of fixed-length (mostly 512) segments, where any information flow across segments is not possible. Consequently, the language models are not able to capture long-term dependencies beyond this fixed-length limit. Moreover, the segmentation procedure builds the segments without paying attention to sentence boundaries. A segment can be absurdly made up of the second half of a sentence and the first half of its successor, hence the language models can miss the necessary contextual information when predicting the next token. This problem is referred to as the *context fragmentation* problem in the literature.

To address and overcome these issues, the Transformer-XL authors (see the paper *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*, 2019) proposed a new transformer architecture that includes a segment-level recurrence mechanism and a new positional encoding scheme. This approach inspired many subsequent models. The largest possible dependency length that the model can attend is limited by the number of layers and segment lengths.

XLNet

Masked Language Modeling (**MLM**) has dominated the pre-training phase of encoder-only transformer architectures. However, it has faced criticism in the past since the masked tokens are present in the pre-training phase but are absent during the fine-tuning phase, which leads to a discrepancy between pre-training and fine-tuning. Because of this absence, the model may not be able to use all of the information learned during the pre-training phase. XLNet (see the paper *XLNet: Generalized Autoregressive Pretraining for Language Understanding*, 2019) replaces MLM with **Permuted Language Modeling** (**PLM**), which is a random permutation of the input tokens to overcome this bottleneck. The permutation language modeling makes each token position utilize contextual information from all positions, leading to capturing bidirectional context. The objective function only permutes the factorization order and defines the order of token predictions, but doesn't change the natural positions of sequences. Briefly, the model chooses some tokens as targets after permutation, and it then tries to predict them conditioned on the remaining tokens and the natural positions of the targets. It makes it possible to use an AR model in a bidirectional fashion.

XLNet takes advantage of both AE and AR models. It is, indeed, a generalized AR model. Besides its objective function, XLNet is made up of two important mechanisms: it integrates the segment-level recurrence mechanism of Transformer-XL into its framework, and it includes the careful design of the two-stream attention mechanism for target-aware representations.

Let's discuss those models using both parts of the Transformers in the next section.

Working with text-to-text models

The left encoder and the right decoder parts of the transformer are connected with cross-attention, which helps each decoder layer attends over the final encoder layer, which means it exploits encoded information accumulated at the encoder layer. This naturally pushes models toward producing output that is closely tied to the original input. The following are popular text-to-text models that keep the encoder and decoder part of the transformer:

- **T5**: Exploring the limits of transfer learning with a unified text-to-text transformer
- **BART**: **Bidirectional and Auto-Regressive Transformer**
- **PEGASUS**: **Pre-training with Extracted Gap-sentences for Abstractive Summarization Sequence-to-Sequence** models

Let's begin to understand and work with T5.

Multi-task learning with T5

Most NLP architectures, ranging from Word2Vec to transformers, learn embeddings and other parameters by predicting the masked words using context (neighbor) words. We treat NLP problems as word prediction problems. While some text-to-text models purely rely on next-word prediction, some, such as T5 and BART, exploit denoising objectives, which maximize the model capacity in terms of reconstructing randomly corrupted text. That is, the model is trained to predict missing or corrupted tokens in the input. T5 (see the paper *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, 2019) emphasized this point, and the authors developed their own objective function based on it. But the main contribution of T5 is its capacity for multi-task learning by rephrasing the task as instruction.

T5 proposed a unifying framework to solve many tasks under the same structure. The idea underlying T5 is to rephrase all NLP tasks to a text-to-text (Seq2Seq) problem where both input and output are a list of tokens. Transformers have been found to be useful in applying the same model to diverse NLP tasks, as seen in GPT-3, from question answering to text summarization, which enables us to employ multi-task training.

The following diagram, which is inspired by the original paper, shows how T5 solves many (four in this figure) different NLP tasks—MT, linguistic acceptability, semantic similarity, and summarization—within a unified framework:

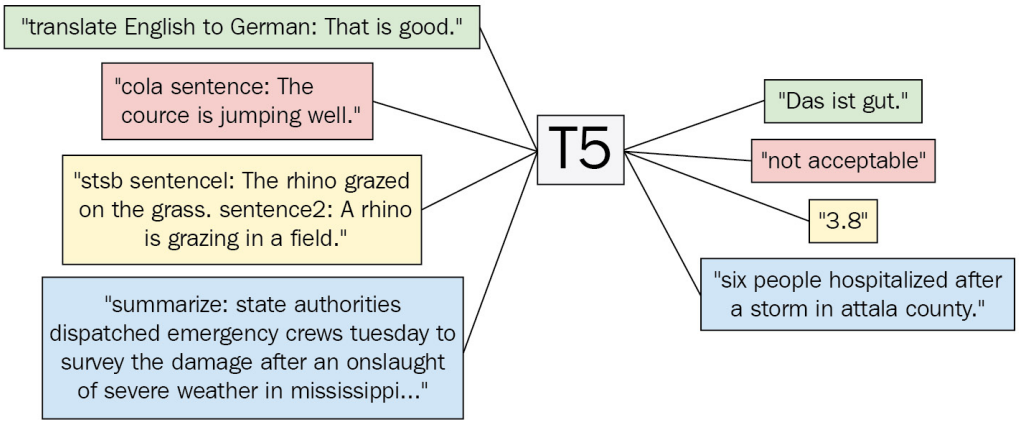


Figure 4.2 – The T5 framework

The T5 model roughly follows the original encoder-decoder transformer model. The modifications are done in the layer normalization, denoising objective function, and position embedding scheme. Instead of using sinusoidal positional embedding or learned embedding, T5 uses relative positional embedding, which is becoming more common in transformer architectures.

T5 casts tasks into a text-to-text format. The model is fed with text that is made up of a task prefix and the input attached to it. We convert a labeled textual dataset to a `{'inputs': '....', 'targets': ...'}` format, where we insert the purpose in the input as a prefix. Then, we train the model with labeled data so that it learns what to do and how to do it. As shown in the preceding diagram, for the English-German translation task, the `"translate English to German: That is good."` input is going to produce `"das is gut."`. Likewise, any input with a `"summarize:"` prefix will be summarized by the model.

It should be noted that the T5 model is not designed for task-agnostic purposes. In the next subsection, we will discuss how to modify it to become task-agnostic, which is referred to as T0.

Multi-task training with T5

Now, let's take a simple look at how to apply multi-task training in a T5 architecture with the `SimpleT5` library. To fully understand and address multitask learning, we'll start with a **proof-of-concept** (**PoC**). Our PoC will be an exercise that's focused on determining whether the multi-tasking idea can be turned into a reality. But it is important to explore all potential implications, of course. To start, model selection, datasets, and task diversity will be kept as simple as possible. We will use two different datasets: the `glue-sst2` dataset for sentiment analysis and the `opus_en-de` (English-German) machine translation dataset from the **Opus project**.

NLP libraries

In this book, we are using various sources and libraries so you can experience different infrastructures. In this way, we will learn about a lot of open-source software. We circulate them for the community. For example, we will now use the `SimpleT5` library for multi-task training. In the next subsection, we will train T5 with the `simpletransformer` library for MT. They are both built on top of Hugging Face's Transformers library.

First, let's install the necessary libraries:

```
pip install simpleT5 datasets
```

For now, let's download the small checkpoint of the T5 model so that we can do faster training. But please, do not forget that we need bigger models to improve performance for real production. This is especially crucial for few-shot problems.

```
from simplet5 import SimpleT5
model = SimpleT5()
model.from_pretrained("t5","t5-small")
```

Now, we will use the `datasets` library to load the datasets and prepare them for modeling. As SimpleT5 requires a pandas DataFrame and for the column names to be `[source_text, target_text]`, we renamed the columns within our pandas DataFrame. We take only the first 1,000 sentences for quick prototyping. Once you have successfully trained the model, you should/can use the entire dataset to improve the accuracy!

```
import pandas as pd
from datasets import load_dataset
sst2_df =pd.DataFrame(
    load_dataset("glue",
        "sst2",
        split="train[:1000]"))
sst2_df=sst2_df[["sentence","label"]]
sst2_df.columns= ["source_text","target_text"]
```

Now we will use prompts to rephrase the tasks so they are suitable for text-to-text. We add the instruction "What is the sentiment? Good or Bad?" to the source sentence. Likewise, we set suitable tokens `"Good"` and `"Bad"` for `1` and `0` for the target text as follows:

```
prompt_sentiment=". What is the sentiment ? "\
    +"Good or Bad ?"
sst2_df["source_text"]= sst2_df\
    .source_text\
    .apply(lambda x: x +prompt_sentiment)
```

```
sst2_df["target_text"]=sst2_df\
    .target_text\
    .apply(lambda x:"Good" if x==1 else "Bad")
```

We will make a similar arrangement in the MT dataset to fit the text-to-text template. Let's load only 1,000 instances from the `opus en-de` dataset first. Notice that we set `source` and `target` as English and German respectively to apply English-to-German translation:

```
opus=load_dataset("opus100","de-en", split="train[:1000]")
opus_df=pd.DataFrame(opus)
opus_df["source_text"]=opus_df.apply(
    lambda x: x.translation["en"], axis=1)
opus_df["target_text"]=opus_df.apply(
    lambda x: x.translation["de"], axis=1)
opus_df= opus_df[["source_text","target_text"]]
```

The prompt that we add at the end of the sentences is `"Translate English to German"`, as follows:

```
prompt_tr=". Translate English to German"
opus_df["source_text"]= opus_df\
    .source_text\
    .apply(lambda x: x +prompt_tr)
```

Our two tasks are ready for training. We merge and shuffle them:

```
merge= opus_df.append(sst2_df).sample(frac=1.0)
merge.head(5)
```

source_text	target_text
a perfect performance . What is the sentiment ...	Good
Aw.. Translate English to German	- Was Neues von Tara?
a clunky tv-movie approach to detailing a chap...	Bad
So what do you want from me?. Translate Englis...	- Nun, das freut mich für sie.
warm water under a red bridge is a celebration...	Good

Figure 4.3 — Two different NLP tasks in one seq-2-seq format

Finally, we have two tasks in one bag in the form of seq-2-seq. We take 1,800 instances for training and 200 for testing:

```
train_df=merge[:1800]
eval_df= merge[1800:]
```

Thanks to the SimpleT5 framework, the training is very simple. We keep the source token length as `512`, which is longer than the target length, `128`:

```
model.train(train_df=train_df,
    eval_df=eval_df,
    source_max_token_len = 512,
    target_max_token_len = 128,
    batch_size = 8,
    max_epochs = 3,
    use_gpu = True,
    precision = 32)
```

Once the training is successfully completed, we can experience the application of multi-task learning with inference. Now we will take the same sentence, `"The cats are fun!"`, for both sentiment analysis and MT using the appropriate prompt.

Let's start with sentiment by just adding a sentiment prompt to the sentence:

```
import torch
model.device= torch.device("cpu")
a_sentence="The cats are fun!"
model.predict(a_sentence+ prompt_sentiment)
Output:['Good']
```

The output is `"Good"`, which is the correct label. But the important thing at this point is that the model understood what the task is. T5 checkpoints (T5-small, T5-base or T5-large) have been already pre-trained with different tasks. So, they have the potential to misunderstand the tasks. We can say now that we have correctly guided the model with prompting. We do not care now if the model classifies perfectly or not. Thus, we do not evaluate the model's performance.

Now, change the instruction to *en-de* translation by adding `prompt_tr` to the sentence and pass the prompt to the model:

```
model.predict(a_sentence+ prompt_tr)
Output: ['Die Katzen sind spass!']
```

Wow! It worked out well. In both cases, the model understood what to do. The model's translation capacity is very good even though we use few instances since the pre-trained T5 checkpoints have been trained on such translation tasks before. We used transfer learning here by fine-tuning! However, let us remind you that we need to take some action to increase the performance of the model:

- First of all, use the entire dataset, not just 1,000 pieces.
- Increase task diversity. Tasks such as NER, question-answering, and summarization can be added.
- Use more than one dataset for each task.
- Increase instruction diversity. Use the same dataset with a different prompt.
- We used T5-small as the initial model. Choose larger models such as T5-base or T5-large as suitable for your hardware.
- Model training can take a lot of time. Focus on parameter-efficient approaches and frameworks.

In this part, we have built a simple multi-task learning model by combining all NLP problems into a single format. Now, let's discuss what T0 is.

Zero-Shot Text Generalization with T0

The T5 framework was successful in the tasks for which it was trained. This success can be attributed to its multi-tasking modeling scheme. In other words, solving different tasks within the same architecture has a positive effect on each task by circulating knowledge between them. However, the T5 framework has a shortcoming in that it is not task-agnostic, which means it is not designed for solutions that were not included in its training.

It is known that scaling the model's parameters, dataset size, and task diversity can lead to breakthroughs in the language understanding capabilities of the models for a variety of difficult tasks, as well as held-out tasks. With this motivation, the T0 team worked on a task-agnostic model that could successfully solve zero-shot generalization. This model, called T0, was trained based on the T5 architecture, which considers scaling the model and the diversity of the data.

Here, you can see a simplified T0 diagram of training and inference:

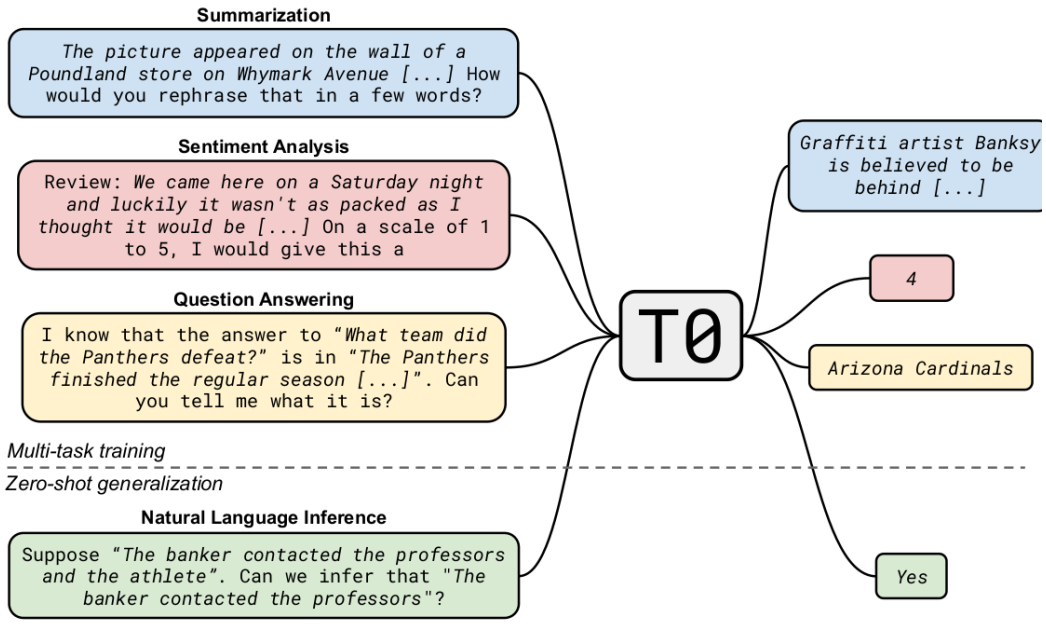


Figure 4.4 – The T0 framework

As shown in the figure, the model is first trained on a diverse mixture of held-in tasks (at the top of the figure). Then, it is evaluated on zero-shot generalization to the held-out tasks that were not seen during training (at the bottom of the figure). There are 62 datasets grouped into 12 tasks, and the T0 model is tested for the zero-shot task generalization problem between tasks, not datasets.

Another Denoising-Based Seq2Seq Model – BART

As with XLNet, the BART model (see the paper *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*, 2019) takes advantage of the schemes of AE and AR models. It uses standard Seq2Seq transformer architecture, with a small modification. However, it has a different training objective than other AR models. It uses a variety of noising approaches that corrupt documents rather than predicting the next word. The main contribution of the model to the field is that it allows us to apply several types of creative corruption schemes, as shown in the following diagram:

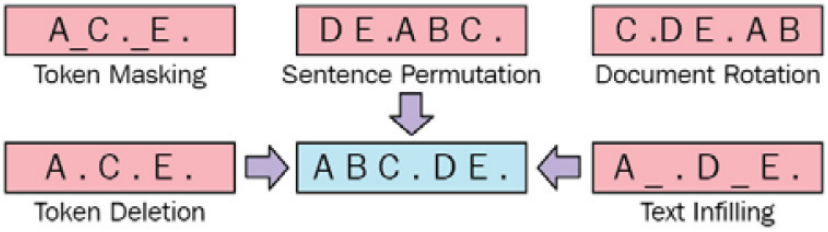


Figure 4.5 – Diagram from the original BART paper

We will look at each scheme in detail, as follows:

- Token masking: Tokens are randomly masked with a `[MASK]` symbol, the same as with the BERT model.
- Token deletion: Tokens are randomly removed from the documents. The model is forced to determine which positions are removed.
- Text infilling: Following **SpanBERT**, a number of text spans are sampled, and then they are replaced by a single `[MASK]` token. There is also `[MASK]` token insertion.
- Sentence permutation: The sentences in the input are segmented and shuffled in a random order.
- Document rotation: The document is rotated so that it begins with a randomly selected token, C in the preceding diagram. The objective is to find the start position of a document.

The **BART** model can be fine-tuned in several ways for downstream applications such as **BERT**.

For the task of sequence classification, the input is passed through an encoder and a decoder, and the final hidden state of the decoder is considered the learned representation. Then, a simple linear classifier can make predictions. Likewise, for token classification tasks, the entire document is fed into the encoder and decoder, and the last state of the final decoder is the representation for each token. Based on these representations, we can carry out token classification tasks such as NER and POS tagging.

For sequence generation, the decoder block of the BART model can be directly fine-tuned for sequence-generation tasks such as abstractive QA or summarization. The BART authors trained the models using two standard summarization datasets: **CNN/DailyMail** and **XSum**. The authors also showed that it is possible to use both the encoder part—which consumes a source language—and the decoder part, which produces the words in the target language as a single pre-trained decoder for MT. They replaced the encoder embedding layer with a new randomly initialized encoder in order to learn words in the source language. Then, the model is trained in an end-to-end fashion, which trains the new encoder to map foreign words into an input that BART can denoise to the target language. The new encoder can use a separate vocabulary, including foreign language, from the original BART model.

In the Hugging Face platform, we can access the original pre-trained BART model with the following line of code:

```
AutoModel.from_pretrained('facebook/bart-large')
```

When we call the standard `summarization` pipeline of the `transformers` library, as shown in the following line of code, a distilled pre-trained BART model is loaded. This call implicitly loads the `sshleifer/distilbart-cnn-12-6` model and the corresponding tokenizers, as follows:

```
summarizer = pipeline("summarization")
```

The following code explicitly loads the same model and the corresponding tokenizer. The code example takes a text to be summarized and outputs the results:

```
from transformers import (
    BartTokenizer, BartForConditionalGeneration, BartConfig)
from transformers import pipeline
model = BartForConditionalGeneration\
    .from_pretrained('sshleifer/distilbart-cnn-12-6')
tokenizer = BartTokenizer\
    .from_pretrained('sshleifer/distilbart-cnn-12-6')
nlp=pipeline("summarization",
     model=model,
     tokenizer=tokenizer)
text='''
We order two different types of jewelry from this
company the other jewelry we order is perfect.
However with this jewelry I have a few things I
don't like. The little Stone comes out of these
and customers are complaining and bringing them
back and we are having to put new jewelry in their
holes. You cannot sterilize these in an autoclave
...[truncated]'''
q=nlp(text)
import pprint
pp = pprint.PrettyPrinter(indent=0, width=100)
pp.pprint(q[0]['summary_text'])
(' The little Stone comes out of these little stones and customers
are complaining and bringing ' 'them back and we are having to put new
jewelry in their holes . You cannot sterilize these in an ' 'autoclave
because it heats up too much and the glue does not hold up so the
second group of ' 'these that we used I did not sterilize them that
way and the stones still came out.')O/
```

In the next section, we look at how to train base models.

GLM training

In this section, you will learn how it is possible to train your own language models. We will start with GPT-2 and get a deeper look inside its different functions for training using the `transformers` library.

You can find any corpus to train your own GPT-2, but for this example, we used *Emma* by Jane Austen, which is a romantic novel. Training on a much bigger corpus is highly recommended to generate more general language.

Before we start, it's good to note that we used TensorFlow's native training functionality to show that all Hugging Face models can be directly trained on TensorFlow or PyTorch if you wish to. Follow these steps:

1. You can download the *Emma* raw text by using the following command:

```
wget https://raw.githubusercontent.com/teropa/nlp/master/
resources/corpora/gutenberg/austen-emma.txt
```

2. The next step is to train the `BytePairEncoding` tokenizer for GPT-2 on a corpus that you intend to train your GPT-2 on. The following code will import the `BPE` tokenizer from the `tokenizers` library:

```
from tokenizers.models import BPE
from tokenizers import Tokenizer
from tokenizers.decoders import ByteLevel as ByteLevelDecoder
from tokenizers.normalizers import Sequence, Lowercase
from tokenizers.pre_tokenizers import ByteLevel
from tokenizers.trainers import BpeTrainer
```

3. As you can see, in this example, we intend to train a more advanced tokenizer by adding more functionality, such as the `Lowercase` normalization. To make a `tokenizer` object, you can use the following code:

```
tokenizer = Tokenizer(BPE())
tokenizer.normalizer = Sequence([Lowercase()])
tokenizer.pre_tokenizer = ByteLevel()
tokenizer.decoder = ByteLevelDecoder()
```

   The first line makes a tokenizer from the `BPE` tokenizer class. For the normalization part, `Lowercase` has been added, and the `pre_tokenizer` attribute is set to `ByteLevel` to ensure we have bytes as our input. The `decoder` attribute must be also set to `ByteLevelDecoder` to be able to decode correctly.

4. Next, the tokenizer will be trained using a `50000` maximum vocabulary size and an initial alphabet from `ByteLevel`, as follows:

```
trainer = BpeTrainer(vocab_size=50000,
    inital_alphabet=ByteLevel.alphabet(),
    special_tokens=[
        "<s>",
        "<pad>",
        "</s>",
        "<unk>",
        "<mask>"
        ])
tokenizer.train(["austen-emma.txt"], trainer)
```

5. It is also necessary to add special tokens to be considered. To save the tokenizer, you are required to create a directory, as follows:

```
!mkdir tokenizer_gpt
```

6. You can save the tokenizer by running the following command:

```
tokenizer.save("tokenizer_gpt/tokenizer.json")
```

7. Now that the tokenizer is saved, it's time to preprocess the corpus and make it ready for GPT-2 training using the saved tokenizer, but first, important imports must not be forgotten. The code to do the imports is illustrated in the following snippet:

```
from transformers import (
    GPT2TokenizerFast, GPT2Config, TFGPT2LMHeadModel)
```

8. And the tokenizer can be loaded by using `GPT2TokenizerFast`, as follows:

```
tokenizer_gpt = GPT2TokenizerFast.from_pretrained(
    "tokenizer_gpt")
```

9. It is also essential to add special tokens with their marks, like this:

```
tokenizer_gpt.add_special_tokens({
    "eos_token": "</s>",
    "bos_token": "<s>",
    "unk_token": "<unk>",
    "pad_token": "<pad>",
    "mask_token": "<mask>"
})
```

10. You can also double-check to see if everything is correct or not by running the following code:

```
tokenizer_gpt.eos_token_id
>> 2
```

This code will output the **End-of-Sentence** (**EOS**) token **Identifier** (**ID**), which is `2` for the current tokenizer.

11. You can also test it for a sentence by executing the following code:

```
tokenizer_gpt.encode("<s> this is </s>")
>> [0, 265, 157, 56, 2]
```

For this output, `0` is the beginning of the sentence, `265`, `157`, and `56` are related to the sentence itself, and the EOS is marked as `2`, which is `</s>`.

12. These settings must be used when creating a configuration object. The following code will create a `config` object and the TensorFlow version of the GPT-2 model:

```
config = GPT2Config(
    vocab_size=tokenizer_gpt.vocab_size,
    bos_token_id=tokenizer_gpt.bos_token_id,
    eos_token_id=tokenizer_gpt.eos_token_id
)
model = TFGPT2LMHeadModel(config)
```

13. On running the `config` object, you can see the configuration in dictionary format, as follows:

```
config
>> GPT2Config { "activation_function": "gelu_new", "attn_
pdrop": 0.1, "bos_token_id": 0, "embd_pdrop": 0.1, "eos_token_
id": 2, "gradient_checkpointing": false, "initializer_range":
0.02, "layer_norm_epsilon": 1e-05, "model_type": "gpt2",
"n_ctx": 1024, "n_embd": 768, "n_head": 12, "n_inner": null,
"n_layer": 12, "n_positions": 1024, "resid_pdrop": 0.1,
"summary_activation": null, "summary_first_dropout": 0.1,
"summary_proj_to_labels": true, "summary_type": "cls_index",
"summary_use_proj": true, "transformers_version": "4.3.2", "use_
cache": true, "vocab_size": 11750}
```

As you can see, other settings are not touched, and the interesting part is that `vocab_size` is set to `11750`. The reason for this is that we set the maximum vocabulary size to be `50000`, but the corpus had fewer words, and its **Byte-Pair Encoding** (**BPE**) token created `11750` words.

14. Now, you can get your corpus ready for pre-training, as follows:

```
with open("austen-emma.txt", "r", encoding='utf-8') as f:
    content = f.readlines()
```

15. The content will now include all raw text from the raw file, but it is required to remove `'\n'` from each line and drop lines with fewer than `10` characters, as follows:

```
content_p = []
for c in content:
    if len(c)>10:
        content_p.append(c.strip())
content_p = " ".join(content_p)+tokenizer_gpt.eos_token
```

16. Dropping short lines will ensure that the model is trained on long sequences in order to be able to generate longer sequences. At the end of the preceding snippet, `content_p` has the concatenated raw file with `eos_token` added to the end. But you can follow different strategies too—for example, you can separate each line by adding `</s>` to each line, which will help the model to recognize when the sentence ends. However, we intend to make it work for much longer sequences without encountering EOS. The code is illustrated in the following snippet:

```
tokenized_content = tokenizer_gpt.encode(content_p)
```

The GPT tokenizer from the preceding code snippet will tokenize the whole text and make it one whole, long sequence of token IDs.

17. Now, it's time to make the samples for training, as follows:

```
sample_len = 100
examples = []
for i in range(0, len(tokenized_content)):
    examples.append(
        tokenized_content[i:i + sample_len])
```

18. The preceding code makes `examples` a size of `100` for each one starting from a given part of text and ending at `100` tokens later:

```
train_data = []
labels = []
for example in examples:
    train_data.append(example[:-1])
    labels.append(example[1:])
```

In `train_data`, there will be a sequence of size `99` from the start to the 99th token, and the labels will have a token sequence from `1` to `100`.

19. For faster training, it is required to put the data in the form of a TensorFlow dataset, as follows:

```
Import tensorflow as tf
buffer = 500
batch_size = 16
dataset = tf.data.Dataset.from_tensor_slices(
    (train_data, labels))
dataset = dataset.shuffle(buffer).batch(batch_size,
    drop_remainder=True)
```

`buffer` is the buffer size used for shuffling data, and `batch_size` is the batch size for training. `drop_remainder` is used to drop the remainder if it is less than `16`.

20. Now, you can specify your `optimizer`, `loss`, and `metrics` properties, as follows:

```
optimizer = tf.keras.optimizers.Adam(
       learning_rate=3e-5,
       epsilon=1e-08, clipnorm=1.0)
loss = tf.keras.losses\
     .SparseCategoricalCrossentropy(from_logits=True)
metric = tf.keras.metrics\
      .SparseCategoricalAccuracy('accuracy')
model.compile(optimizer=optimizer,
       loss=[loss, *[None] * model.config.n_layer],
       metrics=[metric])
```

21. The model is compiled and ready to be trained with the number of epochs you wish, as follows:

```
epochs = 10
model.fit(dataset, epochs=epochs)
```

You will see an output that looks something like this:

```
WARNING:tensorflow:The parameters `output_attentions`, `output_hidden_states` and `use_cache` cannot be updated when callin
g a model.They have to be set to True/False in the config object (i.e.: `config=XConfig.from_pretrained('name', output_atte
ntions=True)`).
WARNING:tensorflow:The parameter `return_dict` cannot be set in graph mode and will always be set to `True`.
83/83 [==============================] - 212s 3s/step - loss: 6.7420 - logits_loss: 6.7420 - logits_accuracy: 0.0992 - past
_key_values_1_accuracy: 0.0026 - past_key_values_2_accuracy: 0.0021 - past_key_values_3_accuracy: 0.0018 - past_key_values_
4_accuracy: 0.0018 - past_key_values_5_accuracy: 0.0027 - past_key_values_6_accuracy: 0.0022 - past_key_values_7_accuracy:
0.0033 - past_key_values_8_accuracy: 0.0032 - past_key_values_9_accuracy: 0.0034 - past_key_values_10_accuracy: 0.0018 - pa
st_key_values_11_accuracy: 0.0039 - past_key_values_12_accuracy: 0.0019
```

Figure 4.6 – GPT-2 training using TensorFlow/Keras

We will now look at NLG using AR models. Now that you have saved the model, it will be used for generating sentences in the next section.

Up until this point, you have learned how it is possible to train your own model for NLG. In the next section, we describe how to utilize NLG models for language generation.

NLG using AR models

In the previous section, you learned how it is possible to train an AR model on your own corpus. As a result, you trained a GPT-2 version of your own. But the answer to the question *How can I use it?* remains. To answer that, let's proceed as follows:

1. Let's start generating sentences from the model you have just trained, as follows:

```
def generate(start, model):
    input_token_ids = tokenizer_gpt.encode(start,
        return_tensors='tf')
    output = model.generate(
        input_token_ids,
        max_length = 500,
        num_beams = 5,
        temperature = 0.7,
        no_repeat_ngram_size=2,
        num_return_sequences=1
    )
    return tokenizer_gpt.decode(output[0])
```

The `generate` function that is defined in the preceding code snippet takes a `start` string and generates sequences following that string. You can change parameters such as `max_length` to a smaller sequence size or `num_return_sequences` to have different generations.

2. Let's just try it with an empty string, as follows:

```
generate(" ", model)
```

We get the following output:

```
'  it was a nervous; and emma could not but it, and made it necessary to be cheerful. his spirits required with th
em; hating change of every kind.  he was by no means yet reconciled to his own daughter\'s marrying, was always di
sagreeable; but with compassion, as the origin of affection, though it had been entirely a mile from his habits of
being never able to part with miss taylor been a great must be accepted in herself as for herself; fond of gentle
selfishness, nor could ever speak of her own, only half a long october and from isabella\'s being now obliged to s
uppose that great deal happier if she was very much beyond her father and he could feel differently from such a go
od was now long ago for him from himself to have had done as sad a thing for her life at hartfield. emma was much
older man in their little too long evening, when and would not have been living together as her friend and a very
early from them, they had spent all the house of having been supplied by any means rank as cheerfully as friend wa
s her but emma smiled and bear the rest of emma had ceased to say her sister\'s marriage, "how she had died too mu
ch disposed to keep him not to think a little children of authority being settled in london was some satisfaction
in great danger of great comfort and chatted as large and with what a house from five years had said at any time.
weston was more than an excellent woman as he had many a man, the want of the with her temper had her advantages,
but little way and her daily but particularly was no companion for even half so unperceived, who had taught and wi
sh she dearly loved her through the actual disparity in mr. woodhouse had such an affection; you have never any re
straint as governess than any odd humours, it is such thoughts; highly her!" "i cannot that other-and you know fro
m a large.--and how was used to see her in affection for having great house; these were first with all his heart a
nd had hardly allowed her pleasant society again. how nursed her many now in consequence of a much have recommende
d him at this is the friendliness of sixteen years old and friend very mutually attached, being left to impose any
disagreeable consciousness were here to sit and november evening must go in the next visit from intellectual such
as few a match of both daughters, very good-people have more the advantage of his life in ways than a melancholy c
hange than her as great with you would be felt every body that her place had miss'
```

Figure 4.7 – GPT-2 text-generation example

As you can see from the preceding output, a long text is generated, even if the semantics of the text is not very pleasing, but the syntax is almost correct in many cases.

3. Now, let's try different starts, with `max_length` set to a lower value such as `30`, as follows:

```
generate("weston was very good")
>> 'weston was very good; but it, that he was a great must be a
mile from them, and a miss taylor in the house;'
```

As you may recall, `weston` is one of the characters from the novel.

4. To save the model, you can use the following code to make it reusable for publishing or different applications:

```
model.save_pretrained("my_gpt-2/")
```

5. To make sure your model is saved correctly, you can try loading it, as follows:

```
model_reloaded = TFGPT2LMHeadModel.from_pretrained("my_gpt-2/")
```

Two files are saved—a `config` file and a `model.h5` file, which is for the TensorFlow version. We can see both of these files in the following screenshot:

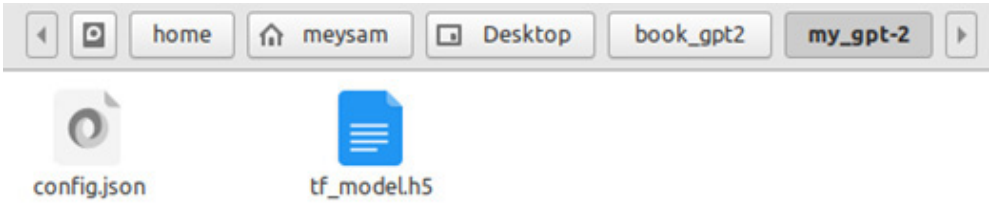


Figure 4.8 – Language model save_pretrained output

6. Hugging Face also has a standard for filenames that must be used—these standard filenames are available by using the following import:

```
from transformers import (WEIGHTS_NAME, CONFIG_NAME,
    TF2_WEIGHTS_NAME)
```

However, when using the `save_pretrained` function, it is not required to put the filenames—just the directory will suffice.

7. Hugging Face also has `AutoModel` and `AutoTokenizer` classes, as you saw in the previous sections. You can also use this functionality to save the model, but before doing that there are still a few configurations that need to be done manually. The first thing is to save the tokenizer in the right format to be used by `AutoTokenizer`. You can do this by using `save_pretrained`, as follows:

```
tokenizer_gpt.save_pretrained("tokenizer_gpt_auto/")
```

This is the output:

```
('tokenizer_gpt_auto/tokenizer_config.json',
 'tokenizer_gpt_auto/special_tokens_map.json',
 'tokenizer_gpt_auto/vocab.json',
 'tokenizer_gpt_auto/merges.txt',
 'tokenizer_gpt_auto/added_tokens.json')
```

Figure 4.9 – Tokenizer save_pretrained output

8. The file list is shown in the directory you specified, but `tokenizer_config` must be manually changed to be usable. First, you should rename it as `config.json`, and secondly, you should add a property in **JavaScript Object Notation** (**JSON**) format, indicating that the `model_type` property is `gpt2`, as follows:

```
{"model_type":"gpt2",
...
}
```

9. Now, everything is ready, and you can simply use these two lines of code to load `model` and `tokenizer`:

```
model = AutoModel.from_pretrained("my_gpt-2/", from_tf=True)
tokenizer = AutoTokenizer.from_pretrained("tokenizer_gpt_auto")
```

However, do not forget to set `from_tf` to `True` because your model is saved in TensorFlow format.

Well done. You have now learned how you can pre-train and save your own text-generation model using TensorFlow and transformers. You have also learned how to train your own AR models.

Summary

In this chapter, we have covered various aspects of GLMs (AR and seq2seq), from pre-training to fine-tuning. We looked at the best features of such models by training GLMs and fine-tuning them on tasks such as MT and multi-task training. We explored the basics of more complex models such as T5, GPT3, and T0 and used this kind of model to perform MT. We used different Python libraries to train T5-based multi-task learning prototyping. We trained GPT-2 on our own corpus and generated text using it. We learned how to save it and use it with AutoModel. We also had a deeper look into how BPE can be trained and used, using the tokenizers library.

In the next chapter, we will see how to fine-tune models for text classification.

References

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI blog, 1(8), 9.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O. and Zettlemoyer, L. (2019). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. arXiv preprint arXiv:1910.13461.
- Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A. and Raffel, C. (2020). *mT5: A massively multilingual pre-trained text-to-text transformer*. arXiv preprint arXiv:2010.11934.
- Raffel, C. , Shazeer, N. , Roberts, A. , Lee, K. , Narang, S. , Matena, M. and Liu, P. J. (2019). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. arXiv preprint arXiv:1910.10683.
- Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R. and Le, Q. V. (2019). XLNet: *Generalized Autoregressive Pretraining for Language Understanding*. arXiv preprint arXiv:1906.08237.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V. and Salakhutdinov, R. (2019). *Transformer-xl: Attentive Language Models Beyond a Fixed-Length Context*. arXiv preprint arXiv:1901.02860.
- Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, Barham P, Chung HW, Sutton C, Gehrmann S, Schuh P. *Palm: Scaling language modeling with pathways*. arXiv preprint arXiv:2204.02311. 2022 Apr 5.
- Sanh V, Webson A, Raffel C, Bach SH, Sutawika L, Alyafeai Z, Chaffin A, Stiegler A, Scao TL, Raja A, Dey M. *Multitask prompted training enables zero-shot task generalization*. arXiv preprint arXiv:2110.08207. 2021 Oct 15.

5

Fine-Tuning Language Models for Text Classification

In this chapter, we will learn how to configure a pre-trained model for text classification and how to fine-tune it for any text classification downstream task, such as sentiment analysis, multi-class classification, or multi-label classification. We will also discuss how to handle sentence-pair and regression problems by covering an implementation example. We will work with well-known datasets such as GLUE, as well as our own custom datasets. We will then take advantage of the `Trainer` class, which deals with the complexity of processes for training and fine-tuning.

First, we will learn how to fine-tune single-sentence binary sentiment classification with the `Trainer` class. Then, we will train for sentiment classification with native PyTorch without the `Trainer` class. In multi-class classification and multi-label classification, more than two classes will be taken into consideration. We will have seven class classification fine-tuning tasks to perform. Later, we will train a text regression model to predict numerical values with sentence pairs.

The following topics will be covered in this chapter:

- Introduction to text classification
- Fine-tuning the BERT model for single-sentence binary classification
- Training a classification model with native PyTorch
- Fine-tuning BERT for multi-class classification with custom datasets
- Fine-tuning BERT for sentence-pair regression
- Multi-label text classification
- Utilizing `run_glue.py` to fine-tune the models

Technical requirements

We will be using Jupyter Notebook to run our coding exercises. You will need **Python 3.6+** for this. Ensure that the following packages are installed:

- `sklearn`
- `transformers 4.0+`
- `datasets`

All the notebooks for the coding exercises in this chapter will be available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

Introduction to text classification

Text classification (also known as **text categorization**) is a way of mapping a document (sentence, Twitter/X post, book chapter, email content, and so on) to a category out of a predefined list (classes). In the case of two classes that have positive and negative labels, we use binary classification – more specifically, sentiment analysis. For more than two classes, we call it **multi-class classification**, where the classes are mutually exclusive, or **multi-label classification**, where the classes are not mutually exclusive, which means a document can receive more than one label. For instance, the content of a news article may be related to sports and politics at the same time. Beyond this classification, we may want to score the documents in a range of `[-1,1]` or rank them in a range of `[1-5]`. We can solve this kind of problem with a regression model, where the type of the output is numeric, not categorical.

Luckily, the transformer architecture allows us to solve these problems efficiently. For sentence-pair tasks such as document similarity or textual entailment, the input is not a single sentence but, rather, two sentences, as illustrated in the following diagram. We can score to what degree two sentences are semantically similar or predict whether they are semantically similar. Another sentence-pair task is textual entailment, where the problem is defined as multi-class classification. Here, two sequences are mapped as in the GLUE benchmark to `entail/contradict/neutral`:

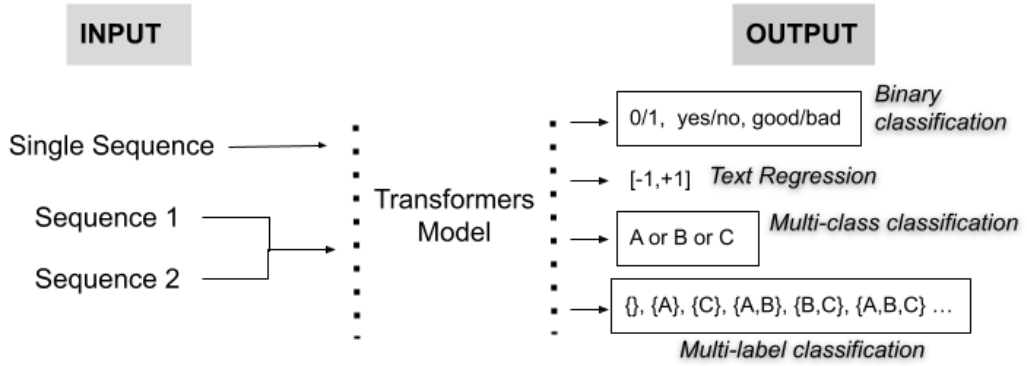


Figure 5.1 – Text classification scheme

Let's start our training process by fine-tuning a pre-trained BERT model. Fine-tuning requires making minor adjustments to the weights of the existing BERT model to adapt it to a different NLP task. We will use a very common task for fine-tuning now: **sentiment analysis**.

Fine-tuning a BERT model for single-sentence binary classification

In this section, we will discuss how to fine-tune a pre-trained BERT model for sentiment analysis by using the popular IMDb sentiment dataset. Working with a GPU will speed up our learning process, but if you do not have such resources, you can work with a CPU as well for fine-tuning. Let's get started.

To learn about and save our current device, we can execute the following lines of code:

```
from torch import cuda
device = 'cuda' if cuda.is_available() else 'cpu'
```

We will use the `DistilBertForSequenceClassification` class here, which is inherited from the `DistilBert` class, with a special sequence classification head at the top. We can utilize this classification head to train the classification model, where the number of classes is `2` by default:

```
from transformers import (
    DistilBertTokenizerFast, DistilBertForSequenceClassification)
model_path= 'distilbert-base-uncased'
tokenizer = DistilBertTokenizerFast.from_pre-trained(model_path)
model = DistilBertForSequenceClassification.from_pre-trained(
    model_path,
    id2label={0:"NEG", 1:"POS"},
    label2id={"NEG":0, "POS":1})
```

Notice that two parameters called `id2label` and `label2id` are passed to the model to use during inference. Alternatively, we can instantiate a particular `config` object and pass it to the model, as follows:

```
config = AutoConfig.from_pre-trained(....)
SequenceClassification.from_pre-trained(.... config=config)
```

Now, let's select a popular sentiment classification dataset called the IMDb dataset. The original dataset consists of two sets of data: 25,000 examples for training and 25 examples for testing. We will split the dataset into test and validation sets. Note that the examples for the first half of the dataset are positive, while the second half's examples are all negative. We can distribute the examples as follows:

```
from datasets import load_dataset
imdb_train= load_dataset('imdb', split="train")
imdb_test= load_dataset('imdb', split="test[:6250]+test[-6250:]")
imdb_val= load_dataset('imdb',
    split="test[6250:12500]+test[-12500:-6250]")
```

Let's check the shape of the dataset:

```
imdb_train.shape, imdb_test.shape, imdb_val.shape
((25000, 2), (12500, 2), (12500, 2))
```

You can take a small portion of the dataset based on your computational resources. For a smaller portion, you should run the following code to select 4,000 examples for training, 1,000 for testing, and 1,000 for validation:

```
imdb_train= load_dataset('imdb',
    split="train[:2000]+train[-2000:]")
imdb_test= load_dataset('imdb',
    split="test[:500]+test[-500:]")
imdb_val= load_dataset('imdb',
    split="test[500:1000]+test[-1000:-500]")
```

Now, we can pass these datasets through the tokenizer model to make them ready for training:

```
enc_train = imdb_train.map(lambda e: tokenizer( e['text'],
    padding=True, truncation=True), batched=True,
    batch_size=1000)
enc_test = imdb_test.map(lambda e: tokenizer( e['text'],
    padding=True, truncation=True), batched=True,
    batch_size=1000)
enc_val = imdb_val.map(lambda e: tokenizer( e['text'],
```

```
    padding=True, truncation=True), batched=True,
    batch_size=1000)
```

Let's see what the training set looks like. The attention mask and input IDs were added to the dataset by the tokenizer so that the BERT model could process them:

```
import pandas as pd
pd.DataFrame(enc_train)
```

The output is as follows:

	attention_mask	input_ids	label	text
0	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 22953, 2213, 4381, 2152, 2003, 1037, 947...	1	Bromwell High is a cartoon comedy. It ran at t...
1	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 11573, 2791, 1006, 2030, 2160, 24913, 20...	1	Homelessness (or Houselessness as George Carli...
2	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 8235, 2058, 1011, 3772, 2011, 23920, 575...	1	Brilliant over-acting by Lesley Ann Warren. Be...
3	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 2023, 2003, 4089, 1996, 2087, 2104, 9250...	1	This is easily the most underrated film inn th...
4	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 2023, 2003, 2025, 1996, 5171, 11463, 837...	1	This is not the typical Mel Brooks film. It wa...
...	...	...	...	...
24995	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 2875, 1996, 2203, 1997, 1996, 3185, 1010...	0	Towards the end of the movie, I felt it was to...
24996	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 2023, 2003, 1996, 2785, 1997, 3185, 2008...	0	This is the kind of movie that my enemies cont...
24997	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 1045, 2387, 1005, 6934, 1005, 2197, 2305...	0	I saw 'Descent' last night at the Stockholm Fi...
24998	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 2070, 3152, 2008, 2017, 4060, 2039, 2005...	0	Some films that you pick up for a pound turn o...
24999	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[101, 2023, 2003, 2028, 1997, 1996, 12873, 435...	0	This is one of the dumbest films, I've ever se...

25000 rows × 4 columns

Figure 5.2 – Encoded training dataset

At this point, the datasets are ready for training and testing. The `Trainer` class (`TFTrainer` for TensorFlow) and the `TrainingArguments` class (`TFTrainingArguments` for TensorFlow) will help us with much of the training complexity. We will define our argument set within the `TrainingArguments` class, which will then be passed to the `Trainer` object.

Let's define what each training argument does:

Argument	Definition
output_dir	Points to the model checkpoints and where the predictions will be saved at the end.
do_train and do_eval	Options for monitoring the model's performance during training.
logging_strategy	The options here are no, epoch, and steps (by default).
logging_steps (default value: 500)	The number of steps between two logs to be saved into the `logging_dir` director.
save_strategy	This is used to save the model checkpoint. The options are `no, epoch, and steps` (default).
save_steps (def. 500)	The number of steps between two checkpoints.
fp16	This is for mixed precision and uses both 16-bit and 32-bit floating-point types to make the model train faster and use less memory.
load_best_model_at_end	As the name suggests, this will load the best model checkpoint in terms of validation loss at the end of training.
logging_dir	TensorBoard log directory.

Table 5.1 – Table of different training argument definitions

For more information, please check the API documentation of `TrainingArguments` or execute the following code in a Python notebook:

```
TrainingArguments?
```

Although deep learning architectures such as LSTM need many epochs (sometimes more than 50), for transformer-based fine-tuning, we will typically be satisfied with an epoch number of `3` due to transfer learning. Most of the time, this number is enough for fine-tuning, as a pre-trained model learns a lot about the language during the pre-training phase, which takes about 50 epochs on average. To determine the correct number of epochs, we need to monitor training and evaluation loss. We will learn how to track training in *Chapter 11*.

This will be enough for many downstream task problems, as we will see here. During the training process, our model checkpoints will be saved under the `./MyIMDBModel` folder every 200 steps:

```
from transformers import TrainingArguments, Trainer
training_args = TrainingArguments(
    output_dir='./MyIMDBModel',
    do_train=True,
    do_eval=True,
```

```
    num_train_epochs=3,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    warmup_steps=100,
    weight_decay=0.01,
    logging_strategy='steps',
    logging_dir='./logs',
    logging_steps=200,
    evaluation_strategy= 'steps',
    fp16= cuda.is_available(),
    load_best_model_at_end=True
)
```

Before instantiating a `Trainer` object, we will define the `compute_metrics()` method, which helps us monitor the progress of the training in terms of particular metrics for whatever we need, such as precision, **RMSE**, Pearson correlation, **BLEU**, and so on. Text classification problems (such as sentiment classification and multi-class classification) are mostly evaluated with micro-averaging or macro-averaging F1 scores. While the macro-averaging method gives equal weight to each class, micro-averaging gives equal weight to each per-text or per-token classification decision. Micro-averaging is equal to the ratio of the number of times the model decides correctly to the total number of decisions that have been made. On the other hand, the macro-averaging method computes the average score of precision, recall, and F1 for each class. For our classification problem, macro-averaging is more convenient for evaluation since we want to give equal weight to each label, as follows:

```
from sklearn.metrics import (
    accuracy_score, Precision_Recall_fscore_support)
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    Precision, Recall, f1, _ = \
        Precision_Recall_fscore_support(labels,
            preds, average='macro')
    acc = accuracy_score(labels, preds)
    return {
        'Accuracy': acc,
        'F1': f1,
        'Precision': Precision,
        'Recall': Recall
    }
```

We are almost ready to start the training process. Now, let's instantiate the `Trainer` object and start it. The `Trainer` class is a very powerful and optimized tool for organizing complex training and evaluation processes for PyTorch and TensorFlow (`TFTrainer` for TensorFlow) thanks to the `transformers` library:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=enc_train,
    eval_dataset=enc_val,
    compute_metrics= compute_metrics
)
```

Finally, we can start the training process:

```
results=trainer.train()
```

The preceding call starts logging metrics, which we will discuss in more detail in *Chapter 11*. The entire IMDb dataset includes 25,000 training examples. With a batch size of `32`, we have *25K/32 ~=782* steps, and 2,346 (782 x 3) steps to go for 3 epochs, as shown in the following progress bar:

Step	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall	Runtime	Samples Per Second
200	0.417800	0.239647	0.900160	0.899943	0.903660	0.900160	58.657100	213.103000
400	0.251100	0.207064	0.918960	0.918960	0.918960	0.918960	58.724400	212.859000
600	0.237300	0.188785	0.926560	0.926554	0.926707	0.926560	58.727300	212.848000
800	0.209200	0.234559	0.923680	0.923621	0.924982	0.923680	58.750400	212.764000
1000	0.128500	0.248400	0.927280	0.927280	0.927286	0.927280	58.717100	212.885000
1200	0.137400	0.251818	0.920000	0.919869	0.922771	0.920000	58.713500	212.898000
1400	0.125900	0.186671	0.930720	0.930707	0.931054	0.930720	58.724900	212.857000
1600	0.111800	0.230385	0.932960	0.932959	0.932980	0.932960	58.695400	212.964000
1800	0.051300	0.255035	0.933440	0.933440	0.933440	0.933440	58.840300	212.440000
2000	0.045200	0.269209	0.934800	0.934795	0.934927	0.934800	58.819400	212.515000
2200	0.053700	0.242861	0.934640	0.934639	0.934661	0.934640	58.836100	212.455000

Figure 5.3 – The output produced by the Trainer object

The `Trainer` object keeps the checkpoint whose validation loss is the smallest at the end. It selects the checkpoint at step 1,400 since the validation loss at this step is the minimum. Let's evaluate the best checkpoint on three (train/test/validation) datasets:

```
q=[trainer.evaluate(eval_dataset=data) for data in\
    [enc_train, enc_val, enc_test]]
pd.DataFrame(q, index=["train","val","test"]).iloc[:,:5]
```

The output is as follows:

	eval_loss	eval_accuracy	eval_f1	eval_precision	eval_recall
train	0.057059	0.98320	0.983199	0.983259	0.98320
val	0.186671	0.93072	0.930707	0.931054	0.93072
test	0.213239	0.92616	0.926128	0.926904	0.92616

Figure 5.4 – Classification model's performance on the train/validation/test datasets

Well done! We have successfully completed the training/testing phase and received 92.6 accuracy and 92.6 F1 for our macro-average. To monitor your training process in more detail, you can call advanced tools such as TensorBoard. These tools parse the logs and enable us to track various metrics for comprehensive analysis. We've already logged the performance and other metrics under `the ./logs` folder. Just running the `tensorboard` function within our Python notebook will be enough, as shown in the following code block (we will discuss TensorBoard and other monitoring tools in detail in *Chapter 11*:

```
%reload_ext tensorboard
%tensorboard --logdir logs
```

Now, we will use the model for inference to check whether it works properly. Let's define a `prediction` function to simplify the prediction steps, as follows:

```
def get_prediction(text):
    inputs = tokenizer(text, padding=True,truncation=True,
        max_length=250, return_tensors="pt").to(device)
    outputs = model(inputs["input_ids"].to(device),
        inputs["attention_mask"].to(device))
    probs = outputs[0].softmax(1)
    return probs, probs.argmax()
```

Now, run the model for inference:

```
text = "I didn't like the movie it bored me "
get_prediction(text)[1].item()
0
```

What we have here is `0`, which is a negative. We have already defined which ID refers to which label. We can use this mapping scheme to get the label. Alternatively, we can simply pass all these boring steps to a dedicated API, namely Pipeline, which we are already familiar with. Before instantiating it, let's save the best model for further inference:

```
model_save_path = "MyBestIMDBModel"
trainer.save_model(model_save_path)
tokenizer.save_pre-trained(model_save_path)
```

The Pipeline API is an easy way to use pre-trained models for inference. We load the model from where we saved it and pass it to the Pipeline API, which does the rest. We can skip this saving step and, instead, directly pass our model and tokenizer objects in memory to the Pipeline API. If you do so, you will get the same result.

As shown in the following code, we need to specify the task name argument of `pipeline` as `sentiment-analysis` when we perform binary classification:

```
from transformers import (
    pipeline, DistilBertForSequenceClassification,
    DistilBertTokenizerFast)
model = DistilBertForSequenceClassification.from_pre-trained("
    MyBestIMDBModel")
tokenizer= DistilBertTokenizerFast.from_pre-trained("MyBestIMDBModel")
nlp= pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)
nlp("the movie was very impressive")
Out: [{'label': 'POS', 'score': 0.9621992707252502}]

>>> nlp("the text of the picture was very poor")
Out: [{'label': 'NEG', 'score': 0.9938313961029053}]
```

The Pipeline API knows how to treat the input and somehow learns which ID refers to which label (`POS` or `NEG`). It also yields the class probabilities.

Well done! We have fine-tuned a sentiment prediction model for the IMDb dataset using the `Trainer` class. In the next section, we will do the same binary classification training but with native PyTorch. We will also use a different dataset.

Training a classification model with native PyTorch

The `Trainer` class is very powerful, and we have the Hugging Face team to thank for providing such a useful tool. However, in this section, we will fine-tune the pre-trained model from scratch to see what happens under the hood. Let's get started.

First, let's load the model for fine-tuning. We will select `DistilBert` here since it is a small, fast, and cheap version of BERT:

```
from transformers import DistilBertForSequenceClassification
model = DistilBertForSequenceClassification\
    .from_pre-trained('distilbert-base-uncased')
```

To fine-tune any model, we need to put it into training mode, as follows:

```
model.train()
```

Now, we must load the tokenizer:

```
from transformers import DistilBertTokenizerFast
tokenizer = DistilBertTokenizerFast.from_pre-trained(
    'bert-base-uncased')
```

Since the `Trainer` class organized the entire process for us, we did not deal with optimization and other training settings in the previous IMDb sentiment classification exercise. Now, we need to instantiate the optimizer ourselves. Here, we must select `AdamW`, which is an implementation of the Adam algorithm but with a weight decay fix. Recently, it has been shown that `AdamW` produces better training loss and validation loss than models trained with Adam. Hence, it is a widely used optimizer within many transformer training processes:

```
from transformers import AdamW
optimizer = AdamW(model.parameters(), lr=1e-3)
```

To design the fine-tuning process from scratch, we must understand how to implement a single step forward and backpropagation. We can pass a single batch through the transformer layer and get the output, which is called forward propagation. Then, we must compute the loss using the output and ground truth label and update the model weight based on the loss. This is called backpropagation.

The following code receives three sentences associated with the labels in a single batch and performs forward propagation. At the end, the model automatically computes the loss:

```
import torch
texts= ["this is a good example",
    "this is a bad example","this is a good one"]
labels= [1,0,1]
labels = torch.tensor(labels).unsqueeze(0)
encoding = tokenizer(texts,
    return_tensors='pt', padding=True,
    truncation=True, max_length=512)
input_ids = encoding['input_ids']
attention_mask = encoding['attention_mask']
outputs = model(input_ids,
```

```
    attention_mask=attention_mask, labels=labels)
loss = outputs.loss
loss.backward()
optimizer.step()
Outputs:
SequenceClassifierOutput(
[('loss', tensor(0.7178, grad_fn=<NllLossBackward>)),
('logits',tensor([[ 0.0664, -0.0161],[ 0.0738, 0.0665], [ 0.0690,
-0.0010]], grad_fn=<AddmmBackward>))])
```

The model takes `input_ids` and `attention_mask`, which were produced by the tokenizer, and computes the loss using ground truth labels. As we can see, the output consists of both loss and logits. Now, `loss.backward()` computes the gradient of the tensor by evaluating the model with the inputs and labels. `optimizer.step()` performs a single optimization step and updates the weight using the gradients that were computed, which is called backpropagation. When we put all these lines into a loop shortly, we will also add `optimizer.zero_grad()`, which clears the gradient of all the parameters. It is important to call this at the beginning of the loop; otherwise, we may accumulate the gradients from multiple steps. The second tensor of the output is `logits`. In the context of deep learning, the term *logits* (short for *logistic units*) is the last layer of the neural architecture and consists of prediction values as real numbers. Logits need to be turned into probabilities by the `softmax` function in the case of classification. Otherwise, they are simply normalized for regression.

If we want to manually calculate the loss, we must not pass the labels to the model. Due to this, the model only yields the logits and does not calculate the loss. In the following example, we are computing the cross-entropy loss manually:

```
from torch.nn import functional
labels = torch.tensor([1,0,1])
outputs = model(input_ids, attention_mask=attention_mask)
loss = functional.cross_entropy(outputs.logits, labels)
loss.backward()
optimizer.step()
loss
Output: tensor(0.6101, grad_fn=<NllLossBackward>)
```

With that, we've learned how batch input is fed in the forward direction through the network in a single step. Now, it is time to design a loop that iterates over the entire dataset in batches to train the model with several epochs. To do so, we will start by designing the `Dataset` class. It is a subclass of `torch.Dataset`, inherits member variables and functions, and implements the `__init__()` and `__getitem()__` abstract functions:

```
from torch.utils.data import Dataset
class MyDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
```

```
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, \
            val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)
```

Let's fine-tune the model for sentiment analysis by taking another sentiment analysis dataset called the **Stanford Sentiment Treebank v2** (`sst2`) dataset. We will also load the corresponding metric for `sst2` for evaluation, as follows:

```
import datasets
from datasets import load_dataset
sst2= load_dataset("glue","sst2")
from datasets import load_metric
metric = load_metric("glue", "sst2")
```

We will extract the sentences and the labels accordingly:

```
texts=sst2['train']['sentence']
labels=sst2['train']['label']
val_texts=sst2['validation']['sentence']
val_labels=sst2['validation']['label']
```

Now, we can pass the datasets through the tokenizer and instantiate the `MyDataset` object to make the BERT models work with them:

```
train_dataset= MyDataset(tokenizer(texts, truncation=True,
    padding=True), labels)
val_dataset=  MyDataset(tokenizer(val_texts,
    truncation=True, padding=True), val_labels)
```

Let's instantiate a `DataLoader` class that provides an interface to iterate through the data samples by loading order. This also helps with batching and memory pinning:

```
from torch.utils.data import DataLoader
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=True)
```

The following lines detect the device and define the `AdamW` optimizer properly:

```
from transformers import AdamW
device = torch.device('cuda') \
    if torch.cuda.is_available() else torch.device('cpu')
model.to(device)
optimizer = AdamW(model.parameters(), lr=1e-3)
```

So far, we know how to implement forward propagation, which is where we process a batch of examples. Here, batch data is fed in the forward direction through the neural network. In a single step, each layer from the first to the final one is processed by the batch data, as per the activation function, and is passed to the successive layer. To go through the entire dataset in several epochs, we designed two nested loops: the outer loop is for the epoch, while the inner loop is for the steps for each batch. The inner part is made up of two blocks; one is for training, while the other one is for evaluating each epoch. As you may have noticed, we called `model.train()` at the first training loop, and when we moved the second evaluation block, we called `model.eval()`. This is important as we put the model into training and inference mode.

We have already discussed the inner block. Note that we track the model's performance by means of the corresponding `metric` object:

```
for epoch in range(3):
    model.train()
    for batch in train_loader:
        optimizer.zero_grad()
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
        outputs = model(input_ids,
            attention_mask=attention_mask, labels=labels)
        loss = outputs[0]
        loss.backward()
        optimizer.step()
    model.eval()
    for batch in val_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
        outputs = model(input_ids,
            attention_mask=attention_mask, labels=labels)
        predictions=outputs.logits.argmax(dim=-1)
        metric.add_batch(
            predictions=predictions,
            references=batch["labels"],
```

```
            )
    eval_metric = metric.compute()
    print(f"epoch {epoch}: {eval_metric}")
OUTPUT:
epoch 0: {'accuracy': 0.9048165137614679}
epoch 1: {'accuracy': 0.8944954128440367}
epoch 2: {'accuracy': 0.9094036697247706}
```

Well done! We've fine-tuned our model and got around 90.94 accuracy. The remaining processes, such as saving, loading, and inference, will be similar to what we did with the `Trainer` class.

With that, we are done with binary classification. In the next section, we will learn how to implement a model for multi-class classification for a language other than English.

Fine-tuning BERT for multi-class classification with custom datasets

In this section, we will fine-tune the Turkish BERT, namely BERTurk, to perform seven-class classification downstream tasks with a custom dataset. This dataset has been compiled from Turkish newspapers and consists of seven categories. We will start by getting the dataset. Alternatively, you can find it in this book's GitHub repository or get it from `https://www.kaggle.com/savasy/ttc4900`.

First, run the following code to get data within a Python notebook:

```
!wget https://raw.githubusercontent.com/savasy/
TurkishTextClassification/master/TTC4900.csv
```

Then, we load the data:

```
import pandas as pd
data= pd.read_csv("TTC4900.csv")
data=data.sample(frac=1.0, random_state=42)
```

Let's organize the IDs and labels with `id2label` and `label2id` to make the model figure out which ID refers to which label. We will also pass the number of labels, `NUM_LABELS`, to the model to specify the size of a thin classification head layer on top of the BERT model:

```
labels=["teknoloji","ekonomi","saglik",
    "siyaset","kultur","spor","dunya"]
NUM_LABELS= len(labels)
id2label={i:l for i,l in enumerate(labels)}
label2id={l:i for i,l in enumerate(labels)}
data["labels"]=data.category.map(lambda x: label2id[x.strip()])
data.head()
```

The output is as follows:

	category	text	labels
4657	teknoloji	acıların kedisi sam çatık kaşlı kedi sam in i...	0
3539	spor	g saray a git santos van_persie den forma ala...	5
907	dunya	endonezya da çatışmalar 14 ölü endonezya da i...	6
4353	teknoloji	emniyetten polis logolu virüs uyarısı telefon...	0
3745	spor	beni türk yapın cristian_baroni yıldırım dan ...	5

Figure 5.5 – Text classification dataset – TTC 4900

Let's count and plot the number of classes using a pandas object:

```
data.category.value_counts().plot(kind='pie')
```

As shown in the following diagram, the dataset classes have been fairly distributed:

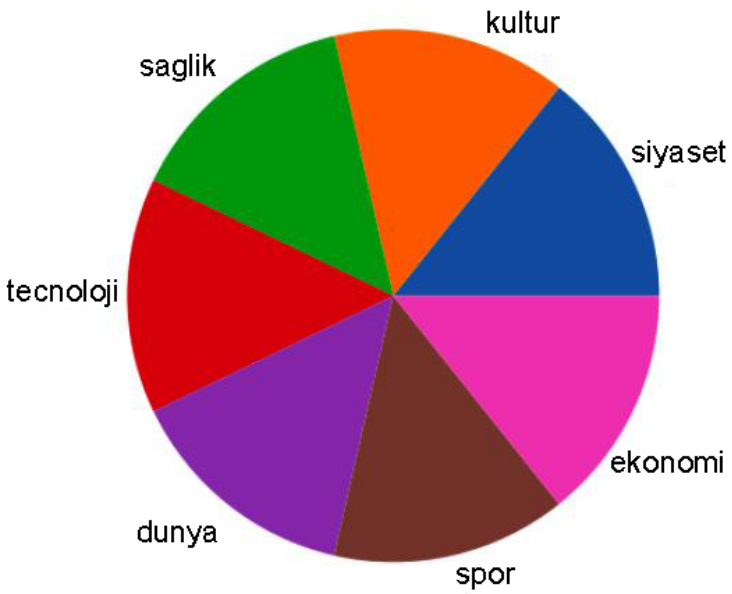


Figure 5.6 – The class distribution

The following execution instantiates a sequence classification model with the number of labels (`7`), label ID mappings, and BERTurk:

```
>>> model
```

The output will be a summary of the model and is too long to show here. Instead, let's turn our attention to the last layer:

```
(classifier): Linear(in_features=768, out_features=7, bias=True)
```

You may have noticed that we did not choose `DistilBert` as there is no pre-trained uncased `DistilBert` for the Turkish language:

```
from transformers import BertTokenizerFast
tokenizer = BertTokenizerFast\
    .from_pre-trained("dbmdz/bert-base-turkish-uncased",
        max_length=512)
from transformers import BertForSequenceClassification
model = BertForSequenceClassification\
    .from_pre-trained("dbmdz/bert-base-turkish-uncased",
        num_labels=NUM_LABELS,
        id2label=id2label, label2id=label2id)
model.to(device)
```

Now, let's prepare the training (50%), validation (25%), and test (25%) datasets, as follows:

```
SIZE= data.shape[0]
## sentences
train_texts= list(data.text[:SIZE//2])
val_texts= list(data.text[SIZE//2:(3*SIZE)//4 ])
test_texts= list(data.text[(3*SIZE)//4:])
## labels
train_labels= list(data.labels[:SIZE//2])
val_labels= list(data.labels[SIZE//2:(3*SIZE)//4])
test_labels= list(data.labels[(3*SIZE)//4:])
## check the size
len(train_texts), len(val_texts), len(test_texts)
(2450, 1225, 1225)
```

The following code tokenizes the sentences of three datasets and their tokens and converts them into integers (`input_ids`), which are then fed into the BERT model:

```
train_encodings = tokenizer(train_texts, truncation=True,
padding=True)
val_encodings = tokenizer(val_texts, truncation=True, padding=True)
test_encodings = tokenizer(test_texts, truncation=True, padding=True)
```

We have already implemented the `MyDataset` class. The class inherits from the abstract `Dataset` class by overwriting the `__getitem__` and `__len__()` methods, which are expected to return the items and the size of the dataset using any data loader, respectively:

```
train_dataset = MyDataset(train_encodings, train_labels)
val_dataset = MyDataset(val_encodings, val_labels)
test_dataset = MyDataset(test_encodings, test_labels)
```

We will keep the batch size at `16` since we have a relatively small dataset. Notice that the other parameters of `TrainingArguments` are almost the same as they were for the previous sentiment analysis experiment:

```
from transformers import TrainingArguments, Trainer
training_args = TrainingArguments(
    output_dir='./TTC4900Model',
    do_train=True,
    do_eval=True,
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    weight_decay=0.01,
    logging_strategy='steps',
    logging_dir='./multi-class-logs',
    logging_steps=50,
    evaluation_strategy="epoch",
    eval_steps=50,
    save_strategy="epoch",
    fp16=True,
    load_best_model_at_end=True)
```

Sentiment analysis and text classification are objects of the same evaluation metrics – that is, macro-averaged F1, precision, and recall. Therefore, we will not define the `compute_metric()` function again. Here is the code for instantiating a `Trainer` object:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics= compute_metrics
)
```

Finally, let's start the training process:

```
trainer.train()
```

The output is as follows:

Step	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall	Runtime	Samples Per Second
50	1.874100	1.706715	0.377143	0.379416	0.553955	0.383715	20.982000	58.383000
100	0.842900	0.327575	0.915102	0.913738	0.914565	0.915279	20.981700	58.384000
150	0.358200	0.281808	0.911020	0.910288	0.912012	0.911213	20.997000	58.342000
200	0.233500	0.366845	0.905306	0.905313	0.916948	0.903440	20.980800	58.387000
250	0.222700	0.292270	0.922449	0.921374	0.921567	0.923131	20.981300	58.385000
300	0.257700	0.280120	0.924898	0.923810	0.924427	0.924510	20.979800	58.390000
350	0.115200	0.292410	0.925714	0.924946	0.924752	0.925454	20.982700	58.381000
400	0.064900	0.322697	0.925714	0.924944	0.925674	0.925265	20.988300	58.366000
450	0.080400	0.297606	0.929796	0.929267	0.929170	0.929497	20.985100	58.375000

Figure 5.7 – The output of the Trainer class for text classification

To check the trained model, we must evaluate the fine-tuned model on three dataset splits, as follows. Our best model is fine-tuned at step 300 with a loss of 0.28012:

```
q=[trainer.evaluate(eval_dataset=data) for data \
    in [train_dataset, val_dataset, test_dataset]]
pd.DataFrame(q, index=["train","val","test"]).iloc[:,:5]
```

The output is as follows:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.091844	0.975510	0.97546	0.975942	0.975535
val	0.280120	0.924898	0.92381	0.924427	0.924510
test	0.280038	0.926531	0.92542	0.927410	0.925425

Figure 5.8 – The text classification model's performance on the train/validation/test dataset

The classification accuracy is around 92.6, while the F1 macro-average is around 92.5. In the literature, many approaches have been tested on this Turkish benchmark dataset. They mostly followed TF-IDF and linear classifier, word2vec embeddings, or an LSTM-based classifier, and got around 90.0 F1 at best. Compared to those approaches, other than the transformer model, the fine-tuned BERT model outperforms them.

As with any other experiment, we can track the experiment via TensorBoard:

```
%load_ext tensorboard
%tensorboard --logdir multi-class-logs/
```

Let's design a function that will run the model for inference. If you want to see a real label instead of an ID, you can use the `config` object of our model, as shown in the following `predict` function:

```
def predict(text):
    inputs = tokenizer(text, padding=True, truncation=True,
        max_length=512, return_tensors="pt").to("cuda")
    outputs = model(**inputs)
    probs = outputs[0].softmax(1)
    return (probs,
        probs.argmax(),
        model.config.id2label[probs.argmax().item()])
```

Now, we are ready to call the `predict` function for text classification inference. The following code classifies a sentence about a football team:

```
text = "Fenerbahçeli futbolcular kısa paslarla hazırlık çalışması
yaptılar"
predict(text)
Output:
(tensor([[5.6183e-04, 4.9046e-04, 5.1385e-04,
    9.94e-04, 3.44e-04, 9.96e-01, 4.061e-04]],
    device='cuda:0',
    grad_fn=<SoftmaxBackward>),
tensor(5, device='cuda:0'),
'spor')
```

As we can see, the model correctly predicted the sentence as sports (spor). Now, it is time to save the model and reload it using the from_`pre-trained()` function. Here is the code:

```
model_path = "turkish-text-classification-model"
trainer.save_model(model_path)
tokenizer.save_pre-trained(model_path)
```

Now, we can reload the saved model and run inference with the help of the `pipeline` class:

```
model_path = "turkish-text-classification-model"
from transformers import (pipeline,
    BertForSequenceClassification,
    BertTokenizerFast)
```

```
model = BertForSequenceClassification.from_pre-trained(model_path)
tokenizer= BertTokenizerFast.from_pre-trained(model_path)
nlp= pipeline("sentiment-analysis",
                model=model, tokenizer=tokenizer)
```

You may have noticed that the task's name is `sentiment-analysis`. This term may be confusing but this argument will actually return `TextClassificationPipeline` at the end. Let's run the pipeline:

```
nlp("Sinemada hangi filmler oynuyor bugün")
[{'label': 'kultur', 'score': 0.9930670261383057}]
nlp("Dolar ve Euro bugün yurtiçi piyasalarda yükseldi")
[{'label': 'ekonomi', 'score': 0.9927696585655212}]
nlp("Bayern Münih ile Barcelona bugün karşı karşıya geliyor. \
    Maçı İngiliz hakem James Watts yönetecek!")
[{'label': 'spor', 'score': 0.9975664019584656}]
```

The model has predicted successfully.

So far, we have implemented two single-sentence tasks – that is, sentiment analysis and multi-class classification. In the next section, we will learn how to handle sentence-pair input and how to design a regression model with BERT.

Fine-tuning the BERT model for sentence-pair regression

The regression model is typically used for classification purposes but, in this case, the last layer consists of only one unit. Instead of being processed by softmax logistic regression, it is normalized. To define the model and incorporate a single-unit head layer at the top, there are two options: directly include the `num_labels=1` parameter in the `BERT.from_pre-trained()` method or pass this information through a `config` object. Initially, this needs to be copied from the `config` object of the pre-trained model, as follows:

```
from transformers import (
    DistilBertConfig, DistilBertTokenizerFast,
    DistilBertForSequenceClassification)
model_path='distilbert-base-uncased'
config = DistilBertConfig.from_pre-trained(model_path,
    num_labels=1)
tokenizer = DistilBertTokenizerFast.from_pre-trained(model_path)
model = DistilBertForSequenceClassification.from_pre-trained(
    model_path, config=config)
```

Well, our pre-trained model has a single-unit head layer thanks to the `num_labels=1` parameter. Now, we are ready to fine-tune the model with our dataset. Here, we will use the **Semantic Textual Similarity Benchmark** (**STS-B**), which is a collection of sentence pairs that have been drawn from a variety of content, such as news headlines. Each pair has been annotated with a similarity score from `1` to `5`. Our task is to fine-tune the BERT model to predict these scores. We will evaluate the model using the Pearson/Spearman correlation coefficients while following the literature. Let's get started.

The following code loads the data. The original data was split into three. However, the test split has no label so we can divide the validation data into two parts, as follows:

```
import datasets
from datasets import load_dataset
stsb_train= load_dataset('glue','stsb', split="train")
stsb_validation = load_dataset('glue','stsb', split="validation")
stsb_validation=stsb_validation.shuffle(seed=42)
stsb_val= datasets.Dataset.from_dict(stsb_validation[:750])
stsb_test=datasets.Dataset.from_dict(stsb_validation[750:])
```

Let's make the `stsb_train` training data neat by wrapping it with pandas:

```
pd.DataFrame(stsb_train)
```

Here is what the training data looks like:

	idx	label	sentence1	sentence2
0	0	5.00	A plane is taking off.	An air plane is taking off.
1	1	3.80	A man is playing a large flute.	A man is playing a flute.
2	2	3.80	A man is spreading shreded cheese on a pizza.	A man is spreading shredded cheese on an uncoo...
3	3	2.60	Three men are playing chess.	Two men are playing chess.
4	4	4.25	A man is playing the cello.	A man seated is playing the cello.
...	...	...	...	...
5744	5744	0.00	Severe Gales As Storm Clodagh Hits Britain	Merkel pledges NATO solidarity with Latvia
5745	5745	0.00	Dozens of Egyptians hostages taken by Libyan t...	Egyptian boat crash death toll rises as more b...
5746	5746	0.00	President heading to Bahrain	President Xi: China to continue help to fight ...
5747	5747	0.00	China, India vow to further bilateral ties	China Scrambles to Reassure Jittery Stock Traders
5748	5748	0.00	Putin spokesman: Doping charges appear unfounded	The Latest on Severe Weather: 1 Dead in Texas ...

5749 rows × 4 columns

Figure 5.9 – STS-B training dataset

Run the following code to check the shape of the three sets:

```
stsb_train.shape, stsb_val.shape, stsb_test.shape
((5749, 4), (750, 4), (750, 4))
```

Run the following code to tokenize the datasets:

```
enc_train = stsb_train.map(lambda e: tokenizer(
        e['sentence1'],e['sentence2'], padding=True,
        truncation=True),
    batched=True, batch_size=1000)
enc_val = stsb_val.map(lambda e: tokenizer( e['sentence1'],
        e['sentence2'], padding=True, truncation=True),
    batched=True, batch_size=1000)
enc_test = stsb_test.map(lambda e: tokenizer(
        e['sentence1'],e['sentence2'], padding=True,
        truncation=True),
    batched=True, batch_size=1000)
```

The tokenizer merges two sentences with the `[SEP]` delimiter and produces single `input_ids` and `attention_mask` for a sentence pair, as shown here:

```
pd.DataFrame(enc_train)
```

The output is as follows:

	idx	label	sentence1	sentence2
0	0	5.00	A plane is taking off.	An air plane is taking off.
1	1	3.80	A man is playing a large flute.	A man is playing a flute.
2	2	3.80	A man is spreading shreded cheese on a pizza.	A man is spreading shredded cheese on an uncoo...
3	3	2.60	Three men are playing chess.	Two men are playing chess.
4	4	4.25	A man is playing the cello.	A man seated is playing the cello.
...	...	...	...	...
5744	5744	0.00	Severe Gales As Storm Clodagh Hits Britain	Merkel pledges NATO solidarity with Latvia
5745	5745	0.00	Dozens of Egyptians hostages taken by Libyan t...	Egyptian boat crash death toll rises as more b...
5746	5746	0.00	President heading to Bahrain	President Xi: China to continue help to fight ...
5747	5747	0.00	China, India vow to further bilateral ties	China Scrambles to Reassure Jittery Stock Traders
5748	5748	0.00	Putin spokesman: Doping charges appear unfounded	The Latest on Severe Weather: 1 Dead in Texas ...

5749 rows × 4 columns

Figure 5.10 – Encoded training dataset

Similar to other experiments, we follow almost the same scheme for the `TrainingArguments` and `Trainer` classes. Here is the code:

```
from transformers import TrainingArguments, Trainer
training_args = TrainingArguments(
    output_dir='./stsb-model',
    do_train=True,
    do_eval=True,
```

```
    num_train_epochs=3,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    warmup_steps=100,
    weight_decay=0.01,
    logging_strategy='steps',
    logging_dir='./logs',
    logging_steps=50,
    evaluation_strategy="steps",
    save_strategy="epoch",
    fp16=True,
    load_best_model_at_end=True
)
```

Another important difference between the current regression task and the previous classification tasks is the design of `compute_metrics`. Here, our evaluation metric will be based on the Pearson correlation coefficient and Spearman's rank correlation following the common practice provided in the literature. We also provide the **Mean Squared Error** (**MSE**), **Root Mean Square Error** (**RMSE**), and **Mean Absolute Error** (**MAE**) metrics, which are commonly used, especially for regression models:

```
import numpy as np
from scipy.stats import pearsonr
from scipy.stats import spearmanr
def compute_metrics(pred):
    preds = np.squeeze(pred.predictions)
    return {"MSE": ((preds - pred.label_ids) ** 2)\
        .mean().item(),
        "RMSE": (np.sqrt (((preds - pred.label_ids)**2)\
        .mean())).item(),
        "MAE": (np.abs(preds-pred.label_ids))\
        .mean().item(),
        "Pearson" : pearsonr(preds,pred.label_ids)[0],
        "Spearman":spearmanr(preds,pred.label_ids)[0]
        }
```

Now, let's instantiate the `Trainer` object:

```
trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=enc_train,
        eval_dataset=enc_val,
        compute_metrics=compute_metrics,
        tokenizer=tokenizer)
```

Run the training, like so:

```
train_result = trainer.train()
```

The output is as follows:

Step	Training Loss	Validation Loss	Mse	Rmse	Mae	Pearson	Spearman's rank	Runtime	Samples Per Second
50	4.973200	2.242550	2.242550	1.497515	1.261815	0.140489	0.138228	0.943900	794.538000
100	1.447300	0.801587	0.801587	0.895314	0.735321	0.808588	0.809430	0.933300	803.602000
150	0.940400	0.693730	0.693730	0.832904	0.675787	0.843234	0.842421	0.930700	805.838000
200	0.736300	0.679696	0.679696	0.824437	0.662136	0.846722	0.843393	0.934700	802.407000
250	0.585400	0.590002	0.590002	0.768116	0.618677	0.859470	0.854824	0.931600	805.067000
300	0.513800	0.584674	0.584674	0.764640	0.610141	0.861033	0.856779	0.942900	795.438000
350	0.488000	0.604512	0.604512	0.777504	0.611338	0.865844	0.861726	0.939600	798.174000
400	0.362900	0.555219	0.555219	0.745130	0.582900	0.868366	0.863372	0.938200	799.379000
450	0.298500	0.544973	0.544973	0.738223	0.576751	0.868145	0.864209	0.938200	799.407000
500	0.270100	0.546966	0.546966	0.739571	0.575326	0.867538	0.864035	0.941900	796.240000

Figure 5.11 – Training result for text regression

The best validation loss that's computed is 0.544973 at step 450. Let's evaluate the best checkpoint model at that step, as follows:

```
q=[trainer.evaluate(eval_dataset=data) for data in \
    [enc_train, enc_val, enc_test]]
pd.DataFrame(q, index=["train","val","test"]).iloc[:,:5]
```

The output is as follows:

[90/90 00:30]

	eval_loss	eval_MSE	eval_RMSE	eval_MAE	eval_Pearson	eval_Spearman's Rank
train	0.232471	0.232471	0.482152	0.372915	0.944844	0.935578
val	0.544973	0.544973	0.738223	0.576751	0.868145	0.864209
test	0.537752	0.537752	0.733316	0.567489	0.875409	0.872858

Figure 5.12 – Regression performance on the training/validation/test dataset

The Pearson and Spearman correlation scores are around 87.54 and 87.28 on the test dataset, respectively. We did not get a state-of-the-art result, but we did get a comparable result for the STS-B task based on the GLUE benchmark leaderboard.

We are now ready to run the model for inference. Let's take the following two sentences, which share the same meaning, and pass them to the model:

```
s1,s2="A plane is taking off.","An air plane is taking off."
encoding = tokenizer(s1,s2, return_tensors='pt',
```

```
    padding=True, truncation=True, max_length=512)
input_ids = encoding['input_ids'].to(device)
attention_mask = encoding['attention_mask'].to(device)
outputs = model(input_ids, attention_mask=attention_mask)
outputs.logits.item()
OUTPUT: 4.033723831176758
```

The following code consumes the negative sentence pair, which means the sentences are semantically different:

```
s1,s2="The men are playing soccer.","A man is riding a motorcycle."
encoding = tokenizer("hey how are you there",
    "hey how are you", return_tensors='pt', padding=True,
    truncation=True, max_length=512)
input_ids = encoding['input_ids'].to(device)
attention_mask = encoding['attention_mask'].to(device)
outputs = model(input_ids, attention_mask=attention_mask)
outputs.logits.item()
OUTPUT: 2.3579328060150146
```

Finally, we will save the model, as follows:

```
model_path = "sentence-pair-regression-model"
trainer.save_model(model_path)
tokenizer.save_pre-trained(model_path)
```

Well done! We can congratulate ourselves since we have successfully completed three tasks: sentiment analysis, multi-class classification, and sentence pair regression. All the tasks are based on a single label. Next, we will learn how to solve a multilabel classification problem.

Multilabel text classification

We had already solved multi-class text classification in this chapter, where a single label is assigned to each text. Now, we will discuss multi-label classification, where a text can have multiple labels. This is common in NLP applications such as news classification. For instance, news can be related to sports and health at the same time. The following figure depicts multi-label classification:

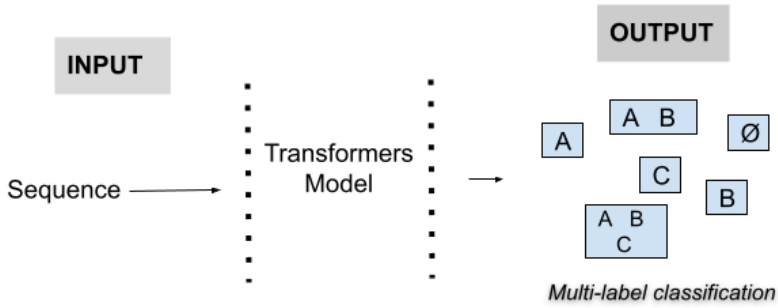


Figure 5.13 – Multi-label classification scheme

Now, we will dive into how to develop a pipeline to apply multi-label classification. To do so, we will take the PubMed dataset, which comprises around 50,000 research articles. The dataset has multiple labels for the articles. Biomedical experts manually annotated these articles with MeSH labels, and each article has been described based on the combination of 14 MeSH labels.

Now, we will start by importing the libraries:

```
import torch, numpy as np, pandas as pd
from datasets import Dataset
from datasets import load_dataset
from transformers import (AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments, Trainer)
```

The dataset is already hosted by the Hugging Face Hub. Let's download it from the Hub as seen in the following link. For quick training, we select only 10 % of the training dataset as follows. If you wish, you can use the entire dataset to get better performance:

```
path="owaiskha9654/PubMed_MultiLabel_Text_Classification_Dataset_MeSH"
dataset = load_dataset(path, split="train[:10%]")
train_dataset=pd.DataFrame(dataset)
text_column='abstractText' # text field
label_names= list(train_dataset.columns[6:])
num_labels=len(label_names)
print('Number of Labels:' , num_labels)
train_dataset[[text_column]+ label_names]
Output: Number of Labels: 14
```

As you can see in the following table, we have an `abstractText` field and 14 possible labels, where `1` and `0` mean the presence and absence of a tag, respectively!

	abstractText	A	B	C	D	E	F	G	H	I	J	L	M	N	Z
0	Fifty-four paraffin embedded tissue sections f...	0	1	1	1	1	0	0	1	0	0	0	0	0	0
1	The present cross-sectional study was conducte...	0	1	1	1	1	1	1	0	1	1	0	1	1	1
2	The occurrence of individual amino acids and d...	1	1	0	1	1	0	1	0	0	0	1	0	0	0
3	In 1980, Lim and Sun introduced a microcapsule...	1	1	1	1	1	0	1	0	0	1	0	0	0	0
4	Substantially improved hydrogel particles base...	1	1	0	1	1	0	1	0	0	1	0	0	0	0

Figure 5.14 – PubMed multi-label dataset

Now, let us compute the label distribution and analyze it with the following code:

```
train_dataset[label_names].apply(lambda x: sum(x),
    axis=0).plot(kind="bar", figsize=(10,6))
```

It plots the following:

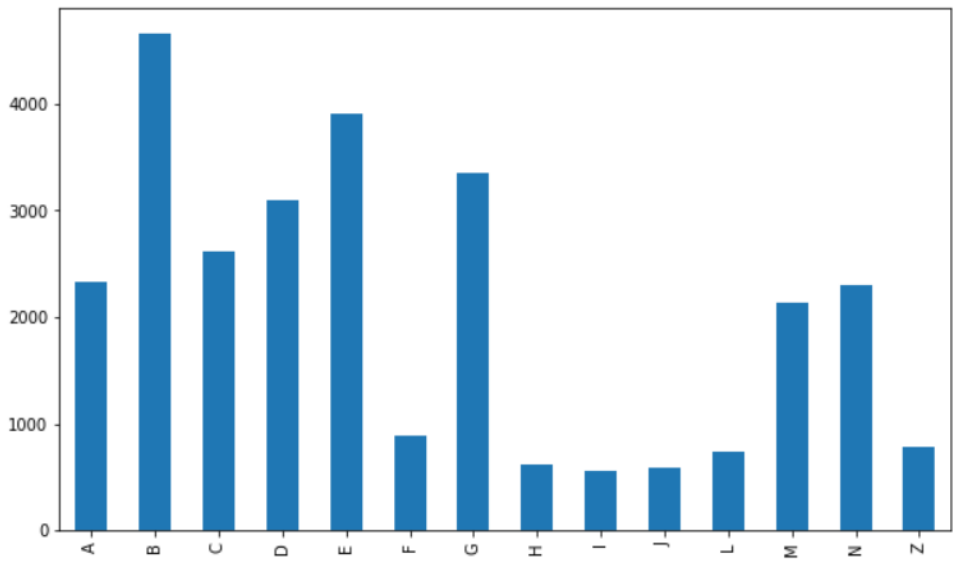


Figure 5.15 – PubMed dataset label distribution

As seen in the imbalanced distribution, we have some minor labels such as `F`, `H`, `I`, `J`, `L`, and `Z`. When the entire dataset is analyzed, a similar distribution comes out!

We need to cast the label columns to a single list as follows:

```
train_dataset["labels"]=train_dataset.apply(
    lambda x: x[label_names].to_numpy(), axis=1)
train_dataset[[text_column, "labels"]]
```

Now, we have the following shape (`abstractText-> labels`), which is needed by the following pipeline:

	abstractText	labels
0	Fifty-four paraffin embedded tissue sections f...	[0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0]
1	The present cross-sectional study was conducte...	[0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1]
2	The occurrence of individual amino acids and d...	[1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0]
3	In 1980, Lim and Sun introduced a microcapsule...	[1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0]
4	Substantially improved hydrogel particles base...	[1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0]

Figure 5.16 – Processed dataset (text->label)

We will fine-tune `distilbert-base-uncased`. As we need to tokenize the text, we first load the `Distilbert` tokenizer as follows:

```
model_path="distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_path)
def tokenize(batch):
    return tokenizer(batch[text_column],
        padding=True,
        truncation=True)
```

We now split the dataset into three subsets (50% training, 25% validation, and 25% test) and tokenize them accordingly:

```
q = train_dataset[[text_column, "labels"]].copy()
CUT= int((q.shape)[0]*0.5)
CUT2= int((q.shape)[0]*0.75)
train_df= q[:CUT] # training set
val_df= q[CUT:CUT2] # validations set
test_df= q[CUT2:]  # test set
train=Dataset.from_pandas(train_df) #Cast to Dataset object
val=Dataset.from_pandas(val_df)
test=Dataset.from_pandas(test_df)
train_encoded = train.map(tokenize, batched=True,
```

```
    batch_size=None)
val_encoded = val.map(tokenize, batched=True,
    batch_size=None)
test_encoded = test.map(tokenize, batched=True,
    batch_size=None)
```

We have three subsets of the dataset that have been encoded by the tokenizer and are ready to be processed by the Transformer model.

Now, we will define a function that deals with the activations from the last layer (`logits`) of the Transformer model to generate a `prediction` vector. If we were doing single-label classification, softmax would be the most appropriate formula since we have a single output. However, since many labels can be correct labels and they are not mutually exclusive, we will pass all labels through the sigmoid function independently, rather than using softmax. By doing this, we will have the chance to select either all labels as predictions at the same time or none of them. We make our decision by passing the output of the sigmoid function through a simple `>0.5` threshold.

Now, we will define the `compute_metric()` function to monitor model performance between the steps during training. The following function gives the precision, recall, and F1 score of the presence of the labels by utilizing the `sklearn` library. Please notice that in the `f1_score()` function, we set `pos_label=1` since we only focus on the presence of labels. If you take both the presence and absence of the labels, you can get artificially high results, and it gets hard to monitor the model performance, especially for label sparsity mode!

Let us define compute metrics function:

```
from sklearn.metrics import (f1_score,precision_score,\
    recall_score)
def compute_metrics(eval_pred):
    y_pred, y_true = eval_pred
    y_pred = torch.from_numpy(y_pred)
    y_true = torch.from_numpy(y_true)
    y_pred = y_pred.sigmoid() >0.5
    y_true=y_true.bool()
    r=recall_score(y_true, y_pred,
    average='micro', pos_label=1)
    p=precision_score(y_true, y_pred,
    average='micro', pos_label=1)
    f1=f1_score(y_true, y_pred,
    average='micro', pos_label=1)
    result={"Recall":r,"Precision":p,"F1":f1}
    return result
```

For single-label multi-class classification, we use softmax activation followed by a cross-entropy loss function. However, for multi-label mode, we need to use different activation and loss functions. During implementation, we keep all other functionalities of the original `Trainer` as is, but we must adjust the loss function. Specifically, we replace the `torch.nn.CrossEntropyLoss()` function in the original loss function of the `Trainer` class with `torch.nn.BCEWithLogitsLoss()`.

As you can see in the following class definition, the `torch.nn.BCEWithLogitsLoss()` function computes the loss between the ground truth labels and the raw logits. The loss function first produces predictions by passing the raw output (`logits`) to the sigmoid function and computes the loss accordingly:

```
class MultilabelTrainer(Trainer):
    def compute_loss(self, model, inputs,
        return_outputs=False):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits
        loss_fct = torch.nn.BCEWithLogitsLoss()
        preds_=logits.view(-1,self.model.config.num_labels)
        labels_=labels.float().view(-1,
            self.model.config.num_labels)
        loss = loss_fct(preds_,labels_)
        return (loss, outputs) if return_outputs else loss
```

At this stage, the `Trainer` instance and the `TrainingArguments` instance are roughly the same as in the previous multi-class mode. Here are the arguments:

```
batch_size=16
num_epoch=3
args = TrainingArguments(
    output_dir="/tmp",
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    num_train_epochs=num_epoch,
    do_train=True,
    do_eval=True,
    load_best_model_at_end=True,
    save_steps=100,
    eval_steps=100,
    save_strategy="steps",
    evaluation_strategy="steps")
```

We will load the `Distilbert` model. Notice that we set the output layer size of the model as 14 (as in `num_labels`) units:

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_path, num_labels=num_labels)
```

Well! We are ready to train. Here we go!

```
multi_trainer = MultilabelTrainer(model, args,
    train_dataset=train_encoded,
    eval_dataset=val_encoded,
    compute_metrics=compute_metrics,
    tokenizer=tokenizer)
multi_trainer.train()
```

If everything goes well, we can test our model on the test dataset as follows:

```
res=multi_trainer.predict(test_encoded)
pd.Series(compute_metrics(res[:2])).to_frame()
```

Here is the output:

	Recall	Precision	F1
0	0.820272	0.846009	0.832942

Figure 5.17 – Performance on the test dataset

Well! We are done. We trained a multi-label classification model with a sampled dataset for a quick run. Please note that training the model with the full dataset gets a better F1 score! Please try it yourself! Next, we will use a script to make the whole process easier.

Utilizing run_glue.py to fine-tune the models

So far, we have designed a fine-tuning architecture from scratch using both native PyTorch and the `Trainer` class. The Hugging Face community also provides another powerful script called `run_glue.py` for the GLUE benchmark and GLUE-like classification downstream tasks. This script can handle and organize the entire training/validation process for us. If you want to do quick prototyping, you should use this script. It can fine-tune any pre-trained models on the Hugging Face Hub. We can also feed it with our own data in any format.

Please go to the following link to access the script and to learn more: `https://github.com/huggingface/transformers/tree/master/examples`.

The script can perform nine different GLUE tasks. With the script, we can do everything that we have done with the `Trainer` class so far. The task name could be one of the following GLUE tasks: `cola`, `sst2`, `mrpc`, `stsb`, `qqp`, `mnli`, `qnli`, `rte`, or `wnli`.

Here is the script scheme for fine-tuning a model:

```
export TASK_NAME= "My-Task-Name"
python run_glue.py \
--model_name_or_path bert-base-cased \
--task_name $TASK_NAME \
--do_train \ --do_eval \
--max_seq_length 128 \
--per_device_train_batch_size 32 \
--learning_rate 2e-5 \
--num_train_epochs 3 \
--output_dir /tmp/$TASK_NAME/
```

The community provides another script called `run_glue_no_trainer.py`. The main difference between the original script and this one is that this `no_trainer` script gives us more chances to change the options for the optimizer or add any customization that we want to do.

Summary

In this chapter, we discussed how to fine-tune a pre-trained model for any text classification downstream task. We fine-tuned the models using sentiment analysis, multi-class classification, and sentence-pair classification – more specifically, sentence-pair regression and multi-label classification. We worked with a well-known IMDb dataset and our own custom dataset to train the models. While we took advantage of the `Trainer` class to cope with much of the complexity of the processes for training and fine-tuning, we learned how to train from scratch with native libraries to understand forward propagation and backpropagation with the `transformers` library. To summarize, we discussed and conducted fine-tuning single-sentence classification with `Trainer`, sentiment classification with native PyTorch without `Trainer`, single-sentence multi-class classification, multi-label classification, and fine-tuning sentence-pair regression.

In the next chapter, we will learn how to fine-tune a pre-trained model for any token classification downstream task, such as parts-of-speech tagging or named entity recognition.

References

- Maas, Andrew, et al. *Learning word vectors for sentiment analysis*. Proceedings of the 49th annual meeting of the Association for Computational Linguistics: Human Language Technologies. 2011.

6

Fine-Tuning Language Models for Token Classification

In this chapter, we will learn about fine-tuning language models for token classification. Tasks such as **Named Entity Recognition** (**NER**), **Part-of-Speech** (**POS**) tagging, and **Question Answering** (**QA**) are explored in this chapter. We will learn how a specific language model can be fine-tuned on such tasks. We will focus on BERT more than other language models. You will learn how to apply POS, NER, and QA using BERT. You will get familiar with the theoretical details of these tasks, such as their respective datasets and how to perform them. After finishing this chapter, you will be able to perform any token classification task using Transformers.

In this chapter, we will fine-tune BERT for the following tasks: fine-tuning BERT for token classification problems such as NER, fine-tuning a language model for an NER problem, and thinking of the QA problem as a start/stop token classification.

The following topics will be covered in this chapter:

- Introduction to token classification
- Fine-tuning language models for NER
- Question answering using token classification
- Question answering for many tasks

Technical requirements

We will be using Jupyter Notebook to run our coding exercises, and Python 3.6+ and the following packages need to be installed:

- `sklearn`
- `transformers 4.0+`

- `Datasets`
- `seqeval`

All notebooks with coding exercises are available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

Introduction to token classification

The task of classifying each token in a token sequence is called **token classification**. This task requires the model to classify each token into a class. POS and NER are two of the most well-known tasks in this criterion. However, QA is also another major **Natural Language Processing** (**NLP**) task that fits in this category. We will discuss the basics of these three tasks in the following sections. It is important to note that tasks such as NER, POS, and QA can be seen from two perspectives: labeling the existing context of the text with respective classes or text generation.

The token classification approach gets the text as input and classifies each respective token into a class. Take the following example:

```
"I went to Berlin."
```

From this sentence, and using token classification-based NER, the output must be in the following form:

```
I = O
Went = O
to = O
Berlin = Location
```

However, generative models solve the problem differently; they accept the input as text and generate the result as text. For example, a T5-based NER model (`https://huggingface.co/dbmdz/t5-base-conll03-english`) uses the following format:

```
I went to [ Berlin | location ]
```

An extra postprocessing step is required to extract the respective entities if required. The same applies to QA as well. In generative QA, the answer is not always completely taken from context; instead, a narrow understanding and rephrasing of the model is also added to the response.

Generative QA can be very useful when not only extraction from text is required but additional knowledge and background wisdom about the problem is also important. However, with generative QA, we cannot always ensure that the generated answer is correct because the words are different from the original answer.

In the following subsection, we will particularly focus on token classification-based approaches that only label the given input text according to the underlying task. For example, QA here only refers to open-book QA using token classification. The same also applies to NER and POS.

Understanding NER

One of the well-known tasks in the category of token classification is NER – the recognition of each token as an entity or not and identifying the type of each detected entity. For example, a text can contain multiple entities at the same time – person names, locations, organizations, and other types of entities. The following text is a clear example of NER:

```
George Washington is one the presidents of the United States of
America.
```

`George Washington` is a person name while `the United States of America` is a location name. A sequence tagging model is expected to tag each word, each containing information about the tag. BIO tags are universally used for standard NER tasks.

The following table is a list of tags and their descriptions:

Tag	Description
O	Out of entity
B-PER	Beginning of Person entity
I-PER	Inside of Person entity
B-LOC	Beginning of Location entity
I-LOC	Inside of Location entity
B-ORG	Beginning of Organization entity
I-ORG	Inside of Organization entity
B-MISC	Beginning of Miscellaneous entity
I-MISC	Inside of Miscellaneous entity

Figure 6.1 – Table of BIOS tags and their descriptions

In this table, *B* indicates the beginning of a tag and *I* denotes the inside of a tag, while *O* is the outside of the entity. This is the reason that this type of annotation is called *BIO*. For example, the sentence shown earlier can be annotated using BIO:

```
[B-PER|George] [I-PER|Washington] [O|is] [O|one] [O|the]
[O|presidents] [O|of] [B-LOC|United] [I-LOC|States] [I-LOC|of]
[I-LOC|America] [O|.]
```

Accordingly, the sequence must be tagged in BIO format. A sample dataset such as CoNLL-2003 (`https://www.clips.uantwerpen.be/conll2003/ner/`) can be in the format shown as follows:

```
SOCCER NN B-NP O
- : O O
JAPAN NNP B-NP B-LOC
GET VB B-VP O
LUCKY NNP B-NP O
WIN NNP I-NP O
, , O O
CHINA NNP B-NP B-PER
IN IN B-PP O
SURPRISE DT B-NP O
DEFEAT NN I-NP O
. . O O

Nadim NNP B-NP B-PER
Ladki NNP I-NP I-PER

AL-AIN NNP B-NP B-LOC
, , O O
United NNP B-NP B-LOC
Arab NNP I-NP I-LOC
Emirates NNPS I-NP I-LOC
1996-12-06 CD I-NP O
```

Figure 6.2 – CoNLL-2003 dataset

In addition to the NER tags we have seen, there are POS tags available for this dataset.

Understanding POS tagging

POS tagging, or grammar tagging, is annotating a word in a given text according to its respective part of speech. As a simple example, in a given text, identification of each word's role in the categories of noun, adjective, adverb, and verb is considered to be POS. However, from a linguistic perspective, there are many roles other than these four.

In the case of POS tags, there are variations, but the **Penn Treebank POS** tagset is one of the most well-known ones. The following screenshot shows a summary and respective descriptions of these roles:

1. CC	Coordinating conjunction	25. TO	*to*
2. CD	Cardinal number	26. UH	Interjection
3. DT	Determiner	27. VB	Verb, base form
4. EX	Existential *there*	28. VBD	Verb, past tense
5. FW	Foreign word	29. VBG	Verb, gerund/present participle
6. IN	Preposition/subordinating conjunction	30. VBN	Verb, past participle
7. JJ	Adjective	31. VBP	Verb, non-3rd ps. sing. present
8. JJR	Adjective, comparative	32. VBZ	Verb, 3rd ps. sing. present
9. JJS	Adjective, superlative	33. WDT	*wh*-determiner
10. LS	List item marker	34. WP	*wh*-pronoun
11. MD	Modal	35. WP$	Possessive *wh*-pronoun
12. NN	Noun, singular or mass	36. WRB	*wh*-adverb
13. NNS	Noun, plural	37. #	Pound sign
14. NNP	Proper noun, singular	38. $	Dollar sign
15. NNPS	Proper noun, plural	39. .	Sentence-final punctuation
16. PDT	Predeterminer	40. ,	Comma
17. POS	Possessive ending	41. :	Colon, semi-colon
18. PRP	Personal pronoun	42. (	Left bracket character
19. PP$	Possessive pronoun	43.)	Right bracket character
20. RB	Adverb	44. "	Straight double quote
21. RBR	Adverb, comparative	45. ‘	Left open single quote
22. RBS	Adverb, superlative	46. “	Left open double quote
23. RP	Particle	47. ’	Right close single quote
24. SYM	Symbol (mathematical or scientific)	48. ”	Right close double quote

Figure 6.3 – Penn Treebank POS tags

Datasets for POS tasks are annotated like the example shown in *Figure 6.2*.

The annotation of these tags is very useful in specific NLP applications and is one of the building blocks of many other methods. Transformers and many advanced models can somehow understand the relationship between words in their complex architecture.

Understanding QA

A QA or reading comprehension task comprises a set of reading comprehension texts with respective questions on them. An exemplary dataset from this scope is **SQUAD**, or the **Stanford Question Answering Dataset**. This dataset consists of Wikipedia texts and the respective questions asked about them. The answers are in the form of segments of the original Wikipedia text.

The following screenshot shows an example of this dataset:

Figure 6.4 – SQUAD dataset example

The segments highlighted in red are the answers, and important parts of each question are highlighted in blue. It is required for a good NLP model to segment text according to the question, and this segmentation can be done in the form of sequence labeling. The model labels the start and end of the segment as answer start and end segments.

Up to this point, you have learned the basics of modern NLP sequence tagging tasks such as QA, NER, and POS. In the next section, you will learn how it is possible to fine-tune BERT for these specific tasks and use the related datasets from the `datasets` library.

Fine-tuning language models for NER

In this section, we will learn how to fine-tune BERT for an NER task. We will first start with the `datasets` library and by loading the `CoNLL-2003` dataset.

The dataset card is accessible at `https://huggingface.co/datasets/conll2003`. The following screenshot shows this model card from the Hugging Face website:

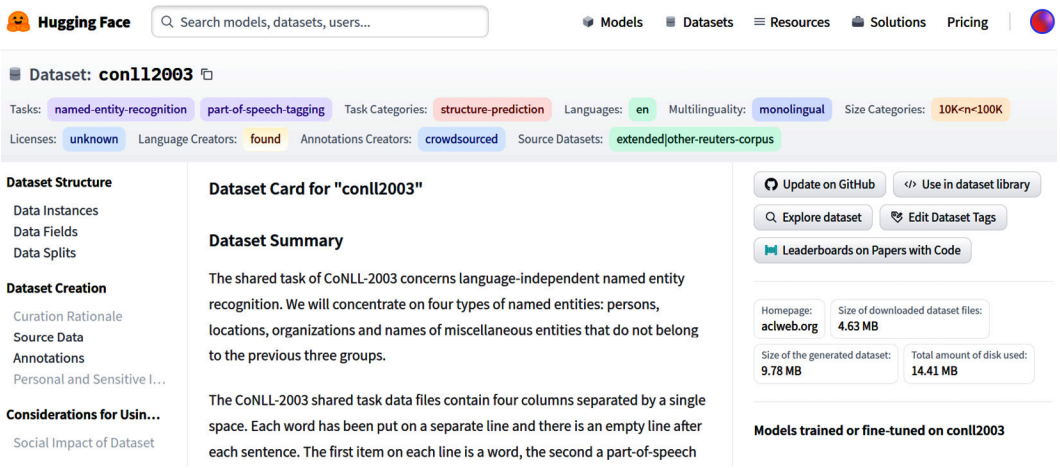


Figure 6.5 – CoNLL-2003 dataset card from Hugging Face

From this screenshot, it can be seen that the model is trained on this dataset and is currently available and listed in the right panel. However, there are also descriptions of the dataset, such as its size and its characteristics, let's dive into those now:

1. To load the dataset, the following commands are used:

```
import datasets
conll2003 = datasets.load_dataset("conll2003")
```

A download progress bar will appear, and after finishing downloading and caching, the dataset will be ready to use. The following screenshot shows the progress bars:

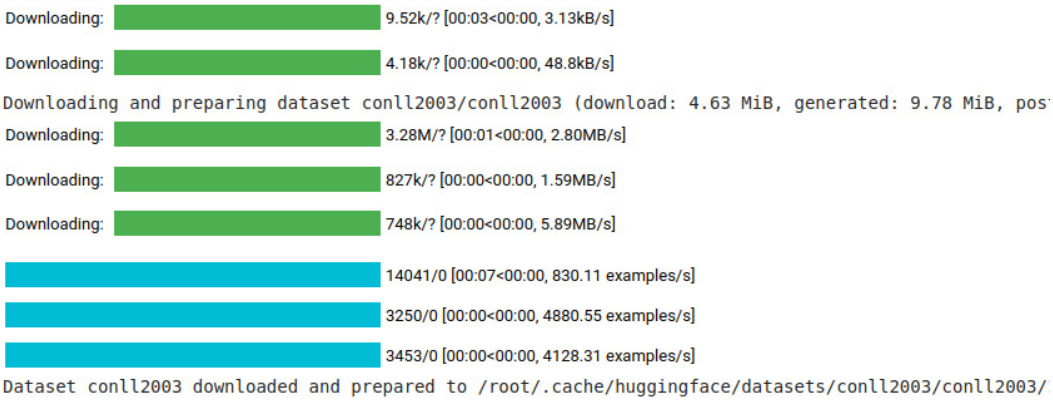


Figure 6.6 – Downloading and preparing the dataset

2. You can easily double-check the dataset by accessing the training samples using the following command:

```
>>> conll2003["train"][0]
```

The following screenshot shows the result:

```
{'chunk_tags': [11, 21, 11, 12, 21, 22, 11, 12, 0],
 'id': '0',
 'ner_tags': [3, 0, 7, 0, 0, 0, 7, 0, 0],
 'pos_tags': [22, 42, 16, 21, 35, 37, 16, 21, 7],
 'tokens': ['EU',
  'rejects',
  'German',
  'call',
  'to',
  'boycott',
  'British',
  'lamb',
  '.']}
```

Figure 6.7 – CoNLL-2003 train samples from the datasets library

3. The respective tags for POS and NER are shown in the preceding screenshot. We will use only NER tags for this example. You can use the following command to get the NER tags available in this dataset:

```
>>> conll2003["train"].features["ner_tags"]
```

4. The result is also shown in *Figure 6.7*. All the BIO tags are shown and there are nine tags in total:

```
>>> Sequence(feature=ClassLabel(num_classes=9, names=['O',
'B-PER', 'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC', 'B-MISC',
'I-MISC'], names_file=None, id=None), length=-1, id=None)
```

5. The next step is to load the BERT tokenizer:

```
from transformers import BertTokenizerFast
tokenizer = BertTokenizerFast.from_pretrained("bert-base-
uncased")
```

6. The `tokenizer` class can also work with whitespace-tokenized sentences. We need to enable our tokenizer to work with whitespace-tokenized sentences, because the NER task has a token-based label for each token. Tokens in this task are usually whitespace-tokenized words rather than **Byte Pair Encoding** (**BPE**) or any other tokenizer tokens. Let's see how `tokenizer` can be used with a whitespace-tokenized sentence:

```
tokenizer(["Oh","this","sentence","is","tokenized", "and",
    "splitted","by","spaces"], is_split_into_words=True)
```

As you can see, by just setting `is_split_into_words` to `True`, the problem is solved.

7. Preprocessing the data before using it for training is required. To do so, we must use the following function and map into the entire dataset:

```
def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(examples["tokens"],
        truncation=True, is_split_into_words=True)
    labels = []
    for i, label in enumerate(examples["ner_tags"]):
        word_ids = \
            tokenized_inputs.word_ids(batch_index=i)
        previous_word_idx = None
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
                    label_ids.append(label[word_idx])
            else:
                    label_ids.append(label[word_idx] if \
                        label_all_tokens else -100)
            previous_word_idx = word_idx
        labels.append(label_ids)
    tokenized_inputs["labels"] = labels
    return tokenized_inputs
```

8. This function will make sure that our tokens and labels are aligned properly. This alignment is required because the tokens are tokenized in pieces, but the words must be one piece. To test and see how this function works, you can run it by giving a single sample to it:

```
q = tokenize_and_align_labels(conll2003['train'][4:5])
print(q)
```

The result is shown as follows:

```
>>> {'input_ids': [[101, 2762, 1005, 1055, 4387, 2000, 1996,
2647, 2586, 1005, 1055, 15651, 2837, 14121, 1062, 9328, 5804,
2056, 2006, 9317, 10390, 2323, 4965, 8351, 4168, 4017, 2013,
3032, 2060, 2084, 3725, 2127, 1996, 4045, 6040, 2001, 24509,
1012, 102]], 'token_type_ids': [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0]], 'attention_mask': [[1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]], 'labels': [[-100, 5,
0, -100, 0, 0, 0, 3, 4, 0, -100, 0, 0, 1, 2, -100, -100, 0, 0,
0, 0, 0, 0, 0, -100, -100, 0, 0, 0, 0, 5, 0, 0, 0, 0, 0, 0, 0,
-100]]}
```

9. But this result is not readable, so you can run the following code to have a readable version:

```
for token, label in zip( tokenizer.convert_ids_to_tokens(
    q["input_ids"][0]),q["labels"][0]):
    print(f"{token:_<40} {label}")
```

The result is shown as follows:

```
[CLS]_____________________________________ -100
germany___________________________________ 5
'_________________________________________ 0
s_________________________________________ -100
representative____________________________ 0
to________________________________________ 0
the_______________________________________ 0
european__________________________________ 3
union_____________________________________ 4
'_________________________________________ 0
s_________________________________________ -100
veterinary________________________________ 0
committee_________________________________ 0
werner____________________________________ 1
z_________________________________________ 2
##wing____________________________________ -100
##mann____________________________________ -100
said______________________________________ 0
on________________________________________ 0
wednesday_________________________________ 0
consumers_________________________________ 0
should____________________________________ 0
buy_______________________________________ 0
sheep_____________________________________ 0
##me_____________________________________ -100
##at_____________________________________ -100
from______________________________________ 0
countries_________________________________ 0
other_____________________________________ 0
than______________________________________ 0
britain___________________________________ 5
until_____________________________________ 0
the_______________________________________ 0
scientific________________________________ 0
advice____________________________________ 0
was_______________________________________ 0
clearer___________________________________ 0
._________________________________________ 0
[SEP]_____________________________________ -100
```

Figure 6.8 – Result of the tokenize and align functions

10. The mapping of this function to the dataset can be done by using the `map` function of the `datasets` library:

```
tokenized_datasets = \
    conll2003.map(tokenize_and_align_labels, batched=True)
```

11. In the next step, we need to load the BERT model with the respective number of labels:

```
from transformers import AutoModelForTokenClassification
model = AutoModelForTokenClassification.from_pretrained(
    "bert-base-uncased", num_labels=9)
```

12. The model will be loaded and ready to be trained. In the next step, we must prepare the trainer and training parameters:

```
from transformers import TrainingArguments, Trainer
args = TrainingArguments(
    "test-ner",
    evaluation_strategy = "epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
)
```

13. Preparing the data collator is required. It will apply batch operations on the training dataset to use less memory and perform faster. You can do so as follows:

```
from transformers import DataCollatorForTokenClassification
data_collator = \
    DataCollatorForTokenClassification(tokenizer)
```

14. To be able to evaluate model performance, there are many metrics available for many tasks in Hugging Face's `datasets` library. We will be using the sequence evaluation metric for NER. `seqeval` is a good Python framework for evaluating sequence tagging algorithms and models. It is necessary to install the `seqeval` library:

```
pip install seqeval
```

15. Afterward, you can load the metric:

```
metric = datasets.load_metric("seqeval")
```

16. It is easily possible to see how the metric works by using the following code:

```
example = conll2003['train'][0]
label_list = \
    conll2003["train"].features["ner_tags"].feature.names
labels = [label_list[i] for i in example["ner_tags"]]
metric.compute(predictions=[labels], references=[labels])
```

The result is as follows:

```
{'MISC': {'f1': 1.0, 'number': 2, 'precision': 1.0, 'recall': 1.0},
 'ORG': {'f1': 1.0, 'number': 1, 'precision': 1.0, 'recall': 1.0},
 'overall_accuracy': 1.0,
 'overall_f1': 1.0,
 'overall_precision': 1.0,
 'overall_recall': 1.0}
```

Figure 6.9 – Output of the seqeval metric

Various metrics, such as accuracy, F1-score, precision, and recall, are computed for the sample input.

17. The following function is used to compute the metrics:

```
import numpy as np def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)
    true_predictions = [
        [label_list[p] for (p, l) in zip(prediction, label) \
            if l != -100]
        for prediction, label in zip(predictions, labels)]
    true_labels = [
        [label_list[l] for (p, l) in zip(prediction,
            label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    results = \
        metric.compute(predictions=true_predictions,
        references=true_labels)
    return {
    "precision": results["overall_precision"],
    "recall": results["overall_recall"],
    "f1": results["overall_f1"],
    "accuracy": results["overall_accuracy"],
    }
```

18. The last step is to make a trainer and train it accordingly:

```
trainer = Trainer(
    model,
    args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)
trainer.train()
```

19. After running the `train` function of `trainer`, the result will be as follows:

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy	Runtime	Samples Per Second
1	0.035800	0.043440	0.937072	0.944800	0.940920	0.988454	17.061700	190.486000
2	0.019100	0.043591	0.939359	0.951531	0.945406	0.989311	16.797100	193.486000
3	0.014500	0.043591	0.939359	0.951531	0.945406	0.989311	16.790100	193.567000

Figure 6.10 – `trainer` results after running train

20. It is necessary to save the model and tokenizer after training:

```
model.save_pretrained("ner_model")
tokenizer.save_pretrained("tokenizer")
```

21. If you wish to use the model with the pipeline, you must read the config file and assign `label2id` and `id2label` correctly according to the labels you have used in the `label_list` object:

```
id2label = {
    str(i): label for i,label in enumerate(label_list)
}
label2id = {
    label: str(i) for i,label in enumerate(label_list)
}
import json
config = json.load(open("ner_model/config.json"))
config["id2label"] = id2label
config["label2id"] = label2id
json.dump(config, open("ner_model/config.json","w"))
```

22. Afterward, it is easy to use the model as in the following example:

```
from transformers import pipeline
model = \
    AutoModelForTokenClassification.from_pretrained("ner_model")
nlp = \
    pipeline("ner", model=mmodel, tokenizer=tokenizer)
example = "I live in Istanbul"
ner_results = nlp(example)
print(ner_results)
```

23. The result will appear as seen here:

```
[{'entity': 'B-LOC', 'score': 0.9983942, 'index': 4, 'word':
'istanbul', 'start': 10, 'end': 18}]
```

Fine-tuning on a specific dataset, domain, or even language makes the model perform much better. However, there are certain pros and cons to this kind of fine-tuning; the dataset should be broad in a sense to cover the actual domain that we are going to use. As an example, the model fine-tuned with the given data (as shown in the previous example) will not perform very well in the medical domain.

Up to this point, you have learned how to apply POS using BERT. You have learned how to train your own POS tagging model using Transformers and you also tested the model. In the next section, we will focus on QA.

Question answering using token classification

A QA problem is generally defined as an NLP problem with a given text and a question for AI, and getting an answer back. Usually, this answer can be found in the original text but there are different approaches to this problem. In the case of **Visual Question Answering** (**VQA**), the question is about a visual entity or visual concept rather than text but the question itself is in the form of text.

Some examples of VQA are as follows:

Figure 6.11 – VQA examples

Most of the models that are intended to be used in VQA are multimodal models that can understand the visual context along with the question and generate the answer properly. However, unimodal fully textual QA or just QA is based on textual context and textual questions with respective textual answers:

SQUAD is one of the most well-known datasets in the field of QA. To see examples of SQUAD and examine them, you can use the following code:

```
from pprint import pprint
from datasets import load_dataset
squad = load_dataset("squad")
for item in squad["train"][1].items():
    print(item[0])
    pprint(item[1])
    print("="*20)
```

The following is the result:

```
answers
{'answer_start': [188], 'text': ['a copper statue of Christ']}
====================
Context
('Architecturally, the school has a Catholic character. Atop the
Main ' "Building's gold dome is a golden statue of the Virgin Mary.
Immediately in " 'front of the Main Building and facing it, is a
copper statue of Christ with ' 'arms upraised with the legend "Venite
Ad Me Omnes". Next to the Main ' 'Building is the Basilica of the
Sacred Heart. Immediately behind the ' 'basilica is the Grotto, a
Marian place of prayer and reflection. It is a ' 'replica of the
grotto at Lourdes, France where the Virgin Mary reputedly ' 'appeared
to Saint Bernadette Soubirous in 1858. At the end of the main drive '
'(and in a direct line that connects through 3 statues and the Gold
Dome), is ' 'a simple, modern stone statue of Mary.')
====================
Id
'5733be284776f4190066117f'
====================
Question
'What is in front of the Notre Dame Main Building?'
====================
Title
'University_of_Notre_Dame'
====================
```

However, there is a version 2 of the SQUAD dataset, which has more training samples, and it is highly recommended to use it. To have an overall understanding of how it is possible to train a model for a QA problem, we will focus on the current part of this problem., training on textual question answering using SQUAD dataset:

1. To start, load SQUAD version 2 using the following code:

```
from datasets import load_dataset
squad = load_dataset("squad_v2")
```

2. After loading the SQUAD dataset, you can see the details of this dataset by using the following code:

```
squad
```

The result is as follows:

```
DatasetDict({
    train: Dataset({
        features: ['id', 'title', 'context', 'question', 'answers'],
        num_rows: 130319
    })
    validation: Dataset({
        features: ['id', 'title', 'context', 'question', 'answers'],
        num_rows: 11873
    })
})
```

Figure 6.12 – SQUAD dataset (version 2) details

The details of the SQUAD dataset will be shown as seen in *Figure 6.12*. As you can see, there are more than 130,000 training samples with more than 11,000 validation samples.

3. As we did for NER, we must preprocess the data to have the right form to be used by the model. To do so, you must first load your tokenizer, which is a pretrained tokenizer as long as you are using a pretrained model and want to fine-tune it for a QA problem:

```
from transformers import AutoTokenizer
model = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model)
```

As you have seen, we are going to use the `distilBERT` model.

For our SQUAD example, we need to input more than one text to the model, one for the question and one for the context. Accordingly, we need our tokenizer to put them side by side and separate them with the special `[SEP]` token because `distilBERT` is a BERT-based model.

There is another problem in the scope of QA, and it is the size of the context. The context size can be longer than the model input size, but we cannot reduce it to the size the model accepts. With some specific NLP tasks we can truncate the input, but in QA, it is possible that the answer is in the truncated part. We will show you an example where we tackle this problem using document stride.

4. The following is an example to show how it works using `tokenizer`:

```
max_length = 384
doc_stride = 128
example = squad["train"][173]
```

```
tokenized_example = tokenizer(
    example["question"],
    example["context"],
    max_length=max_length,
    truncation="only_second",
    return_overflowing_tokens=True,
    stride=doc_stride
)
```

5. The stride is same as the document stride that is used to return the stride for the second part, like a window, while the `return_overflowing_tokens` flag gives the model information on whether it should return the extra tokens. The result of `tokenized_example` is more than a single tokenized output, instead having two input IDs. In the following code snippet, you can see the result:

```
len(tokenized_example['input_ids'])
2
```

6. Accordingly, you can see the full result by running the following `for` loop:

```
for input_ids in tokenized_example["input_ids"][:2]:
    print(tokenizer.decode(input_ids))
    print("-"*50)
```

The result is as follows:

```
[CLS] beyonce got married in 2008 to whom? [SEP] on april
4, 2008, beyonce married jay z. she publicly revealed their
marriage in a video montage at the listening party for her third
studio album, i am... sasha fierce, in manhattan's sony club on
october 22, 2008. i am... sasha fierce was released on november
18, 2008 in the united states. the album formally introduces
beyonce's alter ego sasha fierce, conceived during the making
of her 2003 single " crazy in love ", selling 482, 000 copies
in its first week, debuting atop the billboard 200, and giving
beyonce her third consecutive number - one album in the us. the
album featured the number - one song " single ladies ( put a
ring on it ) " and the top - five songs " if i were a boy " and
" halo ". achieving the accomplishment of becoming her longest
- running hot 100 single in her career, " halo "'s success in
the us helped beyonce attain more top - ten singles on the list
than any other woman during the 2000s. it also included the
successful " sweet dreams ", and singles " diva ", " ego ", "
broken - hearted girl " and " video phone ". the music video
for " single ladies " has been parodied and imitated around the
world, spawning the " first major dance craze " of the internet
age according to the toronto star. the video has won several
awards, including best video at the 2009 mtv europe music
awards, the 2009 scottish mobo awards, and the 2009 bet awards.
at the 2009 mtv video music awards, the video was nominated for
nine awards, ultimately winning three including video of the
```

```
year. its failure to win the best female video category, which
went to american country pop singer taylor swift's " you belong
with me ", led to kanye west interrupting the ceremony and
beyonce [SEP]
--------------------------------------------------
[CLS] beyonce got married in 2008 to whom? [SEP] single ladies
" has been parodied and imitated around the world, spawning the
" first major dance craze " of the internet age according to
the toronto star. the video has won several awards, including
best video at the 2009 mtv europe music awards, the 2009
scottish mobo awards, and the 2009 bet awards. at the 2009 mtv
video music awards, the video was nominated for nine awards,
ultimately winning three including video of the year. its
failure to win the best female video category, which went to
american country pop singer taylor swift's " you belong with
me ", led to kanye west interrupting the ceremony and beyonce
improvising a re - presentation of swift's award during her
own acceptance speech. in march 2009, beyonce embarked on the i
am... world tour, her second headlining worldwide concert tour,
consisting of 108 shows, grossing $ 119. 5 million. [SEP]
--------------------------------------------------
```

As you can see from the preceding output, with a window of 128 tokens, the rest of the context is replicated again in the second output of input IDs.

Another problem is the end span, which is not available in the dataset, but instead, the start span or the start character for the answer is given. It is easy to find the length of the answer and add it to the start span, which would automatically yield the end span.

7. Now that we know all the details of this dataset and how to deal with them, we can easily put them together to make a preprocessing function (`https://github.com/huggingface/transformers/blob/master/examples/pytorch/question-answering/run_qa.py`).
8. This function gets examples as input:

```
def prepare_train_features(examples):
```

9. The next step is to tokenize the examples:

```
    # tokenize examples
    tokenized_examples = tokenizer(
        examples["question" if pad_on_right else "context"],
        examples["context" if pad_on_right else "question"],
        truncation="only_second" if pad_on_right else
            "only_first",
        max_length=max_length,
        stride=doc_stride,
        return_overflowing_tokens=True,
```

```
        return_offsets_mapping=True,
        padding="max_length",
    )
```

10. Now we need to map from a feature to its example:

```
    sample_mapping = \
        tokenized_examples.pop("overflow_to_sample_mapping")
    offset_mapping = tokenized_examples.pop("offset_mapping")
    tokenized_examples["start_positions"] = []
    tokenized_examples["end_positions"] = []
```

Impossible to answer examples should be CLS and also start and end tokens for each one should be added as well:

```
    for i, offsets in enumerate(offset_mapping):
        input_ids = tokenized_examples["input_ids"][i]
        cls_index = input_ids.index(tokenizer.cls_token_id)
        sequence_ids = tokenized_examples.sequence_ids(i)
        sample_index = sample_mapping[i]
        answers = examples["answers"][sample_index]
        if len(answers["answer_start"]) == 0:
            tokenized_examples["start_positions"].\
                append(cls_index)
            tokenized_examples["end_positions"].\
                append(cls_index)
        else:
            start_char = answers["answer_start"][0]
            end_char = \
                start_char + len(answers["text"][0])
            token_start_index = 0
            while sequence_ids[token_start_index] != \
                (1 if pad_on_right else 0):
                token_start_index += 1
            token_end_index = len(input_ids) - 1
            while sequence_ids[token_end_index] != \
                (1 if pad_on_right else 0):
                token_end_index -= 1
            if not (offsets[token_start_index][0] <= \
                start_char and offsets[token_end_index][1] >= \
                end_char):
                tokenized_examples["start_positions"].append(
                cls_index)
```

```
                tokenized_examples["end_positions"].append(
                cls_index)
            else:
                while token_start_index < len(offsets) and \
                offsets[token_start_index][0] <= start_char:
                    token_start_index += 1
                tokenized_examples["start_positions"].
                append(token_start_index - 1)
                while offsets[token_end_index][1] >= end_char:
                    token_end_index -= 1
                tokenized_examples["end_positions"].
                append(token_end_index + 1)
    return tokenized_examples
```

11. Mapping this function to the dataset would apply all the required changes:

```
tokenized_datasets = squad.map(prepare_train_features,
    batched=True, remove_columns=squad["train"].column_names)
```

12. Just like other examples, you can now load a pretrained model to be fine-tuned:

```
from transformers import (
    AutoModelForQuestionAnswering, TrainingArguments, Trainer)
model = AutoModelForQuestionAnswering.from_pretrained(model)
```

13. The next step is to create training arguments:

```
args = TrainingArguments(
    "test-squad",
    evaluation_strategy = "epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
)
```

14. If we are not going to use a data collator, we will give a default data collator to the model trainer:

```
from transformers import default_data_collator
data_collator = default_data_collator
```

15. Now, everything is ready to make the trainer:

```
trainer = Trainer(
    model,
    args,
```

```
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    tokenizer=tokenizer,
)
```

16. The trainer can be used with the `train` function:

```
trainer.train()
```

The result will be something like the following:

Epoch	Training Loss	Validation Loss	Runtime	Samples Per Second
1	1.220600	1.160322	39.574900	272.496000
2	0.945200	1.121690	39.706000	271.596000
3	0.773000	1.157358	39.734000	271.405000

Figure 6.13 – Training results

As you can see, the model is trained with three epochs and the outputs for loss in validation and training are reported.

17. Like any other model, you can easily save this model by using the following function:

```
trainer.save_model("distilBERT_SQUAD")
```

If you want to use your saved model or any other model that is trained on QA, the `transformers` library provides a pipeline that's easy to use and implement with no extra effort.

18. By using this pipeline functionality, you can use any model. The following is an example of using a model with the QA pipeline:

```
from transformers import pipeline
qa_model = pipeline('question-answering',
    model='distilbert-base-cased-distilled-squad',
    tokenizer='distilbert-base-cased')
```

The pipeline just requires two inputs to make the model ready for usage, the model and the tokenizer. However, you are also required to give it a pipeline type, which is QA in the given example.

19. The next step is to give it the inputs it requires, `context` and `question`:

```
question = squad["validation"][0]["question"]
context = squad["validation"][0]["context"]
```

The question and the context can be seen by using following code:

```
print("Question:")
print(question)
print("Context:")
print(context)
Question:
In what country is Normandy located?
Context:
('The Normans (Norman: Nourmands; French: Normands; Latin:
Normanni) were the ' 'people who in the 10th and 11th centuries
gave their name to Normandy, a ' 'region in France. They were
descended from Norse ("Norman" comes from ' '"Norseman") raiders
and pirates from Denmark, Iceland and Norway who, under ' 'their
leader Rollo, agreed to swear fealty to King Charles III of
West ' 'Francia. Through generations of assimilation and mixing
with the native ' 'Frankish and Roman-Gaulish populations, their
descendants would gradually ' 'merge with the Carolingian-
based cultures of West Francia. The distinct ' 'cultural and
ethnic identity of the Normans emerged initially in the first '
'half of the 10th century, and it continued to evolve over the
succeeding ' 'centuries.')
```

20. The model can be used by the following example:

```
qa_model(question=question, context=context)
```

The result can be seen as follows:

```
{'answer': 'France', 'score': 0.9889379143714905, 'start': 159,
'end': 165,}
```

Up to this point, you have learned how you can train on the dataset you want. You have also learned how you can use the trained model using pipelines. In the next section, you will learn how it is possible to apply QA for many tasks as well.

Question answering for many tasks

The very nature of many languages allows us to bring many different NLP tasks into a simple paradigm, QA. For example, given a sentiment classification task, one can argue that instead of classifying input into classes (positive, negative, and neutral), we can use QA-based approaches to solve it. We can redefine the input in this way:

Context: `"I loved this movie!"`

Question: `"What best describes the sentiment of this text (Positive, Negative, Neutral)?"`

Answer: `"Positive"`

In this way, not only a single NLP task but also many other NLP tasks can be combined with only a single token classifier. For example, different questions can be used to handle different NLP tasks. As described in an earlier section, a set of questions and respective answers are required. But in this very specific demonstration, not all of the answers come from the given context, but the answer can be from the question itself!

Another example of this approach is to use QA for pronoun resolution. For example, with the following format, one can use QA for pronoun resolution:

> Context: `"Meysam admired Savas. He was always fascinated about his work."`
>
> Question: `"What does He refer to in this text?"`
>
> Answer: `"Meysam"`

As you have seen, token classification can be used for many tasks. It is also possible to use token classification for even more tasks than QA.

Summary

In this chapter, we discussed how to fine-tune a pretrained model for any token classification task. Fine-tuning models on NER and QA problems was explored. Using pretrained and fine-tuned models on specific tasks with pipelines was detailed with examples. We also learned about various preprocessing steps for these two tasks. Saving pretrained models that are fine-tuned on specific tasks was another major learning point of this chapter. We also saw how it is possible to train models with a limited input size on tasks such as QA that have longer sequence sizes than the model input. Using tokenizers more efficiently to have document splitting with document stride was another important topic of this chapter too.

In the next chapter, we will discuss text representation methods using Transformers. By the end of the chapter, you will learn how to perform zero- or few-shot learning and semantic text clustering.

7
Text Representation

So far, we have addressed the classification and generation problems with the `transformers` library. **Text representation** is another crucial task in modern **natural language processing** (**NLP**), especially for unsupervised tasks such as clustering, semantic search, and **topic modeling**. Representing sentences by using various models such as **Universal Sentence Encoder** (**USE**) and **Sentence-BERT** (**SBERT**) with additional frameworks such as SentenceTransformers will be explained here. **Zero-shot learning** using **BART** will also be explained, and you will learn how to utilize it. **Few-shot learning** methodologies and unsupervised use cases such as **semantic text clustering** and topic modeling will also be described. Finally, **one-shot learning** use cases such as **semantic search** will be covered. Although embedding and text representation models are mostly based on semantic paradigms or specific topics, we will also discuss how instruction fine-tuned embedding models can help solve diverse tasks.

The following topics will be covered in this chapter:

- Introducing **sentence embeddings**
- Benchmarking sentence similarity models
- Using BART for zero-shot learning
- Semantic similarity experiment with **FLAIR**
- Text clustering with Sentence-BERT
- Semantic search with Sentence-BERT
- Instruction fine-tuned embedding models

Technical requirements

We will be using a Jupyter Notebook to run our coding exercises. For this, you will need Python 3.6+ and the following packages:

- `sklearn`
- `transformers >=4.00`
- `datasets`
- `sentence-transformers`
- `tensorflow-hub`
- `flair`
- `umap-learn`
- `bertopic`

All the notebooks for the coding exercises in this chapter will be available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`

Introduction to sentence embeddings

Pretrained BERT models do not produce efficient and independent sentence embeddings as they always need to be fine-tuned in an end-to-end supervised setting. This is because we can think of a pretrained BERT model as an indivisible whole and semantics is spread across all layers, not just the final layer. Without fine-tuning, it may be ineffective to use its internal representations independently. It is also hard to handle unsupervised tasks such as clustering, topic modeling, information retrieval, or semantic search. Because we have to evaluate many sentence pairs during clustering tasks, for instance, this causes massive computational overhead.

Luckily, many modifications have been made to the original BERT model, such as SBERT, to derive semantically meaningful and independent sentence embeddings. SBERT employs a particular objective function in comparison to BERT. In this approach, sentences can be paired via a twin network and evaluated in terms of similarity. We will talk about these approaches in a moment. In the NLP literature, many neural sentence embedding methods have been proposed for mapping a single sentence to a common feature space (**vector space model**) wherein a cosine function (or dot product) is usually used to measure similarity and the Euclidean distance to measure dissimilarity.

The following are some applications that can be efficiently solved with sentence embeddings:

- Sentence-pair tasks
- Information retrieval
- Question answering
- Duplicate question detection
- Paraphrase detection
- Document clustering
- Topic modeling

The simplest but most efficient kind of neural sentence embedding is the average-pooling operation, which is performed on the embeddings of words in a sentence. To get a better representation of this, some early neural methods learned sentence embeddings in an unsupervised fashion, such as `Doc2Vec`, `Skip-Thought`, `FastSent`, and `Sent2Vec`. `Doc2Vec` utilized a token-level distributional theory and an objective function to predict adjacent words, similar to **word2Vec**. The approach injects an additional memory token (called **Paragraph-ID**) into each sentence, which is reminiscent of CLS or SEP tokens in the `transformers` library. This additional token acts as a piece of memory that represents the context or document embeddings. `SkipThought` and `FastSent` are considered sentence-level approaches, where the objective function is used to predict adjacent sentences. These models extract the sentence's meaning to obtain necessary information from the adjacent sentences and their context.

Some other methods, such as `InferSent`, leveraged supervised learning and multi-task transfer learning to learn generic sentence embeddings. `InferSent` trained various supervised tasks to get more efficient embedding. **Recurrent Neural Network** (**RNN**)-based supervised models, such as **GRU**)or **Long Short-Term Memory** (**LSTM**), the last hidden state (or stacked entire hidden states) to obtain sentence embeddings in a supervised setting. We touched on the RNN approach in *Chapter 1*.

Cross-encoder versus bi-encoder

So far, we have discussed how to train transformer-based language models and fine-tune them in semi-supervised and supervised settings, respectively. As we learned in the previous chapters, we got successful results thanks to the transformer architectures. Once a task-specific thin linear layer has been put on top of a pretrained model, all the weights of the network (not only the last task-specific thin layer) are fine-tuned with task-specific labeled data. We also experienced how the BERT architecture has been fine-tuned for two different groups of tasks (single sentence or sentence pair) without any

architectural modifications being required. The only difference is that for the sentence-pair tasks, the sentences are concatenated and marked with a *SEP* token. Thus, self-attention is applied to all the tokens of the concatenated sentences. This is a big advantage of the BERT model, where both input sentences can get the necessary information from each other at every layer. In the end, they are encoded simultaneously. This is called **cross-encoding**.

However, there are two disadvantages regarding the cross-encoders that were addressed by the SBERT authors and *Humeau et al., 2019*, as follows:

- The cross-encoder setup is not convenient for many sentence-pair tasks due to too many possible combinations needing to be processed. For instance, to get the two closest sentences from a list of 1,000 sentences, the cross-encoder model (BERT) requires around 500,000 (*n * (n-1) /2*) inference computations. Therefore, it would be very slow compared to alternative solutions such as SBERT or **Universal Sentence Encoder** (**USE**) and similarity functions can be performed efficiently on modern architectures. Moreover, with the help of an optimized index structure, we can reduce computational complexity from many hours to a few minutes when comparing or clustering many documents.
- Due to its supervised characteristics, the BERT model can't derive independent meaningful sentence embeddings. It is hard to leverage a pretrained BERT model as is for unsupervised tasks such as clustering, semantic search, or topic modeling. The BERT model produces a fixed-size vector for each token in a document. In an unsupervised setting, the document-level representation may be obtained by averaging or pooling token vectors, plus *SEP* and *CLS* tokens. Later, we will see that such a representation of BERT produces below-average sentence embeddings and that its performance scores are usually worse than word embedding pooling techniques such as `Word2Vec`, `FastText`, or `GloVe`.

Alternatively, bi-encoders (such as SBERT) independently map a sentence pair to a semantic vector space, as shown in the following diagram. Since the representations are separate, bi-encoders can cache the encoded input representation for each input, resulting in fast inference time. One of the successful bi-encoder modifications of BERT is SBERT. Based on the **Siamese** and **Triplet** network structures, SBERT fine-tunes the BERT model to produce semantically meaningful and independent embeddings of the sentences.

The following diagram shows the bi-encoder architecture:

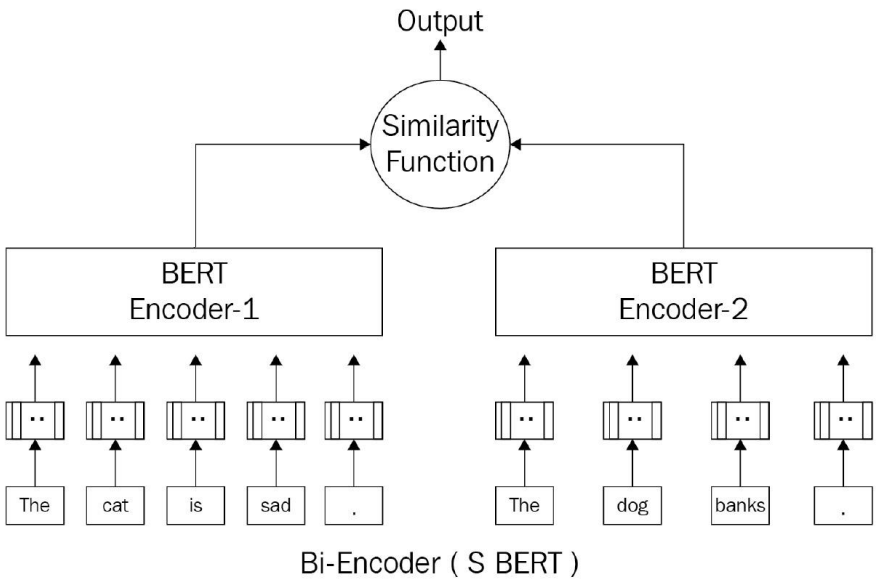


Figure 7.1 – Bi-encoder architecture

You can find hundreds of pretrained SBERT models that have been trained with different objectives at `https://public.ukp.informatik.tu-darmstadt.de/reimers/sentence-transformers/v0.2/`.

We will use some of them in the next section.

Benchmarking sentence similarity models

There are many semantic textual similarity models available, but it is highly recommended that you benchmark and understand their capabilities and differences using metrics. *Papers With Code* provides a list of these datasets at `https://paperswithcode.com/task/semantic-textual-similarity`.

Also, there are many model outputs in each dataset that are ranked by their results. These results have been taken from the aforementioned article.

General Language Understanding Evaluation (**GLUE**) provides most of these datasets and tests, but it is not only for semantic textual similarity. GLUE is a general benchmark for evaluating a model with different NLP characteristics. More details about the GLUE dataset and its usage were provided in *Chapter 2*. Let's take a look at it before we move on:

1. To load the metrics and some datasets, we import datasets functions as follows:

```
from datasets import load_metric, load_dataset
```

2. Let's assume the model produces values and that these values are stored in an array called `predictions`. You can easily use this metric with the predictions to see the F1 and accuracy values:

```
labels = [i['label'] for i in dataset['test']]
metric.compute(predictions=predictions, references=labels)
```

3. Some semantic textual similarity datasets such as **Semantic Textual Similarity Benchmark** (**STSB**) have different metrics. For example, this benchmark uses Spearman and Pearson correlations because the outputs and predictions are between `0` and `5` and are float numbers instead of multiple `0` and `1` values, which is a regression problem. The following code shows an example of this benchmark:

```
metric = load_metric('glue', 'stsb')
metric.compute(predictions=[1,2,3],references=[5,2,2])
```

The predictions are the model outputs, while the references are the dataset labels.

4. To get a comparative result between the two models, we will use a distilled version of **RoBERTa** and test these two on the STSB. To start, you must load both models. The following code shows us how to install the required libraries before loading and using models:

```
pip install tensorflow-hub
pip install sentence-transformers
```

5. As we mentioned previously, the next step is to load the dataset and metric:

```
from datasets import load_metric, load_dataset
stsb_metric = load_metric('glue', 'stsb')
stsb = load_dataset('glue', 'stsb')
```

6. Afterward, we must load both models:

```
import tensorflow_hub as hub
use_model = hub.load(
    "https://tfhub.dev/google/universal-sentence-encoder/4")
from sentence_transformers import SentenceTransformer
distilroberta = SentenceTransformer(
    'stsb-distilroberta-base-v2')
```

7. Both of these models provide embeddings for a given sentence. To compare the similarity between two sentences, we will use cosine similarity. The following function takes sentences as a batch and provides cosine similarity for each pair by utilizing USE:

```
import tensorflow as tf
import math
def use_sts_benchmark(batch):
```

```
    sts_encode1 = tf.nn.l2_normalize(
        use_model(tf.constant(batch['sentence1'])), axis=1)
    sts_encode2 = tf.nn.l2_normalize(
        use_model(tf.constant(batch['sentence2'])),   axis=1)
    cosine_similarities = tf.reduce_sum(
        tf.multiply(sts_encode1,sts_encode2),axis=1)
    clip_cosine_similarities = tf.clip_by_value(
        cosine_similarities,-1.0, 1.0)
    scores = 1.0 - \
        tf.acos(clip_cosine_similarities) / math.pi
return scores
```

8. With small modifications, the same function can be used for RoBERTa, too. These small modifications are only for replacing the embedding function, which is different for TensorFlow Hub models and transformers. The following is the modified function:

```
def roberta_sts_benchmark(batch):
    sts_encode1 = tf.nn.l2_normalize(
        distilroberta.encode(batch['sentence1']), axis=1)
    sts_encode2 = tf.nn.l2_normalize(
        distilroberta.encode(batch['sentence2']), axis=1)
    cosine_similarities = tf.reduce_sum(
        tf.multiply(sts_encode1, sts_encode2),  axis=1)
    clip_cosine_similarities = tf.clip_by_value(
        cosine_similarities, -1.0, 1.0)
    scores = 1.0  - \
        tf.acos(clip_cosine_similarities) / math.pi
return scores
```

9. Applying these functions to the dataset will result in similarity scores for each of the models:

```
use_results = use_sts_benchmark(stsb['validation'])
distilroberta_results = roberta_sts_benchmark(
    stsb['validation'])
```

10. Using metrics on both results produces the Spearman and Pearson correlation values:

```
results = {
    "USE":stsb_metric.compute(
        predictions=use_results,
        references=references),
    "DistilRoberta":stsb_metric.compute(
        predictions=distilroberta_results,
        references=references)
}
```

11. You can simply use `pandas` to see the results as a table in a comparative fashion:

```
import pandas as pd
pd.DataFrame(results)
```

The output is as follows:

	USE	DistillRoberta
pearson	0.810302	0.888461
spearmanr	0.808917	0.889246

Figure 7.2 – STSB validation results on DistilRoBERTa and USE

In this section, you learned about the important benchmarks of semantic textual similarity. Regardless of the model, you learned how to use any of these metrics to quantify model performance. In the next section, you will learn about the few-shot learning models.

Using BART for zero-shot learning

In the field of machine learning, zero-shot learning is referred to as a model that can perform a task without explicitly being trained on it. In the case of NLP, it's assumed that there's a model that can predict the probability of some text being assigned to classes that are given to the model. However, the interesting part about this type of learning is that the model is not trained on these classes.

With the rise of many advanced language models that can perform transfer learning, zero-shot learning came to life. In the case of NLP, this kind of learning is performed by NLP models at test time, where the model sees samples belonging to new classes where no samples of them were seen before.

This kind of learning is usually used for classification tasks, where both the classes and the text are represented and the semantic similarity of both is compared. The represented form of these two is an embedding vector, while the similarity metric (such as cosine similarity or a pretrained classifier such as a dense layer) outputs the probability of the sentence/text being classified as the class.

There are many methods and schemes we can use to train such models, but one of the earliest methods used crawled pages from the internet containing keyword tags in the meta part. For more information, read the following article and blog post at `https://amitness.com/2020/05/zero-shot-text-classification/`.

Instead of using such huge data, there are language models such as BART that use the **Multi-Genre Natural Language Inference** (**MultiNLI**) dataset to fine-tune and detect the relationship between two different sentences. Also, the Hugging Face model repository contains many models that have been implemented for zero-shot learning. They also provide a zero-shot learning pipeline for ease of use.

For example, BART, from **Fundamental AI Research** (**FAIR**) at Meta, is being used in the following code to perform zero-shot text classification:

```
from transformers import pipeline
import pandas as pd
classifier = pipeline("zero-shot-classification",
    model="facebook/bart-large-mnli")
sequence_to_classify = "one day I will see the world"
candidate_labels = ['travel',
    'cooking',
    'dancing',
    'exploration']
result = classifier(sequence_to_classify, candidate_labels)
pd.DataFrame(result)
```

The results are as follows:

	sequence	labels	scores
0	one day I will see the world	travel	0.795756
1	one day I will see the world	exploration	0.199332
2	one day I will see the world	dancing	0.002621
3	one day I will see the world	cooking	0.002291

Figure 7.3 – Results of zero-shot learning using BART

As you can see, the `travel` and `exploration` labels have the highest probability, but the most probable one is `travel`.

However, sometimes, one sample can belong to more than one class (multilabel). Hugging Face provides a parameter called `multi_label` for this. The following example uses this parameter:

```
result = classifier(sequence_to_classify,
    candidate_labels,
    multi_label=True)
Pd.DataFrame(result)
```

Due to this, it is changed to the following:

	sequence	labels	scores
0	one day I will see the world	travel	0.994511
1	one day I will see the world	exploration	0.938389
2	one day I will see the world	dancing	0.005706
3	one day I will see the world	cooking	0.001819

Figure 7.4 – Results of zero-shot learning using BART (multi_label = True)

You can test the results even further and see how the model performs if very similar labels to the `travel` label are used. For example, you can see how it performs if `moving` and `going` are added to the label list.

There are other models that also leverage the semantic similarity between labels and the context to perform zero-shot classification. In the case of few-shot learning, some samples are given to the model, but these samples are not enough to train a model alone. Models can use these samples to perform tasks such as semantic text clustering, which will be explained shortly.

Now that you've learned how to use BART for zero-shot learning, you should learn how it works. BART is fine-tuned on **natural language inference** (**NLI**) datasets such as MultiNLI. These datasets contain sentence pairs and three classes for each pair. The class labels are `Neutral`, `Entailment`, and `Contradiction`. Models that have been trained on these datasets can capture the semantics of two sentences and classify them by assigning a label in the one-hot format. If you take out the `Neutral` labels and only use `Entailment` and Contradiction as your output labels, if two sentences can come after each other, then it means these two are closely related to each other. In other words, you can change the first sentence to the label (`travel`, for example) and the second sentence to the content (`one day I will see the world`, for example). According to this, if these two can come after each other, this means that the label and the content are semantically related. The following code example shows how to directly use the BART model without the zero-shot classification pipeline according to the preceding descriptions:

```
from transformers import (
    AutoModelForSequenceClassification, AutoTokenizer)
nli_model = AutoModelForSequenceClassification\
    .from_pretrained(
    "facebook/bart-large-mnli")
tokenizer = AutoTokenizer\
    .from_pretrained(
    "facebook/bart-large-mnli")
```

```
premise = "one day I will see the world"
label = "travel"
hypothesis = f'This example is {label}.'
x = tokenizer.encode(
    premise,
    hypothesis,
    return_tensors='pt',
    truncation_strategy='only_first')
logits = nli_model(x)[0]
entail_contradiction_logits = logits[:,[0,2]]
probs = entail_contradiction_logits.softmax(dim=1)
prob_label_is_true = probs[:,1]
print(prob_label_is_true)
```

The result is as follows:

```
tensor([0.9945], grad_fn=<SelectBackward>)
```

You can also call the first sentence the hypothesis and the sentence containing the label the premise. According to the result, the premise can entail the hypothesis, which means that the hypothesis is labeled as the premise.

So far, you've learned how to use zero-shot learning by utilizing NLI fine-tuned models. Next, you will learn how to perform few- or one-shot learning using semantic text clustering and semantic search.

Semantic similarity experiment with FLAIR

In this experiment, we will qualitatively evaluate the sentence representation models thanks to the `flair` library, which really simplifies obtaining the document embeddings for us.

We will perform experiments while taking on the following approaches:

- Document average pool embeddings
- RNN-based embeddings
- BERT embeddings
- SBERT embeddings

We need to install these libraries before we can start the experiments:

```
!pip install sentence-transformers
!pip install dataset
!pip install flair
```

For qualitative evaluation, we define a list of similar sentence pairs and a list of dissimilar sentence pairs (five pairs for each). What we expect from the embedding models is that they should measure a high score and a low score, respectively.

The sentence pairs are extracted from the **Semantic Textual Similarity** (**STS**) Benchmark dataset, which we are already familiar with from the sentence-pair regression part of *Chapter 6*. For similar pairs, two sentences are completely equivalent, and they share the same meaning.

The pairs with a similarity score of around `5` in the STS-B dataset are randomly taken, as follows:

```
import pandas as pd
similar=[
      ("A black dog walking beside a pool.",
     "A black dog is walking along the side of a pool."),
      ("A blonde woman looks for medical supplies for work in
a  suitcase. ",
     " The blond woman is searching for medical supplies in
a  suitcase."),
      ("A doubly decker red bus driving down the road.",
     "A red double decker bus driving down a street."),
     ("There is a black dog jumping into a swimming pool.",
     "A black dog is leaping into a swimming pool."),
     ("The man used a sword to slice a plastic bottle.",
     "A man sliced a plastic bottle with a sword.")]
pd.DataFrame(similar, columns=["sen1", "sen2"])
```

The output is as follows:

	sen1	sen2
0	A black dog walking beside a pool.	A black dog is walking along the side of a pool.
1	A blonde woman looks for medical supplies for ...	The blond woman is searching for medical supp...
2	A doubly decker red bus driving down the road.	A red double decker bus driving down a street.
3	There is a black dog jumping into a swimming p...	A black dog is leaping into a swimming pool.
4	The man used a sword to slice a plastic bottle.\t	A man sliced a plastic bottle with a sword.

Figure 7.5 – Similar pair list

Here is the list of dissimilar sentences whose similarity scores are around `0`, taken from the STS benchmark(`https://huggingface.co/datasets/glue/viewer/stsb`):

```
import pandas as pd
dissimilar= [
     ("A little girl and boy are reading books. ",
```

```
        "An older child is playing with a doll while gazing out the
window."),
    ("Two horses standing in a field with trees in the background.",
        "A black and white bird on a body of water with grass in the
background."),
    ("Two people are walking by the ocean.",
        "Two men in fleeces and hats looking at the camera."),
    ("A cat is pouncing on a trampoline.",
        "A man is slicing a tomato."),
    ("A woman is riding on a horse.",
        "A man is turning over tables in anger.")]
pd.DataFrame(dissimilar, columns=["sen1", "sen2"])
```

The output is as follows:

	sen1	sen2
0	A little girl and boy are reading books.	An older child is playing with a doll while ga...
1	Two horses standing in a field with trees in t...	A black and white bird on a body of water with...
2	Two people are walking by the ocean.	Two men in fleeces and hats looking at the cam...
3	A cat is pouncing on a trampoline.	A man is slicing a tomato.
4	A woman is riding on a horse.	A man is turning over tables in anger.

Figure 7.6 – Dissimilar pair list

Now, let's prepare the necessary functions to evaluate the embedding models. The following `sim()` function computes the cosine similarity between two sentences; that is, `s1` and `s2`:

```
import torch, numpy as np
def sim(s1,s2):
    s1=s1.embedding.unsqueeze(0)
    s2=s2.embedding.unsqueeze(0)
    sim=torch.cosine_similarity(s1,s2).item()
    return np.round(sim,2)
```

The document embedding models that were used in this experiment are all pretrained models. We will pass the document `embeddingsmodel` object and sentence pair list (similar or dissimilar) to the following `evaluate()` function, where, once the model encodes the sentence embeddings, it will compute the similarity score for each pair in the list, along with the list average. The definition of the function is as follows:

```
from flair.data import Sentence
def evaluate(embeddings, myPairList):
    scores=[]
```

```
    for s1, s2 in myPairList:
        s1,s2=Sentence(s1), Sentence(s2)
        embeddings.embed(s1)
        embeddings.embed(s2)
        score=sim(s1,s2)
        scores.append(score)
    return scores, np.round(np.mean(scores),2)
```

Now, it is time to evaluate sentence embedding models. We will start with the average pooling method!

Average word embeddings

Average word embeddings (or **document pooling**) apply the mean pooling operation to all the words in a sentence, where the average of all the word embeddings is considered to be sentence embedding. The following execution instantiates a document pool embedding based on **GloVe** vectors. Note that although we will use only GloVe vectors here, the **FLAIRAPI** allows us to use multiple word embeddings. Here is the code definition:

```
from flair.data import Sentence
from flair.embeddings import (
    WordEmbeddings, DocumentPoolEmbeddings)
glove_embedding = WordEmbeddings('glove')
glove_pool_embeddings = DocumentPoolEmbeddings(
    [glove_embedding])
```

Let's evaluate the GloVe pool model on similar pairs, as follows:

```
evaluate(glove_pool_embeddings, similar)
([0.97, 0.99, 0.97, 0.99, 0.98], 0.98)
```

The results seem to be good since those resulting values are very high, which is what we expect. However, the model produces high scores such as `0.94` on average for the dissimilar list as well. Our expectation would be less than `0.4`. We'll talk about why we got this later in this chapter. Here is the execution:

```
evaluate(glove_pool_embeddings, dissimilar)
([0.94, 0.97, 0.94, 0.92, 0.93], 0.94)
```

Next, let's evaluate some RNN embeddings on the same problem.

RNN-based document embeddings

Let's instantiate a GRU model based on GloVe embeddings, where the default model of `DocumentRNNEmbeddings` is a GRU:

```
from flair.embeddings import (
    WordEmbeddings, DocumentRNNEmbeddings)
gru_embeddings = DocumentRNNEmbeddings([glove_embedding])
```

Run the evaluation method:

```
evaluate(gru_embeddings, similar)
([0.99, 1.0, 0.94, 1.0, 0.92], 0.97)
evaluate(gru_embeddings, dissimilar)
([0.86, 1.0, 0.91, 0.85, 0.9], 0.9)
```

Likewise, we get a high score for the dissimilar list. This is not what we want from sentence embeddings.

Transformer-based BERT embeddings

The following execution instantiates a `bert-base-uncased` model that pools the final layer:

```
from flair.embeddings import TransformerDocumentEmbeddings
from flair.data import Sentence
bert_embeddings = TransformerDocumentEmbeddings(
    'bert-base-uncased')
```

Run the evaluation, as follows:

```
evaluate(bert_embeddings, similar)
([0.85, 0.9, 0.96, 0.91, 0.89], 0.9)
evaluate(bert_embeddings, dissimilar)
([0.93, 0.94, 0.86, 0.93, 0.92], 0.92)
```

This is worse! The score of the dissimilar list is higher than that of the similar list.

SBERT embeddings

Now, let's apply SBERT to the problem of distinguishing similar pairs from dissimilar ones, as follows:

1. First of all, a warning: we need to ensure that the `sentence-transformers` package has already been installed:

   ```
   !pip install sentence-transformers
   ```

2. As we mentioned previously, SBERT provides a variety of pretrained models. We will pick the `bert-base-nli-mean-tokens` model for evaluation. Here is the code:

```
from flair.data import Sentence
from flair.embeddings import (
    SentenceTransformerDocumentEmbeddings)
sbert_embeddings = SentenceTransformerDocumentEmbeddings(
    'bert-base-nli-mean-tokens')
```

3. Let's evaluate the model:

```
evaluate(sbert_embeddings, similar)
([0.98, 0.95, 0.96, 0.99, 0.98], 0.97)
evaluate(sbert_embeddings, dissimilar)
([0.48, 0.41, 0.19, -0.05, 0.0], 0.21)
```

Well done! The SBERT model produced better results. The model produced a low similarity score for the dissimilar list, which is what we expect.

4. Now, we will do a harder test, where we pass contradicting sentences to the models. We will define some tricky sentence pairs, as follows:

```
tricky_pairs=[
("An elephant is bigger than a lion",
"A lion is bigger than an elephant") ,
("the cat sat on the mat",
"the mat sat on the cat")]
evaluate(glove_pool_embeddings, tricky_pairs)
([1.0, 1.0], 1.0)
evaluate(gru_embeddings, tricky_pairs)
([0.87, 0.65], 0.76)
evaluate(bert_embeddings, tricky_pairs)
([1.0, 0.98], 0.99)
evaluate(sbert_embeddings, tricky_pairs)
([0.93, 0.97], 0.95)
```

Interesting! The scores are very high since the sentence similarity model works similarly to topic detection and measures content similarity. When we look at the sentences, they share the same content, even though they contradict each other. The content is about a lion and elephant or cat and mat. Therefore, the models produce a high similarity score. Since the GloVe embedding method pools the average of the words without caring about word order, it measures two sentences as being the same. On the other hand, the GRU model produced lower values as it cares about word order. Surprisingly, even the SBERT model does not produce efficient scores. This may be due to the content similarity-based supervision that's used in the SBERT model.

5. To correctly detect the semantics of two sentence pairs with three classes – that is, `Neutral`, `Contradiction`, and `Entailment` – we must use a fine-tuned model on MultiNLI. The following code block shows an example of using **XLM-RoBERTa**, fine-tuned on XNLI with the same examples:

```
from transformers Import (
    AutoModelForSequenceClassification, AutoTokenizer)
nli_model = AutoModelForSequenceClassification\
    .from_pretrained(
        'joeddav/xlm-roberta-large-xnli')
tokenizer = AutoTokenizer.from_pretrained(
    'joeddav/xlm-roberta-large-xnli')
import numpy as np
for permise, hypothesis in tricky_pairs:
    x = tokenizer.encode(premise,
        hypothesis,
        return_tensors='pt',
        truncation_strategy='only_first')
    logits = nli_model(x)[0]
    print(f"Permise: {permise}")
    print(f"Hypothesis: {hypothesis}")
    print("Top Class:")
    print(nli_model.config.id2label[np.argmax(
        logits[0].detach().numpy()). ])
    print("Full softmax scores:")
    for i in range(3):
        print(nli_model.config.id2label[i],
            logits.softmax(dim=1)[0][i].detach().numpy())
    print("="*20)
```

6. The output will show the correct labels for each:

```
Premise: An elephant is bigger than a lion
Hypothesis: A lion is bigger than an elephant
Top Class:
contradiction
Full softmax scores:
contradiction 0.7731286
neutral 0.2203285
entailment 0.0065428796
====================
Premise: the cat sat on the mat
Hypothesis: the mat sat on the cat
Top Class:
```

```
entailment
Full softmax scores:
contradiction 0.49365467
neutral 0.007260764
entailment 0.49908453
====================
```

The model proved successful for the first example, but it was inconclusive for the second example. Now, we will apply clustering algorithms as unsupervised learning models to texts using SBERT.

Text clustering with Sentence-BERT

For clustering algorithms, we will need a model that's suitable for textual similarity. Let's use the `paraphrase-distilroberta-base-v1` model here for a change. We will start by loading the **Amazon Polarity** dataset for our clustering experiment. This dataset includes Amazon web page reviews spanning a period of 18 years up to March 2013. The original dataset includes over 35 million reviews. These reviews include product information, user information, user ratings, and user reviews. Let's get started:

1. First, randomly select 10 K reviews by shuffling, as follows:

```
import pandas as pd, numpy as np
import torch, os, scipy
from datasets import load_dataset
dataset = load_dataset("amazon_polarity",split="train")
corpus=dataset.shuffle(seed=42)[:10000]['content']
```

2. The corpus is now ready for clustering. The following code instantiates a `sentence-transformer` object using the pretrained `paraphrase-distilroberta-base-v1` model:

```
from sentence_transformers import SentenceTransformer
model_path="paraphrase-distilroberta-base-v1"
model = SentenceTransformer(model_path)
```

3. The entire corpus is encoded with the following execution, where the model maps a list of sentences to a list of embedding vectors:

```
corpus_embeddings = model.encode(corpus)
corpus_embeddings.shape
(10000, 768)
```

4. Here, the vector size is `768`, which is the default embedding size of the BERT base model. From now on, we will proceed with traditional clustering methods. We will choose *k*-means here since it is a fast and widely used clustering algorithm. We just need to set the cluster number (*k*) to

5. Actually, this number may not be optimal. There are many techniques that can determine the optimal number of clusters, such as the **elbow** or **silhouette method**. However, let's leave these issues aside. Here is the execution:

```
from sklearn.cluster import KMeans
K=5
kmeans = KMeans(
    n_clusters=5,
    random_state=0).fit(corpus_embeddings)
cls_dist=pd.Series(kmeans.labels_).value_counts()
cls_dist
3 2772
4 2089
0 1911
2 1883
1 1345
```

Here, we have obtained five clusters of reviews. As we can see from the output, we have fairly distributed clusters. Another issue with clustering is that we need to understand what these clusters mean. As a suggestion, we can apply topic analysis to each cluster or check cluster-based **TF-IDF** (**Term Frequency – Inverse Document Frequency**)to understand the content. Now, let's look at another way to do this based on the cluster centers. The *k*-means algorithm computes cluster centers, called centroids, that are kept in the `kmeans.cluster_centers_` attribute. The centroids are simply the average of the vectors in each cluster. Therefore, they are all imaginary points, not the existing data points. Let's assume that the sentences closest to the centroid will be the most representative example for the corresponding cluster.

Let's try to find only one real sentence embedding, closest to each centroid point. If you like, you can capture more than one sentence. Here is the code:

```
distances = scipy.spatial.distance.cdist(
    kmeans.cluster_centers_, corpus_embeddings)
centers={}
print("Cluster", "Size", "Center-idx",
    "Center-Example", sep="\t\t")
for i,d in enumerate(distances):
    ind = np.argsort(d, axis=0)[0]
    centers[i]=ind
    print(i,cls_dist[i], ind, corpus[ind] ,sep="\t\t")
```

The output is as follows:

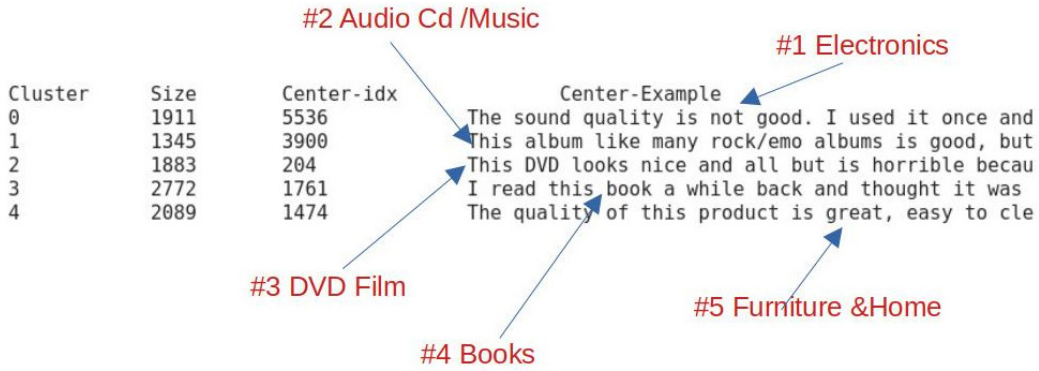


Figure 7.7 – Centroids of the cluster

From these representative sentences, we can reason about the clusters. It seems to be that *k*-means clusters the reviews into five distinct categories: *Electronics*, *Audio CD/Music*, *DVD/Film*, *Books*, and *Furniture and Home*. Now, let's visualize both sentence points and cluster centroids in a 2D space. We will use the **Uniform Manifold Approximation and Projection** (**UMAP**) library to reduce dimensionality. Other widely used dimensionality reduction techniques in NLP that you can use include **t-SNE** (**t-distributed Stochastic Neighbor Embedding**) and **principal component analysis**, or **PCA**.

1. We need to install the `umap` library, as follows:

```
!pip install umap-learn
```

2. The following execution reduces all the embeddings and maps them into a 2D space:

```
import matplotlib.pyplot as plt
import umap
X = umap.UMAP(
    n_components=2,
    min_dist=0.0).fit_transform(corpus_embeddings)
labels= kmeans.labels_fig, ax = plt.subplots(figsize=(12,8))
plt.scatter(X[:,0], X[:,1], c=labels, s=1, cmap='Paired')
for c in centers:
    plt.text(X[centers[c],0], X[centers[c], 1],"CLS-"+ str(c),
        fontsize=18)
    plt.colorbar()
```

The output is as follows:

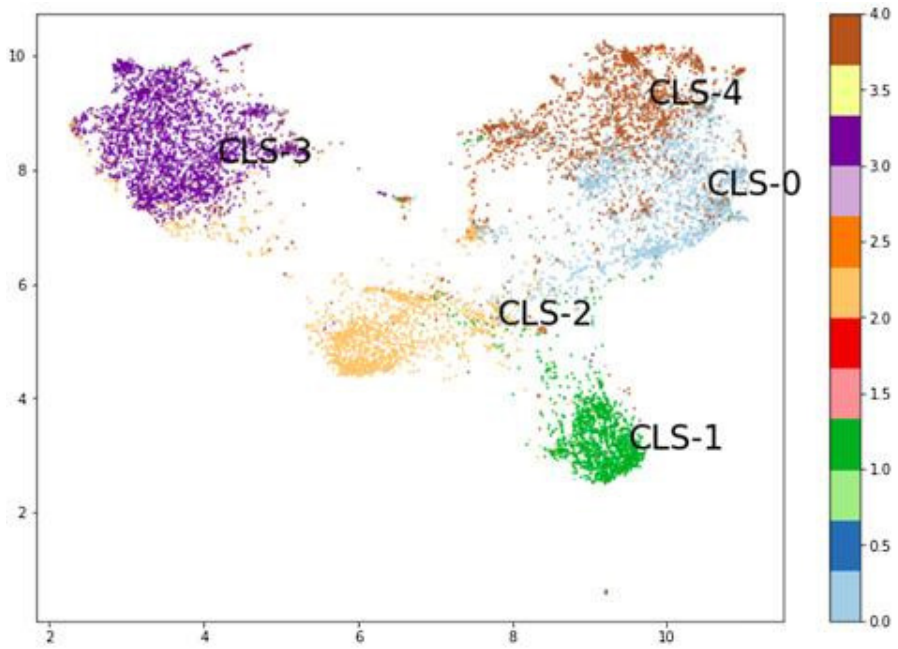


Figure 7.8 – Cluster points visualization

In the preceding output, the points have been colored according to their cluster membership and centroids. It looks like we have picked the right number of clusters.

To capture the topics and interpret the clusters, we located the sentences (one single sentence for each cluster) close to the centroids of the clusters. Now, let's look at a more accurate way of capturing the topic with topic modeling.

Topic modeling with BERTopic

You may be familiar with many unsupervised topic modeling techniques that are used to extract topics from documents; **latent-Dirichlet allocation** (**LDA**) topic modeling and **non-negative matrix factorization** (**NMF**) are well-applied traditional techniques in the literature. BERTopic and top2Vec are two important transformer-based topic modeling projects. In this section, we will apply the BERTopic model to our Amazon corpus. It leverages BERT embeddings and the class-based TF-IDF method to get easily interpretable topics.

First, the BERTopic model begins by encoding the sentences using sentence transformers or any sentence embedding model. This is followed by the clustering step, which has two phases: reducing the dimensionality of the embeddings using UMAP, and then clustering the reduced vectors using

hierarchical density-based spatial clustering of applications with noise (**HDBSCAN**). This process groups similar documents together. In the final stage, the topics are captured using cluster-wise TF-IDF. Instead of extracting the most important words per document, the model extracts the most important words per cluster, providing descriptions of the topics for each cluster. Let's get started:

First, let's install the necessary library, as follows:

```
!pip install bertopic
```

> **Important note**
>
> You may need to restart the runtime since this installation will update some packages that have already been loaded. So, from the Jupyter Notebook, go to `Runtime` | `Restart Runtime`.

If you want to use your own embedding model, you need to instantiate and pass it through the BERTopic model. We will instantiate a `SentenceTransformer` model and pass it to the constructor of BERTopic, as follows:

```
from bertopic import BERTopic
sentence_model = SentenceTransformer(
    "paraphrase-distilroberta-base-v1")
topic_model = BERTopic(embedding_model=sentence_model)
topics, _ = topic_model.fit_transform(corpus)
topic_model.get_topic_info()[:6]
```

The output is as follows:

	Topic	Count	Name
0	4	3086	4_book_read_books_who
1	-1	1818	-1_product_my_use_have
2	7	1499	7_movie_film_dvd_watch
3	5	1327	5_album_cd_songs_music
4	24	274	24_toy_daughter_we_loves
5	2	235	2_game_games_play_graphics

Figure 7.9 – BERTopic results

Please note that BERTopic runs with the same parameters can yield different results since the UMAP model is stochastic. Now, let's see the word distribution of the fifth topic, as follows:

```
topic_model.get_topic(5)
```

The output is as follows:

```
[('album', 0.021777776441862785),
 ('cd', 0.0216003728561258),
 ('songs', 0.015716979809362878),
 ('music', 0.015336261401310738),
 ('song', 0.012883049138010031),
 ('band', 0.008790916825825062),
 ('great', 0.006907063839145953),
 ('good', 0.006594220889305517),
 ('he', 0.006428544176459775),
 ('albums', 0.006402900278216675)]
```

Figure 7.10 – The fifth topic's words of the topic model

The topic words are those words whose vectors are close to the topic vector in the semantic space. In this experiment, we did not cluster the corpus; instead, we applied the technique to the entire corpus. In our previous example, we analyzed the clusters with the closest sentence. Now, we can find the topics by applying the topic model separately to each cluster. This is pretty straightforward, and you can run it yourself.

Please see the top2Vec project for more details and interesting topic modeling applications at https://github.com/ddangelov/Top2Vec.

In the next subsection, we will employ SBERT for semantic search.

Semantic search with SBERT

We may already be familiar with keyword-based search (**Boolean model**), where, for a given keyword or pattern, we can retrieve the results that match the pattern. Alternatively, we can use regular expressions, where we can define advanced patterns such as the **lexico-syntactic** pattern. These traditional approaches cannot handle synonyms (for example, *car* is the same as *automobile*) or word-sense problems (for example, *bank* as the side of a river or *bank* as a financial institute). While the first synonym case causes low recall due to missing the documents that shouldn't be missed, the second causes low precision due to catching the documents that were not being caught.

Let's set up a case study for **frequently asked questions** (**FAQs**) that are idle on websites. We will exploit FAQ resources within a semantic search problem. We will be using the FAQ from the **World Wide Fund for Nature** (**WWF**), a nature non-governmental organization (`https://www.wwf.org.uk/`).

Given these descriptions, it is easy to understand that performing a semantic search using semantic models is very similar to a one-shot learning problem, where we just have a single shot of the class (a single sample), and we want to reorder the rest of the data (sentences) according to it. You can redefine the problem as searching for samples that are semantically close to the given sample, or a

binary classification according to the sample. Your model can provide a similarity metric, and the results for all the other samples will be reordered using this metric. The final ordered list is the search result, which is reordered according to semantic representation and the similarity metric.

WWF has 18 questions and answers on their web page. We defined them as a Python list object called `wf_faq` for this experiment:

- I haven't received my adoption pack. What should I do?
- How quickly will I receive my adoption pack?
- How can I renew my adoption?
- How do I change my address or other contact details?
- Can I adopt an animal if I don't live in the UK?
- If I adopt an animal, will I be the only person who adopts that animal?
- My pack doesn't contain a certificate?
- My adoption is a gift but won't arrive on time. What can I do?
- Can I pay for an adoption with a one-off payment?
- Can I change the delivery address for my adoption pack after I've placed my order?
- How long will my adoption last for?
- How often will I receive updates about my adopted animal?
- What animals do you have for adoption?
- How can I find out more information about my adopted animal?
- How is my adoption money spent?
- What is your refund policy?
- An error has been made with my Direct Debit payment; can I receive a refund?
- How do I change how you contact me?

Users are free to ask any question they want. We need to evaluate which question in the FAQ is the most similar to the user's question, which is the objective of the `quora-distilbert-base` model. There are two options in the SBERT Hub – one is for English and another for multilingual, as follows:

- `quora-distilbert-base`: This is fine-tuned for Quora Duplicate Questions detection retrieval.
- `quora-distilbert-multilingual`: This is a multilingual version of `quora-distilbert-base`. It's fine-tuned with parallel data for 50+ languages.

Let's build a semantic search model by following these steps:

1. The following is the SBERT model's instantiation:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('quora-distilbert-base')
```

2. Let's encode the FAQ, as follows:

```
faq_embeddings = model.encode(wwf_faq)
```

3. Let's prepare five questions so that they are similar to the first five questions in the FAQ, respectively; that is, our first test question should be similar to the first question in the FAQ, the second question should be similar to the second question, and so on, so that we can easily follow the results. Let's define the questions in the `test_questions` list object and encode them as follows:

```
test_questions=["What should be done, if the adoption pack did
not reach to me?",
" How fast is my adoption pack delivered to me?",
"What should I do to renew my adoption?",
"What should be done to change address and contact details ?",
"I live outside of the UK, Can I still adopt an animal?"]
test_q_emb= model.encode(test_questions)
```

4. The following code measures the similarity between each test question and each question in the FAQ and then ranks them:

```
from scipy.spatial.distance import cdist
for q, qe in zip(test_questions, test_q_emb):
    distances = cdist([qe], faq_embeddings, "cosine")[0]
    ind = np.argsort(distances, axis=0)[:3]
    print("\n Test Question: \n "+q)
    for i,(dis,text) in enumerate(zip(
        distances[ind],
        [wwf_faq[i] for i in ind])):
        print(dis,ind[i],text, sep="\t")
```

The output is as follows:

```
 Test Question:
 What should be done, if the adoption pack did not reach to me?
0.1494580342947357      0       I haven't received my adoption pack. What should I do?
0.24940214249978787     7       My adoption is a gift but won't arrive on time. What can I do?
0.3669761157176866      1       How quickly will I receive my adoption pack?

 Test Question:
  How fast is my adoption pack delivered to me?
0.16582390267585112     1       How quickly will I receive my adoption pack?
0.3470478678903325      0       I haven't received my adoption pack. What should I do?
0.3511114386193057      7       My adoption is a gift but won't arrive on time. What can I do?

 Test Question:
 What should I do to renew my adoption?
0.04168242777718267     2       How can I renew my adoption?
0.2993018812386016      12      What animals do you have for adoption?
0.3014071168242859      0       I haven't received my adoption pack. What should I do?

 Test Question:
 What should be done to change adress and contact details ?
0.276601898726506       3       How do I change my address or other contact details?
0.352868128705782       17      How do I change how you contact me?
0.4393553216276348      2       How can I renew my adoption?

 Test Question:
 I live outside of the UK, Can I still adopt an animal?
0.16945626472973518     4       Can I adopt an animal if I don't live in the UK?
0.200544029334076       12      What animals do you have for adoption?
0.28782233378715627     13      How can I nd out more information about my adopted animal?
```

Figure 7.11 – Question-question similarity

Here, we can see the `0`, `1`, `2`, `3`, and `4` indexes in order, which means the model successfully found similar questions as expected.

5. For the deployment, we can design the following `getBest()` function, which takes a question and returns the `K` most similar questions in the FAQ:

```
def get_best(query, K=5):
    query_emb = model.encode([query])
    distances = cdist(query_emb,faq_embeddings,"cosine")[0]
    ind = np.argsort(distances, axis=0)
    print("\n"+query)
    for c,i in list(zip(distances[ind], ind))[:K]:
        print(c,wwf_faq[i], sep="\t")
```

6. Let's ask a question:

```
get_best("How do I change my contact info?",3)
```

The output is as follows:

```
How do I change my contact info?
0.05676792449319612     How do I change my address or other contact details?
0.185665422885958       How do I change how you contact me?
0.32408327251343816     How can I renew my adoption?
```

Figure 7.12 – Similar question similarity results

7. What if a question that's used as input is not similar to one from the FAQ? Here is such a question:

```
get_best("How do I get my plane ticket \
    if I bought it online?")
```

The output is as follows:

```
How do I get my plane ticket if I bought it online?
0.35947505490536136     How do I change how you contact me?
0.3680785568009698      How do I change my address or other contact details?
0.4306634329555338      My adoption is a gift but won't arrive on time. What can I do?
```

Figure 7.13 – Dissimilar question similarity results

The best dissimilarity score is `0.35`. So, we need to define a threshold such as `0.3` so that the model ignores such questions that are higher than that threshold and says `no similar answers found`.

Other than question-question symmetric search similarity, we can also utilize SBERT's question-answer asymmetric search models, such as `msmarco-distilbert-base-v3`, which is trained on a dataset of around 500 K Bing search queries. It is known as **Passage Ranking**. This model helps us measure how related the question and context are and checks whether the answer to the question is in the passage.

Now, we will discuss instruction fine-tuned embeddings. These embeddings are mostly concerned about not only embedding the text but also enabling us to use instructions as well.

Instruction fine-tuned embedding models

A single text-representation model can be trained on a specific task. For example, a model that is trained on the semantic representation of the given text might not be a very good choice for representing scientific texts, because the nature of similarity in this scope is different from semantic similarity. Instruction fine-tuning is one of the approaches that has been proposed in recent years and many new methods have adapted this approach to solve diverse sets of problems in a single model. The Instructor model is one of these models. In this section, we will show a few examples of how this model works.

To import the model, you can either use Hugging Face transformers or the `InstructEmbedding` library. Both provide the same results, but the latter is much more convenient.

To install it, simply use the `pip` command:

```
pip install InstructorEmbedding
```

Please note that the sentence-transformers library also must be installed:

```
pip install sentence-transformers
```

To load the model, you can simply follow this code:

```
from InstructorEmbedding import INSTRUCTOR
model = INSTRUCTOR('hkunlp/instructor-large')
```

This will automatically load the model, such as Hugging Face, as they used the same in the back.

They trained the model so that it follows the following instruction format:

```
Represent the domain text_type for task_objective:
```

In this format, `text_type` refers to the type of text (financial, scientific, etc.) while the objective is used for various tasks, for example, clustering, semantic similarity, and so on.

For example, you can use the following format to represent medical sentences for clustering:

```
import sklearn.cluster
sentences = [['Represent the Medical sentence for clustering: ','COVID
has been striking all over globe from 2019.'],
    ['Represent the Medical sentence for clustering: ','The
coronavirus that causes COVID-19 is officially called SARS-CoV-2,
which stands for severe acute respiratory syndrome coronavirus 2.'],
    ['Represent the Medical sentence for clustering: ','The name of
the illness caused by the coronavirus SARS-CoV-2. COVID-19 stands for
"coronavirus disease 2019.'],
    ['Represent the Medical sentence for clustering: ',"HIV (human
immunodeficiency virus) is a virus that attacks the body's immune
system."],
    ['Represent the Medical sentence for clustering: ','If HIV is not
treated, it can lead to AIDS (acquired immunodeficiency syndrome).']]
embeddings = model.encode(sentences)
clustering_model = sklearn.cluster.MiniBatchKMeans(n_clusters=2)
clustering_model.fit(embeddings)
cluster_assignment = clustering_model.labels_
print(cluster_assignment)
```

As a result, you should get the following:

```
[0 0 0 1 1]
```

Also, it is possible to use it for a very different task on a very different topic such as matching questions and answers from Wikipedia pages. This task is an information retrieval-based task in which the closest answer to the given question must be retrieved. As an example, you can follow this code:

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
query = [['Represent the Wikipedia question for retrieving supporting
documents: ','What is sea sheep?']]
```

```
corpus = [['Represent the Wikipedia document for retrieval:
','Costasiella kuroshimae—also known as a "leaf slug", "leaf sheep",
or "salty ocean caterpillar"—is a species of sacoglossan sea slug.'],
    ['Represent the Wikipedia document for retrieval: ',"Artificial
intelligence (AI) is intelligence demonstrated by machines, as opposed
to intelligence of humans and other animals."],
    ['Represent the Wikipedia document for retrieval: ','In the fields
of medicine, biotechnology and pharmacology, drug discovery is the
process by which new candidate medications are discovered.']]
query_embeddings = model.encode(query)
corpus_embeddings = model.encode(corpus)
similarities = cosine_similarity(query_embeddings,corpus_embeddings)
retrieved_doc_id = np.argmax(similarities)
print(retrieved_doc_id)
```

You should get the following as the result:

```
0
```

This is the first item in the corpus of answers. As you have seen from this example, embedding for questions is done with a slightly different instruction compared to answers.

Instruction fine-tuning open a new way of solving diverse problems with a single model. This single model has the capacity to understand and solve different problems with the given instructions rather than being a static problem solver for a single use case.

Summary

In this chapter, we learned about text representation methods. We learned how it is possible to perform tasks such as zero-, few-, and one-shot learning using different and diverse semantic models. We also learned about NLI and its importance in capturing the semantics of text. Moreover, we looked at some useful use cases such as semantic search, semantic clustering, and topic modeling using Transformer-based semantic models. We learned how to visualize the clustering results and understood the importance of centroids in such problems. We also described instruction-tuned multitask models that can create representations according to the given instructions.

In the next chapter, you will learn about efficient Transformer models. You will learn about distillation, pruning, and quantizing Transformer-based models. You will also learn about different and efficient Transformer architectures that make improvements to computational and memory efficiency, as well as how to use them in NLP problems.

Further reading

- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., and Zettlemoyer, L. (2019). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. arXiv preprint arXiv:1910.13461.
- Pushp, P. K., and Srivastava, M. M. (2017). *Train Once, Test Anywhere: Zero-Shot Learning for Text Classification*. arXiv preprint arXiv:1712.05972.
- Reimers, N., and Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv preprint arXiv:1908.10084.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, M., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv preprint arXiv:1907.11692.
- Williams, A., Nangia, N., and Bowman, S. R. (2017). *A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference*. arXiv preprint arXiv:1704.05426.
- Cer, D., Yang, Y., Kong, S. Y., Hua, N., Limtiaco, N., John, R. S., Constant, N., Guajardo-Cespedes, M., Yuan, S., Tar, C., Sung, Y., Strope, B., and Kurzweil, R. (2018). *Universal Sentence Encoder*. arXiv preprint arXiv:1803.11175.
- Yang, Y., Cer, D., Ahmad, A., Guo, M., Law, J., Constant, N., Abrego, G. H., Yuan, S., Tar C., Sung, Y., Strope, B., and Kurzweil, R. (2019). *Multilingual Universal Sentence Encoder for Semantic Retrieval*. arXiv preprint arXiv:1907.04307.
- Humeau, S., Shuster, K., Lachaux, M. A., and Weston, J. (2019). *Poly-encoders: Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring*. arXiv preprint arXiv:1905.01969.
- Su, H., Shi, W., Kasai, J., Wang, Y., Hu, Y., Ostendorf, M., Yih, W., Smith, N. A., Zettlemoyer, L., and Yu, T. (2022). *One Embedder, Any Task: Instruction-Finetuned Text Embeddings*. arXiv. /abs/2212.09741

8

Boosting Model Performance

Up to this point, we have solved many tasks using common approaches and achieved some success. However, we can increase our task performance by utilizing specific techniques. There are several approaches to improving the performance of Transformer models in the literature. In this chapter, we will explore some of these techniques and demonstrate how to boost the model beyond the vanilla training pipeline, such as with data augmentation or domain adaptation. **Data augmentation** is a powerful technique and is widely used for improving the accuracy of deep learning models. By augmenting the data points, the deep learning model can capture the underlying patterns and relationships in the data more effectively. Another method to improve model performance is **domain adaptation**. Since large language models are trained on general-purpose and diverse texts, there may be a discrepancy when applied to a specific domain. We may need to adjust these language models according to specific application areas by incorporating many factors. **Hyperparameter optimization** (**HPO**) is another widely used technique in every subfield of deep learning for achieving optimum performance. When training a deep learning model, certain parameters (called **hyperparameters**) cannot be learned during the training process. HPO helps us find these parameters

In this chapter, we will cover the following topics:

- Improving performance with data augmentation
- Adapting the model to the domain
- Optimizing parameters with HPO

Technical requirements

We will need the following Python libraries to be installed: `nlpaug`, `datasets`, `transformers`, `sacremoses`, and `optuna`.

Improving performance with data augmentation

Data augmentation is a common technique in deep learning to improve the success of a model. It involves replicating a data point by making changes that do not significantly alter its meaning. This technique has been widely used in image processing and **Natural Language Processing** (**NLP**) as training the neural model with more data typically leads to better performance.

We can introduce noise or perturbations to the existing data, create new variations or aspects of the same data, or generate entirely new data points through interpolation, where new data is generated based on a couple of existing nearby data points. Data augmentation is especially useful when training data is limited. It can make the model more robust and improve its performance on downstream tasks. There are several ways to perform data augmentation. In image processing, this may include flipping or changing the brightness of images. In NLP, it may include removing or swapping words, injecting random characters/words, changing the order of words in a sentence, and many advanced techniques.

The process of duplication depends entirely on creativity and the NLP task at hand. If you are working on a classification task for linguistic acceptability, shuffling words may not be a useful augmentation. If you are working on sentiment classification, this type of shuffling process can be practical. The critical issue is to evaluate each augmentation technique separately to determine which techniques work and which techniques are harmful because it is hard to know which techniques will help us without trial.

As a final remark, we can say that this process of augmentation is also widely used in the field of **contrastive learning**. The contrastive learning method allows us to easily create positive and negative examples for self-supervised learning using such augmentation. So, we can benefit from data augmentation not only in terms of increasing the number of data but also in terms of **self-supervision**.

Now, let's work on some data augmentation techniques by using the IMDb sentiment classification example. First, let's discuss some data augmentation techniques and use them to improve model performance.

Character-level augmentation

Only manipulations on characters fall into this category. We have the opportunity to simply augment the data by manipulating characters. While it may not be feasible to discuss every method in detail, let's highlight some of the most important ones. Here are some examples:

Character-Level Augmentation		**Original**	**Augmented**
Keyboard Augmenter	Substitute character by keyboard distance	*I love cats and dogs*	*I lobe cats and dogx*
Random Augmenter	Insert/swap/delete a character randomly	*I love cats and dogs*	*I love cats an doggs*
OCR error	Simulate OCR errors: *e -> o*, *l -> 1*	*branches*	*branchos*

Table 8.1 – Character-level data augmentation

We can add many more methods to this category. Let's move on to the word level.

Word-level augmentation

The word-level approach has more possibilities than the previous model. We can rely on word semantics at this point, which gives us many techniques for augmentation. The techniques used under this category are mostly based on a predefined list of words and are easy to apply.

Word-Level Augmentation		**Original**	**Augmented**
Spelling Errors Dictionary	We use a predefined dictionary of constantly recurring spelling mistakes	achieve across	acheive accross
Contextual Word Embeddings Augmenter	Replacing a word with the closest word in contextual meaning to that word using deep learning models such as word2vec or BERT	Banana Astronaut	Apple Moon
Synonym Replacement	Based on a synonym dictionary	automobile large	car big
Antonym Replacement	Based on an antonym dictionary	large	small
Random Word	Deletion/insertion	I love cats and dogs I love cats and dogs	I love cats and I love table cats and dogs
Word Shuffling	Changing the order of words	I love cats and dogs	love and cats dogs I

Table 8.2 – Word-level augmentation

While word-level augmentation can be useful, it does have its limitations, as word-level operations operate independently of context and cannot take advantage of semantic information from the sentence. That's why we should consider using more advanced techniques based on sentence semantics, as outlined in the next section.

Sentence-level augmentation

Sentence-level augmentation is a method of data augmentation that considers the entire sentence and its meaning. As language models continue to evolve, this technique has become more and more common. We will discuss two examples.

Back-translation is translating a text to another language (English to French, for example) and then translating it back to the original language (French to English). Another example is translating a text from language A to language B, then to language C, and then back to the original language in the same way (C to B to A).

Paraphrasing is another method of semantically re-generating text, accomplished through the use of trained checkpoints. These models take a text and generate another text that has the same meaning. Therefore, this method is very suitable for data augmentation. However, for the two approaches, the augmentation process takes a bit longer than previous techniques due to the inference time of the neural model.

Here are some examples of sentence-level augmentation:

Sentence-Level Augmentation		**Original**	**Augmented**
Back-translation	Translating a text to another language (EN- FRA) and then back to the original (FRA-EN)	You have to think a lot to understand	It takes a lot of thought to understand
Paraphrasing	Using a text-to-text model	Life is the future, not the past	The future, not the past, is where life lies

Table 8.3 – Sentence-level augmentation

Well done! We are ready to apply these techniques for any NLP task and see whether they improve performance. We will see this in the next part.

Boosting IMDB text classification with augmentation

Now, we will improve the model performance of the IMDB sentiment task by examining some of these augmentation methods by utilizing the `nlpaug` library. Please check the following link for more information: `https://github.com/makcedward/nlpaug`.

This Python library helps us with augmenting textual data points for IMDB datasets (`https://huggingface.co/datasets/imdb`). You can find many more data augmentation methods in this library. We have only applied some randomly selected ones for the demonstration here but it is important to conduct a thorough analysis to determine which set of techniques is best suited to address your NLP task.

We start by installing the following necessary packages:

```
pip install nlpaug datasets transformers sacremoses
```

Here are the imports for augmentation:

```
import nlpaug.augmenter.char as nac
import nlpaug.augmenter.word as naw
import nlpaug.augmenter.sentence as nas
import nlpaug.flow as nafc
from nlpaug.util import Action
```

In *Chapter 5*, we used a small portion of the IMDb dataset for two reasons: to enable quick prototyping and to evaluate the effect of augmentation techniques on model performance.

The following code demonstrates how we select 2,000 examples for training data:

```
from datasets import load_dataset
imdb_train= load_dataset('imdb',
    split="train[:1000]+train[-1000:]")
imdb_test= load_dataset('imdb',
    split="test[:500]+test[-500:]")
imdb_val= load_dataset('imdb',
    split="test[500:1000]+test[-1000:-500]")
imdb_train.shape, imdb_test.shape, imdb_val.shape
OUTPUT> ((2000, 2), (1000, 2), (1000, 2))
```

We define some necessary functions that will also be used in the examples that we will see later in this chapter:

```
from sklearn.metrics import (accuracy_score,
precision_recall_fscore_support)
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    precision, recall, f1, _ = \
    precision_recall_fscore_support(labels,preds,
        average='macro')
    acc = accuracy_score(labels, preds)
    return {
        'Accuracy': acc,
        'F1': f1,
        'Precision': precision,
        'Recall': recall
    }
```

```
def tokenize_it(e):
    return tokenizer(e['text'],
        padding=True,
        truncation=True)
```

Now, we instantiate 10 random different augmentation objects using the `nlpaug` library as follows:

```
import nlpaug.augmenter.word as naw
#substitute character by keyboard distance
aug1 = nac.KeyboardAug(aug_word_p=0.2,
    aug_char_max=2,
    aug_word_max=4)
# random insert/swap/delete
aug2 = nac.RandomCharAug(action="insert", aug_char_max=1)
aug3 = nac.RandomCharAug(action="swap", aug_char_max=1)
aug4 = nac.RandomCharAug(action="delete", aug_char_max=1)
# spelling error
aug5 = naw.SpellingAug()
# contextual word insertion / substitute
aug6 = naw.ContextualWordEmbsAug(
    model_path='bert-base-uncased',
    action="insert")
aug7 = naw.ContextualWordEmbsAug(
    model_path='bert-base-uncased',
    action="substitute")
# wordnet-based synonym replacement
aug8 = naw.SynonymAug(aug_src='wordnet')
# random word deletion
aug9 = naw.RandomWordAug()
# back-translation
aug10 = naw.BackTranslationAug(
    from_model_name='facebook/wmt19-en-de',
    to_model_name='facebook/wmt19-de-en', device='cuda')
```

Now let's write a wrapper function, `augment_it()`, that incorporates and applies 10 of these techniques. The parameters are the original text to be augmented (`text`) and the original label (`label`). We replicate the original label for each augmented text:

```
def augment_it(text, label):
    result= [eval("aug"+str(i)).augment(text)[0]
    for i in range(1,11) ]
    return result, [label]* len(result)
```

Here is an example run:

```
augment_it("i like cats and dogs",1)
OUTPUT:
(['i li.W cats and dogs',
'i lBike cats and zdogs',
'i liek ctas and dogs',
'i ike ats and dogs',
'in like cats and gogs',
'mostly i like my cats and dogs',
'i like this no dogs',
'i like cats and dog',
'like cats and',
'I like cats and dogs'],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

Notice that back-translation does not produce a new sentence and returns the same sentence since the given example is very simple for translation. For a more complex example, we can get different returns. We also get 10 replications of label `1` returned as expected.

With the following code snippet, a random 10% sample of the IMDB dataset is being duplicated. Ten augmented sentences will be generated for each selected example. Then, the input dataset will double in size. Since this code snippet uses BERT-like models, the generation process will be a bit slow.

We can observe the effect on performance by tweaking the degree of augmentation (`frac-> [0.1 - 1.0]`) using up to the entire dataset or increasing the number of augmentation techniques:

```
import pandas as pd
imdb_df=pd.DataFrame(imdb_train)
texts=[]
labels=[]
for r in imdb_df.sample(frac=0.1).itertuples(index=False):
    t,l=augment_it(r.text, r.label)
    texts+= t
    labels+=l
aug_df=pd.DataFrame()
aug_df["text"]= texts
aug_df["label"]= labels
imdb_augmented=pd.concat([imdb_df, aug_df])
imdb_df.shape, imdb_augmented.shape
OUTPUT: ((2000, 2), (4000, 2))
```

We doubled the data size (2K to 4K) and obtained `imdb_augmented`.

The following code contains two datasets: one normal dataset (`imdb_train`) and one augmented dataset (`imdb_augmented`). The normal dataset is currently selected, and the augmented dataset is commented out. When running the code a second time, you should swap them (selecting the `imdb_augmented` dataset by commenting out the normal dataset):

```
from transformers import (
    BertTokenizerFast, BertForSequenceClassification)
model_path= 'bert-base-uncased'
tokenizer = BertTokenizerFast.from_pretrained(model_path)
#imdb train data with augmentation
#imdb_augmented2= Dataset.from_pandas(imdb_augmented)
#enc_train=imdb_augmented2.map(tokenize_it,
#batched=True, batch_size=1000)
# imdb train data without augmentation
enc_train=imdb_train.map(tokenize_it,
    batched=True, batch_size=1000)
enc_test=imdb_test.map(tokenize_it,
    batched=True, batch_size=1000)
enc_val=imdb_val.map(tokenize_it,
    batched=True, batch_size=1000)
model_path= "bert-base-uncased"
model = BertForSequenceClassification.from_pretrained(
    model_path,
    id2label={0:"NEG", 1:"POS"},
    label2id={"NEG":0, "POS":1})
```

With this code, we fine-tune the `bert-base-uncased` model. The following code snippet is exactly the same as the one I applied in *Chapter 5*:

```
from transformers import TrainingArguments, Trainer
training_args = TrainingArguments(
    output_dir='./MyIMDBModel',
    do_train=True,
    do_eval=True,
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    fp16=True,
    load_best_model_at_end=True)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=enc_train,
    eval_dataset=enc_val,
```

```
    compute_metrics= compute_metrics)
trainer.train()
q=[trainer.evaluate(eval_dataset=data) \
    for data in [enc_train, enc_val, enc_test]]
pd.DataFrame(q, index=["train","val","test"]).iloc[:,:5]
```

Without augmentation, we get the following output:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.089077	0.9735	0.973494	0.973899	0.9735
val	0.298978	0.8860	0.885406	0.894174	0.8860
test	0.331224	0.8690	0.868001	0.880521	0.8690

Figure 8.1 – The performance of the IMDB dataset without data augmentation

With augmentation, we get the following results:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.028604	0.99325	0.993250	0.993271	0.993239
val	0.391425	0.89600	0.895973	0.896406	0.896000
test	0.431911	0.88700	0.886905	0.888306	0.887000

Figure 8.2 – The performance of the IMDB dataset with data augmentation

As we can see, the `eval_F1` score has increased from 86.8 to 88.6 (+1.8). Additionally, there appears to be an improvement in other parameters. To improve performance, we can scale up augmentation techniques and conduct a detailed analysis of different augmentation approaches for selection.

Well done! We will try another technique to improve the model performance with domain adaption in the next section!

Adapting the model to the domain

Although the fine-tuning approach for Transformer architectures usually performs well, differences in the distribution of source and target data can have a significant impact on the effectiveness of fine-tuning (*Transfer Learning in Natural Language Processing*, Ruder et al., NAACL 2019). If the source and target datasets are substantially different, fine-tuning may struggle to learn without adaptation.

Previous studies have shown that pretrained Transformers are the most robust architectures for handling word distribution shifts and have greater generalization capacity than other architectures (Yildirim, Savas, et al. *Adaptive Fine-tuning for Multiclass Classification over Software Requirement Data*, arXiv preprint arXiv:2301.00495 (2023).). However, there is still room for improvement (*Don't*

Stop Pretraining: Adapt Language Models to Domains and Tasks, Gururangan et al., ACL 2020), using additional in-domain data. By specializing the model to the target data, we expect to improve its performance on downstream tasks. That is, a pretrained model is further trained using the pretraining objective on target data that is expected to be closer to the target distribution. Finally, we will have another version of the pretrained model (`adapted-bert`, for instance), which still requires fine-tuning for a downstream task. The following diagram summarizes our adaptive fine-tuning framework:

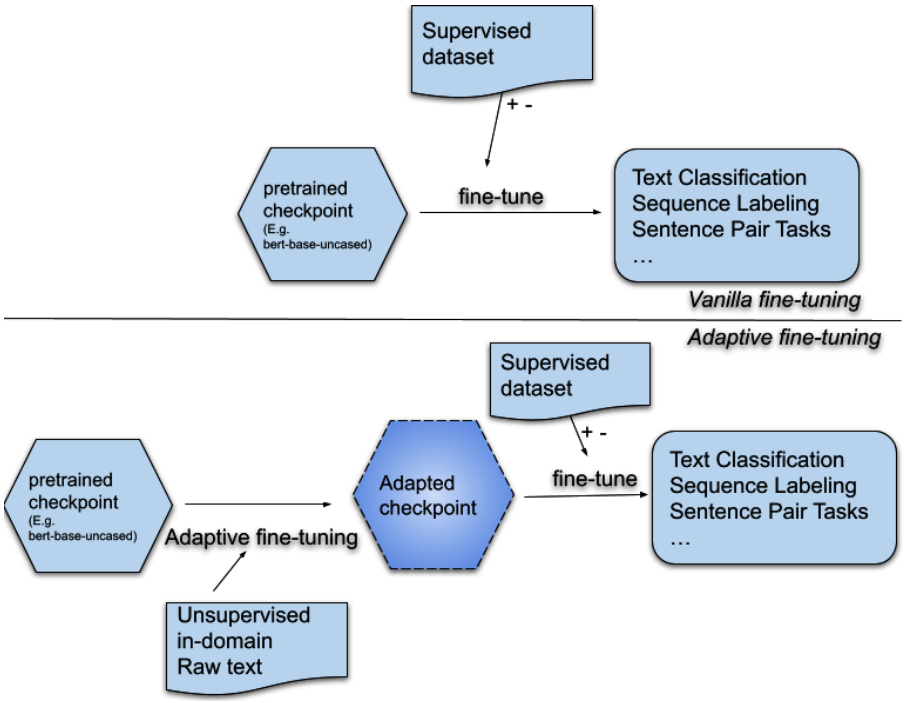


Figure 8.3 – Domain-adaptive fine-tuning

As shown at the bottom of the diagram, adaptive fine-tuning consists of three phases. The objective function is typically the **masked language model** (**MLM**). For the first step, there are already many pretrained checkpoints available in the literature, such as `bert-base-uncased` and `roberta-large`. Therefore, we do not need to pretrain a model from scratch. These pretrained models have already been trained on a large variety of data and have a sufficient depth of syntactic and semantic knowledge encoded in their parameters, as discussed in *Chapters 3* to *5*.

In the second phase, a pretrained model trains with the same source objective MLM but with the target (in-domain) unsupervised datasets for adaptation. It is important to note that other kinds of auxiliary objective functions can be utilized to improve adaptation. During this phase, we do not change the objective of the vanilla BERT model architecture (MLM) and do not add any additional parameters. After this process, we will have an adapted checkpoint to be ready for fine-tuning any downstream tasks, as described in *Chapter 5*.

Let's adapt the BERT model with domain data and then fine-tune it. We will use the same IMDB dataset. We define `model_path` and load the tokenizer first, as follows:

```
from transformers import (BertTokenizerFast,
BertForSequenceClassification)
model_path= 'bert-base-uncased'
tokenizer = BertTokenizerFast.from_pretrained(model_path)
```

The following code loads a dataset consisting of 4K training, 1K validation, and 1K test. You can scale it up to the entire dataset if you like:

```
from datasets import load_dataset
imdb_train= load_dataset('imdb',
    split="train[:2000]+train[-2000:]")
imdb_test= load_dataset('imdb',
    split="test[:500]+test[-500:]")
imdb_val= load_dataset('imdb',
    split="test[500:1000]+test[-1000:-500]")
imdb_train.shape, imdb_test.shape, imdb_val.shape
OUTPUT:
((4000, 2), (1000, 2), (1000, 2))
```

And we tokenize and encode them to prepare our dataset for training as follows:

```
def tokenize_it(e):
    return tokenizer(e['text'],
    padding=True,
    truncation=True)
enc_train=imdb_train.map(tokenize_it,
    batched=True, batch_size=1000)
enc_test=imdb_test.map(tokenize_it,
    batched=True, batch_size=1000)
enc_val=imdb_val.map(tokenize_it,
    batched=True, batch_size=1000)
```

At this point, we will adapt the `bert-base-uncased` model with all raw texts in our available training dataset. No supervised method will be used here. You can use all the raw text data in your domain at this stage. Therefore, only the text part of the IMDB training data will be used. Data size is important for good adaptation; therefore, we will take all the training data and perform adaptation.

Let's load the entire training data as follows:

```
dataset_for_adaptation= load_dataset('imdb', split="train")
imdb_sentences=dataset_for_adaptation["text"]
train_sentences=imdb_sentences[:20000]
dev_sentences=imdb_sentences[20000:]
```

Then, load the model:

```
from transformers import AutoModelForMaskedLM, AutoTokenizer
model = AutoModelForMaskedLM.from_pretrained(model_path)
```

Some hyperparameters for the adaptation phase are as follows:

```
batch_size = 16
num_train_epochs = 15
max_length = 100
mlm_prob = 0.25
```

Here, we keep the epoch number longer than the normal fine-tuning process. On the other hand, the original MLM rate value of BERT is `0.15`. The MLM rate is a value indicating the percentage of input tokens to be masked. In some studies, it has been shown that this value is more efficient if it is between 15 and 40 percent. As the number of masked tokens increases, the model can extract stronger representations to capture the meaning. In parallel, it will be necessary to keep the step or epoch values longer. Therefore, it will be beneficial to apply trial-and-error methods or HPO, which we will see in the next section. In this study, we will keep the MLM value at `0.25`. You can increase the maximum length value and batch value depending on your GPU capacity.

We borrow the `TokenizedSentencesDataset` class definition from the SBERT GitHub repository. You can also find command-line script code there to directly run the Python code without diving into the Jupyter code block (`https://www.sbert.net/examples/unsupervised_learning/MLM/README.html`).

We can see the dataset class definition in the following code:

```
class TokenizedSentencesDataset:
    def __init__(self, sentences, tokenizer,
        max_length, cache_tokenization=False):
        self.tokenizer = tokenizer
        self.sentences = sentences
        self.max_length = max_length
        self.cache_tokenization = cache_tokenization
    def __getitem__(self, item):
        if not self.cache_tokenization:
            return self.tokenizer(self.sentences[item],
                add_special_tokens=True,
                truncation=True,
                max_length=self.max_length,
                return_special_tokens_mask=True)
```

```
        if isinstance(self.sentences[item], str):
            self.sentences[item] =\
                self.tokenizer(self.sentences[item],
                add_special_tokens=True,
                truncation=True,
                max_length=self.max_length,
                return_special_tokens_mask=True)
        return self.sentences[item]
    def __len__(self):
        return len(self.sentences)
```

Let's tokenize our dataset for adaptation as follows:

```
train_dataset = TokenizedSentencesDataset(train_sentences,
    tokenizer, max_length)
dev_dataset = TokenizedSentencesDataset(dev_sentences,
    tokenizer, max_length)
```

Now, we use the `Trainer` class to prepare the running environment. We pass `mlm_prob = 0.25` to the `DataCollator` object as follows:

```
data_collator = \
    DataCollatorForLanguageModeling(tokenizer=tokenizer,
    mlm=True,
    mlm_probability=mlm_prob)
training_args = TrainingArguments(
    num_train_epochs=num_train_epochs,
    evaluation_strategy="steps",
    per_device_train_batch_size=batch_size,
    prediction_loss_only=True,
    fp16=True)
trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=train_dataset,
    eval_dataset=dev_dataset)
```

Let's run the adaptation:

```
trainer.train()
```

Now, we can see the output of the training process as follows:

Step	Training Loss	Validation Loss
1000	1.649100	2.502931
2000	1.661200	2.487715
3000	1.728200	2.455443

Figure 8.4 – Domain adaptation training with MLM (truncated output)

We can see only 3 training epochs and validation loss in the preceding figure, but you will see 15 epochs for your run. It can take a long time depending on your GPU capacity and dataset.

Well done! We will save our adapted model to be used later, as follows:

```
adapted_model_path="adapted-bert"
model.save_pretrained(adapted_model_path)
tokenizer.save_pretrained(adapted_model_path)
```

Now, we have two checkpoints, as follows:

- `bert-base-uncased # (vanilla bert)`
- `adapted-bert # (adapted bert)`

In the following code, we comment out `vanilla bert` and first fine-tune `adapted-bert` and then the `vanilla bert` checkpoints by swapping commenting:

```
model_path="adapted-bert" # 1) Adapted model
#model_path= "bert-base-uncased" # 2)vanilla bert
model = BertForSequenceClassification.from_pretrained(
    model_path,
    id2label={0:"NEG", 1:"POS"},
    label2id={"NEG":0, "POS":1})
```

In the following step, we create a `TrainingArguments` and `Trainer` object in the same way as we did in the *Improving performance with data augmentation* section. Let's show the code of the fine-tuning phase in a truncated way:

```
training_args = TrainingArguments(...
trainer = Trainer(…
trainer.train()
```

Let's compare the `vanilla bert` and `adapted-bert` fine-tuning performance as follows. The following output is vanilla fine-tuning:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.150218	0.95675	0.956747	0.956874	0.95675
val	0.249726	0.91500	0.914990	0.915201	0.91500
test	0.245597	0.91000	0.909997	0.910059	0.91000

Figure 8.5 – Vanilla BERT performance on the IMDB dataset

The following is the output of adapted training:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.116748	0.96175	0.961746	0.961964	0.96175
val	0.244279	0.90500	0.904950	0.905859	0.90500
test	0.218436	0.91400	0.913978	0.914424	0.91400

Figure 8.6 – Adapted BERT performance on the IMDB dataset

As seen in the figures, the adapted model is slightly better than the `bert-base` (vanilla) model in all metrics. We addressed the issue of adaptation in a straightforward manner. There are various ways, of course, to enhance the adaptation process.

Now, let's list some observations and tips about adaptive learning that you can benefit from:

- One point to be aware of is **catastrophic forgetting**, which can result in the loss of a significant amount of information in the trained model weights. Therefore, if you encounter such a situation, it may be beneficial to keep the learning rate value lower and carefully monitor the validation loss value. Failure to pay attention to these factors can hinder the improvement we are now targeting.
- Currently, our transfer learning strategy has a sequential feature. Therefore, we can add this training step to the pipeline as often as we want.
- We already performed unsupervised domain-adaptive training by using domain-specific texts. Then, we conducted task-adaptive training with a specific task objective. You can add some tasks similar to your task before your last fine-tuning. Let's say we want to solve a particular **Question Answering** (**QA**) problem in the medical domain. We have common datasets such as PudMed raw text, **Stanford Question Answering Dataset** (**SquAD**)-like doctor-patient conversations and our own QA task in medicine. We first adapt the model with the unsupervised PudMed dataset, then train it with the supervised doctor-patient dataset, and finally, train our QA task. These types of sequential training (domain-adaptive and task-adaptive) increase the success of the model performance.

- As another example, let's say we want to solve a sequence labeling task such as detecting sensitive information (ID, bank information, etc.) in a text. There are already many checkpoints trained for **Named Entity Recognition** (**NER**) tasks as popular sequence labeling tasks. We can directly use those already-trained NER checkpoints instead of `bert-base`. This will be an important advantage provided by the sequential transfer learning method.

In the previous sections, we mostly chose the default parameters. In the next section, we'll explore whether we can use a more efficient selection strategy.

Optimizing the parameters with HPO

Hyperparameters are the only parameters that deep learning models cannot learn during training. So, we need to follow a way to specify them. The first way is usually to follow common practices in the literature. For example, the typical epoch value for fine-tuning a BERT model is around `3`. One strategy can be manually adjusting these parameters while monitoring model performance – in particular, monitoring validation loss.

Furthermore, it is possible to systematically determine the optimal set of parameters using certain algorithms. One approach to achieve this is to explore all possibilities by searching over a predefined set of hyperparameters, a naive process known as **grid search**. However, a grid search-like model is not always practical and can be tedious since training a single Transformer model is a lengthy procedure. Alternatively, a **random search** can be preferred. It achieves similar results with specific randomization while reducing the complexity of grid search. This approach randomly samples the search space and continues until the stopping criteria are met.

There are many modern algorithms for HPO such as grid search, Bayesian optimization, and particle swarm optimization. One such algorithm is the **Tree-structured Parzen Estimator** (**TPE**) approach, which is a Bayesian optimization algorithm that uses a probabilistic model to evaluate the performance of different hyperparameters. Specifically, it sequentially constructs models based on historical measurements, and then subsequently chooses new hyperparameters. For details, please see the following paper: *Bergstra, James, et al., Algorithms for hyper-parameter optimization*, Advances in neural information processing systems 24 (2011).

Now, we will implement HPO using the Optuna framework (`https://github.com/pfnet/optuna/`). **Optuna** is a framework designed for HPO that contains various optimization approaches. We will use the TPE algorithm, which is the default option.

We start by installing the necessary packages:

```
pip install datasets transformers optuna
```

Loading the same IMDB dataset, we take 2K for training, 1K for testing, and 1K for validation, as follows:

```
from datasets import load_dataset
imdb_train= load_dataset('imdb',
    split="train[:1000]+train[-1000:]")
imdb_test= load_dataset('imdb',
    split="test[:500]+test[-500:]")
imdb_val= load_dataset('imdb',
    split="test[500:1000]+test[-1000:-500]")
imdb_train.shape, imdb_test.shape, imdb_val.shape
```

The following functions are from the previous sections (*Improving performance with data augmentation* and *Adapting the model to the domain*) in this chapter:

```
def compute_metrics(pred):
def tokenize_it(e):
```

As we have optimized the `bert-base-uncased` model, we tokenize the dataset according to it. We are already familiar with the following code:

```
from transformers import (BertTokenizerFast,
        BertForSequenceClassification)
model_path= 'bert-base-uncased'
tokenizer = BertTokenizerFast.from_pretrained(model_path)

enc_train=imdb_train.map(tokenize_it,
    batched=True, batch_size=1000)
enc_test=imdb_test.map(tokenize_it,
    batched=True, batch_size=1000)
enc_val=imdb_val.map(tokenize_it,
    batched=True, batch_size=1000)
```

We can optimize many hyperparameters. The most important hyperparameters for transformers are as follows:

- Learning rate
- The number of epochs
- Batch size
- Optimization algorithm
- Scheduler type

For the sake of simplicity, we will only take the *learning rate* value and *batch size* as example parameters to demonstrate an HPO application.

We will use a `trial` object to create our hyperparameter sampling. We can utilize the following Optuna features:

- **Categorical**: `optuna.trial.Trial.suggest_categorical()`
- **Integer**: `optuna.trial.Trial.suggest_int()`
- **Floating point**: `optuna.trial.Trial.suggest_float()`

We will search `learning-rate` in the range `[1e-6 – 1e-4]` from the log domain, which means the possible values are `1e-6`, `1e-5`, and `1e-4`. If the log value selected is `false` (default), as it is searched in the linear domain, you will need to specify a step for discretization. For `batch_size`, we provide a categorical list rather than a range, as follows:

```
def hp_space(trial):
    hp={"learning_rate":
        trial.suggest_float("learning_rate",
            1e-6, 1e-4,
            log=True),
    "per_device_train_batch_size":
        trial.suggest_categorical(
        "per_device_train_batch_size",
        [8, 16]),
    }
    return hp
```

At the end of the code, the returning object, `hp`, is a type of dictionary for search space. Our `Trainer` object will use it for the optimization.

From now on, the following code snippets will be exactly the same as in the previous sections (*Improving performance with data augmentation* and *Adapting the model to the domain*):

```
training_args = TrainingArgument(...
trainer = Trainer(...
```

We will only run the `trainer` call differently. Instead of calling `trainer.train()`, we call `trainer.hyperparameter_search()`, as follows:

```
best_run= trainer.hyperparameter_search(
    direction="maximize",
    hp_space=hp_space)
```

With the `direction` parameter, we target to maximize the `compute_metric()` function. When you run the following code, you will see that a fine-tuning process is repeated several times. In the end, Optuna will give us the optimized parameters. When you repeat the experiment by adding more

parameters to be optimized and increasing the search space, the entire process will take a bit longer. Let's see what Optuna optimizes by checking `best_run` as follows:

```
best_run
BestRun(run_id='11', objective=3.63,
    hyperparameters={
        'learning_rate': 2.25e-05,
        'per_device_train_batch_size': 16})
```

The optimized learning rate and the batch size are `2.25e-05` and `16`, respectively. Recall that we had selected the default value for the learning rate, which is `5e-5`, and the batch size of `16` in another experiment. Let's compare them in terms of task performance. When we train the model with the default parameters and optimized one, we will get the following performance tables.

Without HPO and taking `lr` as `5e-05` by default, we obtain the following output:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.058251	0.9835	0.983500	0.983504	0.9835
val	0.357040	0.9000	0.899974	0.900410	0.9000
test	0.329510	0.9010	0.900964	0.901580	0.9010

Figure 8.7 – Normal BERT fine-tuning

With HPO and taking `lr` as `2.25e-05`, we get the following output:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.086472	0.9765	0.976500	0.976504	0.9765
val	0.288456	0.8970	0.896955	0.897702	0.8970
test	0.281609	0.9030	0.902852	0.905467	0.9030

Figure 8.8 – BERT fine-tuning with HPO

When we look at the test data results, the experiment conducted with HPO is slightly better than vanilla training with a very small difference. All values are slightly better at the level of 0.2 (90.09 F1 to 90.28). The difference is very small and needs to be statistically tested. For example, McNemar's test, t-test, or 5×2 cross-validation can be utilized to compare two machine learning models. However, we kept our HPO experiment very minimal. If we work in a little more detail, we can achieve better results in terms of statistical significance.

Well done! So far, we have tried to improve our models from three different approaches and have been successful in all three. We kept our experiments as simple as possible, but let's not forget that it would be beneficial to work more comprehensively. It would be useful to test and analyze different techniques for data augmentation, improve the quality of data in domain adaptation, experiment with various checkpoints, and expand the search space for HPO.

Summary

In this chapter, we have examined techniques to improve model performance. Our focus has been on data augmentation, domain adaptation, and HPO, all of which contribute to an improvement in our classification task. However, we are not limited to these methods alone. There are several approaches to improving model performance that can be found in the literature. In this chapter, we have only covered the most common ones. We have updated entire model parameters when fine-tuning so far.

In the upcoming chapter, we will discuss how to enhance the parameter efficiency of model fine-tuning with special parameter-efficient techniques to reduce time complexity.

9

Parameter Efficient Fine-Tuning

Fine-tuning has become a prevalent modeling paradigm in AI, particularly in transfer learning. All of the experiments in this book up to this chapter have been based on updating all parameters. Therefore, it is more accurate to use the term **full fine-tuning** from now on (also called **full model fine-tuning** or **full parameter fine-tuning**).

In this chapter, we will look at partial fine-tuning strategies. As we consider the continuously increasing parameters of **large language models** (**LLMs**), fine-tuning and the inference of LLMs become prohibitively expensive. Full fine-tuning requires a complete update of all parameters, saving large models separately for each task. Unfortunately, this process is computationally expensive in terms of both memory and running time. For example, **Bidirectional Encoder Representations from Transformers** (**BERT**) has 300 million parameters, T5 has up to 11 billion, GPT has 175 billion, **Pathways Language Model** (**PaLM**) has 540 billion parameters, and so forth. Therefore, we need to think much more about **Parameter-Efficient Fine-Tuning** (**PEFT**).

In this chapter, we will cover the following topics:

- What is PEFT?
- Hands-on PEFT experiments

Technical requirements

We will need the following packages:

- `adapter-transformers==3.2.1`
- `transformers==4.29.0`
- `peft==0.3.0`
- `datasets==2.12.0`

Introduction to PEFT

Before we start, let's ask ourselves the following question: In the era of ChatGPT, we know that LLMs can solve many problems without the need for any additional updates or fine-tuning operations; so, do we still need fine-tuning operations?

Yes! We currently use models such as ChatGPT to efficiently solve general problems such as sentiment analysis, **named-entity recognition** (**NER**), and summarization. However, our industry or academia requires very specific **natural language processing** (**NLP**) tasks that are influenced by factors such as culture, domain, time, and geography. Studies (see *Is ChatGPT a General-Purpose Natural Language Processing Task Solver?, Chengwei Qin et al.*) have shown that fine-tuning a model outperforms ChatGPT-like language models. Therefore, it is better to fine-tune the model.

Additionally, it is possible to use relatively smaller models such as BERT or T5 by fine-tuning them for on-premises use within the company. This approach mitigates the risk of information security breaches. In summary, for now, it is necessary to use fine-tuning to meet our NLP needs.

The advantages of PEFT can be summarized as follows:

- When dealing with multiple tasks, it is essential to use PEFT methods. Full fine-tuning produces distinct fine-tuned model parameters for each task, which can be costly when serving models that handle numerous tasks. PEFT makes it easy to switch tasks in deployment.
- Parameter-efficient tuning allows for quick adaptation to new tasks without **catastrophic forgetting**, in addition to parameter savings. Since we only update the task-related parameters without modifying the parameters of the main model, the main models do not undergo considerable forgetting.
- PEFT methods provide us with the chance to manage complicated procedures such as **hyperparameter optimization** when it comes to time and memory complexity. Even with high-performance hardware, performing full fine-tuning is not feasible in resource-intensive applications such as HPO.
- **Multi-task learning** can be applied more easily using PEFT methods. Frameworks such as **AdapterFusion** leverage the knowledge from multiple source tasks to enhance performance on a target task.

In this chapter, we address PEFT methods, run a couple of experiments, and see the difference in terms of model performance and training efficiency. We also address the problem of whether we can achieve the same performance as full fine-tuning by training some of the parameters or other efficient fine-tuning methods. Let us first discuss PEFT approaches.

Understanding Types of PEFT

Several approaches have been developed to democratize LLMs during inference and fine-tuning. Some approaches other than partial fine-tuning are quantizations such as int8, distillation, and sparsification. These approaches reduce memory requirements for inference and fine-tuning. Recently, we have seen a great effort in the modification of models. these efforts, which are called PEFT, aim to keep the number of parameters updated as low as possible while maintaining comparable performance with full training. Although some PEFT studies have shown better results than full fine-tuning, it has been found that these successes can be sensitive to hyperparameters.

There are various PEFT approaches in the literature such as adapters, prompt-tuning, prefix-tuning, $(IA)^3$, BitFit, and **Low-rank Adaptation of Large Language Models** (**LoRA**) to name a few. These approaches have been classified into three main categories in a survey (see *Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning, Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky*):

- Additive methods:
- Selective methods
- Low-rank fine-tuning:

Additive methods

Additive methods add new task-specific neural weights (parameters) to transformers. The weights of the original model are kept frozen and just shared between tasks. Only these added weights are trained during the fine-tuning stage, which leads to < %1 parameter efficiency, which also supports multi-task learning. To reduce the parameters to be updated, bottleneck architecture can be applied in this model family, projecting the original dimension to a lower degree and mapping back to the original again. As the added weights can be shared specifically for tasks, it is an easy-to-use method. `adapter-transformers` is a powerful library for this purpose. We will use it for hands-on study.

Adapters can be roughly classified as **serial** and **parallel**. Serial adapters are the adapter layers added in a sequential manner. Adapter tuning (described in the paper *Parameter-efficient transfer learning for NLP, Neil Houlsby et al., International Conference on Machine Learning, PMLR*) tuning (described in *Prefix-tuning: Optimizing continuous prompts for generation, Xiang Lisa Li and Percy Liang*) and prompt-tuning (described in *The power of scale for parameter-efficient prompt tuning, Brian Lester, Rami Al-Rfou, and Noah Constant*) attach additional tokens to input on the hidden layer of the model in a parallel manner. While prefix-tuning appends a learnable vector to the attention head at each transformer layer, prompt-tuning appends this vector only to the input embedding. Prefix-tuning and prompt-tuning utilize soft-prompting, which means a trainable continuous prompt is added instead of adding a discrete word or token as we do during the prompt engineering phase. With training, we expect the model to estimate the embedding of this soft (or continuous) prompt. The disadvantage of such affixes (prefix, infix, and suffix) reduce the model's usable sequence length.

Selective methods

Approaches in this family do not add a new parameter to language models and update only the selected parameter based on certain methods. Bias-only, **BitFit**, for instance, trains only bias terms on the network while freezing all other parameters. Sparse fine-tuning methods, such as **FISH Mask**, select some parameters of the model based on a formula such as Fisher information in FISH Mask. **SparseAdapter** applies the sparsification method to added adapters instead of the main model. Thus, it is considered both a selective and additive method.

Low-rank fine-tuning

The low-rank structure is very common in the AI field. Many tasks have a certain low-rank structure, which helps to perform various computations rapidly in a low-rank subspace. The most important PEFT method of this family is LoRA (see *LoRA: Low-Rank Adaptation of Large Language Models, Edward J. Hu, et al.*). It performs low-rank fine-tuning by training trainable pairs of rank decomposition matrices for W_q (query) and W_v (value) matrices in the self-attention mechanism. All pretrained model parameters are kept frozen, and only these two W_q (query) and W_v (value) matrices are left trainable with reparameterization. LoRA decomposes them into a product of two low-rank matrices.

This reparameterization process reduces the number of trainable parameters significantly due to its selective and low-rank nature, while also achieving performance levels comparable to full training. The main motivation for this approach is the latency problem for additive adapter models. Adapter-based models raise inference latency problems as they have to process sequentially and LLM relies on hardware parallelism. Since the adapter layer must be computed in addition to the base model, this inevitably introduces additional latency. These serial adapter models are suitable for model fine-tuning, especially on a single GPU.

The authors of AdaLoRA (see *Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning, Qingru Zhang et al.*) proposed a variant of LoRA to enhance the LoRA approach in two ways. Firstly, AdaLoRA fine-tunes LLM by attributing different importance to matrices and pruning redundant singular values. This approach underlines that weight matrices have varying levels of importance across different modules and layers when fine-tuning pretrained models. Secondly, while LoRA applies **SVD** (**Singular Value Decomposition**) to query and value projections only, AdaLoRA applies it to all weight matrices to improve its performance.

Now, it is time to get our hands dirty with some experiments. We will do this in the next section.

Hands-on PEFT experiments

In this section, you may encounter some problems or errors when using the CPU. We suggest you use GPU for training. We will solve two different classification problems using PEFT. Throughout the book, we have mentioned the sentiment analysis problem several times. Now, to diversify our experiments, let's also include the NLI (Natural Language Inference) problem.

We will conduct two experiments as follows:

- Efficiently fine-tuning the BERT model for an IMDb sentiment dataset with `adapter-transformers`
- Efficiently fine-tune FLAN-T5 for an NLI task with the LoRA framework

Fine-tuning a BERT checkpoint with adapter tuning

For the first problem, we will specifically address the IMDb sentiment task that we already fine-tuned with full fine-tuning methods before. This way, we will have the opportunity to compare:

1. Start with installing the necessary packages as follows:

```
!pip install datasets==2.12.0 adapter-transformers==3.2.1
```

   As you can see, we do not install HF's `transformers` packages since `adapter-transformers` is used as a drop-in replacement for HF's `transformers` library.

2. Now, we will import the necessary modules:

```
import torch, os
from torch import cuda
import numpy as np
from transformers import AdapterTrainer
from transformers import (BertTokenizerFast,
    BertForSequenceClassification Trainer)
```

3. `TrainingArgumentsfrom` datasets import `load_dataset`. We will fine-tune a `bert-base-uncased` model:

```
model_path= 'bert-base-uncased'
tokenizer =BertTokenizerFast.from_pretrained(model_path)
```

4. As in the previous hands-on experiment, we load smaller portions from the IMDb dataset: 4K for training, 1K for testing, and 1K for validation:

```
imdb_train= load_dataset('imdb',
   split="train[:2000]+train[-2000:]")
imdb_test= load_dataset('imdb',
   split="test[:500]+test[-500:]")
imdb_val= load_dataset('imdb',
    split="test[500:1000]+test[-1000:-500]")
```

5. We define `tokenize_it()` function to encode the datasets as follows:

```
def tokenize_it(e):
    return tokenizer(e['text'],
```

```
        padding=True,
        truncation=True)
enc_train=imdb_train.map(tokenize_it,
    batched=True, batch_size=1000)
enc_test=imdb_test.map(tokenize_it,
    batched=True, batch_size=1000)
enc_val=imdb_val.map(tokenize_it,
    batched=True, batch_size=1000)
```

Training argument initialization will be the same as in the previous vanilla training. The only different hyperparameter is `learning_rate` set as `2e-4`. Recall that in our other vanilla fine-tuning process, we mostly tended to select a lower `learning_rate` of around `2e-5`. Selecting higher `learning_rate` values can be risky for full fine-tuning as we update entire parameters, which leads to catastrophic forgetting. But now, we are safe as we only update some added parameters and do not touch the entire BERT model. So, you can select a higher `learning_rate` if you want.

6. We define the training arguments as follows:

```
training_args = TrainingArguments(
    "/tmp",
    do_train=True,
    do_eval=True,
    num_train_epochs=3,
    learning_rate=2e-4,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    fp16=True,
    load_best_model_at_end=True
    )
```

7. We will now define the metrics that we want to monitor during training in the `compute()` function. For now, let's keep it simple and monitor the `Accuracy` metric only:

```
def compute_acc(p):
    preds = np.argmax(p.predictions, axis=1)
    acc={"Accuracy": (preds == p.label_ids).mean()}
    return acc
```

8. We load the `bert-base-uncased` checkpoint as follows:

```
from transformers import BertModelWithHeads
model = BertModelWithHeads.from_pretrained(model_path)
```

9. Now, we will define an adapter named `"imdb_sentiment"`. Thus, we will freeze all other original parameters in modeling and only allow the update of the added parameters:

```
model.add_adapter("imdb_sentiment")
```

10. We will add a trainable classification head and associate it with an added adapter, namely `imdb_sentiment`. For this head, we will also specify how many classes it will have:

```
model.add_classification_head("imdb_sentiment",num_labels=2)
```

11. Now, we inform the training process that the added adapter will be trained as in the following code:

```
model.train_adapter("imdb_sentiment")
```

We do this because, in some cases, we may not want to train the added parameters. We may especially need this during the multi-task learning phase.

Let us see the parameter efficiency as follows:

```
trainable_params=model\
    .num_parameters(only_trainable=True)/(2**20)
all_params=model.num_parameters() /2**20
print(f"{all_params=:.2f} M\n"+
f"{trainable_params=:.2f} M\n"+
f"The efficiency ratio is \
    {100*trainable_params/all_params:.2f}%")
```

The output is as follows:

```
all_params=105.83 M
trainable_params=1.42 M
the efficiency ratio = 1.34%
```

As you can see, we have a parameter gain of about `1.34%`.

12. We define `AdapterTrainer` instead of HF's vanilla `Trainer` class. The following code triggers the training process:

```
trainer = AdapterTrainer(
    model=model,
    args=training_args,
    train_dataset=enc_train,
    eval_dataset=enc_val,
    compute_metrics=compute_acc)
trainer.train()
```

The output of the training is as follows:

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	0.259850	0.885000
2	0.321100	0.267514	0.893000
3	0.321100	0.260782	0.903000

Figure 9.1 – Training with adapter-transformers

13. As we can see, we fine-tune the model in 1.34 minutes for 3 epochs. Keep them in mind. Let's look at the overall accuracy performance:

```
import pandas as pd
q=[trainer.evaluate(eval_dataset=data) for data in\
    [enc_train, enc_val, enc_test]]
pd.DataFrame(q, index=["train","val","test"]).iloc[:,:5]
```

The output of the execution is as follows:

	eval_loss	eval_Accuracy
train	0.209338	0.919
val	0.259850	0.885
test	0.235628	0.912

Figure 9.2 – Performance with adapter-transformers

Well, our test accuracy performance is about 0.912.

The results suggest that the model has been fine-tuned quickly and successfully. But is it really fast and successful? We need to look at the results of experiments with vanilla fine-tuning. We have already conducted this IMDb experiment identically in *Chapter 8*. Let's just recall the results of this experiment. The following screenshot shows the outputs generated by the vanilla fine-tuning process for three epochs:

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.296200	0.252173	0.901000	0.900988	0.901194	0.901000
2	0.189700	0.297335	0.897000	0.896809	0.899958	0.897000
3	0.052300	0.422962	0.910000	0.909999	0.910026	0.910000

Figure 9.3 – Training with vanilla Transformers (without adapters)

With vanilla full fine-tuning, the training process takes 8.15 minutes while the fine-tuning of the model takes 1.34 minutes. As we can see, the accuracy performances are comparable. So, we can safely say that PEFT fine-tuning is really fast and successful!

Efficiently fine-tune FLAN-T5 for an NLI task with Lora

NLI involves determining whether a given hypothesis is true (**entailment**), false (**contradiction**), or undetermined (**neutral**) given a corresponding premise.

We will use the SNLI project for the task. The SNLI project (`https://nlp.stanford.edu/projects/snli/`) is a collection of 570,000 sentence pairs annotated with the labels contradiction, entailment, and neutral.

An example for each category is as follows:

`premise`	`hypothesis`	`label`
A man inspects the uniform of a figure in a country.	The man is sleeping.	Contradiction
An older and younger man smiling.	Two men are smiling and laughing at the cats playing on the floor.	Neutral
A soccer game with multiple males playing.	Some men are playing a sport.	Entailment

Table 9.1 – Three examples from the Original SNLI dataset

As seen in the preceding table, we are given `premise` and `hypothesis` and the task is to map the sentence pair to a proper category. We will efficiently train the FLAN-T5 model for this purpose starting with installing the necessary libraries:

1. We start by installing the necessary Python modules:

```
!pip install transformers==4.29.0 peft==0.3.0 datasets==2.12.0
```

2. Let us import the modules:

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
from transformers import (default_data_collator,
get_linear_schedule_with_warmup)
from peft import (get_peft_config, get_peft_model,
    get_peft_model_state_dict,LoraConfig, TaskType)
import torch, os
from torch.utils.data import DataLoader
from tqdm import tqdm
import pandas as pd
```

```
from datasets import (load_dataset,
    Dataset, DatasetDict)
```

3. There are 570K examples in the SNLI dataset. We will work with a 1% sample for faster prototyping. The same code can be used for the entire dataset later.

```
dataset = load_dataset("snli")
snli_sampled=pd.DataFrame(dataset["train"])
snli_sampled=snli_sampled.sample(frac=0.01)
```

4. We will see label distribution of the sampled SNLI dataset as follows:

```
snli_sampled.label.value_counts()
```

The output is as follows:

```
2 1870
1 1825
0 1798
-1 9
Name: label, dtype: int64
```

5. Eliminate unlabeled instances (with the `-1` label):

```
snli_sampled= snli_sampled[snli_sampled.label>-1]
```

6. The label names are as follows:

```
names=dataset["train"].features["label"].names
Names
Output: ['entailment', 'neutral', 'contradiction']
```

7. We will need a dictionary of `id2label`:

```
mapp=dict(enumerate(names))
mapp
OUTPUT:   {0: 'entailment',
1: 'neutral',
2: 'contradiction'}
```

The dataset is ready. However, the current dataset format is suitable for text classification. We could fine-tune a BERT model for the task, but in this experiment, we will treat the task differently and fine-tune a text-to-text model, `flan-t5`, by rephrasing instances. We already introduced `flan-t5` as a generative model in *Chapter 4*. FLAN Framework (`https://github.com/google-research/FLAN`) was able to model 1,800 different instruction fine-tuning tasks simultaneously without changing the model architecture. The framework explored the T5 model's capacity for improving the ability of language models at scale to perform zero-shot tasks based purely on instructions.

As FLAN-T5 is a text-to-text model, we need to convert our current NLP task (`text1, text2-> label`) into a text-to-text instruction format (`text-in->text-out`) by rephrasing it. Therefore, we need to cast both the input and output as text using a template. For this, we will use the following prompt method. You can improve this prompt method with more interesting and alternative expressions and repeat the experiment.

8. Let's run the following rephrasing code and examine the new form of the data:

```
snli_sampled_df= pd.DataFrame(snli_sampled)
snli_sampled_df["text"]= snli_sampled_df\
    .apply(lambda x: "S1:" +x.premise
    +"S2:"+x.hypothesis+
    ". The relation between S1 and S2 is labeled "+
    "as entailment, neutral or contradiction ?",
    axis=1)
snli_sampled_df["label"]=snli_sampled_df\
    .apply(lambda x: f"It is {mapp[x.label]}",
    axis=1)
```

The output looks like the following:

premise	hypothesis	label	text
Two firefighters clad in protective gear are entering a house and one of them is checking his hose system.	Two firefighters are entering a house.	It is entailment	S1:Two firefighters clad in protective gear are entering a house and one of them is checking his hose system. S2:Two firefighters are entering a house.. The relation between S1 and S2 is labeled as entailment, neutral or contradiction ?
Two men work together on a construction project.	Two men are working.	It is entailment	S1:Two men work together on a construction project. S2:Two men are working.. The relation between S1 and S2 is labeled as entailment, neutral or contradiction ?
Three men in uniform walk around town.	Three men rob the residents.	It is contradiction	S1:Three men in uniform walk around town. S2:Three men rob the residents.. The relation between S1 and S2 is labeled as entailment, neutral or contradiction ?

Figure 9.4 – Rephrasing the task as task-instruction of SNLI dataset

9. Now, we have two new `text` and `label` String where `text` includes a rephrased form of `premise` and `hypothesis`. Likewise, `label` is a rephrased form of the original label (`1-> "it is neutral"`). Let us examine the last resulting `text` as follows:

```
S1:Three men in uniform walk around town. S2:Three men rob
the residents. The relation between S1 and S2 is labeled as
entailment, neutral or contradiction ?.
```

The `text` field has both `premise` and `hypothesis`. We named them `S1` and `S2` in order, but we could have named their original names, `premise` and `hypothesis`. You can try it differently. We also inject a question (e.g., `The relation between S1 and S2 I... ?`) to direct the T5 model to the targeted output. To direct the model's output to desired selections, we provided hints (... `is labeled as entailment, neutral,`

`or contradiction ?`) for potential tags that might be outputted by the model because generative language models are prone to producing random words or hallucinations. We try to narrow down the output space.

10. We divide the dataset into two parts: training (70%) and validation (30%) and store them in the `snli_sampled_dict` dictionary object as follows:

```
CUT=snli_sampled_df.shape[0]*7//10
print(f"Training set size is {CUT}")
print(f"Validation set size is \
    {snli_sampled_df.shape[0]-CUT}")
print(f"Total size is {snli_sampled_df.shape[0]}")
snli_sampled_dict= DatasetDict({
    "train":
    Dataset.from_pandas(snli_sampled_df[:CUT])
    "validation":
    Dataset.from_pandas(snli_sampled_df[CUT:])
    })
```

The output is as follows:

```
Training set size is 3847
Validation set size is 1650
Total size is 5497
```

11. The tokenization and encoding parts of our work are slightly different from our text (sentiment) and token classification (NER) problems. Because both the input and the output are natural language, we need to pass both parts through the tokenizer to encode them. The following `preprocess_function()` function performs this operation. We usually apply this process in all text-to-text problems:

```
def preprocess_function(examples):
    inputs = examples["text"]
    targets = examples["label"]
    model_inputs = tokenizer(inputs,
        max_length=max_length,
        padding="max_length",
        truncation=True,
        return_tensors="pt")
    labels = tokenizer(targets,
        max_length=max_target_len,
        padding="max_length",
        truncation=True,
        return_tensors="pt")
    labels = labels["input_ids"]
```

```
    labels[labels == tokenizer.pad_token_id] = -100
    model_inputs["labels"] = labels
    return model_inputs
```

12. Now, it's time to choose the model. Model selection is a critical step because it must be compatible with your hardware resources. So, how do we choose a model based on hardware capacity? We can follow a rough formula for calculation. The formula is so simple that you will only need a GPU support of 10 times the number of parameters of a language model. For example, a BERT base model has 110 million parameters. It will cost you (110Mx10) 1.2 GB GPU. This is why the BERT base model is very comfortable with the latest resources.

    If we use the 3-billion-parameter `google/flan-t5-xl` model, the cost will be 3Bx10 = ~30 GB GPU (one parameter requires a 10-byte GPU). This is a very rough calculation. Likewise, `flan-t5-base` with 250M parameters (~2.5 GB GPU is needed) or `flan-t5-large` with 780M parameters (~8 GB GPU is needed) are the other calculations. As 16 GB GPU has become a widely common resource recently, we can start by training `flan-t5-base` in this experiment. We have commented out the other two bigger alternatives now. As I now have a NVIDIA A100 40 GB GPU, I can train the pipeline with `flan-t5-xlarge` later.

13. Now, we load `tokenizer` as follows:

```
model_name_or_path="google/flan-t5-base" # 250M parameters
#model_name_or_path="google/flan-t5-large" #780M parameters
#model_name_or_path="google/flan-t5-xl" # 3B parameters
tokenizer = AutoTokenizer.from_pretrained(model_name_or_path)
```

14. We will pass the `snli_sample_dict` object through the process function that I have prepared. There is a sequence length issue that needs to be considered carefully here: input and output size. For efficiency, I am trying to keep these values as small as possible but not damage the dataset, so that the model runs quickly. When we examine the dataset, we can see that the safe values for input and output sizes are around 150 and 10, respectively, based on the maximum token count. There is no input or output sequence exceeding these values. We can use the `tokenizer.tokenize()` function to measure it. We skip this calculation now.

15. Let's run the following code:

```
max_length = 150
max_target_len=10
snli_processed = snli_sampled_dict.map(
    preprocess_function,
    batched=True,
    num_proc=1,
    remove_columns=snli_sampled_dict["train"].column_names,
    load_from_cache_file=False,
    )
```

```
train_dataset = snli_processed["train"]
eval_dataset = snli_processed["validation"]
```

16. Let's have a quick look at the encoded dataset:

```
pd.DataFrame(train_dataset).head(3)
```

The output is as follows:

input_ids	attention_mask	labels
[180, 536, 10, 188, 3202, 5119, 3, 9, 4459, 32...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[94, 19, 7163, 1, -100, -100, -100, -100, -100...
[180, 536, 10, 3774, 1725, 5234, 4805, 2100, 5...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[94, 19, 27252, 1, -100, -100, -100, -100, -10...
[180, 536, 10, 188, 1021, 388, 19, 479, 300, 3...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[94, 19, 27252, 1, -100, -100, -100, -100, -10...

Figure 9.5 – A processed dataset with tokenizer

As you can see, the input and label are converted from text (string) to a list of token IDs.

17. Now, we will define `DataLoader` in the following code. When selecting the batch size (32 here), we were generous because we have plenty of space in our GPU capacity. If you receive a memory error with this value, choose a smaller batch size. However, keep in mind that smaller batch size values may affect modeling performance. The recent report suggests that a batch size of 16 or 32 gives the optimum performance. But it is subject to your task:

```
batch_size = 32
train_dataloader = DataLoader(
    train_dataset,
    shuffle=True,
    collate_fn=default_data_collator,
    batch_size=batch_size,
    pin_memory=True)
eval_dataloader = DataLoader(
    eval_dataset,
    collate_fn=default_data_collator,
    batch_size=batch_size,
    pin_memory=True)
```

18. In the following code snippet, we will train our model with LoRA by controlling the `with_peft=True` variable. Then, for comparison, we will perform vanilla training without using LoRA. The main difference between the two trainings is that the `get_peft_model()` method indicates that the T5 model will be trained with LoRA. We arrange the necessary

information regarding configuration with the `LoraConfig` object. Another difference is the `learning_rate` value. In our previous experiment with adapter-transformers, we kept the `learning_rate` higher when applying PEFT. In our current experiment, the `learning_rate` value is kept at `1e-3` with PEFT (Lora), while it is `5e-5` without it. For now, we will run this code with the `with_peft=True` setting. Then, we will set this Boolean value to `False`, which means we apply vanilla training to see the difference between the two:

```
with_peft=True
model = AutoModelForSeq2SeqLM\
    .from_pretrained(model_name_or_path)
    lr=2e-5
if with_peft:
    lr=1e-3
    peft_config = LoraConfig(
        task_type=TaskType.SEQ_2_SEQ_LM,
        inference_mode=False,
        r=8,
        lora_alpha=32,
        lora_dropout=0.1)
    model = get_peft_model(model, peft_config)
    model.print_trainable_parameters()
OUTPUT:
trainable params: 884K || all params: 248M || trainable: 0.35%
```

If the `r` variable in this code controls the degree of the low-rank subspace, the recommended value in the LoRA paper *(Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." arXiv preprint arXiv:2106.09685 (2021))* for this variable is `8`. Accordingly, the number of trainable parameters is 884,736 for `flan-t5-base`. This accounts for 0.35% of the entire model parameters. If you set the `r` variable to `4` and try again, you will see that the number of trainable parameters decreases to 442,368 (0.17%). However, we need to control whether we sacrifice success or not while changing it.

19. We will use a classic nested training and evaluation loop as suggested in the legacy code of the repository (`https://huggingface.co/docs/peft`) as follows:

```
device="cuda"
model = model.to(device)
num_epochs = 3
optimizer = torch.optim.AdamW(model.parameters(), lr=lr)

lr_scheduler = get_linear_schedule_with_warmup(
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=\
        (len(train_dataloader) * num_epochs),)
```

```
import time
st = time.time()
for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for step, batch in enumerate(tqdm(train_dataloader)):
        batch={k:v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        total_loss += loss.detach().float()
        loss.backward()
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
model.eval()
eval_loss = 0
eval_preds = []
for step, batch in enumerate(tqdm(eval_dataloader)):
    batch = {k: v.to(device) for k, v in batch.items()}
    with torch.no_grad():
        outputs = model(**batch)
        loss = outputs.loss
        eval_loss += loss.detach().float()
        eval_preds.extend(
            tokenizer.batch_decode(
            torch.argmax(outputs.logits, -1)\
            .detach().cpu().numpy(),
            skip_special_tokens=True)
        )
eval_loss_avg = eval_loss / len(eval_dataloader)
train_loss_avg = total_loss / len(train_dataloader)
print(f"{epoch=}-> {train_loss_avg=}\t {eval_loss_avg=}")
et = time.time()
elapsed_time = et - st
```

20. Now, we will see the overall performance of PEFT with the following code:

```
zipped=zip(eval_preds,
snli_sampled_dict["validation"]["label"])
q=[real.strip() in pred.strip()\
    for pred,real in zipped]
```

```
print(f"{model_name_or_path=}")
print(f"{num_epochs=}")
print(f"{elapsed_time=:.2f} seconds" \
    + (" with PEFT" if with_peft else " without   PEFT"))
print(f"Accuracy={sum(q)/len(q):.2f}")
```

The output of the code is as follows:

```
model_name_or_path='google/flan-t5-base'
num_epochs = 3
elapsed_time = 90.51 seconds with PEFT
Accuracy = 0.86
```

21. Now, let's run the same code with `with_peft=False`. We will get the following output. As you can see, the training process without LoRA is worse than with LoRA in terms of running time:

    The output is as follows:

```
model_name_or_path='google/flan-t5-base'
num_epochs=3
elapsed_time=117.82 seconds without PEFT
Accuracy=:84
```

When we try scaling the model size, we obtain a result that looks like the following table:

		With LoRA		Without LoRA	
Model	# params	Time (sec.)	Accuracy	Time	Accuracy
`flan-t5-base`	250M	91	86	117	84
`flan-t5-large`	780M	280	90	369	90
`flan-t5-xlarge`	3B	879	92	OOM	OOM

Table 9.2 – Performance comparison of the FLAN-t5 checkpoints

Note

For `flan-t5-xlarge`, an **out-of-memory** (**OOM**) error was encountered with a batch size of 32. Lowering the batch size would resolve this error, but in this case, the comparison would become invalid.

This table tells us two things. First, training models with LoRA show good performance and efficiency in terms of speed without sacrificing accuracy. On the other hand, as we scale the model parameters by using larger models, our training slows down while our model performance begins to improve. As you can see, we obtained accuracy values of `86`, `90`, and `92`, respectively for the `base`, `large`, and `xlarge` models of FLAN. But we could not say the model training with LoRA outperforms vanilla training, at least for now.

1. Now, we will save the model and see the memory used:

```
peft_model_path="my_lora_model"
model.save_pretrained(peft_model_path)
!ls -lh $peft_model_path
total 9.2M
-rw-r--r-- 1 root root 333 May 25 20:22 adapter_config.json
-rw-r--r-- 1 root root 3.5M May 25 20:22 adapter_model.bin
```

> **Note**
>
> LoRA is a smaller model that can be trained more quickly. However, loading the base model and the LoRA model separately may cause delays in inference. To minimize these delays, the PEFT library includes a function called `merge_and_unload()`. This function merges the adapter weights with the base model, making it easier to use the merged model as a standalone model.

2. Merging and saving the model is as follows:

```
model = model.merge_and_unload()
model.save_pretrained("my_lora_model_merged")
```

3. Loading the merged and saved model is pretty straightforward:

```
model = AutoModelForSeq2SeqLM\
    .from_pretrained("my_lora_model_merged",
        load_in_8bit=True)
```

4. As you can see, the adapter model only allocates 3.5M instead of a 500 MB file. Let's load the saved model and infer it as follows:

```
from peft import PeftModel, PeftConfig
config = PeftConfig.from_pretrained(peft_model_path)
model = AutoModelForSeq2SeqLM\
    .from_pretrained(config.base_model_name_or_path)
model = PeftModel.from_pretrained(model,peft_model_path)
model.eval()
```

```
my_text= snli_sampled_dict["validation"]["text"][0]
my_label= snli_sampled_dict["validation"]["label"][0]
print(f"{my_text=}")
print(f"{my_label=}")
```

The output is as follows:

```
My_text: S1:A girl wearing a black jacket and pink boots is
poking in the water of the creek with a stick, from the creek
bank.S2:The girl is poking around in the creek trying to find
something that she lost. The relation between S1 and S2 is
labeled as entailment, neutral or contradiction ?
My_label: It is neutral.
```

5. Let us predict `my_text`:

```
inputs = tokenizer(my_text, return_tensors="pt")
with torch.no_grad():
    outputs = model.generate(
        input_ids=inputs["input_ids"],
        max_new_tokens=10)
print(tokenizer.batch_decode(
    outputs.detach().cpu().numpy(),
    skip_special_tokens=True))
['It is neutral'] # prediction
```

We successfully predicted the case. We will now discuss **quantized LoRA**.

Tuning with QLoRA

With the increasing number of parameters in LLMs, there has been a growing push to find more efficient and memory-friendly methods. One recent study proposed the QLoRA model (see *QLoRA: Efficient fine-tuning of quantized LLMs, Tim Dettmers et al.*), which builds upon the already efficient LoRA model. This approach has gained widespread use in LLMs. QLoRA operates by quantizing the LLM to 4 bits, which significantly reduces the model's memory usage compared to LoRA. The quantized LLM is then fine-tuned using the LoRA approach, which enables the refined model to maintain most of the accuracy of the original LLM while being considerably faster and smaller.

To use QLoRA, simply initialize the model with the following code snippet. The remaining code is identical to the preceding code. Please note that the code requires the `bitsandbytes` module, which can be installed as follows:

```
!pip install bitsandbytes
```

Replace the last two lines of code from *Step 17* with the following code:

```
from transformers import BitsAndBytesConfig
from peft import prepare_model_for_kbit_training
from transformers import AutoModelForSeq2SeqLM

model_name = "google/flan-t5-base"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
model = AutoModelForSeq2SeqLM\
    .from_pretrained(model_name_or_path,
    quantization_config=bnb_config,
    device_map={"":0})
model.gradient_checkpointing_enable()
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, peft_config)
```

Please restart this process from *Step 17* and see how we can get a better running time while maintaining most of the accuracy of the original model and LoRA.

Well done! Congratulations! With the help of PEFT, we have successfully solved two different problems, achieving the same level of success without having to train the entire language model and saving time in the process.

Summary

In this chapter, we discussed how to make the fine-tuning process more effective with PEFT. We covered three different PEFT methods: additive, selective, and low-rank. We utilized the adapter-transformers and HF's PEFT framework for the hands-on experiments. We took text classification and NLI tasks and solved them with two Python PEFT libraries.

Besides their benefits, LLMs place big barriers in front of us such as training, fine-tuning, and inference. How to overcome these barriers is an important field of study. In the future, we will focus on how to handle, control, and utilize LLMs, as well as how to democratize them. In this chapter, we only focused on how to efficiently fine-tune them, but there are many aspects that we can work on!

In the next chapter, we will discuss how to work with LLMs.

References

- Zaken, Elad Ben, Shauli Ravfogel, and Yoav Goldberg. "*Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models.*" arXiv preprint arXiv:2106.10199 (2021).
- Sung, Yi-Lin, Varun Nair, and Colin A. Raffel. "*Training neural networks with fixed sparse masks.*" *Advances in Neural Information Processing Systems* 34 (2021): 24193-24205.
- He, Shwai, et al. "*Sparseadapter: An easy approach for improving the parameter-efficiency of adapters.*" arXiv preprint arXiv:2210.04284 (2022).

Part 3: Advanced Topics

In this part, we delve into a variety of advanced topics that are crucial for the field of NLP. We will explore advanced model architecture designs, including **Large Language Models** (**LLMs**) and efficient transformers, as well as multilingual LMs. Additionally, we will cover other technical subjects such as **Explainable AI** (**XAI**), model tracking, deployment techniques, and serving models in a production environment.

This part has the following chapters:

- *Chapter 10, Large Language Models*
- *Chapter 11, Explainable AI (XAI) for NLP*
- *Chapter 12, Working with Efficient Transformers*
- *Chapter 13, Cross-Lingual and Multilingual Language Modeling*
- *Chapter 14, Serving Transformer Models*
- *Chapter 15, Model Tracking and Monitoring*

10
Large Language Models

The field of **large language models** (**LLMs**) has made significant progress in recent years with the development of models such as **GPT-3 (175B)**, **PaLM (540B)**, **BLOOM (175B), LLaMA(65B)**, **Falcon(180B)**, **Mistral (7B)**, and many others. These models have shown impressive abilities in various natural language tasks. It may be challenging to cover such an important topic in a short book section. We have already covered many aspects of the topic, especially in *Chapter 4*. Moreover, throughout the book, we have discussed the paradigm of neural language models and their training process.

In this chapter, we discuss LLMs and run a couple of experiments. We will also show that it is possible to fine-tune LLMs using a similar process as in the previous chapters, with only minor differences.

We will discuss the following in detail:

- Why large language models?
- Fine-tuning large language models

Technical requirements

We will need the following packages:

- `transformers==4.35.2`
- `accelerate==0.25.0`
- `bitsandbytes== 0.42.0`
- `peft==0.7.1`
- `trl== 0.7.9`

Why large language models?

What exactly defines a "normal" language model when we talk about a large language model? Sometimes people use the terms "language model" and "large language model" interchangeably, but they may actually refer to different things as every LLM is an LM, but not every LM is an LLM.

In the past, we referred to models that calculate the likelihood of the next word in a sequence using n-grams as "language models." Nowadays, "large language models" typically refers to transformer-based neural models with billions of parameters trained on massive datasets such as **ChatGPT** and **LLaMA**. These models are all generative language models. Encoder-only language models with around 100 million parameters, such as BERT, and simple n-gram models can be considered **language models** (**LMs**).

To simplify things, we can say that the term LLM refers to decoder-only models with more than 1 billion parameters and a generative feature. These two features emerged because recent studies have shown that when the models are scaled and trained with large datasets, they perform very well. Specifically, their zero- and few-shot performances improve, and they are able to follow given instructions very effectively with respect to size. Additionally, the generative feature allows us to approach any type of problem within this framework, as the input and output are in a free-text format, which has popularized **prompt engineering**. We can use an LLM to generate numeric predictions or even computer code for various NLP problems.

Current trends in LLMs include **Reinforcement Learning from Human Feedback** (**RLHF**).

We can formalize the steps of the RLHF paradigm as follows:

- **Self-supervised**: Let the model gain knowledge on its own
- **Supervised**: Provide information for the model to learn
- **Human feedback**: Observe the model and correct its behavior

As shown in the following figure (*Figure 10.1*), the initial stage involves training vanilla **pre-training language models** (**PTLMs**). In this stage, we use a specific self-supervised objective function, such as next-word prediction, to enhance the model's language understanding capacity. In the second step, we adapt the model to follow natural language instructions by utilizing instruction-following datasets. Through this process, the language model gains knowledge of various tasks and becomes capable of solving unfamiliar problems.

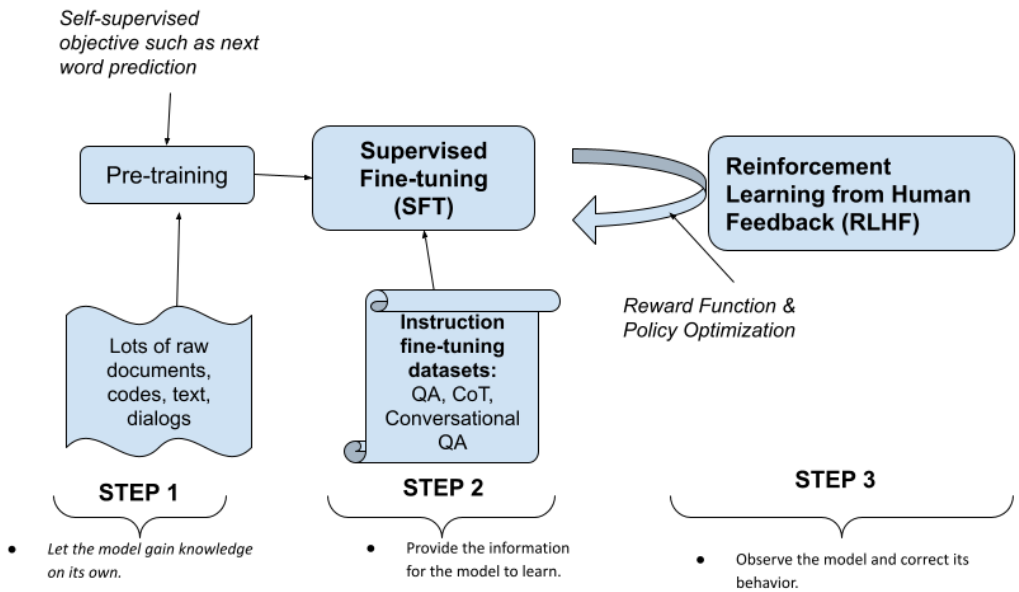


Figure 10.1 – A high-level representation of RLHF

In the final stage (step 3), a distinct feature is introduced compared to the traditional pre-training and fine-tuning two-stage process. This last stage highlights the importance of a **reward function** and **policy optimization** for **reinforcement learning** (**RL**). Human annotators actively participate in experiments with the model and rank its results. These rankings are used to evaluate the text outputs generated by the language model. In the field of RLHF, one of the primary research areas focuses on developing a reward model that accurately represents these human preferences.

Importance of reward function

With the release of ChatGPT, there has been a great deal of excitement both within and beyond the field regarding LLMs. To put it simply, a large language model is essentially just a bigger version of a language model, as the name suggests. However, the exact size of "large" is still unclear. To give you an idea of scale, models with fewer than billions of parameters are not considered large today. It's important to note that this notion of scale is completely relative. A billion-parameter model may be considered large now, but in a few years or even months, it may no longer hold true.

Not only the scale but also other metrics affect the quality of these models. The quality, in this case, is not quantified as numbers reported in scientific articles because there is currently no reliable measurement for understanding the quality of the output. However, we can still find ways to measure it. Let's consider a scenario where you want an **Open Book Question Answering** (**OBQA**) model to be able to answer questions related to a specific context. In this case, you may expect the model to provide exact answers directly taken from the respective context. However, for other use cases, you may prefer the answers to be more conversational and delivered in a specific tone of voice rather than being extracted from the context.

Based on the example provided, if you require a conversational OBQA model, you may reject a model with higher accuracy and instead choose one that offers the desired tone of voice and writing style.

In the following example, we see the output of an encoder-based QA model fine-tuned with the **SQuAD** dataset (`https://huggingface.co/datasets/squad`):

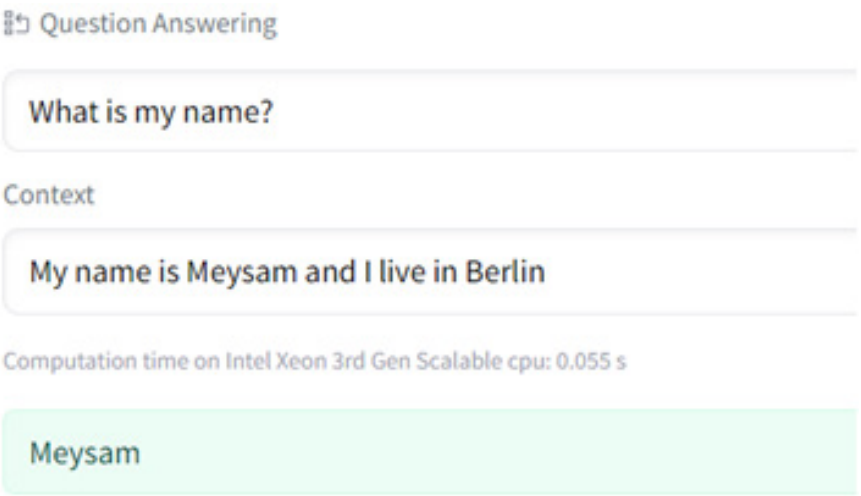


Figure 10.2 – Question answering using https://huggingface.co/distilbert/distilbert-base-cased-distilled-squad

The following example is taken from the ChatGPT interface:

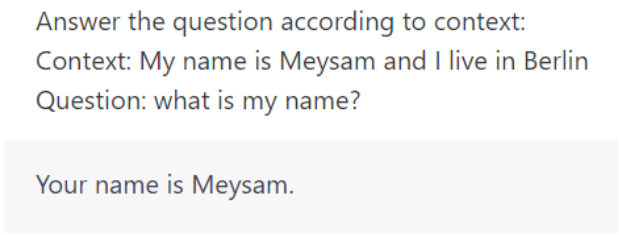


Figure 10.3 – Question answering using ChatGPT

As you can see from the preceding figure, the first model is just a question-answering fine-tuned model but the second one is an LLM that also can understand the instructions you have given to it.

Supervised learning, depicted as step 2 in *Figure 10.1*, has proven to be beneficial in the development of models such as GPT and T5. These models are capable of learning multiple tasks by utilizing prompts. However, a new training scheme, shown in step 3, has been introduced to enhance the models' ability to produce results in a more desired manner. This approach incorporates human feedback to guide the models. Similar to many reinforcement learning approaches, a reward function is utilized to accomplish this task. The reward function plays a crucial role in guiding the model's learning process and enabling it to generate the desired output. In the context of NLP, it is important to model the reward function itself.

A model trained on textual examples determines preferences, which requires training using labeled data. Without a reward model, an LM that solely relies on human-labeled data would be insufficient for training. However, only using the reward model to fine-tune results leads to lower ratings for larger models. An LM can take advantage of this and generate misleading outputs, which highlights the need for an original model to stay aligned and prevent such situations. The best approach is to apply a penalty based on the KL divergence between the original model and the **reinforcement learning** (**RL**) tuned model. However, the story behind LLMs is not only about this. Very large models come with risks; when trained on open web data, they naturally contain biases. For production purposes, it is important to augment these models with another model that can identify harmful, toxic, or abusive content. Otherwise, even with a well-rated and conversational model, there is still a possibility of unintentionally offending individuals.

The instruction-following ability of LLMs

`T0` is a family of open-source models that are vastly trained on many tasks. In the following figure, you can see the related training sets used to train them.

Model	Training datasets
T0	- Multiple-Choice QA: CommonsenseQA, DREAM, QUAIL, QuaRTz, Social IQA, WiQA, Cosmos, QASC, Quarel, SciQ, Wiki Hop - Extractive QA: Adversarial QA, Quoref, DuoRC, ROPES - Closed-Book QA: Hotpot QA*, Wiki QA - Structure-To-Text: Common Gen, Wiki Bio - Sentiment: Amazon, App Reviews, IMDB, Rotten Tomatoes, Yelp - Summarization: CNN Daily Mail, Gigaword, MultiNews, SamSum, XSum - Topic Classification: AG News, DBPedia, TREC - Paraphrase Identification: MRPC, PAWS, QQP
T0p	Same as T0 with additional datasets from GPT-3's evaluation suite: - Multiple-Choice QA: ARC, OpenBook QA, PiQA, RACE, HellaSwag - Extractive QA: SQuAD v2 - Closed-Book QA: Trivia QA, Web Questions
T0pp	Same as T0p with a few additional datasets from SuperGLUE (excluding NLI sets): - BoolQ - COPA - MultiRC - ReCoRD - WiC - WSC
T0_single_prompt	Same as T0 but only one prompt per training dataset
T0_original_task_only	Same as T0 but only original tasks templates
T0_3B	Same as T0 but starting from a T5-LM XL (3B parameters) pre-trained model

Figure 10.4 – T0 training sets from https://huggingface.co/bigscience/T0

As an example, we will try to use `T0_3B`, which is a 3-billion-parameter version of the `T0` series. To demonstrate its capabilities, we will try to examine two different case scenarios: coreference resolution and sentiment analysis.

You need to have good GPU capacity, such as **A100**, to run the following code. First, install the necessary libraries:

```
!pip install transformers sentencepiece
```

Second, we need to load the model and the respective tokenizer:

```
from transformers import(AutoTokenizer,
AutoModelForSeq2SeqLM)
tokenizer = AutoTokenizer.from_pretrained("bigscience/T0_3B")
model = AutoModelForSeq2SeqLM.from_pretrained("bigscience/T0_3B")
```

Afterward, we can use it with the respective instruction prompt for sentiment analysis:

```
inputs = tokenizer.encode("Is this review positive or negative?
Review: This book covers many different ascpect of NLP. I highly
recommend buying it!", return_tensors="pt")
outputs = model.generate(inputs)
print(tokenizer.decode(outputs[0]))
Output: Positive
```

As you can see from the instruction prompt, it is a sentiment analysis task. For coreference resolution, we get the following output:

```
Input: Meysam lives in Germany. He has been living in Berlin for a
long time. In the previous sentence, decide who 'he' is referring to.
Output: Meysam
As you can see from the example, the model is capable of understanding
the instructions that you give it. Natural instructions and
instruction fine-tuning with the help of using different prompt
alternatives (instead of only using the same prompt for training)
broadens the model's capabilities.
```

Well done! Now we will fine-tune an LLM, LLaMA, in the next section.

Fine-tuning large language models

The main purpose of LLMs is to provide zero-shot task generalization with prompt engineering. However, we may still want to fine-tune them for specific tasks. Here, we will fine-tune the most popular open source language model, LLaMA, with the SQuAD dataset. Please note that we can merge different datasets into a single dataset with formatting and train the model. If you want, you can try this in a multi-task experiment. Here, for the sake of simplicity, we are proceeding with a single dataset.

We will use the following techniques for fine-tuning an LLM:

- **PEFT**: As we discussed in *Chapter 9*, we will update fewer parameters in the model instead of the entire model using `LoRA`. The key benefit of **Parameter Efficient Fine-Tuning** (**PEFT**) is its significant reduction in the number of parameters to be trained. This reduction accelerates and simplifies the training process – a valuable attribute for large-scale machine learning projects such as LLM where time and efficiency are vital.
- **Model quantization**: As we discussed in *Chapter 9*, we will apply methods that use less memory, such as using 4-bit variables instead of 32-bit variables in model parameters using `bitsandbytes`. Quantization, a method that reduces bit numbers, optimizes computational and memory costs by using low-precision data types such as 8-bit integers rather than 32-bit floating points. This results in less memory usage, lower energy consumption, and faster operations. It also supports embedded devices' integer data types. Essentially, it's about transitioning from a high-precision to a low-precision data type.
- **TRL**: We will have a simpler, faster fine-tuning experience with the TRL package with efficient memory usage. The TRL library, supported by the Hugging Face team, is a full stack tool for fine-tuning and aligning transformer language and diffusion models using methods such as **Supervised Fine-Tuning** (**SFT**), **Reward Modeling** (**RM**), and **Proximal Policy Optimization** (**PPO**), as well as **Direct Preference Optimization** (**DPO**). Install the necessary packages as follows:

  ```
  !pip install accelerate ==0.25.0 bitsandbytes ==0.42.0   peft
  ==0.7.1 transformers ==4.35.2 trl ==0.7.9
  ```

  And import the functions and classes:

  ```
  import os, torch
  from trl import SFTTrainer
  from peft import LoraConfig, PeftModel
  from datasets import load_dataset
  from transformers import (
      AutoModelForCausalLM,
      AutoTokenizer,
      TrainingArguments,
      BitsAndBytesConfig,
      HfArgumentParser,
      pipeline,
  )
  ```

Building instruction-following datasets is an important step when training an LLM. The quality and variety of data are essential. Having data of various types and fields will enhance the model in terms of multi-task objectives and zero-shot generalization. At this stage, we will perform a training experiment using the SQuAD dataset to maintain simplicity in the process. However, in your own scenarios, you can incorporate different tasks and expand the range of data.

In the following code, we take 1,000 samples for training and 350 samples for evaluation. You can use the entire dataset in your run, but it will take a long time. Let's do the following:

```
import pandas as pd
dataset = load_dataset("squad")
train=pd.DataFrame(dataset["train"].select(range(1000)))
val=pd.DataFrame(dataset["train"].select(range(1000,1350)))
train.iloc[:,2:].head()
```

context	question	answers
Architecturally, the school has a Catholic character. Atop the Main Building's gold dome is a golden statue of the Virgin Mary. Immediately in front of the Main Building and facing it, is a copper statue of Christ with arms upraised with the legend "Venite Ad Me Omnes". Next to the Main Building is the Basilica of the Sacred Heart. Immediately behind the basilica is the Grotto, a Marian place of prayer and reflection. It is a replica of the grotto at Lourdes, France where the Virgin Mary reputedly appeared to Saint Bernadette Soubirous in 1858. At the end of the main drive (and in a direct line that connects through 3 statues and the Gold Dome), is a simple, modern stone statue of Mary.	To whom did the Virgin Mary allegedly appear in 1858 in Lourdes France?	{'text': ['Saint Bernadette Soubirous'], 'answer_start': [515]}
Architecturally, the school has a Catholic character. Atop the Main Building's gold dome is a golden statue of the Virgin Mary. Immediately in front of the Main Building and facing it, is a copper statue of Christ with arms upraised with the legend "Venite Ad Me Omnes". Next to the Main Building is the Basilica of the Sacred Heart.	What is in front of the Notre	{'text': ['a copper statue of Christ'],

Figure 10.5 – SQuAD dataset screenshot

In the output, you can see the presence of context, question, and answer. It is important to remember that during the fine-tuning of the T5 model in the preceding chapters, we transformed the dataset into a text-to-text format. Now, we have just a single text. We must structure the dataset according to the LLaMA template as outlined here:

```
<s>[INST] Context:  ... Question ? [/INST] Answer... </s>
```

We transform every triple (context, question, answer) in a string like `<s> [INST] ... [/INST] ... </s>`. Let us transform the dataset:

```
train["text"]=train.apply(lambda x:
    f"<s>[INST]Context: {x.context} \
        Question: {x.question}[/INST] \
        {x.answers['text'][0]} </s>",
    axis=1)
val["text"]=val.apply(lambda x:
    f"<s>[INST]Context: {x.context} \
        Question: {x.question}[/INST] \
        {x.answers['text'][0]} </s>",
    axis=1)
```

```
from datasets import Dataset
train_dataset=Dataset.from_pandas(train[["text"]])
eval_dataset=Dataset.from_pandas(val[["text"]])
```

Please note that in the preceding code, we have added a new column called `"text"`. We have re-expressed all the content (context, question, answer) in that column, which will be the main focus of the model.

Now we will set up `LoRA`. We previously used `LoRA` in *Chapter 9* to optimize the T5 model. It is a technique that focuses on reducing the size of models, enabling calculations on less memory. We set it as follows:

```
peft_config = LoraConfig(
    lora_alpha=16,
    lora_dropout=0.1,
    r=64,
    bias="none",
    task_type="CAUSAL_LM",
)
```

Quantization is a technique that reduces numerical precision in models. Instead of using high-precision data types such as 32-bit floating-point numbers, quantization uses lower-precision data types such as 8-bit or 4-bit integers. This reduces memory usage and speeds up model execution while maintaining accuracy. We set it as follows:

```
compute_dtype = getattr(torch, "float16")
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type=»nf4»,
    bnb_4bit_compute_dtype=compute_dtype,
    bnb_4bit_use_double_quant=False,
)
```

Now we get the LLaMA model. To access the LLaMA model, we follow these steps:

1. LLaMa is a gated model, which means that access needs to be granted before it can be used. To request access, please follow this link: `https://huggingface.co/meta-llama/Llama-2-7b-chat-hf`.
2. Once access has been granted, you will need to authenticate with the Hugging Face Hub. To do this, create a token in your **Settings** tab (`https://huggingface.co/settings/tokens`) or navigate to **Hugging Face Settings** > **Access Tokens** > **New token** and generate a new read token. Make sure to copy this access token as instructed.

You will encounter the following screen:

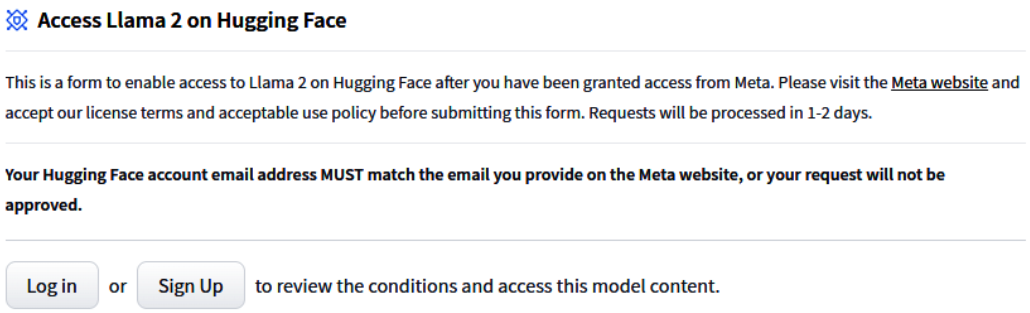


Figure 10.6 – Access LLaMA 2 on Hugging Face

We pass our `read` token to get the permission as follows:

```
from huggingface_hub import login
access_token_read = "hf_BLA_BLA" #<-paste your token here
login(token = access_token_read)
```

3. Now it is time to load the LLaMA model. We load a particular LLaMA checkpoint, `meta-llama/Llama-2-7b-chat-hf`, and its tokenizer as follows:

```
# LLaMA model
model_name="meta-llama/Llama-2-7b-chat-hf"
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map={"": 0}
)
model.config.use_cache = False
model.config.pretraining_tp = 1
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name,
    trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
```

4. We will use the `SFTTRainer` class of the `trl` library, which is a very talented wrapper function. We just pass the PEFT and quantization objects that we have defined to this trainer. We also tell the trainer to focus on the `text` column and not something else:

```
training_arguments = TrainingArguments(
    output_dir="my_llama",
    num_train_epochs=3,
```

```
    per_device_train_batch_size=4,
    gradient_accumulation_steps=1,
    optim="paged_adamw_32bit",
    evaluation_strategy="epoch",
    learning_rate=2e-4,
    weight_decay=0.001,
    fp16=True,
    bf16=True, # if you have A100 resource, you can set it
    lr_scheduler_type="linear"
)
from trl import SFTTrainer
trainer = SFTTrainer(
    model=model,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    max_seq_length=None,
    tokenizer=tokenizer,
    args=training_arguments)
```

5. Now we start the training process as follows:

```
trainer.train()
```

Great job! You may see that the loss keeps increasing, which is not a good sign for training. This is because we are using a small portion of the dataset. When we use the entire dataset, we may see a better training process. We have just fine-tuned the LLaMA model.

Now let's try the model with the inference code that follows. We pass the context and question as follows:

```
prompt='''
Context:Architecturally, the school has a Catholic character. Atop the
Main Building's gold dome is a golden statue of the Virgin Mary.
Immediately in front of the Main Building and facing it, is a copper
statue of Christ with arms upraised with the legend "Venite Ad Me
Omnes".
Next to the Main Building is the Basilica of the Sacred Heart.
Immediately behind the basilica is the Grotto, a Marian place of
prayer and reflection.
It is a replica of the grotto at Lourdes, France where the Virgin Mary
reputedly appeared to Saint Bernadette Soubirous in 1858.
At the end of the main drive (and in a direct line that connects
through 3 statues and the Gold Dome),
is a simple, modern stone statue of Mary.
Question:To whom did the Virgin Mary allegedly appear in 1858 in
Lourdes France?
```

```
'''
# Expected answer:  Saint Bernadette Soubirous
squad_llama = pipeline(task="text-generation",
                       model=model,
                       tokenizer=tokenizer,
                       max_length=350)
result = squad_llama(f"<s>[INST] {prompt} [/INST]")
print(result[0]['generated_text'])
Output: Saint Bernadette Soubirous
```

Perfect! We have got what we expected (`Saint Bernadette Soubirous`). It is just a straightforward test with a basic example. We need to assess LLM using effective evaluation techniques. Currently, our main focus is not on evaluating LLM, as it is beyond the scope of this chapter.

Summary

In this chapter, we discussed the concept of LLMs. We explored how models such as T5 can generate diverse responses when given different prompts. Additionally, we successfully trained LLaMA, an open-source language model, using the PEFT and quantization techniques. Although we didn't extensively delve into LLMs in this chapter, we touched upon them to some extent. Looking ahead, the next chapter will delve into explainable artificial intelligence, specifically from the perspective of natural language processing.

11
Explainable AI (XAI) in NLP

The increasing availability of **large language models** (**LLMs**) has brought the trade-off between the accuracy and interpretability of a model's output to the forefront. The biggest challenge in **explainable artificial intelligence** (**XAI**) studies is dealing with the large number of layers and parameters present in deep neural models. So, how are we going to solve this problem? Will we be able to find a way to understand how a deep model makes a decision? The simple answer is no, but the other answer is somewhat.

In this chapter, we will only approach this problem in terms of Transformers. It will be examined from two perspectives. First of all, the self-attention mechanism, which is the most relevant part of the Transformer architecture regarding explainability, will be examined. This mechanism can make how the Transformer model processes an input understandable to humans. Various attention visualization tools will be used for this purpose by providing important functions for interpretability and explainability. First, we will discuss how to visualize the inner parts of attention and will interpret the learned representations, which help us understand the information encoded by self-attention heads in the Transformer. The most interesting characteristic of attention is that certain heads correspond to a certain aspect of syntax or semantics. Secondly, we will try to explain how a transformer model makes a decision with two important approaches, **LIME** and **SHapley Additive exPlanations** (**SHAP**).

Briefly, we will cover the following topics in this chapter:

- Interpreting attention heads
- Explaining a decision by the LIME approach
- Explaining a decision by the SHAP approach

Technical requirements

The code for this chapter is found at `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`, which is the GitHub repository for this book. We will be using Jupyter Notebook to run our coding exercises that require `Python 3.6.0` or above, and the following packages will need to be installed:

- `tensorflow`
- `pytorch`
- `transformers >=4.00`
- `bertviz`
- `ipywidgets`
- `lime`
- `shape`

Interpreting attention heads

As with most **deep learning** (**DL**) architectures, both the success of Transformer models and how they learn have been not fully understood, but we know that Transformers—remarkably—learn many linguistic features of the language. A significant amount of learned linguistic knowledge is distributed both in the hidden state and in the self-attention heads of the pretrained model. Recent substantial studies have been published and many tools have been developed to understand and better explain the phenomena.

Thanks to some **natural language processing** (**NLP**) community tools, we can interpret the information learned by the self-attention heads in a Transformer model. The heads can be interpreted naturally, thanks to the weights between tokens. We will soon see that in further experiments in this section, certain heads correspond to a certain aspect of syntax or semantics. We can also observe surface-level patterns and many other linguistic features.

In the following subsections, we will conduct some experiments using community tools to observe these patterns and features in the attention heads. Recent studies have already revealed many of the features of self-attention. Let's highlight some of them before we get into the experiments. For example, most of the heads attend to delimiter tokens such as **separator** (**SEP**) and **classification** (**CLS**), since these tokens are never masked out and bear segment-level information. Another observation is that most heads pay little attention to the current token, but some heads specialize in only attending to the next or previous tokens, especially in earlier layers.

Here is a list of other patterns found in recent studies that we can easily observe in our experiments:

- Attention heads in the same layer show similar behavior
- Particular heads correspond to specific aspects of syntax or semantic relations
- Some heads encode so that the direct objects tend to attend to their verbs, such as `<lesson, take>` or `<car, drive>`
- In some heads, the noun modifiers attend to their noun (for example, the hot water; the next layer), or the possessive pronoun attends to the head (for example, her car)
- Some heads encode so that passive auxiliary verbs attend to a related verb, such as `Been damaged` and `was taken`
- In some heads, coreferential mentions attend to themselves, such as `talks-negotiation`, `she-her`, and `President-Biden`
- The lower layers usually have information about word positions
- Syntactic features are observed earlier in the transformer, while high-level semantic information appears in the upper layers
- The final layers are the most task-specific and are therefore very effective for downstream tasks

To observe these patterns, we can use two important tools, **exBERT** and **BertViz**, here. These tools have almost the same functionality. We will start with exBERT.

Visualizing attention heads with exBERT

The exBERT is a visualization tool to see the inner parts of Transformers. We will use it to visualize the attention heads of the BERT-base-cased model, which is the default model in the exBERT interface. Unless otherwise stated, the model we will use in the following examples is BERT-base-cased. This contains 12 layers and 12 self-attention heads in each layer, which makes for 144 self-attention heads.

We will learn how to utilize exBERT step by step, as follows:

1. Let's click on the exBERT link hosted by Hugging Face: `https://huggingface.co/exbert`.
2. Enter the sentence `The cat is very sad .` and see the output, as follows:

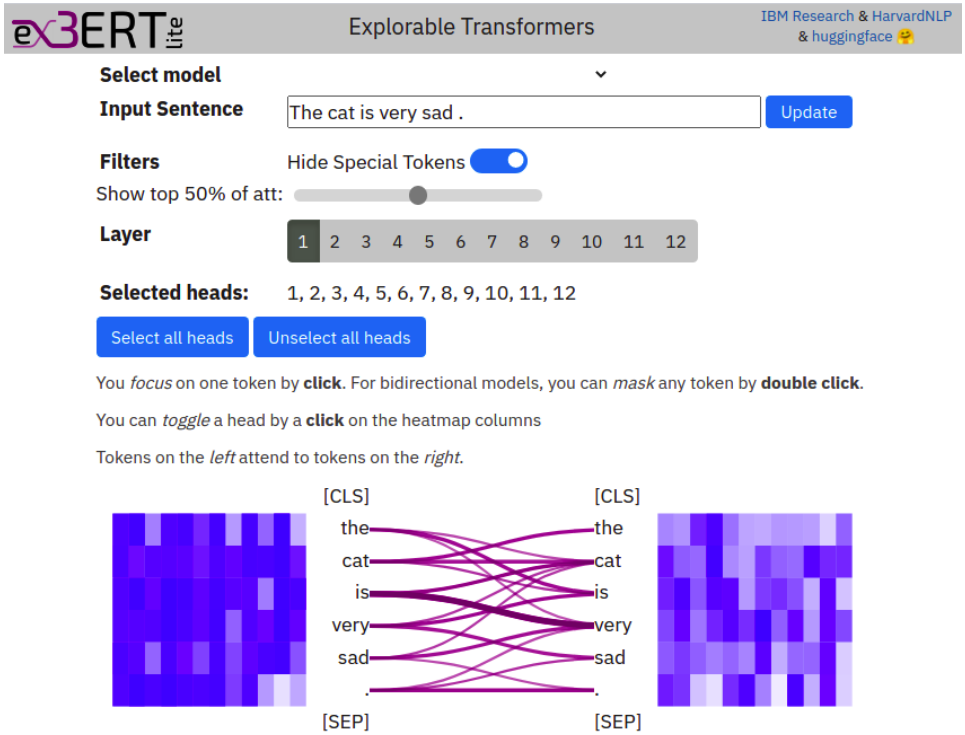


Figure 11.1 – exBERT interface

In the preceding screenshot, the left tokens attend to the right tokens. The thickness of the lines represents the value of the weights. Because CLS and SEP tokens have very frequent and dense connections, we cut off the links associated with them for simplicity. Please see the **Hide Special Tokens** toggle switch. What we see now is the attention mapping at layer **1**, where the lines correspond to the sum of the weights on all heads. This is called a **multi-head attention mechanism**, in which 12 heads work in parallel to each other. This mechanism allows us to capture a wider range of relationships than is possible with single-head attention. This is why we see a broadly attending pattern in *Figure 11.1*. We can also observe any specific head by clicking the `Head` column.

If you hover over a token on the left, you will see the specific weights of that token connecting to the right ones. For more detailed information on using the interface, read the paper *exBERT: A Visual Analysis Tool to Explore Learned Representations in Transformer Models, Benjamin Hoover, Hendrik Strobelt, Sebastian Gehrmann, 2019*, or watch the video at the following link: `https://github.com/bhoov/exbert`.

3. Now, we will try to support the findings of other researchers addressed in the introductory part of this section. Let's take some heads specialized in only attending the next or previous tokens, especially in earlier layers patterns, and see whether there's a head that supports this.

4. We will use the `<Layer-No, Head-No>` notation to denote a certain self-attention head for the rest of the chapter, where the indices start at `1` for exBERT and start at `0` for BertViz—for example, `<3,7>` denotes the seventh head at the third layer for exBERT. When you select the `<2,5>` (or `<4,12>` or `<6,2>`) head, you will get the following output, where each token attends to the previous token only:

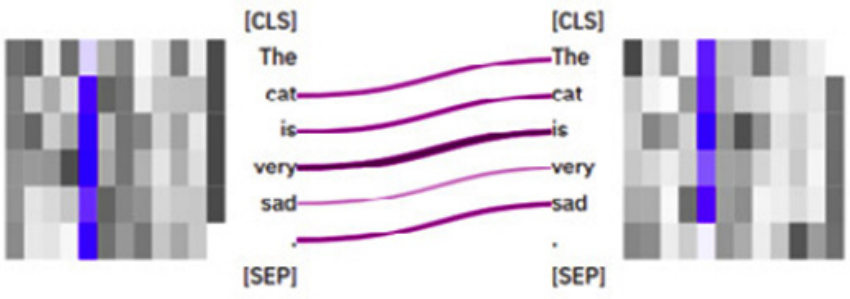


Figure 11.2 – Previous-token attention pattern

5. For the `<2,12>` and `<3,4>` heads, you will get a pattern whereby each token attends to the next token, as follows:

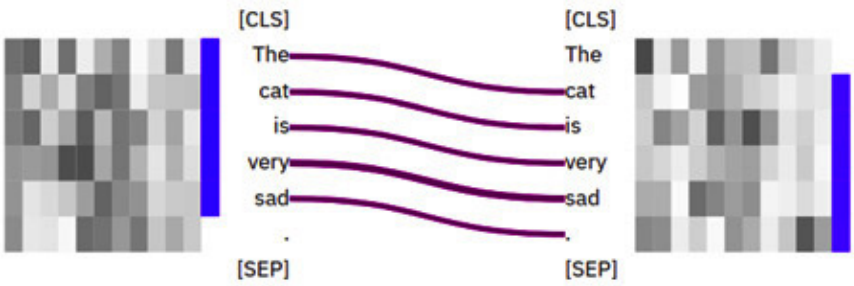


Figure 11.3 – Next-token attention pattern

These heads serve the same functionality for other input sentences—that is, they work independently of the input. You can try different sentences yourself.

We can use an attention head for advanced semantic tasks such as pronoun resolution using a probing classifier. First, we will qualitatively check whether the internal representation has such a capacity for pronoun resolution (or coreference resolution) or not. Pronoun resolution is considered a challenging semantic relation task since the distance between the pronoun and its antecedent is usually very long.

6. Now, we take the sentence, "*The cat is very sad, because it could not find food to eat.*" When you check each head, you will notice that the `<9,9>` and `<9,12>` heads encode the pronoun relation. When hovering over it at the `<9,9>` head, we get the following output:

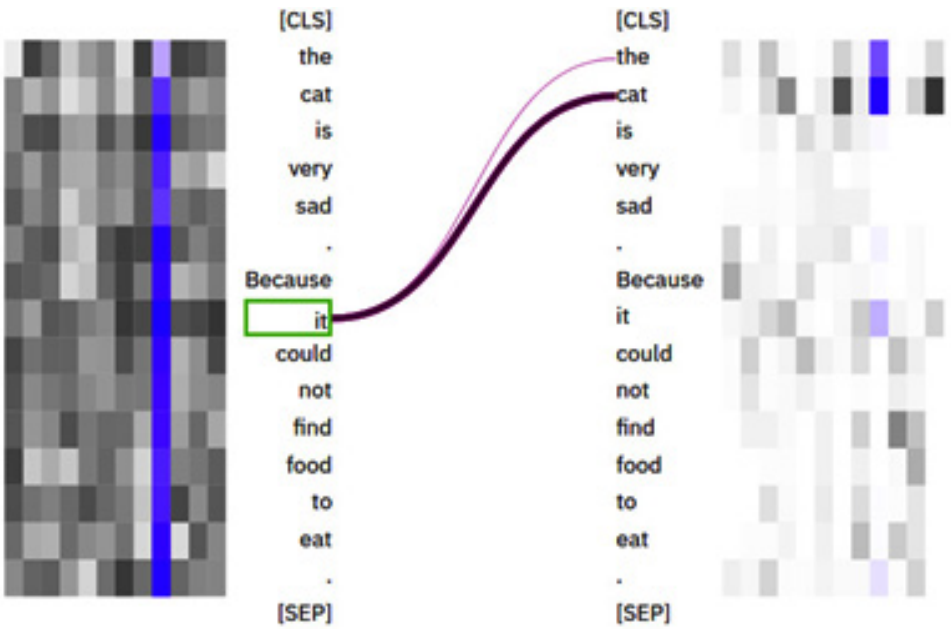


Figure 11.4 – The coreference pattern at the <9,9> head

The `<9,12>` head also works for pronoun relation. Again, on hovering over it, we get the following output:

Figure 11.5 – The coreference pattern at the <9,12> head

From the preceding screenshot, we see that the `it` pronoun strongly attends to its antecedent, `cat`. We change the sentence a bit so that the `it` pronoun now refers to the `food` token instead of the `cat` token, as in, "The cat did not eat the food because it was not fresh." As seen in the following screenshot, which relates to the `<9,9>` head, it properly attends to its antecedent, `food`, as expected:

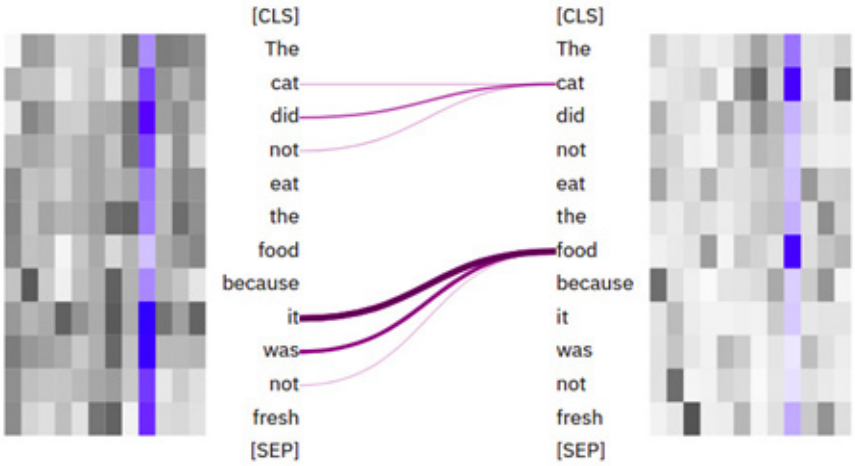


Figure 11.6 – The pattern at the <9,9> head for the second example

7. Let's look at another run, where the pronoun refers to the `cat` token, as in "The cat did not eat the food because it was very angry." In the `<9,9>` head, the `it` token mostly attends to the `cat` token, as shown in the following screenshot:

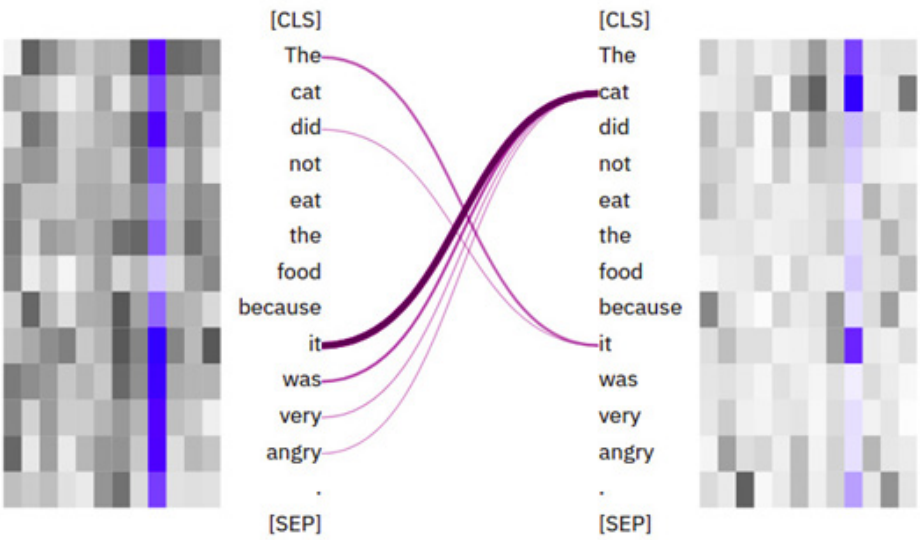


Figure 11.7 – The pattern at the <9,9> head for the second input

8. I think those are enough examples. Now, we will use the exBERT model differently to evaluate the model capacity. Let's restart the exBERT interface, select the last layer (layer **12**), and keep all heads. Then, enter the sentence `The cat did not eat the food.` and mask out the `food` token. Double-clicking masks the `food` token out, as follows:

Figure 11.8 – Evaluating the model by masking

When you hover over that masked token, you can see the prediction distribution of the BERT-base-cased model, as shown in the preceding screenshot. The first prediction is `food`, which is expected. Nowadays, we exploit such features of the BERT model for data augmentation. For instance, you can replace `food` with `meat` as a second prediction and you will have a new instance similar to the original. For more detailed information about the tool, you can refer to exBERT's web page: `https://exbert.net/`.

Well done! In the next section, we will work with BertViz and write some Python code to access the attention heads.

Multiscale visualization of attention heads with BertViz

Now, we will write some code to visualize heads with BertViz, which is a tool used to visualize attention in the Transformer model, as is exBERT. It was developed by Jesse Vig in 2019 (see *A Multiscale Visualization of Attention in the Transformer Model, Jesse Vig, 2019*). It is the extension of the work of the Tensor2Tensor visualization tool (Jones, 2017). We can monitor the inner parts of a model with multiscale qualitative analysis. The advantage of BertViz is that we can work with most Hugging-Face-hosted models (such as BERT, GPT, and XLM through the Python API). Therefore, we will be able to work with non-English models as well, or any pretrained model. You can access BertViz resources and other information from the following GitHub link: `https://github.com/jessevig/bertviz`.

Like exBERT, BertViz visualizes attention heads in a single interface. Additionally, it supports a bird's-eye view and a low-level neuron view, where we observe how individual neurons interact to build attention weights. A useful demonstration video can be found at the following link: `https://vimeo.com/340841955`.

Before starting, we need to install the necessary libraries, as follows:

```
!pip install bertviz ipywidgets transformers
```

We then import the following modules:

```
from bertviz import head_view
from transformers import BertTokenizer, BertModel
```

BertViz supports three views: a **head view**, a **model view**, and a **neuron view**. Let's examine these views one by one. First of all, though, it is important to point out that we started from `1` for index layers and heads in exBERT. But in BertViz, we start from `0` for indexing, as in Python programming. If I say a `<9,9>` head in exBERT, its BertViz counterpart is `<8,8>`.

Let's start with the head view.

Attention head view

The head view is the BertViz equivalent of what we have experienced so far with exBERT in the previous section. The attention head view visualizes the attention patterns based on one or more attention heads in a selected layer:

First, we define a `get_bert_attentions()` function to retrieve attentions and tokens for a given model and a given pair of sentences. The function definition is shown in the following code block:

```
def get_bert_attentions(model_path,sentence_a,sentence_b):
    model = BertModel.from_pretrained(model_path,
        output_attentions=True)
    tokenizer = BertTokenizer.from_pretrained(model_path)
    inputs = tokenizer.encode_plus(sentence_a,
        sentence_b,
        return_tensors='pt',
        add_special_tokens=True)
    token_type_ids = inputs['token_type_ids']
    input_ids = inputs['input_ids']
    attention = model(input_ids,
        token_type_ids=token_type_ids)[-1]
    input_id_list = input_ids[0].tolist()
    tokens = tokenizer.convert_ids_to_tokens(input_id_list)
    return attention, tokens
```

In the following code snippet, we load the `bert-base-cased` model and retrieve the tokens and corresponding attentions of the given two sentences. We then call the `head_view()` function at the end to visualize the attentions. Here is the code execution:

```
model_path = 'bert-base-cased'
sentence_a = "The cat is very sad."
sentence_b = "Because it could not find food to eat."
attention, tokens=get_bert_attentions(model_path,
    sentence_a, sentence_b)
head_view(attention, tokens)
```

The code output is an interface, as displayed here:

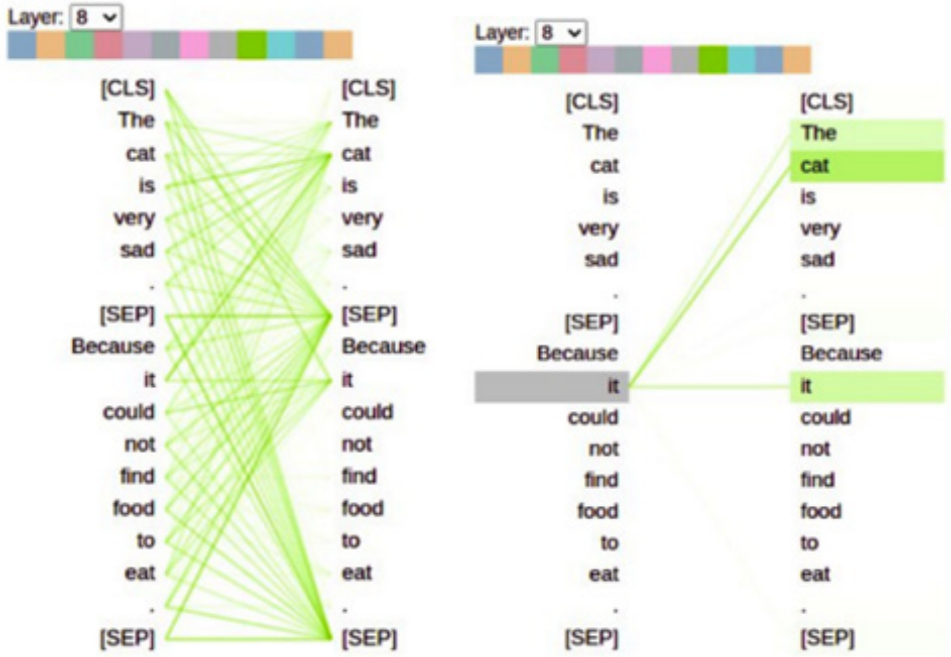


Figure 11.9 – Head-view output of BertViz

The interface on the left of *Figure 11.9* comes first. Hovering over any token on the left will show the attention going from that token. The colored tiles at the top correspond to the attention head. Double-clicking on any of them will make a selection and discard the rest. The thicker attention lines denote higher attention weights.

Please remember that in the preceding exBERT examples, we observed that the `<9,9>` head (the equivalent in BertViz is `<8,8>`, due to indexing) bears a pronoun-antecedent relationship. We observe the same pattern in *Figure 11.9*, selecting **Layer 8** and head 8. Then, we see the interface on the right of *Figure 11.9* when we hover over it, and we will see that it strongly attends to the `cat` and `it` tokens

(itself). So, can we observe these semantic patterns in other pretrained language models? Although the heads are not exactly the same in other models, some heads can encode these semantic properties. We also know from recent works that semantic features are mostly encoded in the higher layers.

Let's look for a coreference pattern in a Turkish language model. The following code loads a Turkish `bert-base-cased` model and takes a sentence pair. We observe here that the `<8,8>` head has the same semantic feature in Turkish as in the English model, as follows:

```
model_path = 'dbmdz/bert-base-turkish-cased'
sentence_a = "Kedi çok üzgün."
sentence_b= "Çünkü o her zamanki gibi çok fazla yemek yedi."
attention, tokens= get_bert_attentions(model_path,
    sentence_a, sentence_b)
head_view(attention, tokens)
```

From the preceding code, `sentence_a` and `sentence_b` mean "The cat is sad" and "Because it ate too much food" as usual, respectively. When hovering over `o` (it), it attends to `Kedi` (cat), as follows:

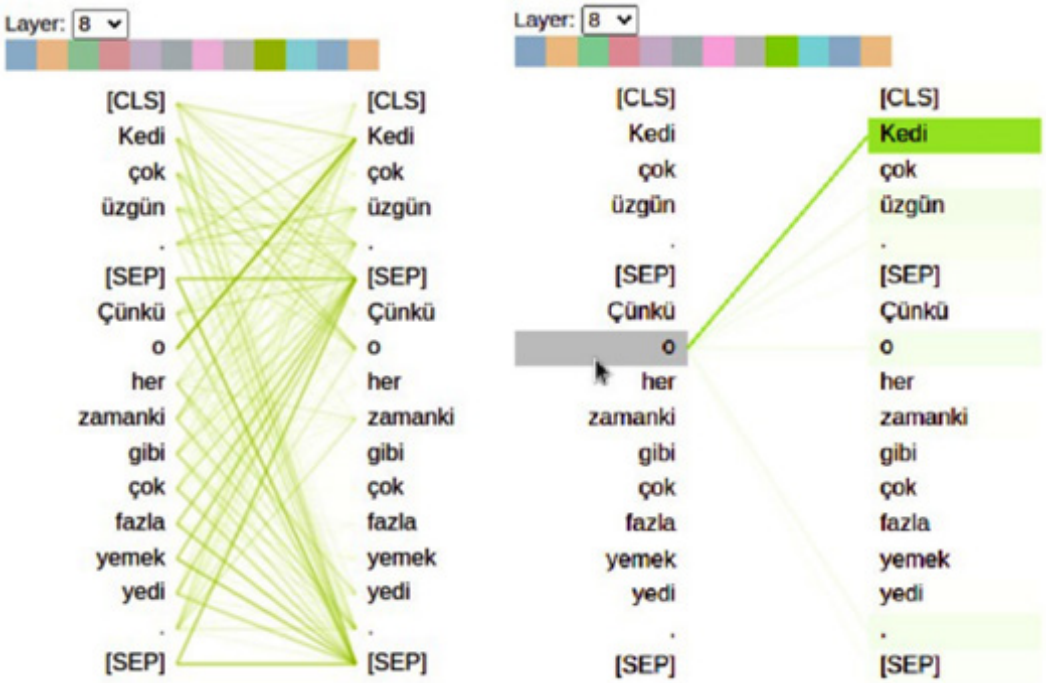


Figure 11.10 – Coreference pattern in the Turkish language model

All other tokens except `o` (it) mostly attend to the SEP delimiter token, which is a dominant behavior pattern in all heads in the BERT architecture.

As a final example for the head view, we will interpret another language model and move on to the model view feature. This time, we choose the `bert-base-german-cased` German language model and visualize it for the input—that is, the German equivalent of the same-sentence pair we used for Turkish.

The following code loads a German model, consumes a pair of sentences, and visualizes them:

```
model_path = 'bert-base-german-cased'
sentence_a = "Die Katze ist sehr traurig."
sentence_b = "Weil sie zu viel gegessen hat"
attention, tokens= get_bert_attentions(model_path,
    sentence_a, sentence_b)
head_view(attention, tokens)
```

When we examine the heads, we can see the coreference pattern in the 8th layer again, but this time in the 11th head. To select the `<8,11>` head, pick layer **8** from the drop-down menu and double-click on the last head, as follows:

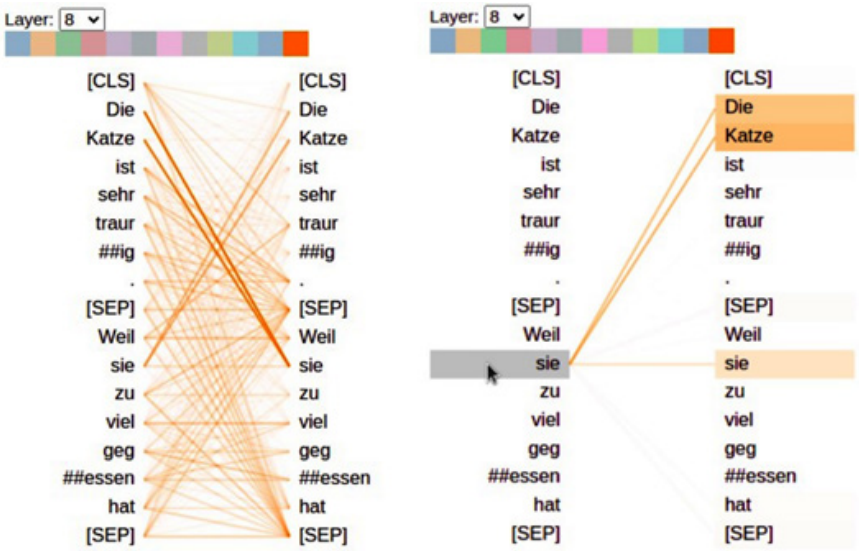


Figure 11.11 – Coreference relation pattern in the German language model

As you see, when hovering over `sie`, you will see strong attentions to `Die Katze`. While this `<8,11>` head is the strongest one for coreference relations (known as **anaphoric relations** in computational linguistics literature), this relationship may have spread to many other heads. To observe it, we will have to check all the heads one by one.

On the other hand, BertViz's model view feature gives us a basic bird's-eye view to see all heads at once. Let's take a look at it in the next section.

Model view

Model view allows us to have a bird's-eye view of attentions across all heads and layers. Self-attention heads are shown in tabular form, with rows and columns corresponding to layers and heads, respectively. Each head is visualized in the form of a clickable thumbnail that includes the broad shape of the attention model.

The view can tell us how BERT works and makes it easier to interpret. Many recent studies, such as *A Primer in BERTology: What We Know About How BERT Works, Anna Rogers, Olga Kovaleva, Anna Rumshisky*, found some clues about the behavior of the layers and came to a conclusion. We already listed some of them in the *Interpreting attention heads* section. You can test these facts yourself using BertViz's model view.

Let's view the German language model that we just used, as follows:

1. First, import the following modules:

```
from bertviz import model_view
from Transformers import BertTokenizer, BertModel
```

2. Now, we will use a `show_model_view()` wrapper function developed by Jesse Vig. You can find the original code at the following link: `https://github.com/jessevig/bertviz/blob/master/notebooks/model_view_bert.ipynb`.

   `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`. We are just dropping the function header here:

```
def show_model_view(model, tokenizer, sentence_a,
    sentence_b=None, hide_delimiter_attn=False,
    display_mode="dark"):
. . .
```

3. Let's load the German model again. If you have already loaded it, you can skip the first five lines. Here is the code you'll need:

```
model_path='bert-base-german-cased'
sentence_a = "Die Katze ist sehr traurig."
sentence_b = "Weil sie zu viel gegessen hat"
model = BertModel.from_pretrained(model_path,
    output_attentions=True)
tokenizer = BertTokenizer.from_pretrained(model_path)
show_model_view(model, tokenizer, sentence_a,
    sentence_b,
    hide_delimiter_attn=False,
    display_mode="light")
```

This is the output:

Figure 11.12 – The model view of the German language model

This view helps us easily observe many patterns such as next-token (or previous-token) attention patterns. As we mentioned earlier in the *Interpreting attention heads* section, tokens often tend to attend to delimiters—specifically, CLS delimiters at lower layers and SEP delimiters at upper layers. Because these tokens are not masked out, they can ease the flow of information. In the last layers, we only observe SEP-delimiter-focused attention patterns. It could be speculated that SEP is used to collect segment-level information, which can be used then for inter-sentence tasks such as **Next Sentence Prediction** (**NSP**) or for encoding sentence-level meaning.

On the other hand, we observe that coreference relation patterns are mostly encoded in the `<8,1>`, `<8,11>`, `<10,1>`, and `<10,7>` heads. Again, it can be clearly said that the `<8,11>` head is the strongest head that encodes the coreference relation in the German model, which we already discussed.

When you click on that thumbnail, you will see the same output, as follows:

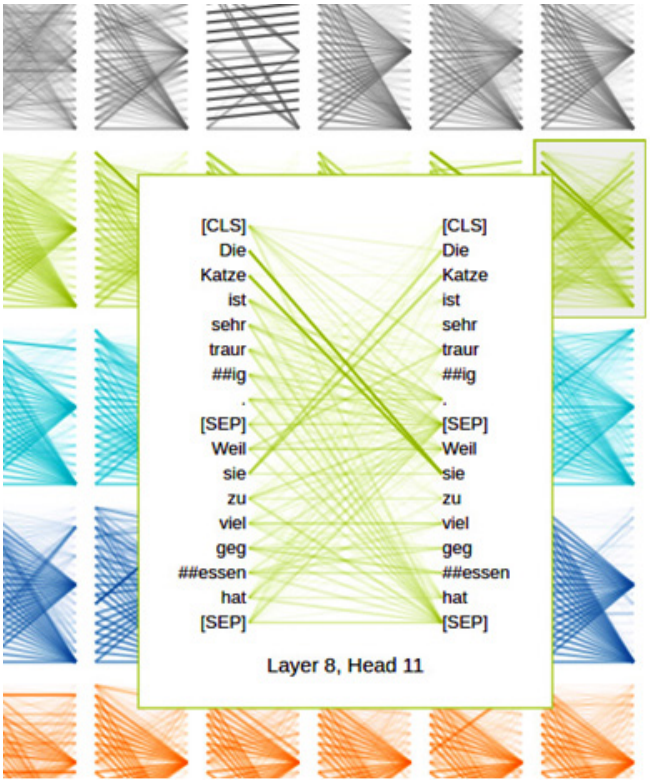


Figure 11.13 – Close-up of the <8,11> head in model view

Again, you can hover over the tokens and see the mappings.

I think that's enough work for the head view and the model view. Now, let's deconstruct the model with the help of the neuron view and try to understand how these heads calculate weights.

Neuron view

So far, we have visualized computed weights for a given input. The neuron view visualizes the neurons and the key vectors in a query and how the weights between tokens are computed based on interactions. We can trace the computation phase between any two tokens.

Again, we will load the German model and visualize the same-sentence pair we just worked with. We execute the following code:

```
from bertviz.Transformers_neuron_view import BertModel, BertTokenizer
from bertviz.neuron_view import show
model_path='bert-base-german-cased'
```

```
sentence_a = "Die Katze ist sehr traurig."
sentence_b = "Weil sie zu viel gegessen hat"
model = BertModel.from_pretrained(model_path,
    output_attentions=True)
tokenizer = BertTokenizer.from_pretrained(model_path)
model_type = 'bert'
show(model, model_type, tokenizer,
    sentence_a, sentence_b, layer=8, head=11)
```

This is the output:

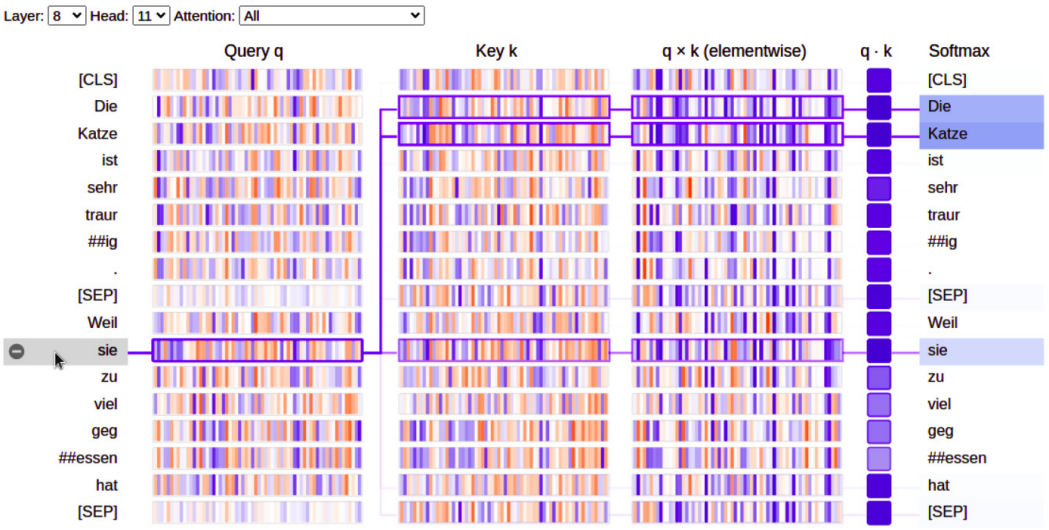


Figure 11.14 – Neuron view of the coreference relation pattern (the <8,11> head)

The view helps us trace the computation of attention from the `sie` token that we selected on the left to the other tokens on the right. Positive values are blue and negative values are orange. Color intensity represents the magnitude of the numerical value. The query of `sie` is very similar to the keys of `Die` and `Katze`. If you look at the patterns carefully, you will notice how similar these vectors are. Therefore, their dot product goes higher than the other comparison, which establishes strong attention between those tokens. We also trace the dot product and the softmax function output as we go to the right. When clicking on the other tokens on the left, you can trace other computations as well.

Now, let's select a head-bearing next-token attention pattern for the same input, and trace it. To do so, we select the `<2,6>` head. In this pattern, virtually all the attention is focused on the next word. We click the `sie` token once again, as follows:

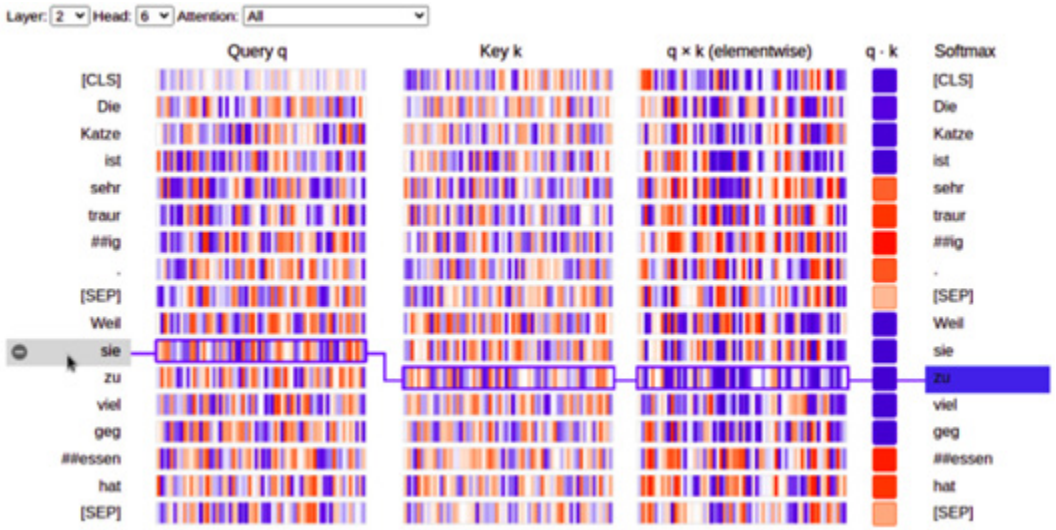


Figure 11.15 – Neuron view of next-token attention patterns (the <2,6> head)

Now, the `sie` token is focused on the next token instead of its own antecedent (`Die Katze`). When we carefully look at the query and the candidate keys, the most similar key to the query of `sie` is the next token, `zu`. Likewise, we observe how the dot product and softmax function are applied in order.

In the next section, we will briefly talk about probing classifiers for interpreting Transformers.

Understanding the inner parts of BERT with probing classifiers

The opacity of what DL learns has led to a number of studies on the interpretation of such models. We attempt to answer the question of which parts of a Transformer model are responsible for certain language features, or which parts of the input lead the model to make a particular decision. To do so, other than visualizing internal representations, we can train a classifier on the representations to predict some external morphological, syntactic, or semantic properties. Hence, we can determine whether we associate internal representations with external properties. The successful training of the model would be quantitative evidence of such an association—that is, the language model has learned information relevant to an external property. This approach is called a **probing-classifier approach**, which is a prominent analysis technique in NLP and other DL studies. An attention-based probing classifier takes an attention map as input and predicts external properties such as coreference relations or head-modifier relations.

As seen in the preceding experiments, we get the self-attention weights for a given input with the `get_bert_attention()` function. Instead of visualizing these weights, we can directly transfer them to a classification pipeline. So, with supervision, we can determine which head is suitable for which semantic feature—for example, we can figure out which heads are suitable for coreference with labeled data.

However, the framework for probing classifiers reveals that it is more complex than it may seem at first. It is crucial to clearly define both the original dataset and model, as well as the probing dataset and classifier. Depending on your objectives, you may want to evaluate the complexity and ease of extracting information from the probe.

Now, let's move on to the model-tracking part, which is crucial for building efficient models.

Explain the model decision

Even if we cannot fully understand the decisions or results of LLMs, we can understand certain aspects that led to this decision. But first, we need to answer and understand what we are trying to explain here. We have three aspects for it as shown in the following taxonomy diagram:

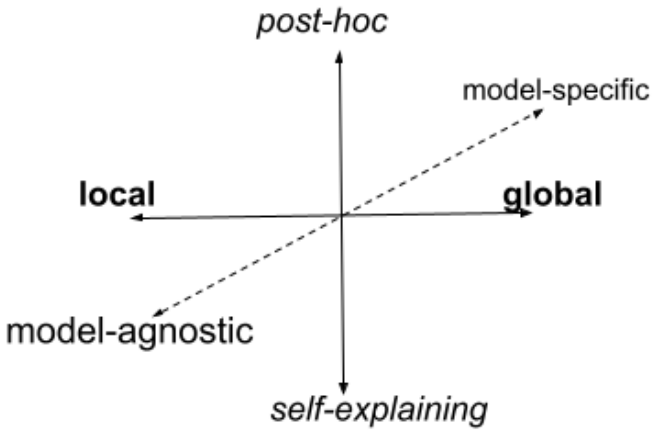


Figure 11.16 – Taxonomy of XAI in 3D

First, are we trying to understand either how a model makes decisions as a whole or its behavior in a specific input? This distinction is referred to as the **global explanation** and **local explanation**. While local explainers deal with the decision on a specific input, global explainers try to explain the model's prediction process as a whole.

The second distinction between XAI models is whether they are self-explaining or post-hoc models. In the former approach, also known as **self-explainers**, models make both predictions and explanations as seen in self-attention mechanisms. However, the self-attention mechanism is used for building contextual word and sentence embeddings, not for a decision. Decision trees, as well-known machine learning models, and other rule-based models are good examples of global self-explainers. The latter, post-hoc explainability models, need additional processes to understand the model decision behavior where a surrogate model is trained to explain the main model. This post-hoc process can be global and local. Although LIME is a good example of producing local explanations using a surrogate model, SHAP can function in both ways.

Another aspect of XAI is the differentiation between model-specific and model-agnostic interpretation tools. Model-specific tools can only be used with specific models, while model-agnostic tools are capable of explaining any black-box model.

As recent trends in machine learning are based on large neural network-based models such as LLMs, which are often considered black-box models, the popularity of surrogate and model-agnostic approaches is increasing. These surrogate models can help explain the behavior of the neural network models for decisions.

In the industry, we have seen very good open-source implementations sorted by popularity as follows:

- `SHAP`
- `LIME`
- `LIT`
- `ELI-5`
- `Transformers Interpret`
- `Captum`

We first take a look at the LIME and SHAP models to understand the Transformers model decisions. Let's start with SHAP.

Interpret Transformers' decision with LIME

LIME is an example of a model-agnostic, local, and post-hoc approach. The main idea behind LIME is input perturbation. Random perturbations are applied to the input (a sentence in our case), and their effects on the output (sentiment or category) are measured. This process uses a simple trainable machine learning model as the main XAI surrogate model.

In the LIME process, an explanation is defined as a local linear approximation of a model's behavior. This is because many models are globally complex, so it is easier to approximate them in the immediate vicinity of a particular instance. To achieve this, the instance to be explained is perturbed, and a sparse linear model (also called a surrogate model) is learned in the vicinity of that instance. Please see the paper "*Why Should I Trust You?": Explaining the Predictions of Any Classifier, Ribeiro et al.*) for more details.

We can simplify the LIME process as follows:

1. First, select the single prediction to be explained.
2. Perturbate the input by randomly hiding features and collect the results of the black-box model, which generates neighborhood data around the original single sample.
3. Assign weights to the new neighborhood samples based on how closely they resemble the original prediction.

4. Train a less complex, interpretable linear model on these sample variations, and a surrogate model is obtained.
5. Finally, predict the explanations by the surrogate model for this local prediction.

Now, let's understand the LIME process with a hands-on example using the LIME and Transformer libraries.

Let us start with installing the necessary packages:

```
!pip install lime transformers
```

We will take our Turkish Text Classification model, which has been fine-tuned on CH05, and load it from the Hugging Face hub as seen in the following:

```
import lime
from lime.lime_text import LimeTextExplainer
from transformers import (AutoTokenizer,
    AutoModelForSequenceClassification)
model_path = 'savasy/bert-turkish-text-classification'
tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForSequenceClassification\
    .from_pretrained(model_path)
class_names =model.config.id2label.values()
```

We have seven classes, as follows:

```
class_names
Output: dict_values(['world', 'economy', 'culture', 'health',
'politics', 'sport', 'technology'])
```

Now, we will pass a Turkish text and interpret the classification. The text is `Ünlü futbolcu Ahmet Yıldız sene sonunda yapılacak turnuva maçları hakkında konuştu`, which means, "The famous football player Ahmet Yıldız talked about the tournament matches to be held at the end of the year." It is obvious that it is about sports.

The following code classifies the input and plots the class probability distribution:

```
import pandas as pd
import torch.nn.functional as Func
text = 'Ünlü futbolcu Ahmet Yıldız sene sonunda yapılacak turnuva
maçları hakkında konuştu'
outputs = model(**tokenizer(text,
    return_tensors="pt",
    padding=True))
probabilities = Func.softmax(outputs[0]).detach().numpy()
q=dict(zip(class_names,probabilities[0]))
pd.Series(q).plot(kind="bar")
```

This plots the following distribution where "sport" is selected out of seven class labels.

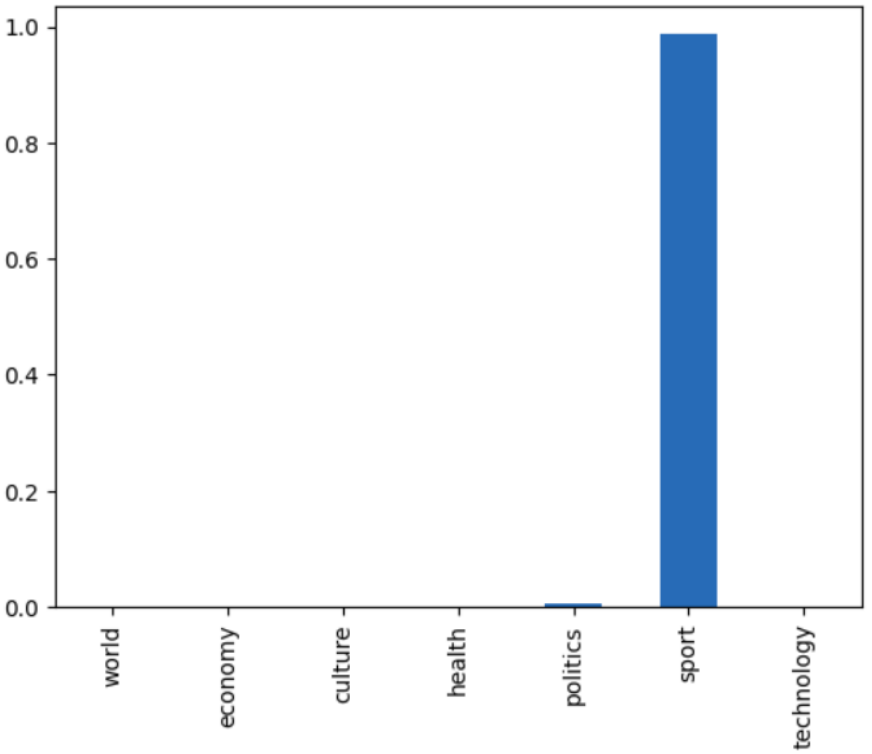


Figure 11.17 – Class probability distribution for Turkish Text Classification

Now, we will generate an explanation for this decision. To do so, we need to write these lines in a function that computes the class probability distribution. The function will be used by the LIME XAI model to generate neighbor samples and to train a linear XAI classifier. Here is the function:

```
def pred_prob(text):
outputs = model(**tokenizer(text,
    return_tensors="pt",
    padding=True))
ps = Func.softmax(outputs[0]).detach().numpy()
return ps
```

We are ready to run the Lcode. As seen in the following, we defined a `LimeTextExplainer` to which class names are passed. Then, we pass the text and `pred_prob` class probability function , and select the fifth label, which is sports. The `num_features` parameter tells us how many features the LIME model uses to explain a decision. In our text classification case, it means how many words the LIME model selects for the decision explanation. The `num_samples` parameter tells us how

many neighbor samples will be generated. We normally keep this number high, such as 1K or 5K, to train a linear classifier efficiently:

```
explainer = LimeTextExplainer(class_names=class_names)
exp = explainer.explain_instance(text,
    pred_prob,
    labels= [5],
    num_features=7,
    num_samples=1000)
exp.show_in_notebook(text=text)
```

With the execution, we see the following output. It explains that the words in brown are indicative of the text belonging to the sports category.

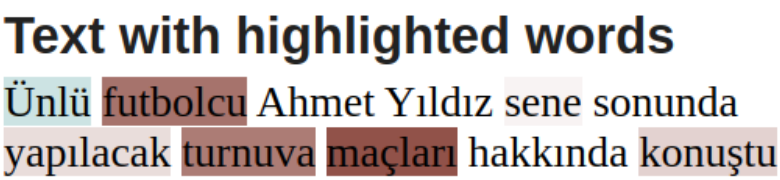


Figure 11.18 – Explanation by LIME output for Turkish Text Classification

The pastel blue represents non-sports content. As seen in the output, the text classification model cares about the sport-related terms during mapping this text to the sports category. The darker the words, the more salient the decision. Now, let us look at another example for English. The following code will load the Emotion classification model for English. It is a model for multi-label text classification with 28 labels. We used it as a single-label classification model with the softmax function. The process will be the same as the Turkish pipeline:

```
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
tokenizer = AutoTokenizer\
    .from_pretrained("SamLowe/roberta-base-go_emotions")
model = AutoModelForSequenceClassification\
    .from_pretrained("SamLowe/roberta-base-go_emotions")
class_names =model.config.id2label.values()
```

We will classify the text, `"I have tried different models, but I have not yet made a decision."` The model will predict it as confusion (id=6):

```
text= "I have tried different models, but I have not yet made a
decision."
explainer = LimeTextExplainer(class_names=class_names)
exp = explainer.explain_instance(text,
    pred_prob,
```

```
    labels= [6],
    num_features=7,
    num_samples=1000)
exp.show_in_notebook(text=text)
```

It outputs the following interpretation:

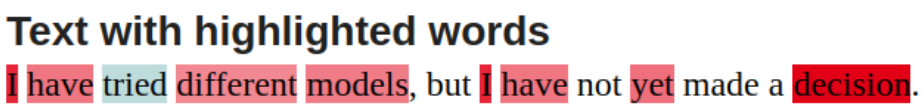


Figure 11.19 – LIME Interpretation for English Emotion Classification

Just as in the previous examples, each color has a meaning. The color red represents a confusion label, while the color green represents a non-confusion label. Here, we cropped the output of LIME. When you run the code, you will see lots of information about color coding.

One disadvantage of LIME is that its surrogate model is based on sampled data points, making it very sensitive to the sampling strategy used. Different sampling methods around the same local data can lead to very different explanation results. Second, the LIME approach is based on the assumption that a local instance can be interpreted linearly.

Now, we will do the same thing with the SHAP model!

Interpret Transformers' decision with SHAP

One of the most common and widely used techniques in the field of XAI is SHAP. It is based on the concept of the Shapley value from game theory. In contrast to LIME, SHAP does not always construct a local interpretable model. Instead, it utilizes the black-box model to compute the marginal contribution of each feature to the prediction. You can see the paper *A Unified Approach to Interpreting Model Predictions*, by *Scott M. Lundberg and Su-In Lee*.

By doing so, it provides insights by identifying the most important features that are driving the model's decision-making process both globally and locally. It calculates SHAP values for the feature importance of words at the local level, then provides global feature importance by adding up the absolute SHAP values for each individual prediction. The SHAP technique is gaining popularity in the field of XAI due to its ability to provide accurate and interpretable explanations for complex DL models.

While LIME assumes that the local model needs to be linear, SHAP does not make any such assumptions. However, a drawback of SHAP is that the calculation of SHAP values can be computationally expensive and time-consuming. This is because it checks all the possible combinations, and it does it through Monte Carlo simulations rather than brute force.

Now, let's learn about the SHAP process with the following code example, using the same as in the LIME example.

Install the necessary packages:

```
!pip install shap transformers
```

Let us load the previous 28-class English Emotion classification model:

```
from transformers import (AutoTokenizer,
 AutoModelForSequenceClassification)
tokenizer = AutoTokenizer\
    .from_pretrained("SamLowe/roberta-base-go_emotions")
model = AutoModelForSequenceClassification\
    .from_pretrained("SamLowe/roberta-base-go_emotions")
class_names =model.config.id2label.values()
```

We wrap them in a pipeline object as follows:

```
from transformers import pipeline
classifier= pipeline("text-classification",
    model=model,
    tokenizer=tokenizer)
```

Let us classify the text first:

```
text= "I have tried different models, but I have not yet made a decision."
classifier(text)
OUTPUT: {'label': 'confusion','score': 0.528}]
```

It decides on the `confusion` label. We will explain the decision by SHAP as follows:

```
import shap
explainer = shap.Explainer(classifier)
shap_values = explainer([text])
```

We specify the target label as `confusion` so that we see which feature directs the model towards that decision. We will choose bar plot visualization as it is more readable than others:

```
shap.plots.bar(shap_values[0,:,"confusion"])
```

It outputs the following diagram:

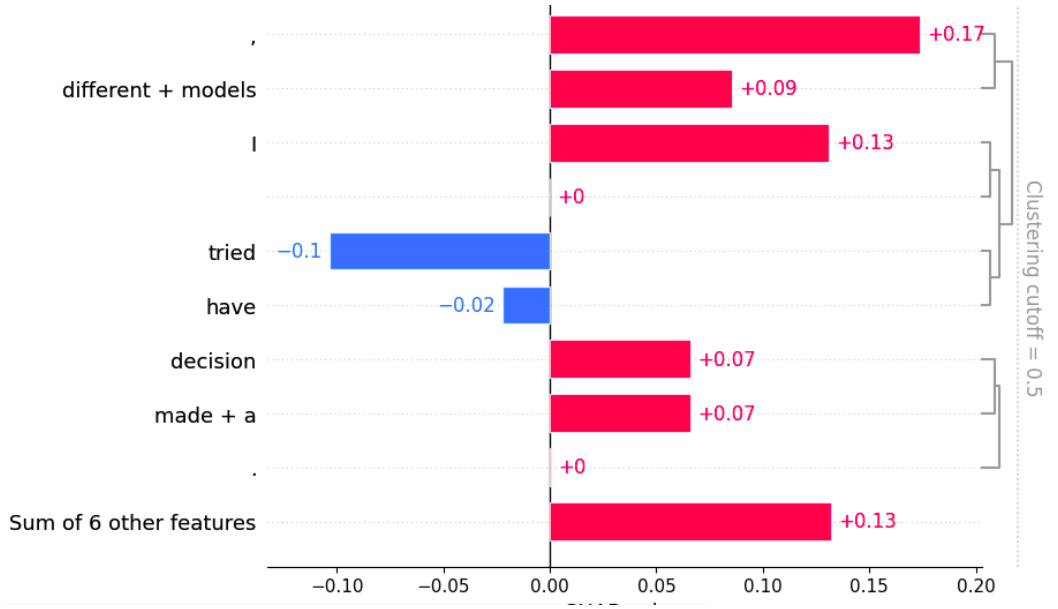


Figure 11.20 – SHAP interpretations for English Emotion Classification

We will see a similar interpretation with LIME, more or less. As a last remark, SHAP is widely used and perhaps more widely accepted due to its theoretical argument based on the game theory concept of Shapley values and its simplicity. However, in LIME, we must define how we consider a neighbor. Additionally, we build a linear local model that may not be linear in a highly complex decision surface. In SHAP, we do not have such an assumption.

Summary

In this chapter, we discussed one of the most important problems in front of AI: XAI. Explainability becomes a serious issue as language models grow with a strong trend. We addressed two topics in terms of Transformers only. First, we examined the self-attention mechanism in the architecture. We tried to understand the internal process of these mechanisms with various visualization tools. Secondly, we interpreted the decision-making process of Transformer architectures. We worked with two model-agnostic approaches: LIME and SHAP. With these two techniques, we monitor how models give importance to parts of inputs (words) in a simple text classification process.

In the next chapter, we will focus on a special kind of transformer namely the **efficient transformer**.

12
Working with Efficient Transformers

So far, you have learned how to design a **Natural Language Processing** (**NLP**) architecture to achieve successful task performance with transformers. In this chapter, you will learn first how to make efficient models out of trained models using distillation, pruning, and quantization. Second, you will also gain knowledge about efficient sparse transformers such as **Linformer**, **BigBird**, and **Performer**. You will see how they perform on various benchmarks, such as memory versus sequence length and speed versus sequence length. You will also see the practical use of model size reduction.

The importance of this chapter came to light as it is getting difficult to run large neural models under limited computational capacity. It is important to have a light general-purpose language model such as **DistilBERT**. This model can then be fine-tuned with good performance, like its non-distilled counterparts. Transformer-based architectures face complexity bottlenecks due to the quadratic complexity of the attention dot product in the transformers, especially for long-context NLP tasks. Character-based language models, speech processing, and long documents are among the long-context problems. In recent years, we have seen a lot of progress in making self-attention more efficient, such as **Reformer**, **Performer**, and **BigBird**, as a solution to complexity. We will also briefly introduce **bitsandbytes** for more efficient and easier quantization.

In short, in this chapter, you will learn about the following topics:

- Introduction to efficient, light, and fast transformers
- Implementation for model size reduction
- Working with efficient self-attention
- Using **bitsandbytes** for quantization

Technical requirements

We will be using the Jupyter Notebook to run our coding exercises, which require Python 3.6+, and the following packages need to be installed:

- `tensorflow`
- `pytorch`
- `transformers >=4.00`
- `Datasets`
- `sentence-transformers`
- `py3nvml`
- `bitsandbytes`

All notebooks with coding exercises are available at the following GitHub link:

`https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`

Introduction to efficient, light, and fast transformers

Transformer-based models have achieved state-of-the-art results in many NLP problems at the cost of quadratic memory and computational complexity. We can highlight the issues regarding complexity as follows:

- The models are not able to efficiently process long sequences due to their self-attention mechanism, which scales quadratically with the sequence length.
- An experimental setup using a typical GPU with 16 GB can handle the sentences of 512 tokens for training and inference. However, longer entries can cause problems.
- NLP models keep growing in size, from the 110 million parameters of BERT-Base to the 175 billion parameters of GPT-3 and the 540 billion parameters of PaLM. This notion raises concerns about computational and memory complexity.
- We also need to care about costs, production, reproducibility, and sustainability. Hence, we need faster and lighter transformers, especially on-edge devices.

Several approaches have been proposed to reduce computational complexity and memory footprint. Some of these approaches focus on changing the architecture and some do not alter the original architecture but instead make improvements to the trained model or to the training phase. We will divide them into two groups, model size reduction and efficient self-attention.

Model size reduction can be accomplished using three different approaches:

- Knowledge distillation
- Pruning
- Quantization

Each of these approaches has its own way of reducing the size model, which we will describe briefly in the *Implementation for model size reduction* section.

In knowledge distillation, a smaller transformer (student) can transfer the knowledge of a big model (teacher). We train the student model so that it can mimic the teacher's behavior or produce the same output for the same input. The distilled model may underperform the teacher. There is a trade-off between compression, speed, and performance.

Pruning is a model compression technique in machine learning that is used to reduce the size of the model by removing a section of the model that contributes little to producing results. The most typical example is decision tree pruning, which helps to reduce the complexity and increase the generalization capacity of the model. Quantization changes model weight types from higher resolutions to lower resolutions. For example, we use a typical floating-point number (`float64`) consuming 64 bits of memory for each weight. Instead, we can use `int8` in quantization, which consumes 8 bits for each weight, and naturally has less accuracy in presenting numbers.

Self-attention heads are not optimized for long sequences. In order to solve this issue, many different approaches have been proposed. The most efficient approach is **self-attention sparsification**, which we will discuss soon. The other most widely used approach is **memory-efficient backpropagation**. This approach strikes a balance between the caching of intermediate results and re-computing. Intermediate activations computed during forward propagation are needed to compute gradients during backward propagation. Gradient checkpoints can reduce a substantial amount of memory footprint and computation. Another approach is **pipeline parallelism algorithms**. Mini-batches are split into micro-batches, and the parallelism pipeline takes advantage of using the waiting time during the forward and backward operations while transferring the batches to deep learning accelerators such as **Graphics Processing Units** (**GPUs**) or **Tensor Processing Units** (**TPUs**).

Parameter sharing can be counted as one of the first approaches towards efficient deep learning. The most typical example is RNN, as depicted in *Chapter 1*, where the units of unfolded representation use shared parameters. Hence, the number of trainable parameters is not affected by the input size. Some shared parameters, which are also called weight tying or weight replication, spread the network so that the number of trainable parameters is reduced. For instance, Linformer shares projection matrices across heads and layers. Reformer shares the query and key at the cost of performance loss.

Now, let's try to understand these issues with practical examples.

Implementation for model size reduction

Even though transformer-based models achieve state-of-the-art results in many aspects of NLP, they usually share the very same problem: they are big models and are not fast enough to be used. In business cases where it is necessary to embed them inside a mobile application or in a web interface, it is impossible if you try to use the original models.

In order to improve the speed and size of these models, some techniques have been proposed, which are listed here:

- Distillation (also known as knowledge distillation)
- Pruning
- Quantization

For each of these techniques, we provide a separate subsection to address the technical and theoretical insights.

Working with DistilBERT for knowledge distillation

The process of transferring knowledge from a bigger model to a smaller one is called **knowledge distillation**. In other words, there is a teacher model and a student model; the teacher is typically a bigger and stronger model while the student is smaller and weaker.

This technique is used in various problems, from vision to acoustic models and NLP. A typical implementation of this technique is shown in *Figure 12.1*:

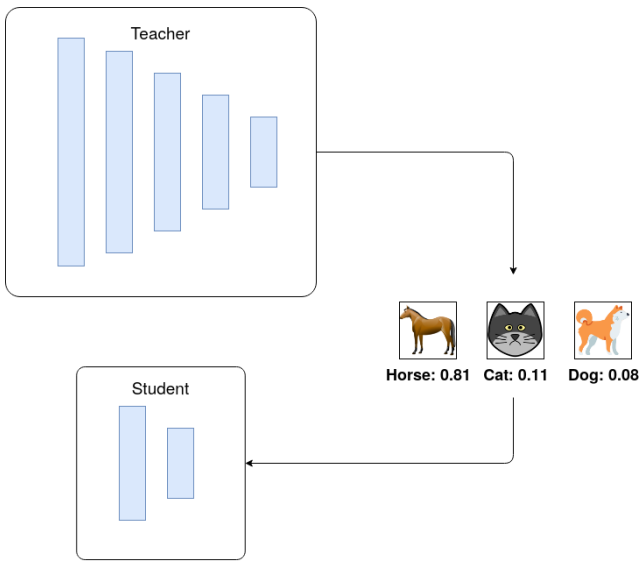


Figure 12.1 – Knowledge distillation for image classification

DistilBERT is one of the most important models in this field and has gotten the attention of researchers and even businesses. This model, which tries to mimic the behavior of BERT-Base, has 50% fewer parameters and achieves 95% of the teacher model's performance.

Here's some information about DistilBERT. All figures are in comparison with the original BERT:

- DistilBert is 1.7x compressed and 1.6x faster with 97% relative performance
- Mini-BERT is 6x compressed, 3x faster, and has 98% relative performance
- TinyBERT is 7.5x compressed, has 9.4x speed, and 97% relative performance

The distillation training step that is used to train the model is very simple using PyTorch (the original description and code is available at `https://medium.com/huggingface/distilbert-8cf3380435b5`):

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim import Optimizer
KD_loss = nn.KLDivLoss(reduction='batchmean')
def kd_step(teacher: nn.Module,
    student: nn.Module,
    temperature: float,
    inputs: torch.tensor,
    optimizer: Optimizer):
    teacher.eval()
    student.train()
    with torch.no_grad():
        logits_t = teacher(inputs=inputs)
    logits_s = student(inputs=inputs)
    loss = KD_loss(input=F.log_softmax(
        logits_s/temperature,
        dim=-1),
        target=F.softmax(
            logits_t/temperature,
            dim=-1))
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

This model-supervised training provides us with a smaller model that is very similar to the base model in behavior. The loss function used here is Kullback-Leibler loss to ensure that the student model mimics the good and bad aspects of the teacher model with no modification to the decision on the last `softmax` logits. This loss function shows how different two distributions are from each other; a greater difference means a higher loss value. The reason for using this loss function is to make the student model try to completely mimic the behavior of the teacher. The **GLUE** macro scores for BERT and DistilBERT are just 2.8% different.

Pruning transformers

Pruning is the process of setting weights at the layers to zero based on a pre-specified criterion. This method eliminates weights that are too low in value and do not affect the results too much. Likewise, we prune some redundant parts of the transformer network. The pruned networks are more likely to generalize better than the original one. We have seen a successful pruning operation because the pruning process probably keeps the true underlying explanatory factors and discards the redundant subnetwork. Then, the less salient weights or units whose removals have a small effect on the model performance are discarded.

There are two approaches:

- **Unstructured pruning**: Individual weights with a small saliency (or the least weight magnitude) are removed no matter which part of the neural network they are located in
- **Structured pruning**: This approach prunes heads or layers

However, the pruning process has to be compatible with modern GPUs.

Most libraries, such as Torch and TensorFlow, come with this capability. We will describe how it is possible to prune a model using Torch. There are many different methods to use for pruning (magnitude-based or mutual information-based). One of the simplest ones to understand and implement is the L1 pruning method. This method takes the weights of each layer and zeros out the ones with the lowest L1-norm. You can also specify what percentage of your weights must be converted to zero after pruning. In order to make this example more understandable and show its impact on the model, we'll use the text representation example from *Chapter 7*. We will prune the model and see how it performs after pruning:

1. We will use the Roberta model. You can load the model using the following code:

```
from sentence_transformers import SentenceTransformer
distilroberta = SentenceTransformer(
    'stsb-distilroberta-base-v2')
```

2. You will also need to load metrics and datasets for evaluation:

```
from datasets import load_metric, load_dataset
stsb_metric = load_metric('glue', 'stsb')
stsb = load_dataset('glue', 'stsb')
```

3. In order to evaluate the model, just like in *Chapter 7*, you can use the following function:

```
import math
import tensorflow as tf
def roberta_sts_benchmark(batch):
    sts_encode1 = tf.nn.l2_normalize(
        distilroberta.encode(batch['sentence1']),
        axis=1)
    sts_encode2 = tf.nn.l2_normalize(
        distilroberta.encode(batch['sentence2']), axis=1)
    cosine_similarities = tf.reduce_sum(
        tf.multiply(sts_encode1, sts_encode2), axis=1)
    clip_cosine_similarities = tf.clip_by_value(
        cosine_similarities,-1.0,1.0)
    scores = 1.0 -\
        tf.acos(clip_cosine_similarities) / math.pi
return scores
```

4. And of course, it is required to set labels:

```
references = stsb['validation'][:]['label']
```

5. And then we need to run the base model with no changes in it:

```
distilroberta_results = \
    roberta_sts_benchmark(stsb['validation'])
```

6. After all these things are done, this is the step where we start to prune our model:

```
from torch.nn.utils import prune
pruner = prune.L1Unstructured(amount=0.2)
```

7. The previous code makes a pruning object using L1-norm pruning with 20% of the weights in each layer. To apply it to the model, you can use the following code:

```
state_dict = distilroberta.state_dict()
for key in state_dict.keys():
    if "weight" in key:
        state_dict[key] = pruner.prune(state_dict[key])
```

It will iteratively prune all layers that have weight in their name; in other words, we will prune all weight layers and not touch the layers that are biased. Of course, you can try that, too, for experimentation purposes.

8. And again, it is good to reload the state dictionary to the model:

```
distilroberta.load_state_dict(state_dict)
```

9. Now that we have done everything, we can test the new model:

```
distilroberta_results_p = \
    roberta_sts_benchmark(stsb['validation'])
```

10. In order to have a good visual representation of the results, you can use the following code:

```
import pandas as pd
pd.DataFrame({
"DistilRoberta":stsb_metric.compute(
    predictions=distilroberta_results,
    references=references),
"DistilRobertaPruned":stsb_metric.compute(
    predictions=distilroberta_results_p,
    references=references)
})
```

The following screenshot shows the results:

	DistillRoberta	DistillRobertaPruned
pearson	0.888461	0.849915
spearmanr	0.889246	0.849125

Figure 12.2 – Comparison between original and pruned models

We have eliminated 20% of the weights in the model, reduced its size and computation cost, and lost 4% in performance. However, this step can be combined with other techniques, such as quantization, which is explored in the next subsection.

This type of pruning is applied to some of the weights in a layer; however, it is also possible to completely drop some parts or layers of transformer architectures. For example, it is possible to drop some of the attention heads and track the changes too.

There are also other types of pruning algorithms available in PyTorch, such as iterative and global pruning, which are worth trying.

Quantization

Quantization is a signal processing and communication term that is generally used to denote how much accuracy is presented by the data provided. More bits mean more accuracy and precision in terms of data resolution. For example, if you have a variable that is presented by 4 bits and you want to quantize it to 2 bits, it means you have to drop the accuracy of your resolution. With 4 bits, you can specify 16 different states, while with 2 bits you can distinguish 4 states. In other words, by reducing the resolution of your data from 4 to 2 bits, you are saving 50% more space and complexity.

Many popular libraries, such as TensorFlow, PyTorch, and MXNET, support mixed-precision operations. Recall the `fp16` parameter used in the `TrainingArguments` class in *Chapter 5*. `fp16` increases computational efficiency since modern GPUs provide higher efficiency for reduced precision math, but the results are accumulated in `fp32`. Mixed-precision can reduce the memory usage required for training, which allows us to increase the batch size or model size.

Quantization can be applied to model weights to reduce their resolution and save computation time, memory, and storage. In this subsection, we will try to quantize the model that we pruned in the previous section:

1. In order to do so, you can use the following code to quantize your model in 8-bit integer representation instead of float:

```
import torch
distilroberta = torch.quantization.quantize_dynamic(
    model=distilroberta,
    qconfig_spec = {
    torch.nn.Linear : \
        torch.quantization.default_dynamic_qconfig,
        },
    dtype=torch.qint8)
```

2. Afterwards, you can get the evaluation results by using the following code:

```
distilroberta_results_pq = \
    roberta_sts_benchmark(stsb['validation'])
```

3. And as before, you can view the results:

```
pd.DataFrame({
"DistilRoberta":stsb_metric.compute(
    predictions=distilroberta_results,
    references=references),
"DistillRobertaPruned":stsb_metric.compute(
    predictions=distilroberta_results_p,
    references=references),
```

```
"DistilRobertaPrunedQINT8":stsb_metric.compute(
    predictions=distilroberta_results_pq,
    references=references)
})
```

The results can be seen as follows:

	DistillRoberta	DistillRobertaPruned	DistillRobertaPrunedQINT8
pearson	0.888461	0.849915	0.826784
spearmanr	0.889246	0.849125	0.824857

Figure 12.3 – Comparison between the original, pruned, and quantized models

4. So far, you have used a distilled model, pruned it, and then quantized it to reduce its size and complexity. Let's see how much space you have saved by saving the model:

```
distilroberta.save("model_pq")
```

Use the following code in order to see the model size:

```
ls model_pq/0_Transformer/ -l --block-size=M | grep pytorch_
model.bin
-rw-r--r-- 1 root 191M May 23 14:53 pytorch_model.bin
```

As you can see, the size is 191 MB. The initial size of the model was 313 MB, which means we managed to decrease the size of the model to 61% of its original size and just lost 6%–6.5% in terms of performance. Please note that the `block-size` parameter may fail on a Mac, and you need to use `-lh` instead.

Up to this point, you have learned about pruning and quantization in terms of practical model preparation for industrial usage. However, you also gained information about the distillation process and how it can be useful. There are many other ways to perform pruning and quantization, which can be a good step to take in after reading this section. For more information and guides, you can take a look at **movement pruning** at `https://github.com/huggingface/block_movement_pruning`. This kind of pruning is a simple and deterministic first-order weight pruning approach. It uses the weight changes in training to find out which weights are more likely to be unused to have less effect on the result. In the next subsection, we will explore ways to enhance the efficiency of the transformer.

Working with efficient self-attention

Efficient approaches restrict the attention mechanism to get an effective transformer model because the computational and memory complexity of a transformer is mostly due to the self-attention mechanism. The attention mechanism scales quadratically with respect to the input sequence length. For short input, quadratic complexity may not be an issue. However, to process longer documents, we need to improve the attention mechanism that scales linearly with sequence length.

We can roughly group the efficient attention solutions into three types:

- Sparse attention with fixed patterns
- Learnable sparse patterns
- Low-rank factorization/kernel function

Let's begin with sparse attention based on a fixed pattern next.

Sparse attention with fixed patterns

Recall that the attention mechanism is made up of a query, a key, and values, as roughly formulated here:

$$Attention\,(Q,K,V)\;=\;Score\,(Q,K)\,.V$$

Here, the score function, which is usually `softmax`, performs multiplication that requires $O(n^2)$ memory and computational complexity since a token position attends to all other token positions in full self-attention mode to build its embeddings. We repeat the same process for all token positions to get their embeddings, leading to a quadratic complexity problem. It is a very expensive way of learning, especially for long-context NLP problems. It is natural to ask the question, "Do we need such a dense interaction, or is there a cheaper way to do the calculations?" Many researchers have addressed this problem and employed a variety of techniques to mitigate the complexity burden and reduce the quadratic complexity of the self-attention mechanism. They have mostly made a trade-off between performance, computation, and memory, especially for long documents.

The simplest way to reduce complexity is to sparsify the full self-attention matrix or find another cheaper way to approximate full attention. Sparse attention patterns formulate how to connect/disconnect certain positions without disturbing the flow of information through layers, which helps the model to track long-term dependency and build sentence-level encoding.

Full self-attention and sparse attention are depicted in *Figure 12.4* in that order; the rows correspond to output positions and the columns are for the inputs. A full self-attention model would directly transfer the information between any two positions. On the other hand, in localized sliding window attention, which is sparse attention, as shown on the right of the figure, the empty cells mean that there is no interaction between the corresponding input-output position. The sparse model in the figure is based on fixed patterns that are certain manually designed rules. More specifically, it is localized sliding window attention that was one of the first proposed methods, also known as the local-based fixed pattern approach. The assumption behind it is that useful information is located in each position neighbor. Each query token attends to `window/2` key tokens to the left and `window/2` key tokens to the right of that position. In the following example, the window size is set to 4. This rule applies to every layer in the transformer in the same way. In some studies, the window size is increased as it moves towards the layers further.

The following figure simply depicts the difference between full and sparse attention:

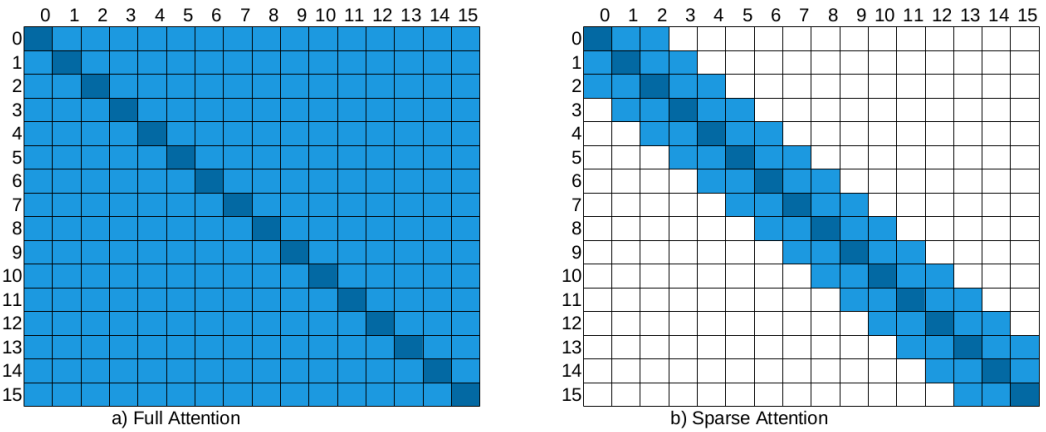


Figure 12.4 – Full attention versus sparse attention

In sparse mode, the information is transmitted through connected nodes (non-empty cells) in the model. For example, output position 7 of the sparse attention matrix cannot directly attend to input position 3 (please see the sparse matrix at the right of *Figure 12.4*) since cell (7,3) is empty. However, position 7 indirectly attends to position 3 via the token position 5, that is (7->5, 5->3 => 7->3). The figure also illustrates that while the full self-attention incurs a total of n^2 active cells (vertex), the sparse model does roughly $5 \times n$.

Another important type is global attention. A few selected tokens or a few injected tokens are used as global attention that can attend to all other positions and be attended by them. Hence, the maximum path distance between any two token positions is equal to 2. Suppose we have a sentence *[GLB, the, cat, is, very, sad]*, where *GLB* is an injected global token and the window size is 2, which means a token can attend to only its immediate left and right tokens and to GLB as well. There is no direct interaction between *cat* and *sad*. But we can follow *cat-> GLB, GLB-> sad* interactions, which creates a hyperlink through the GLB token. The global tokens can be selected from existing tokens or added tokens such as *(CLS)*. As shown in the following screenshot, the first two token positions are selected as global tokens:

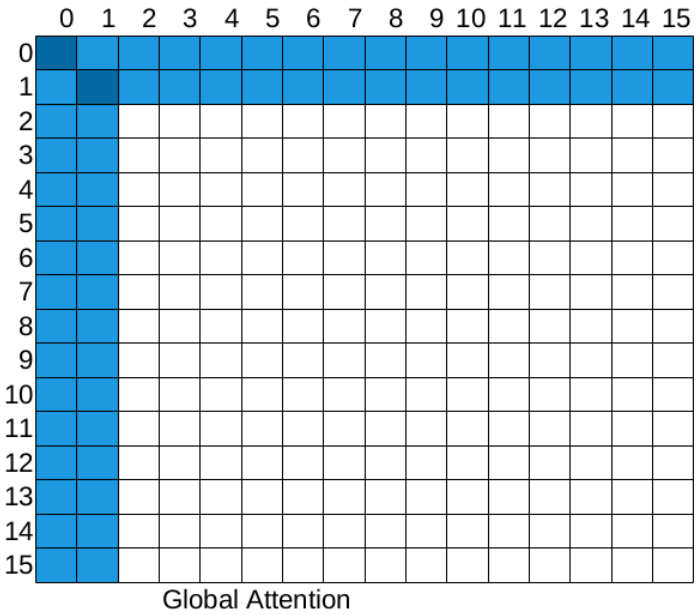


Figure 12.5 – Global attention

These global tokens don't have to be at the beginning of the sentence either. For example, the **Longformer** model randomly selects global tokens in addition to the first two tokens.

There are four more widely seen patterns. **Random attention** (the first matrix in *Figure 12.6*) is used to ease the flow of information by randomly selecting from existing tokens. But most of the time, we employ random attention as part of a **combined pattern** (the bottom-left matrix) that consists of a combination of other models. **Dilated attention** is similar to the sliding window, but some gaps are put in the window, as shown at the top right of *Figure 12.6*:

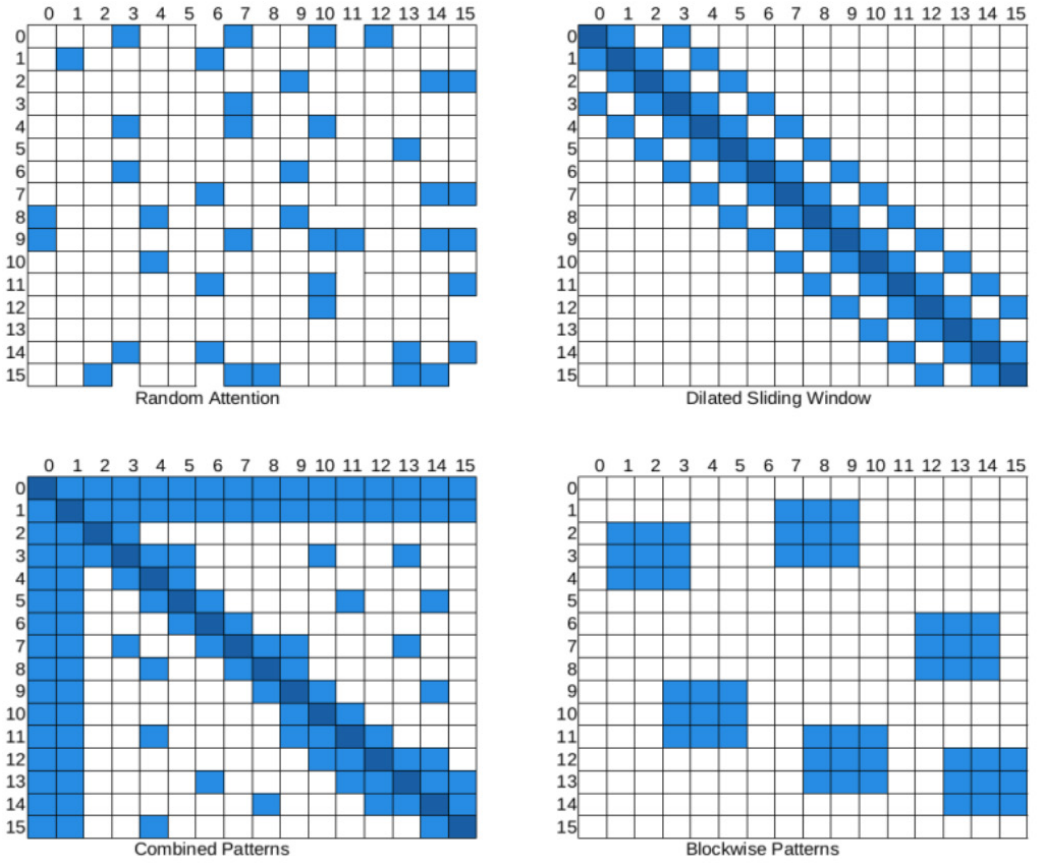


Figure 12.6 – Random, Dilated, Combined, and Blockwise

The **Blockwise Pattern** (the bottom right of *Figure 12.6*) provides a basis for other patterns. It chunks the tokens into a fixed number of blocks, which is especially useful for long-context problems. For example, when a 4,096x4,096 attention matrix is chunked using a block size of 512, then 8 (512x512) query blocks and key blocks are obtained. Many efficient models, such as **BigBird** and **Reformer**, chunk tokens into blocks to reduce the complexity.

It is important to note that the proposed patterns must be supported by accelerators and libraries. Some attention patterns, such as dilated patterns, require a special matrix multiplication that is not directly supported in current deep learning libraries such as PyTorch or TensorFlow as of writing this chapter.

We are ready to run some experiments for efficient transformers. We will proceed with models that are supported by the Transformers library and that have checkpoints on the Hugging Face platform. **Longformer** is one of the models that uses sparse attention. It uses a combination of a sliding window and global attention. It supports dilated sliding window attention as well.

Before we start, we need to install the `py3nvml` package for benchmarking. Please recall that we already discussed how to apply benchmarking in *Chapter 2*:

```
!pip install py3nvml
```

We also need to check our devices to ensure that there is no running process (within Jupyter Notebook):

```
!nvidia-smi
```

The output is as follows:

```
Sat May 22 13:43:18 2021
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 465.19.01    Driver Version: 460.32.03    CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla P100-PCIE...  Off  | 00000000:00:04.0 Off |                    0 |
| N/A   36C    P0    26W / 250W |      0MiB / 16280MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

Figure 12.7 – GPU usage

Currently, Longformer authors have shared a couple of checkpoints. The following code snippet loads the Longformer checkpoint `allenai/longformer-base-4096` and processes a long text:

```
from transformers import (
    LongformerTokenizer, LongformerForSequenceClassification)
import torch
tokenizer = LongformerTokenizer.from_pretrained(
    'allenai/longformer-base-4096')
model=LongformerForSequenceClassification.from_pretrained(
    'allenai/longformer-base-4096')
sequence= "hello "*4093
inputs = tokenizer(sequence, return_tensors="pt")
print("input shape: ",inputs.input_ids.shape)
outputs = model(**inputs)
```

The output is as follows:

```
input shape:  torch.Size([1, 4096])
```

As seen, Longformer can process a sequence with a length of up to `4096`. When we pass a sequence whose length is more than `4096`, which is the limit, you will get the error `IndexError: index out of range in self`.

Longformer's default `attention_window` is `512`, which is the size of the attention window around each token. With the following code, we instantiate two Longformer configuration objects, where the first one is the default Longformer, and the second is a lighter one where we set the window size to a smaller value such as 4 so that the model becomes lighter:

Please pay attention to the following examples. We will always call `XformerConfig.from_pretrained()`. This call does not download the actual weights of the model checkpoint, but instead only downloads the configuration from the Hugging Face Hub. Throughout this section, since we will not fine-tune, we only need the configuration:

```
from transformers import (LongformerConfig,
    PyTorchBenchmark, PyTorchBenchmarkArguments)
config_longformer=LongformerConfig.from_pretrained(
    "allenai/longformer-base-4096")
config_longformer_window4=LongformerConfig.from_pretrained(
    "allenai/longformer-base-4096",
     attention_window=4)
```

With these configuration instances, you can train your Longformer language model with your own datasets, passing the configuration object to the Longformer model as follows:

```
from transformers import LongformerModel
model = LongformerModel(config_longformer)
```

Other than training a Longformer model, you can also fine-tune a trained checkpoint to a downstream task. To do so, you can continue by applying the code as shown in *Chapter 3* for language model training and *Chapters 5 and 6* for fine-tuning.

We will now compare the time and memory performance of these two configurations with various lengths of input `[128, 256, 512, 1024, 2048, 4096]` by utilizing `PyTorchBenchmark` as follows:

```
sequence_lengths=[128,256,512,1024,2048,4096]
models=["config_longformer","config_longformer_window4"]
configs=[eval(m) for m in models]
benchmark_args = PyTorchBenchmarkArguments(
    sequence_lengths= sequence_lengths,
    batch_sizes=[1],
```

```
    models= models)
benchmark = PyTorchBenchmark(
    configs=configs,
    args=benchmark_args)
results = benchmark.run()
```

The output is the following:

```
1 / 2
2 / 2

====================       INFERENCE - SPEED - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length     Time in s
--------------------------------------------------------------------------------
      config_longformer              1              128            0.036
      config_longformer              1              256            0.036
      config_longformer              1              512            0.036
      config_longformer              1              1024           0.064
      config_longformer              1              2048           0.117
      config_longformer              1              4096           0.226
  config_longformer_window4          1              128            0.019
  config_longformer_window4          1              256            0.022
  config_longformer_window4          1              512            0.028
  config_longformer_window4          1              1024           0.044
  config_longformer_window4          1              2048           0.074
  config_longformer_window4          1              4096           0.136
--------------------------------------------------------------------------------

====================      INFERENCE - MEMORY - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length    Memory in MB
--------------------------------------------------------------------------------
      config_longformer              1              128             1595
      config_longformer              1              256             1595
      config_longformer              1              512             1595
      config_longformer              1              1024            1679
      config_longformer              1              2048            1793
      config_longformer              1              4096            2089
  config_longformer_window4          1              128             1525
  config_longformer_window4          1              256             1527
  config_longformer_window4          1              512             1541
  config_longformer_window4          1              1024            1561
  config_longformer_window4          1              2048            1643
  config_longformer_window4          1              4096            1763
--------------------------------------------------------------------------------
```

Figure 12.8 – Benchmark results

Some hints for `PyTorchBenchmarkArguments`: if you like to see the performance for training as well as inference, you should set the argument `training` to `True` (the default is `False`). You may also want to see information about your current environment. You can do so by setting `no_env_print` to `False`; the default is `True`.

Let's visualize the performance to make it more interpretable. To do so, we define a `plotMe()` function since we will need that function for further experiments as well. The function plots the inference performance in terms of both running time complexity and memory complexity.

Here is the function definition:

```
import matplotlib.pyplot as plt
def plotMe(results,title="Time"):
    plt.figure(figsize=(8,8))
    fmts= ["rs--","go--","b+-","c-o"]
    q=results.memory_inference_result
    if title=="Time":
        q=results.time_inference_result
    models=list(q.keys())
    seq=list(q[models[0]]['result'][1].keys())
    models_perf=[list(q[m]['result'][1].values()) \
        for m in models]
    plt.xlabel('Sequence Length')
    plt.ylabel(title)
    plt.title('Inference Result')
    for perf,fmt in zip(models_perf,fmts):
        plt.plot(seq, perf,fmt)
    plt.legend(models)
    plt.show()
```

Let's see the computational performance of two Longformer configurations, as follows:

```
plotMe(results)
```

It plots the following:

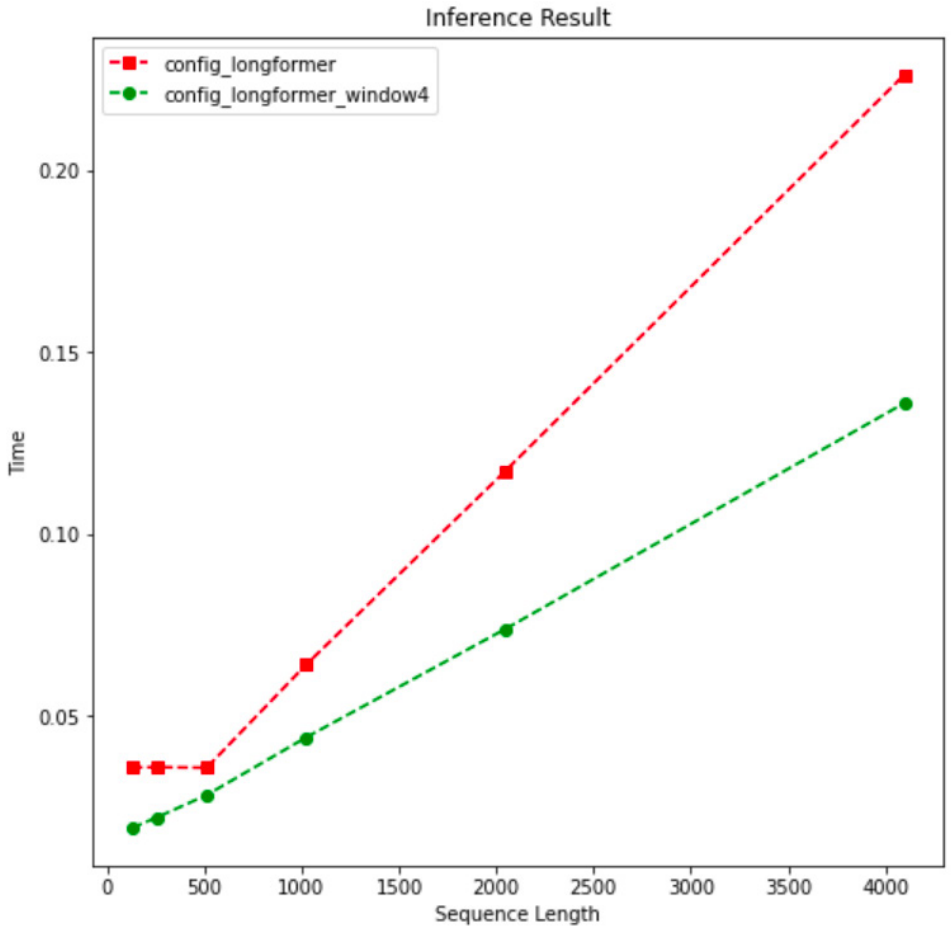


Figure 12.9 – Speed performance over sequence length (Longformer)

In this and the next examples, we see the main differentiation between a heavy model and a light model starting from length 512. The preceding figure shows the lighter Longformer model in green (the one with a window length of 4) performs better in terms of time complexity as expected. We also see that two Longformer models process the input with linear time complexity.

Let's evaluate these two models in terms of memory performance:

```
plotMe(results, "Memory")
```

It plots the following:

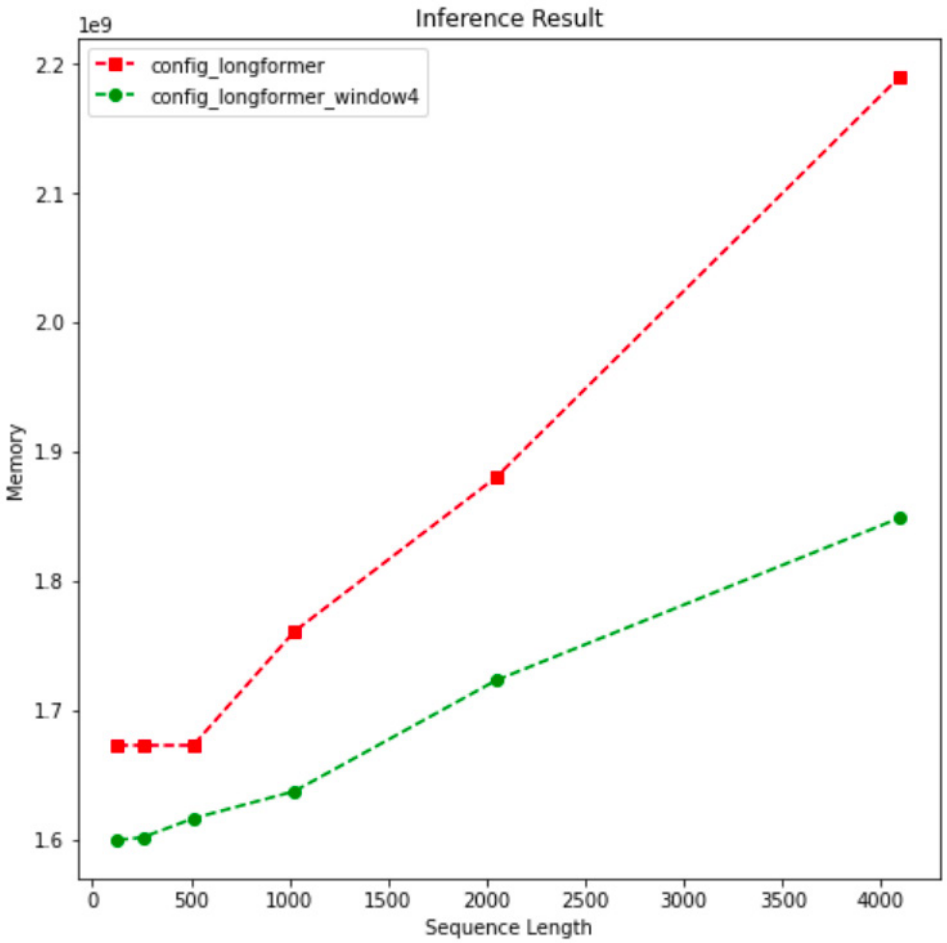


Figure 12.10 – Memory performance over sequence length (Longformer)

Again, up to length 512, there is no substantial differentiation. For the rest, we see a similar memory performance to the time performance. It is clear to say that the memory complexity of the Longformer self-attention is linear. On the other hand, we are not saying anything about model task performance yet.

Thanks to the `PyTorchBenchmark` script, we have cross-checked these models. This script is very useful when we choose which configuration the language model should be trained with. It will be vital before starting the real language model training and fine-tuning.

Another best-performing model exploiting sparse attention is **BigBird** (Zohen et al. 2020). The authors claimed that their sparse attention mechanism (they called it a generalized attention mechanism) preserves all the functionality of the full self-attention mechanism of vanilla transformers in linear time. The authors treat the attention matrix as a directed graph so that they can leverage graph theory algorithms. They took inspiration from the graph sparsification algorithm, which approximates a given graph `G` by graph `G'` with fewer edges or vertices.

BigBird is a block-wise attention model and can handle sequences up to a length of `4096`. It first blocks the attention pattern by packing queries and keys together and then defines attention on these blocks. They utilize random, sliding window, and global attention.

Let's load and use BigBird model checkpoint configuration just like the Longformer transformer model. A couple of BigBird checkpoints are shared by the developers in the Hugging Face Hub. We will select the original BigBird model, `google/bigbird-roberta-base`, which is based on a RoBERTa checkpoint. Once again, we're not downloading the model checkpoint weights, but the configuration instead. The `BigBirdConfig` implementation allows us to compare full self-attention and sparse attention. Thus, we can check whether the sparsification will reduce the full-attention $O(n^2)$ complexity to a lower level. Once again, up to a length of 512, we do not clearly observe quadratic complexity. We can see the complexity from this level on. Setting the attention type to `original_full` will give us a full self-attention model. For comparison, we created two types of configurations: the first one is BigBird's original sparse approach and the second is a model that uses the full self-attention model.

We call them `sparseBird` and `fullBird`, as follows:

```
from transformers import BigBirdConfig
# Default Bird with num_random_blocks=3, block_size=64
sparseBird = BigBirdConfig.from_pretrained(
    "google/bigbird-roberta-base")
fullBird = BigBirdConfig.from_pretrained(
    "google/bigbird-roberta-base",
    attention_type="original_full")
```

Please notice that for smaller sequence lengths up to 512, the BigBird model works as full self-attention mode due to `block-size` and `sequence-length` inconsistency:

```
sequence_lengths=[256,512,1024,2048, 3072, 4096]
models=["sparseBird","fullBird"]
configs=[eval(m) for m in models]
benchmark_args = PyTorchBenchmarkArguments(
    sequence_lengths=sequence_lengths,
    batch_sizes=[1],
    models=models)
```

```
benchmark = PyTorchBenchmark(
    configs=configs,
    args=benchmark_args)
results = benchmark.run()
```

The output is as follows:

```
====================       INFERENCE - SPEED - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length     Time in s
--------------------------------------------------------------------------------
          sparseBird                 1              256            0.014
          sparseBird                 1              512            0.029
          sparseBird                 1              1024           0.112
          sparseBird                 1              2048           0.288
          sparseBird                 1              3072           0.217
          sparseBird                 1              4096           0.279
           fullBird                  1              256            0.014
           fullBird                  1              512            0.024
           fullBird                  1              1024           0.049
           fullBird                  1              2048           0.123
           fullBird                  1              3072           0.232
           fullBird                  1              4096           0.348
--------------------------------------------------------------------------------

====================      INFERENCE - MEMORY - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length    Memory in MB
--------------------------------------------------------------------------------
          sparseBird                 1              256             1495
          sparseBird                 1              512             1571
          sparseBird                 1              1024            1777
          sparseBird                 1              2048            2195
          sparseBird                 1              3072            2591
          sparseBird                 1              4096            2969
           fullBird                  1              256             1495
           fullBird                  1              512             1571
           fullBird                  1              1024            1801
           fullBird                  1              2048            2257
           fullBird                  1              3072            2955
           fullBird                  1              4096            3835
--------------------------------------------------------------------------------
```

Figure 12.11 – Benchmark results (BigBird)

Again, we plot the time performance as follows:

```
plotMe(results)
```

It plots the following:

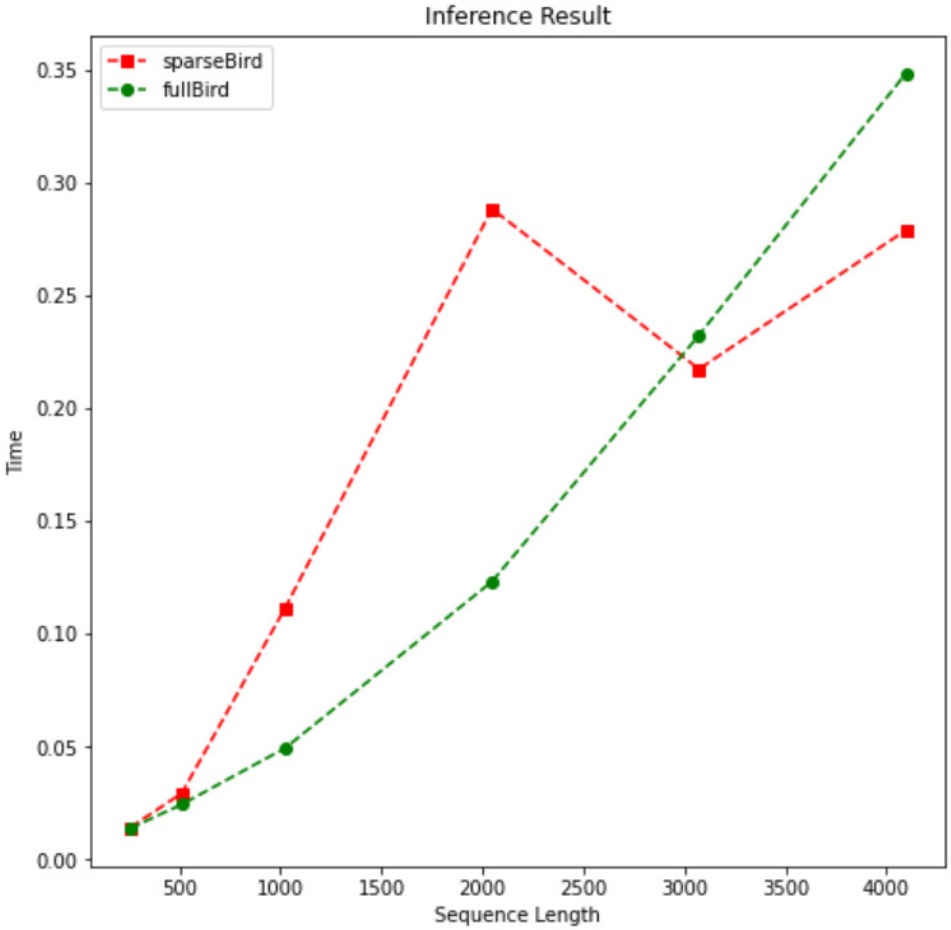


Figure 12.12 – Speed performance (BigBird)

To a certain extent, the full self-attention model performs better than a sparse model. However, we can observe the quadratic time complexity for `fullBird`. Hence, after a certain point, we also see that the sparse attention model abruptly outperforms it, when coming to an end.

Let's check the memory complexity as follows:

```
plotMe(results,"Memory")
```

Here is the output:

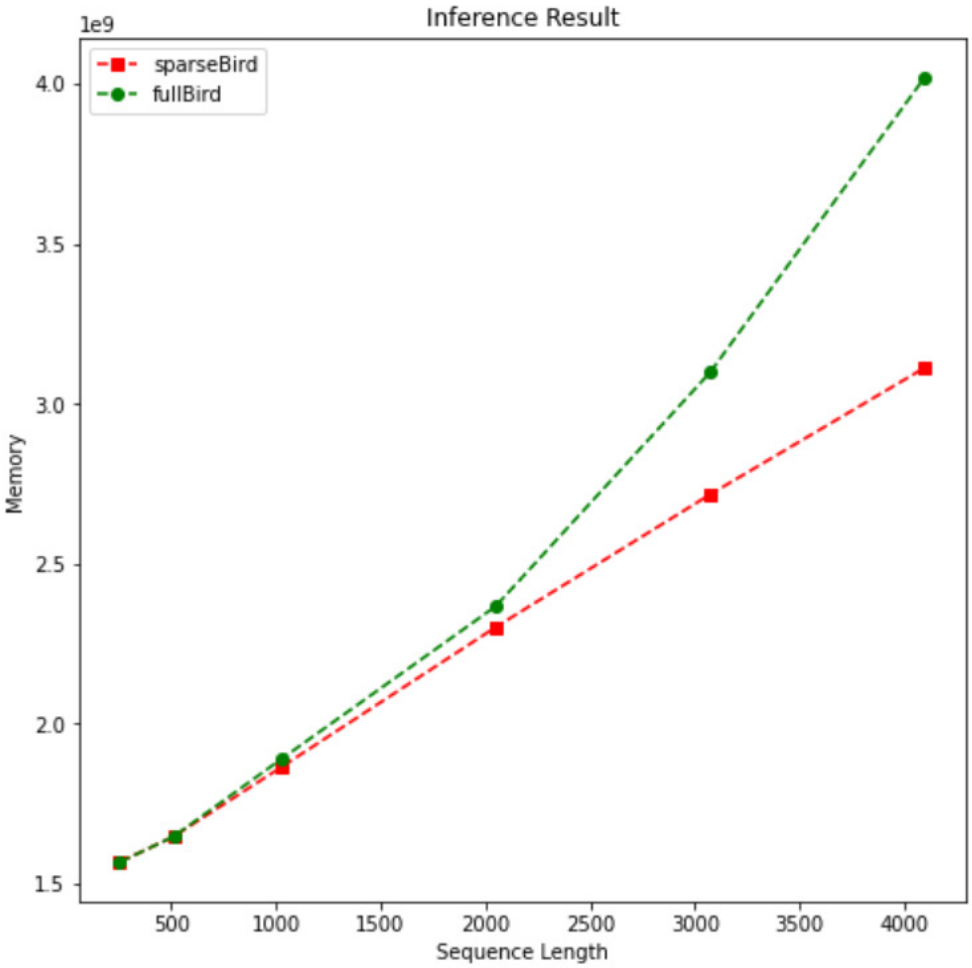


Figure 12.13 – Memory performance (BigBird)

In the preceding figure, we can clearly see linear and quadratic memory complexity. Once again, up to a certain point (a length of 2,000 in this example), we cannot make a clear distinction.

Next, let's discuss learnable patterns and work with models that can process longer input.

Learnable patterns

Learning-based patterns are the alternatives to fixed (predefined) patterns. These approaches extract the patterns in an unsupervised data-driven fashion. They leverage some techniques to measure the similarity between the queries and the keys to properly cluster them. This transformer family learns first how to cluster the tokens and then restrict the interaction to get an optimum view of the attention matrix.

Now, we will do some experiments with Reformer as one of the important efficient models based on learnable patterns. Before that, let's address what the Reformer model contributes to the NLP field.

It employs **Local Self Attention** (**LSA**) that cuts the input into *n* chunks to reduce the complexity bottleneck. But this cutting process makes the boundary token unable to attend to its immediate neighbors. For example, in the chunks `[a,b,c]` and `[d,e,f]`, the token `d` cannot attend to its immediate context `c`. As a remedy, Reformer augments each chunk with the parameters that control the number of previous neighboring chunks.

The most important contribution of Reformer is to leverage the **Locality Sensitive Hashing** (**LSH**) function, which assigns the same value to similar query vectors. Attention could be approximated by only comparing the most similar vectors, which helps us reduce the dimensionality and then sparsify the matrix. It is a safe operation since the `softmax` function is highly dominated by large values and can ignore dissimilar vectors. Additionally, instead of finding the relevant keys for a given query, only similar queries are found and clustered. That is, the position of a query can only attend to the positions of other queries to which it has a high cosine similarity.

To reduce the memory footprint, **Reformer** model uses reversible residual layers. It does not need to store the activations of all the layers. Because, the activations of any layer can be recovered from the activation of the following layer. A similar idea has been applied in **Reversible Residual Network** (**RevNet**).

It is important to note that the Reformer model and many other efficient transformers are criticized because, in practice, they are only more efficient than the vanilla transformer when the input length is very long (*Efficient Transformers: A Survey*, Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler). We made similar observations in our earlier experiments (please see the BigBird and Longformer experiment).

Now we will conduct some experiments with Reformer. Thanks to the Hugging Face platform, the Transformers library provides us with Reformer implementation and its pre-trained checkpoints. We will load the configuration of the original checkpoint, `google/reformer-enwik8`, and also tweak some settings to work in full self-attention mode. When we set `lsh_attn_chunk_length` and `local_attn_chunk_length` to `16384`, which is the maximum length that Reformer can

process, the Reformer instance will have no chance of local optimization and will automatically work like a vanilla transformer with full attention. We call it `fullReformer`. As for the original Reformer, we instantiate it with default parameters from the original checkpoint and call it `sparseReformer` as follows:

```
from transformers import ReformerConfig
fullReformer = ReformerConfig\
    .from_pretrained("google/reformer-enwik8",
        lsh_attn_chunk_length=16384,
        local_attn_chunk_length=16384)
sparseReformer = ReformerConfig\
    .from_pretrained("google/reformer-enwik8")
sequence_lengths=[256, 512, 1024, 2048, 4096, 8192, 12000]
models=["fullReformer","sparseReformer"]
configs=[eval(e) for e in models]
```

Please note that the Reformer model can process sequences up to a length of `16384`. But for the full self-attention mode, due to the accelerator capacity of our environment, the attention matrix does not fit on GPU, and we get a CUDA out of memory warning. Hence, we set the max length as `12000`. If your environment is suitable, you can increase it.

Let's run the benchmark experiments as follows:

```
benchmark_args = PyTorchBenchmarkArguments(
    sequence_lengths=sequence_lengths,
    batch_sizes=[1],
    models=models)
benchmark = PyTorchBenchmark(
    configs=configs,
    args=benchmark_args)
result = benchmark.run()
```

The output is as follows:

```
====================       INFERENCE - SPEED - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length     Time in s
--------------------------------------------------------------------------------
         fullReformer                1              256            0.024
         fullReformer                1              512            0.036
         fullReformer                1             1024            0.075
         fullReformer                1             2048            0.196
         fullReformer                1             4096            0.529
         fullReformer                1             8192            1.722
         fullReformer                1            12000            3.443
        sparseReformer               1              256            0.026
        sparseReformer               1              512            0.049
        sparseReformer               1             1024            0.084
        sparseReformer               1             2048            0.165
        sparseReformer               1             4096            0.296
        sparseReformer               1             8192            0.576
        sparseReformer               1            12000            0.869
--------------------------------------------------------------------------------

====================      INFERENCE - MEMORY - RESULT       ====================
--------------------------------------------------------------------------------
          Model Name             Batch Size     Seq Length    Memory in MB
--------------------------------------------------------------------------------
         fullReformer                1              256             1511
         fullReformer                1              512             1551
         fullReformer                1             1024             1671
         fullReformer                1             2048             2115
         fullReformer                1             4096             3791
         fullReformer                1             8192             8415
         fullReformer                1            12000            16191
        sparseReformer               1              256             1509
        sparseReformer               1              512             1705
        sparseReformer               1             1024             1885
        sparseReformer               1             2048             2339
        sparseReformer               1             4096             3143
        sparseReformer               1             8192             4803
        sparseReformer               1            12000             6367
--------------------------------------------------------------------------------
```

Figure 12.14 – Benchmark results

Let's visualize the time performance result as follows:

```
plotMe(result)
```

The output is the following:

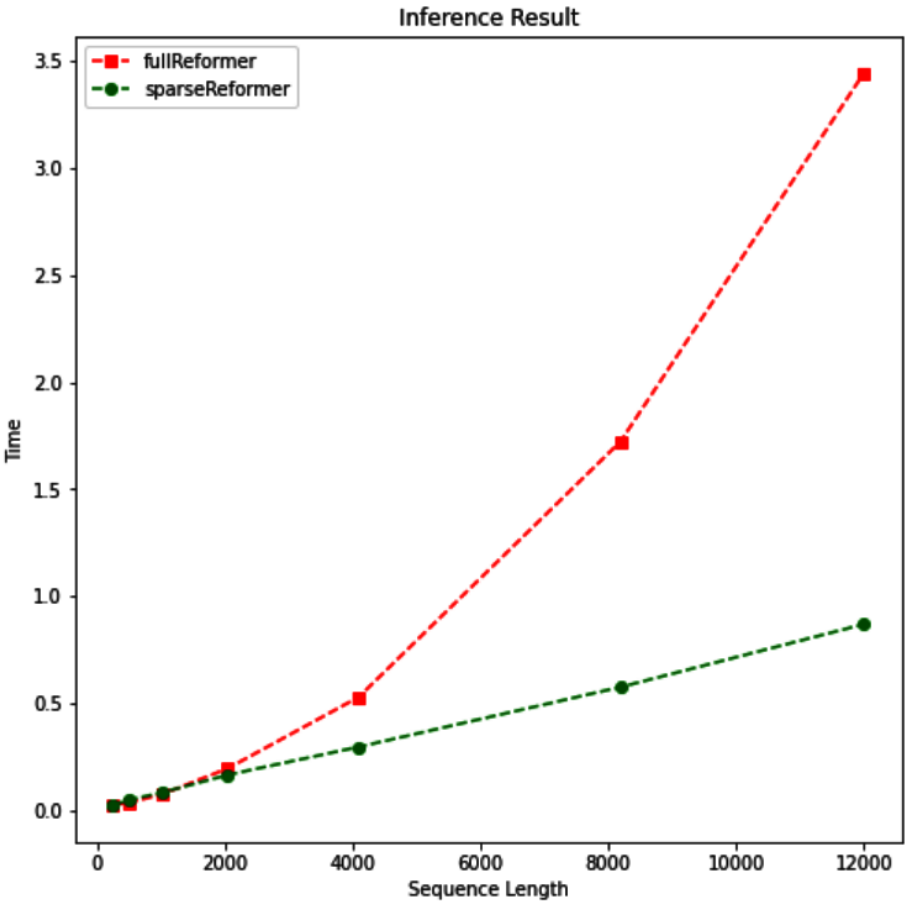


Figure 12.15 – Speed performance (Reformer)

We can see the linear and quadratic complexity of the models. We observe similar characteristics for the memory footprint by running the following line:

```
plotMe(result,"Memory Footprint")
```

It plots the following:

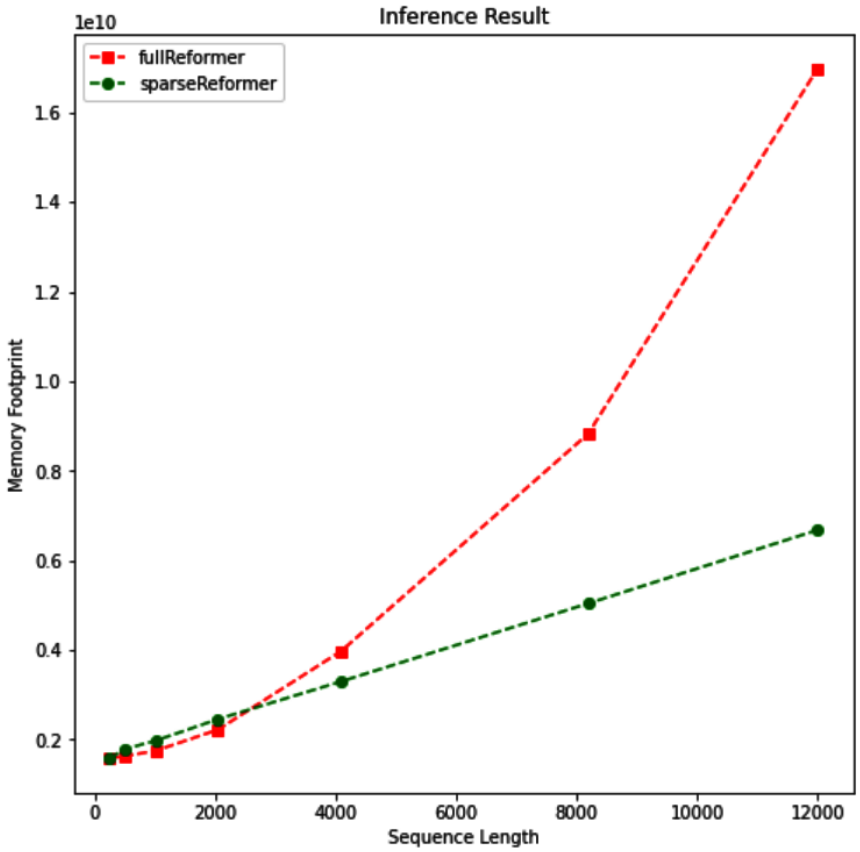


Figure 12.16 – Memory usage (Reformer)

Just as expected, Reformer with sparse attention produces a lightweight model. However, as mentioned before, we have difficulty observing the quadratic/linear complexity up to a certain length. As all these experiments indicate, efficient transformers can mitigate the time and memory complexity for longer text. What about task performance? How accurate would they be for classification or summarization tasks? To answer this, we will either start an experiment or have a look at the performance reports in the paper on Reformer model. For the experiment, you can repeat the code in *Chapter 4* and *Chapter 5* by instantiating an efficient model instead of a vanilla transformer. And you can track model performance and optimize it by using model tracking tools that we will discuss in detail in *Chapter 15*.

Let's take a look alternative types of efficient transformers, in addition to sparsification.

Low-rank factorization, kernel methods, and other approaches

The latest trend of the efficient model is to leverage low-rank approximations of the full self-attention matrix. These models are considered to be the lightest since they can reduce the self-attention complexity from $O(n^2)$ to O(n) in both computational time and memory footprint. Choosing a very small projection dimension k, such that $k << n$, then the memory and space complexity is highly reduced. **Linformer** and **Synthesizer** are the models that efficiently approximate the full attention with a low-rank factorization. They decompose the dot-product $N \times N$ attention of the original transformer through linear projections.

Kernel attention is another method family that we have seen lately to improve efficiency by viewing the attention mechanism through **kernelization**. A kernel is a function that takes two vectors as arguments and returns the product of their projection with a feature map. It enables us to operate in high-dimensional feature space without even computing the coordinates of the data in that high-dimensional space, because computations within that space become more expensive. This is when the kernel trick comes into play. The efficient models based on kernelization enable us to re-write the self-attention mechanism to prevent the explicit computation of the N×N matrix. In the realm of machine learning, the algorithm that frequently employs the kernel trick is the Support Vector Machines. This is where kernels such as the radial basis function or the polynomial kernel come into play, particularly when dealing with nonlinearity. For transformers, the most notable examples are **Performer** and **Linear Transformers**.

Easier quantization using bitsandbytes

Although quantization provides smaller models with reducing the accuracy, it is always important to use GPU friendly functions to get the most out of it. One of very useful libraries that implements NVidia CUDA custom functions for this specific use case (8-bit quantization) is **bitsandbytes**.

Hugging face's Transformers library also integrated this functionality for easier use. All you must do is to first change your runtime to GPU and then install bitsandbytes and accelerator:

```
pip install bitsandbytes
pip install accelerate
```

And then you can easily load any model you want using 8-bit quantization:

```
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained(
    'decapoda-research/llama-7b-hf',
    load_in_8bit=True)
```

As you can see, using the quantization with transformers is very easy; just setting `load_in_8bit` to `True` will do the job.

This is a useful technique for loading big models on machines with low memory and low computation power. However, it should be taken into consideration that this will decrease the model accuracy as well.

Summary

In this chapter, we have learned how to mitigate the burden of running large models with limited computational capacity. We first discussed how to make efficient models out of trained models using distillation, pruning, and quantization. It is important to pre-train a small general-purpose language model such as DistilBERT. This light model can then be fine-tuned with good performance on a wide variety of problems compared to their non-distilled counterparts.

Second, we have learned about efficient sparse transformers that replace the full self-attention matrix with a sparse one using approximation techniques such as Linformer, BigBird, and Performer. We have seen how they perform on various benchmarks, such as computational complexity and memory complexity. The examples showed us that these approaches can reduce quadratic complexity to linear complexity without sacrificing performance.

As we gather more data over time, we aim for our models to operate more quickly. In this regard, efficient transformers play a crucial role.

In the next chapter, we will discuss how to work with multiple languages.

References

- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108.
- Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., & Weller, A. (2020). *Rethinking attention with performers*. arXiv preprint arXiv:2009.14794.
- Wang, S., Li, B., Khabsa, M., Fang, H., & Ma, H. (2020). *Linformer: Self-attention with linear complexity*. arXiv preprint arXiv:2006.04768.
- Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., ... & Ahmed, A. (2020). *Big bird: Transformers for longer sequences*. arXiv preprint arXiv:2007.14062.
- Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2020). *Efficient transformers: A survey*. arXiv preprint arXiv:2009.06732.

- Tay, Y., Bahri, D., Metzler, D., Juan, D. C., Zhao, Z., & Zheng, C. (2020). *Synthesizer: Rethinking self-attention in transformer models*. arXiv preprint arXiv:2005.00743.
- Kitaev, N., Kaiser, Ł., & Levskaya, A. (2020). *Reformer: The efficient transformer*. arXiv preprint arXiv:2001.04451.
- Fournier, Q., Caron, G. M., & Aloise, D. (2021). *A Practical Survey on Faster and Lighter Transformers*. arXiv preprint arXiv:2103.14636.

13

Cross-Lingual and Multilingual Language Modeling

Up to this point, you have learned a lot about transformer-based architectures, from encoder-only models to decoder-only models and from efficient transformers to long-context transformers. You also learned about semantic text representation based on a Siamese network. However, we discussed all these models in terms of monolingual problems. We assumed that these models just understand a single language and are not capable of having a general understanding of text, regardless of the language itself. In fact, some of these models have multilingual variants: multilingual bidirectional encoder representations from transformers (mBERT), multilingual text-to-text transfer transformer (mT5), and multilingual bidirectional and auto-regressive transformer (mBART), to name but a few. On the other hand, some models are specifically designed for multilingual purposes and are trained with cross-lingual objectives. For example, cross-lingual language model (XLM) is such a method, and this will be described in detail in this chapter.

In this chapter, the concept of knowledge sharing between languages will be presented, and the impact of byte-pair encoding (BPE) on the tokenization part will also be covered in order to achieve better input. Cross-lingual sentence similarity using the Cross-Lingual Natural Language Inference (XNLI) corpus will be detailed. Tasks such as cross-lingual classification and the utilization of cross-lingual sentence representation for training on one language and testing on another one will be presented by giving concrete examples of real-life problems in natural language processing (NLP), such as multilingual intent classification.

Massive multilingual translation is also another important part of this chapter. Multilingual models, such as mBART and M2M100, can perform translation tasks in a massive number of languages as well.

In short, you will learn the following topics in this chapter:

- Translation language modeling and cross-lingual knowledge sharing
- XLM and mBERT
- Cross-lingual similarity tasks

- Cross-lingual classification
- Cross-lingual zero-shot learning
- Massive multilingual translation

Technical requirements

The code for this chapter is found in the repo at `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`, which is in the GitHub repository for this book. We will be using Jupyter Notebook to run our coding exercises that require `Python 3.6.0+`, and the following packages will need to be installed:

- `tensorflow`
- `pytorch`
- `transformers >=4.00`
- `datasets`
- `sentence-transformers`
- `umap-learn`
- `openpyxl`

Translation language modeling and cross-lingual knowledge sharing

So far, you have learned about **masked language modeling** (**MLM**) as a cloze task. However, language modeling using neural networks is divided into three categories based on the approach itself and its practical usage, as follows:

- **MLM**
- **Causal language modeling** (**CLM**)
- **Translation language modeling** (**TLM**)

It is also important to note that there are other pretraining approaches, such as next-sentence prediction (NSP) and sentence order prediction (SOP), but we only consider token-based language modeling. These three are the main approaches that are used in the literature. MLM, which is described and detailed in previous chapters, is a very close concept to a cloze task in language learning.

CLM is defined by predicting the next token, which is followed by some previous tokens. For example, if you see the following context, you can easily predict the next token:

```
<s> Transformers changed the natural language …
```

As you see, only the last token is masked, and the previous tokens are given to the model to predict the last one. This token would be **processing**, and if the context with this token is given to you again, you might end it with an "</s>" token. In order to have good training in this approach, it is required not to mask the first token because the model would have just a sentence start token to make a sentence out of it. This sentence can be anything! Here's an example:

```
<s> …
```

What would you predict from this? It can be literally anything. To obtain better training and better results, it is required to give at least the first token, such as this:

```
<s> Transformers …
```

The model is required to predict change; after giving it Transformers changed ..., it is required to predict the, and so on. This approach is very similar to N-grams and long-short-term memory (LSTM)-based approaches because it is left-to-right modeling based on the probability P(wn|wn-1, wn-2 ,…,w0), where wn is the token to be predicted, and the rest represents the tokens before it. The token with the maximum probability is the predicted one.

These are the objectives used for monolingual models. So, what can be done for cross-lingual models? The answer is TLM, which is very similar to MLM, with a few changes. Instead of giving a sentence from a single language, a sentence pair is given to a model in different languages, separated by a special token. The model is required to predict the masked tokens, which are randomly masked in any of these languages.

The following sentence pair is an example of such a task:

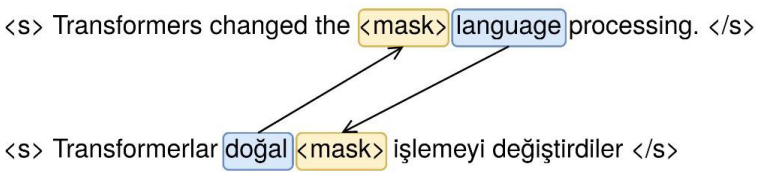


Figure 13.1 – Cross-lingual relation example between Turkish and English

Given these two masked sentences, the model is required to predict the missing tokens. In this task, on some occasions, the model has access to the tokens (for example, *doğal* and language, respectively, in the sentence pair in *Figure 13.1*) that are missing from one of the languages in the pair.

As another example, you can see the same pair in the Persian and Turkish sentences. In the second sentence, the *değiştirdiler* token can be attended by multiple tokens (one is masked) in the first sentence. In the following example, the word تغییر is missing, but the meaning of *değiştirdiler* is تغییر دادند:

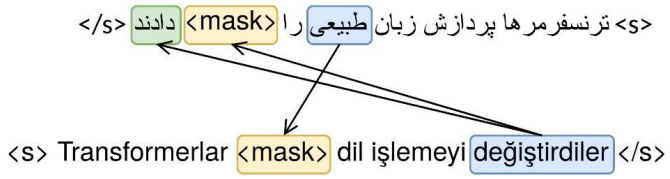


Figure 13.2 – Cross-lingual relation example between Persian and Turkish

Accordingly, a model can learn the mapping between these meanings. Just as with a translation model, our TLM must also learn these complexities between languages because machine translation (MT) is more than a token-to-token mapping.

So far, we learned how cross-lingual models can learn via translation language modeling. In the next part, two of the well-known models in this field will be explained further.

XLM and mBERT

In this section, we will delve into the explanation of two models: mBERT and XLM. The rationale behind choosing these models lies in their representation as the foremost examples of multilingual models; mBERT is a multilingual model trained on a different corpus of various languages using MLM modeling. It can operate separately for many languages. On the other hand, XLM is trained on different corpora using MLM, CLM, and TLM language modeling, and it can solve cross-lingual tasks. For instance, it can measure the similarity of the sentences in two different languages by mapping them in a common vector space, which is not possible with mBERT.

mBERT

You are familiar with the BERT autoencoder model from *Chapter 3*, and how to train it using MLM on a specified corpus. Imagine a case where a wide and huge corpus is provided not from a single language but from 104 languages instead. Training on such a corpus would result in a multilingual version of BERT. However, training using such a wide variety of languages would increase the model size, and this is inevitable in the case of BERT. The vocabulary size would be increased, and accordingly, the size of the embedding layer would be larger because of more vocabulary.

Compared to a monolingual pretrained BERT, this new version is capable of handling multiple languages inside a single model. However, the downside of this kind of modeling is that this model is not capable of mapping between languages. This means that the model, in the pretraining phase, does not learn anything about these mappings between the semantic meanings of the tokens from different languages. In order to provide cross-lingual mapping and understanding for this model, it is necessary to train it on some of the cross-lingual supervised tasks, such as those available in the **XNLI** dataset.

Using this model is as easy as working with the models you have used in the previous chapters (see `https://huggingface.co/bert-base-multilingual-uncased` for more details). Here's the code you'll need to get started:

```
from transformers import pipeline
unmasker = pipeline('fill-mask', model='bert-base-
    multilingual-uncased')
sentences = [
    "Transformers changed the [MASK] language processing",
    "Transformerlar [MASK] dil işlemeyi değiştirdiler",
    "ترنسفرمرها پردازش زبان [MASK] را تغییر دادند"
]
for sentence in sentences:
    print(sentence)
    print(unmasker(sentence)[0]["sequence"])
    print("="*50)
```

The output will then be presented, as shown in the following code snippet:

```
Transformers changed the [MASK] language processing
transformers changed the english language processing
==================================================
Transformerlar [MASK] dil işlemeyi değiştirdiler
transformerlar bu dil islemeyi degistirdiler
==================================================
ترنسفرمرها پردازش زبان [MASK] را تغییر دادند
ترنسفرمرها پردازش زبانی را تغییر دادند
==================================================
```

As you can see, it can perform fill-mask for various languages.

XLM

The cross-lingual pretraining of language models, such as that shown with an XLM approach, is based on three different pretraining objectives. MLM, CLM, and TLM are used to pretrain the XLM model. The sequential order of this pretraining is performed using a shared BPE tokenizer between all languages. The reason that tokens are shared is that the shared tokens provide fewer tokens in

the case of languages that have similar tokens or subwords, and, on the other hand, these tokens can provide shared semantics in the pretraining process. For example, some tokens have remarkably similar writing and meaning across many languages, and accordingly, these tokens are shared by BPE for all. In addition, some tokens that are spelled the same in different languages can have different meanings—for example, "was" is shared in German and English contexts. Luckily, self-attention mechanisms help us to disambiguate the meaning of "was" using the surrounding context.

Another major improvement of this cross-lingual modeling is that it is also pretrained on CLM, which makes it more reasonable for inferences where sentence prediction or completion is required. In other words, this model has an understanding of the languages and is capable of completing sentences, predicting missing tokens, and predicting missing tokens by using other language sources.

The following diagram shows the overall structure of cross-lingual modeling. You can read more at `https://arxiv.org/pdf/1901.07291.pdf`:

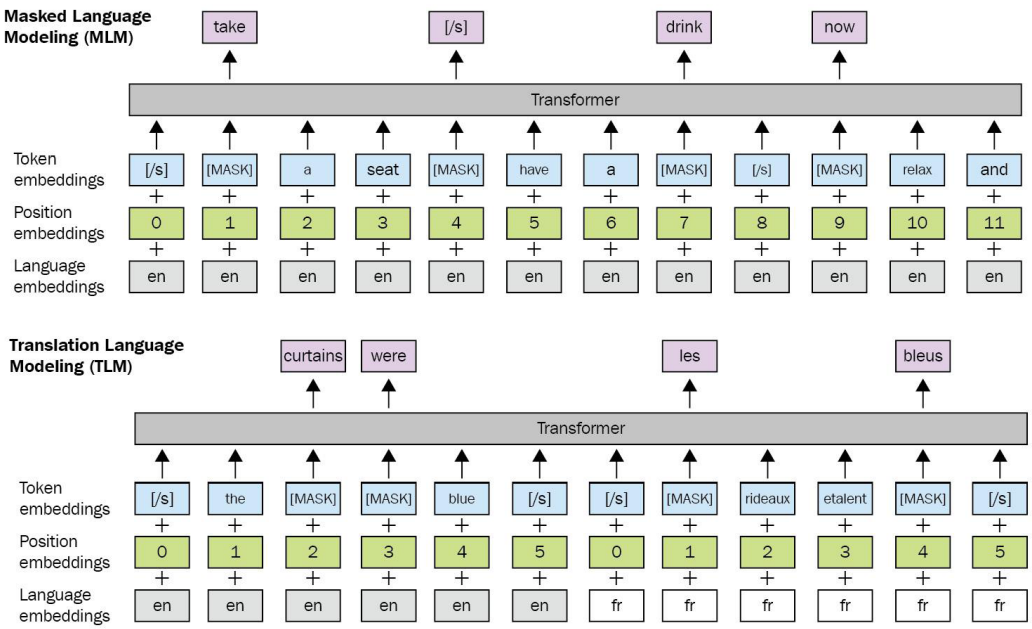


Figure 13.3 – MLM and TLM pretraining for cross-lingual modeling

A newer version of the XLM model was also released: XLM-R, which has minor changes in the training and corpus used. XLM-R is identical to the XLM model but is trained on more languages and a much bigger corpus. The CommonCrawl and Wikipedia corpus are aggregated, and the XLM-R is trained for MLM on this. However, the XNLI dataset is also used for TLM. The following diagram shows the amount of data used by XLM-R pretraining:

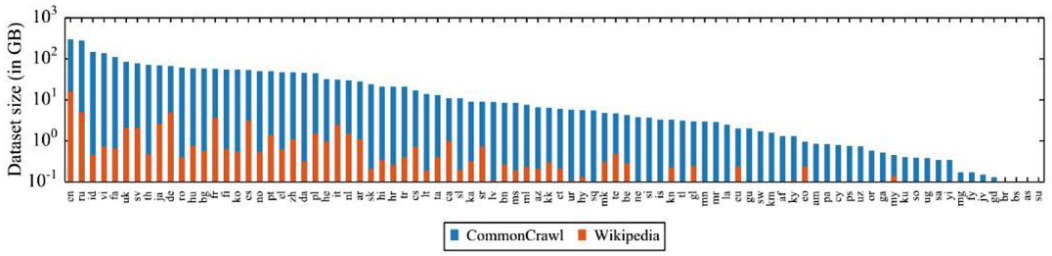


Figure 13.4 – Amount of data in gigabytes (GB) (log-scale)

There are many upsides and downsides when adding new languages for training data; for example, adding new languages may not always improve the overall model of natural language inference (NLI). The XNLI dataset is usually used for multilingual and cross-lingual NLI. From previous chapters, we have seen the Multi-Genre NLI (MNLI) dataset for English; the XNLI dataset is almost identical to it but has more languages, and it also has sentence pairs. However, training only on this task is not enough, and it will not cover TLM pretraining. For TLM pretraining, much broader datasets, such as the parallel corpus of OPUS (short for Open Source Parallel Corpus), are used. This dataset contains subtitles from different languages, which are aligned and cleaned, with the translations provided by many software sources such as Ubuntu.

The following screenshot shows OPUS (`https://opus.nlpl.eu/trac/`) and its components for searching and getting information about the dataset:

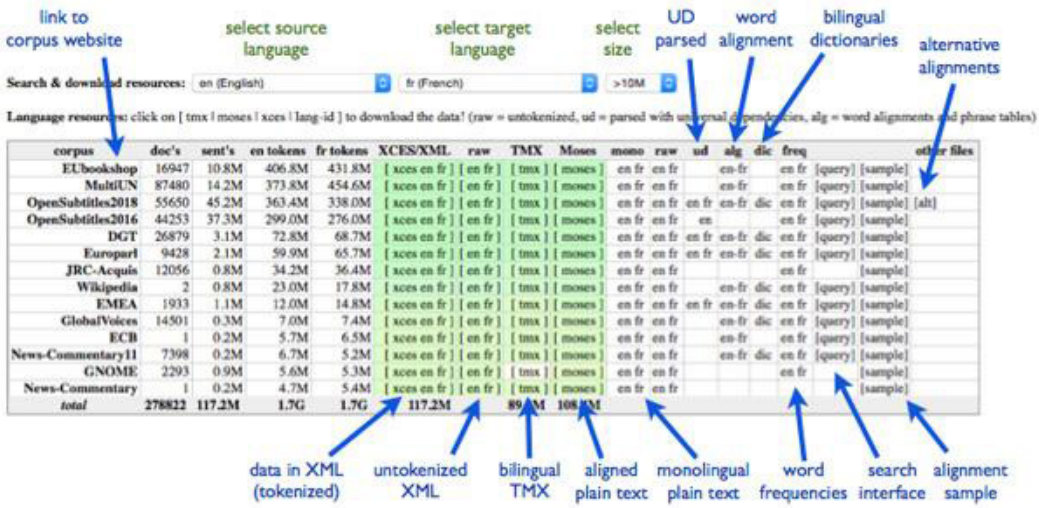


Figure 13.5 – An overview of OPUS

The steps for using cross-lingual models are described here:

1. Simple changes to the previous code can show you how XLM-R performs mask filling. First, you must change the model as follows:

```
unmasker = pipeline('fill-mask', model='xlm-roberta-base')
```

2. Afterward, you need to change the mask token from [MASK] to <mask>, which is a special token for XLM-R (or simply call tokenizer.mask_token). Here's the code to accomplish this:

```
sentences = [
"Transformers changed the <mask> language processing",
"Transformerlar <mask> dil işlemeyi değiştirdiler",
"ترنسفرمرها پردازش زبان <mask> را تغییر دادند"
]
```

3. Then, you can run the same code as follows:

```
for sentence in sentences:
    print(sentence)
    print(unmasker(sentence)[0]["sequence"])
    print("="*50)
```

4. The results will appear:

```
Transformers changed the <mask> language processing
Transformers changed the human language processing
==================================================
Transformerlar <mask> dil işlemeyi değiştirdiler
Transformerlar, dil işlemeyi değiştirdiler
================================================== ترنسفرمرها پردازش
زبان [MASK] را تغییر دادند
ترنسفرمرها پردازش زبانی را تغییر دادند
==================================================
```

5. However, as you can see from the Turkish and Persian examples, the model still made mistakes; for example, in the Persian text, it just added ی, and in the Turkish version, it added ,. For the English sentence, it added human, which is not what was expected. The sentences are not wrong, but they are not what we expected. However, this time, we have a cross-lingual model that is trained using TLM, so let's use it by concatenating two sentences and giving the model some extra hints. Here we go:

```
print(unmasker("Transformers changed the natural language
processing. </s> Transformerlar <mask> dil işlemeyi
değiştirdiler.")[0]["sequence"])
```

6. The results will be shown as follows:

```
Transformers changed the natural language processing.
Transformerlar doğal dil işlemeyi değiştirdiler.
```

7. That's it! The model has now made the right choice. Let's play with it a bit more and see how it performs:

```
print(unmasker("Earth is a great place to live in. </s> زمین جای
خوبی برای <mask> 0](".کردن است][["sequence"]))
```

Here is the result:

```
Earth is a great place to live in. زمین جای خوبی برای زندگی کردن است .
```

Well done! So far, you have learned about multilingual and cross-lingual models such as mBERT and XLM. In the next section, you will learn how to use such models for multilingual text similarity. You will also see some use cases, such as multilingual plagiarism detection.

Cross-lingual similarity tasks

Cross-lingual models are capable of representing text in a unified form, where sentences are taken from different languages but those with close meaning are mapped to similar vectors in vector space. XLM-R, as was detailed in the previous section, is one of the successful models in this scope. Now, let's look at some applications of this.

Cross-lingual text similarity

In the following example, you will see how it is possible to use a cross-lingual language model pre-trained on the XNLI dataset to find similar texts from different languages. A use-case scenario is where a plagiarism detection system is required for this task. We will use sentences from the Azerbaijani language and see whether XLM-R finds similar sentences from English—if there are any. The sentences from both languages are identical. Here are the steps to take:

1. First, you need to load a model for this task as follows:

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer("stsb-xlm-r-multilingual")
```

2. Afterward, we assume that we have sentences ready in the form of two separate lists, as illustrated in the following code snippet:

```
azeri_sentences = ['Pişik çöldə oturur',
    'Bir adam gitara çalır',
    'Mən makaron sevirəm',
    'Yeni film möhtəşəmdir',
    'Pişik bağda oynayır',
```

```
    'Bir qadın televizora baxır',
    'Yeni film çox möhtəşəmdir',
    'Pizzanı sevirsən?']
english_sentences = ['The cat sits outside',
    'A man is playing guitar',
    'I love pasta',
    'The new movie is awesome',
    'The cat plays in the garden',
    'A woman watches TV',
    'The new movie is so great',
    'Do you like pizza?']
```

3. The next step is to represent these sentences in vector space by using the XLM-R model. You can do this by simply using the encode function of the model as follows:

```
azeri_representation = model.encode(azeri_sentences)
english_representation = model.encode(english_sentences)
```

4. In the final step, we will search for semantically similar sentences of the first language against the other language's representations as follows:

```
results = []
for azeri_sentence, query in zip(azeri_sentences,
    azeri_representation):
    id_, score = util.semantic_search(
        query,english_representation)[0][0].values()
    results.append({
        "azeri": azeri_sentence,
        "english": english_sentences[id_],
        "score": round(score, 4)
    })
```

5. In order to see a clear form of these results, you can use a pandas DataFrame, as follows:

```
import pandas as pd
pd.DataFrame(results)
```

You will see the results with their matching score as follows:

	azeri	english	score
0	Pişik çöldə oturur	The cat sits outside	0.5969
1	Bir adam gitara çalır	A man is playing guitar	0.9939
2	Mən makaron sevirəm	I love pasta	0.6878
3	Yeni film möhtəşəmdir	The new movie is so great	0.9757
4	Pişik bağda oynayır	A man is playing guitar	0.2695
5	Bir qadın televizora baxır	A woman watches TV	0.9946
6	Yeni film çox möhtəşəmdir	The new movie is so great	0.9797
7	Pizzanı sevirsən?	Do you like pizza?	0.9894

Figure 13.6 – Plagiarism detection results (XLM-R)

The model made mistakes in one case (row number **4**) if we accept the maximum scored sentence to be paraphrased or translated, but it is useful to have a threshold and accept values higher than it. We will show more comprehensive experimentation in the following sections.

On the other hand, alternative bi-encoders are available, too. Such approaches provide a pair encoding of two sentences and classify the result to train the model. In such cases, language-agnostic bert sentence embedding (LaBSE) may be a good choice, too, and it is available in the sentence-transformers library and in TensorFlow Hub, too. LaBSE is a dual encoder based on transformers, which is similar to Sentence-BERT, where two encoders that have the same parameters are combined with a loss function based on the dual similarity of two sentences.

Using the same example, you can change the model to LaBSE in a very simple way and rerun the previous code (Step 1), as follows:

```
model = SentenceTransformer("LaBSE")
```

The results are shown in the following screenshot:

	azeri	english	score
0	Pişik çöldə oturur	The cat sits outside	0.8686
1	Bir adam gitara çalır	A man is playing guitar	0.9143
2	Mən makaron sevirəm	I love pasta	0.8888
3	Yeni film möhtəşəmdir	The new movie is so great	0.9107
4	Pişik bağda oynayır	The cat sits outside	0.6761
5	Bir qadın televizora baxır	A woman watches TV	0.9359
6	Yeni film çox möhtəşəmdir	The new movie is so great	0.9258
7	Pizzanı sevirsən?	Do you like pizza?	0.9366

Figure 13.7 – Plagiarism detection results (LaBSE)

As you see, LaBSE performs better in this case, and the result in row number **4** is correct this time. LaBSE authors claim that it works very well in finding translations of sentences, but it is not so good at finding sentences that are not completely identical. For this purpose, it is a very useful tool for finding plagiarism in cases where a translation is used to steal intellectual material. However, there are many other factors that change the results, too—for example, the resource size for the pre-trained model in each language and the nature of the language pairs is also important. For a reasonable comparison, we need a more comprehensive experiment, and we should consider many factors.

Visualizing cross-lingual textual similarity

Now, we will measure and visualize the degree of textual similarity between two sentences, one of which is a translation of the other. Tatoeba is a free collection of such sentences and translations, and it is part of the XTREME benchmark. The community aims to obtain high-quality sentence translation with the support of many participants. We'll now take the following steps:

1. We will take Russian and English sentences out of this collection. Make sure the following libraries are installed before you start working:

```
!pip install sentence_transformers datasets transformers umap-
learn
```

2. Load the sentence pairs as follows:

```
from datasets import load_dataset
import pandas as pd
data=load_dataset("xtreme","tatoeba.rus",
    split="validation")
pd.DataFrame(data)[["source_sentence","target_sentence"]]
```

3. Let's look at the output:

	source_sentence	target_sentence
0	Я знаю много людей, у которых нет прав.\n	I know a lot of people who don't have driver's...
1	У меня много знакомых, которые не умеют играть...	I know a lot of people who don't know how to p...
2	Мой начальник отпустил меня сегодня пораньше.\n	My boss let me leave early today.\n
3	Я загорел на пляже.\n	I tanned myself on the beach.\n
4	Вы сегодня проверяли почту?\n	Have you checked your email today?\n
...	...	...
995	Что сказал врач?\n	What did the doctor say?\n
996	Я рад, что ты сегодня здесь.\n	I'm glad you're here today.\n
997	Фермеры пригнали в деревню пять волов, девять ...	The farmers had brought five oxen and nine cow...
998	Жужжание пчёл заставляет меня немного нервнича...	The buzzing of the bees makes me a little nerv...
999	С каждым годом они становились всё беднее.\n	From year to year they were growing poorer.\n

1000 rows × 2 columns

Figure 13.8 – Russian–English sentence pairs

4. First, we will take the first K=30 sentence pairs for visualization, and later, we will run an experiment for the entire set. Now, we will encode them with the sentence transformers that we already used for the previous example. Here is the execution of the code:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("stsb-xlm-r-multilingual")
K=30
q=data["source_sentence"][:K] + data["target_sentence"][:K]
emb=model.encode(q)
len(emb), len(emb[0])
Output: (60, 768)
```

5. We now have 60 vectors of length 768. We will reduce the dimensionality to 2 by using uniform manifold approximation and projection (UMAP), which we have already encountered in previous chapters. We visualized sentences that are translations of each other, marking them with the same color and code. We also drew a dashed line between them to make the link more obvious. The code is illustrated in the following snippet:

```
import matplotlib.pyplot as plt
import numpy as np
import umap
import pylab
```

```
X= umap.UMAP(n_components=2, random_state=42).fit_transform(emb)
    idx= np.arange(len(emb))
fig, ax = plt.subplots(figsize=(12, 12))
ax.set_facecolor('whitesmoke')
cm = pylab.get_cmap("prism")
colors = list(cm(1.0*i/K) for i in range(K))
for i in idx:
    if i<K:
        ax.annotate("RUS-"+str(i), # text
            (X[i,0], X[i,1]), # coordinates
            c=colors[i]) # color
        ax.plot((X[i,0],X[i+K,0]),(X[i,1],X[i+K,1]),"k:")
    else:
        ax.annotate("EN-"+str(i%K),
            (X[i,0], X[i,1]),
            c=colors[i%K])
```

6. Here is the output of the preceding code:

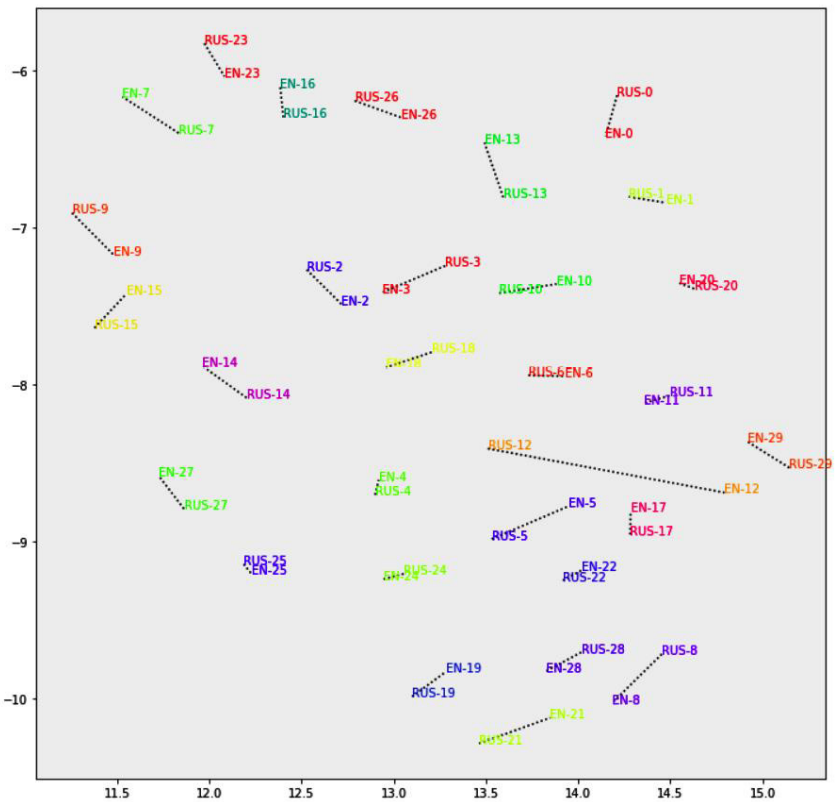


Figure 13.9 – An overview of Russian–English sentence similarity visualization

As we expected, most sentence pairs are located close to each other. Inevitably, certain pairs (such as id 12) insist on not being close.

7. For a comprehensive analysis, let's now measure the entire dataset. We encode all of the source and target sentences—1K pairs—as follows:

```
source_emb=model.encode(data["source_sentence"])
target_emb=model.encode(data["target_sentence"])
```

8. We calculate the cosine similarity between all pairs, save them in the sims variable, and plot a histogram as follows:

```
from scipy import spatial
sims=[ 1 - spatial.distance.cosine(s,t) \
    for s,t in zip(source_emb, target_emb)]
plt.hist(sims, bins=100, range=(0.8,1))
plt.show()
```

9. Here is the output:

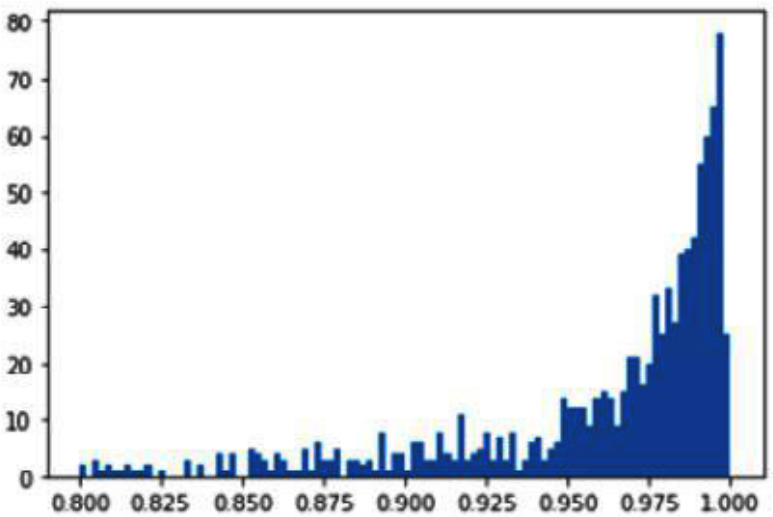


Figure 13.10 – Similarity histogram for the English and Russian sentence pairs

10. As can be seen, the scores are very close to `1`. This is what we expect from a good cross-lingual model. The mean and standard deviation of all similarity measurements also support the cross-lingual model performance, as follows:

```
>>> np.mean(sims), np.std(sims)
(0.946, 0.082)
```

11. You can run the same code yourself for languages other than Russian. As you run it with French (fra), Tamil (tam), and so on, you will get the following resulting table. The table indicates that in your experiment, you will see that the model works well in many languages but fails in others, such as Afrikaans or Tamil:

Language	Code	Mean	Std
French	fra	0.94	0.087
Afrikaans	afr	0.79	0.18
Arabic	ara	0.94	0.08
Korean	kor	0.92	0.11
Tamil	tam	0.77	0.19

Table 13.1 – Cross-lingual model performance for other languages

In this section, we applied cross-lingual models to measure similarity between different languages. In the next section, we'll make use of cross-lingual models in a supervised way.

Cross-lingual classification

So far, you have learned that cross-lingual models are capable of understanding different languages in semantic vector space where similar sentences, regardless of their language, are close in terms of vector distance. However, how it is possible to use this capability in use cases where we have few samples available?

For example, you are trying to develop an intent classification for a chatbot in which there are few samples or no samples available for the second language; however, for the first language—let's say English—you do have enough samples. In such cases, it is possible to freeze the cross-lingual model itself and just train a classifier for the task. A trained classifier can be tested on a second language instead of the language it is trained on.

In this section, you will learn how to train a cross-lingual model in English for text classification and test it in other languages. We have selected a very low-resource language known as Khmer, which is spoken by 16 million people in Cambodia, Thailand, and Vietnam. It has few resources on the internet, and it is hard to find good datasets with which to train your model. However, we have access to a good Internet Movie Database (IMDb) sentiment dataset of movie reviews for sentiment analysis. We will use that dataset to find out how our model performs in a language on which it is not trained.

The following diagram nicely depicts the kind of flow we will follow. The model is trained with training data on the left, and this model is applied to the test sets on the right. Please notice that MT and sentence-encoder mappings play a significant role in the flow:

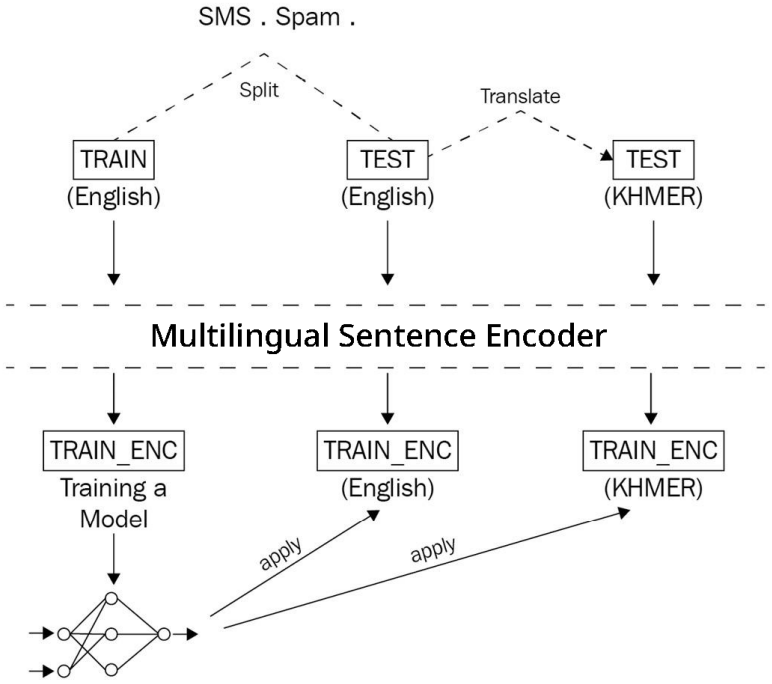


Figure 13.11 – Flow of cross-lingual classification

The required steps to load and train a model for cross-lingual testing are outlined here:

1. The first step is to load the dataset as follows:

```
from datasets import load_dataset
sms_spam = load_dataset("imdb")
```

2. You need to shuffle the dataset to shuffle the samples before using them, as follows:

```
imdb = imdb.shuffle()
```

3. The next step is to make a good test split out of this dataset, which is in the Khmer language. In order to do so, you can use a translation service such as Google Translate. First, you should save this dataset in Excel format as follows:

```
imdb_x = [x for x in imdb['train'][:1000]['text']]
labels = [x for x in imdb['train'][:1000]['label']]
import pandas as pd
pd.DataFrame(imdb_x,
    columns=["text"]).to_excel(
    "imdb.xlsx",
    index=None)
```

4. Afterward, you can upload it to Google Translate and get the Khmer translation of this dataset (`https://translate.google.com/?sl=en&tl=km&op=docs`), as illustrated in the following screenshot:

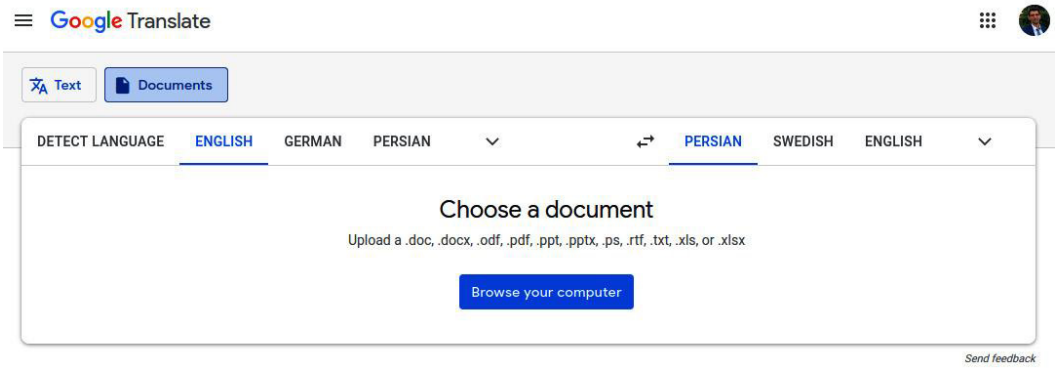


Figure 13.12 – Google document translator

5. After selecting and uploading the document, you will obtain the translated version in Khmer, which you can copy and paste into an Excel file. It is also required to be saved in Excel format again. The result is an Excel document that is a translation of the original spam/ham English dataset. You can read it using pandas by running the following command:

```
pd.read_excel("KHMER.xlsx")
```

The following results will be seen:

	text
0	ទៅរហូតដល់ចំណុចចុងឆ្នូត .. អាចប្រើបានតែនៅថៃហា...
1	អ្នកឡូ ... ចូជឺ wif u oni ...
2	ធ្វើសារដោយឥតគិតថ្លៃក្នុងលេខ ២ ដោយឥតគិតថ្លៃដើម...
3	U dun និយាយអញ្ចឹងមុនគេ ... U c និយាយរួចទៅហើយ ...។
4	ទេខ្ញុំមិនគិតថាគាត់ទៅកេរយើងទេគាត់រស់នៅទីនេះ
...	...
5569	នេះជាលើកទី ២ ហើយដែលយើងបានព្យាយាមទំនាក់ទំនង ២ ។...
5570	តើប៉ុនឹងទៅផ្ទះ esplanade fr?
5571	គួរឱ្យអាណិត, * គឺនៅក្នុងអារម្មណ៍សម្រាប់រឿងនោះ។...
5572	បុរសម្នាក់នេះបានធ្វើឱ្យចែចង់ខ្លះប៉ុន្តែខ្ញុំធ្...
5573	Rofl ។ វាជាឈ្មោះពិត

5574 rows × 1 columns

Figure 13.13 – IMDB dataset in KHMER language

6. However, we only need to obtain text, so you should use the following code:

```
imdb_khmer = list(pd.read_excel("KHMER.xlsx").text)
```

7. Now that you have text for both languages and the labels, you can split the train and test validations as follows:

```
from sklearn.model_selection import train_test_split
train_x, test_x, train_y, test_y, khmer_train, khmer_test = \
    train_test_split(imdb_x, labels, imdb_khmer,
        test_size = 0.2, random_state = 1)
```

8. The next step is to provide the representation of these sentences using the XLM-R cross-lingual model. First, you should load the model as follows:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("stsb-xlm-r-multilingual")
```

9. Now, you can obtain the representations:

```
encoded_train = model.encode(train_x)
encoded_test = model.encode(test_x)
encoded_khmer_test = model.encode(khmer_test)
```

10. However, you should not forget to convert the labels into the NumPy format because TensorFlow and Keras only deal with NumPy arrays when using the fit function of the Keras models. Here's how to do it:

```
import numpy as np
train_y = np.array(train_y)
test_y = np.array(test_y)
```

11. Now that everything is ready, let's make a very simple model for classifying the representations:

```
import tensorflow as tf
input_ = tf.keras.layers.Input((768,))
classification = tf.keras.layers.Dense(
    1, activation="sigmoid")(input_)
classification_model = \
    tf.keras.Model(input_, classification)
classification_model.compile(
    loss=tf.keras.losses.BinaryCrossentropy(),
    optimizer="Adam",
    metrics=["accuracy", "Precision", "Recall"])
```

12. You can fit your model using the following function:

```
classification_model.fit(
    x = encoded_train,
    y = train_y,
    validation_data=(encoded_test, test_y),
    epochs = 10)
```

13. The results for 20 epochs of training are shown here:

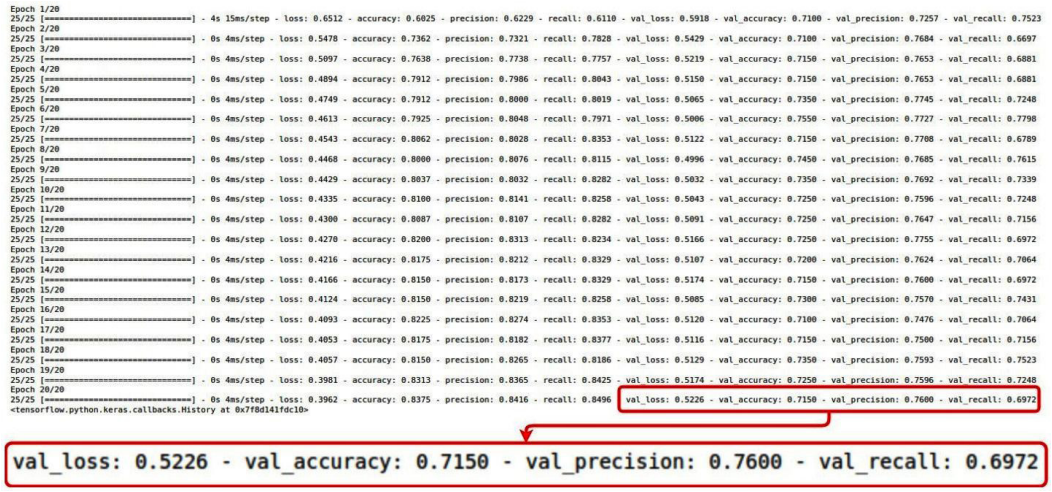


Figure 13.14 – Training results on the English version of the IMDb dataset

14. As you have seen, we used an English test set to see the model performance across epochs, and it is reported in the final epoch as follows:

```
val_loss: 0.5226
val_accuracy: 0.7150
val_precision: 0.7600
val_recall: 0.6972
```

15. Now, we have trained our model and tested it on English; let's test it on the Khmer test set, as our model had not seen any of the samples in either English or Khmer. Here's the code to accomplish this:

```
classification_model.evaluate(x = encoded_khmer_test,
    y = test_y)
```

Here are the results:

```
loss: 0.5949
accuracy: 0.7250
precision: 0.7014
recall: 0.8623
```

So far, you have learned how it is possible to leverage the capabilities of cross-lingual models in low-resource languages. It makes a huge impact and difference when you can use such a capability in cases where there are very few samples or no samples on which to train the model. In the next section, you will learn how it is possible to use zero-shot learning where there are no samples available, even for high-resource languages such as English.

Cross-lingual zero-shot learning

In the previous sections, you learned how to perform zero-shot text classification using monolingual models. Using XLM-R for multilingual and cross-lingual zero-shot classification is identical to the approach and code used previously, so we will use mT5 here.

mT5, which is a massively multilingual pretrained language model, is based on the encoder-decoder architecture of Transformers and is also identical to T5. T5 is pretrained on English and mT5 is trained on 101 languages from Multilingual Common Crawl (mC4).

The fine-tuned version of mT5 on the XNLI dataset is available from the Hugging Face repository (`https://huggingface.co/alan-turing-institute/mt5-large-finetuned-mnli-xtreme-xnli`).

The T5 model and its variant, mT5, are completely text-to-text models, which means they will produce text for any task given, even if the task is classification or NLI. So, in the case of inferring this model, extra steps are required. We'll take the following steps:

1. The first step is to load the model and the tokenizer as follows:

```
from torch.nn.functional import softmax
from transformers import(
    MT5ForConditionalGeneration, MT5Tokenizer)
model_name = "alan-turing-institute/mt5-large-finetuned-mnli-
xtreme-xnli"
tokenizer = MT5Tokenizer.from_pretrained(model_name)
model = MT5ForConditionalGeneration\
    .from_pretrained(model_name)
```

2. In the next step, let's provide samples to be used in zero-shot classification—a sentence and labels—as follows:

```
sequence_to_classify = \
    "Wen arden Sie bei der nächsten Wahl wählen? "
candidate_labels = ["spor", "ekonomi", "politika"]
hypothesis_template = "Dieses Beispiel ist {}."
```

As you can see, the sequence itself is in German ("Who will you vote for in the next election?") but the labels are written in Turkish ("spor", "ekonomi", "politika"). `hypothesis_template` says: "this example is ..." in German.

3. The next step is to set the label identifiers (IDs) of the entailment, CONTRADICTS, and NEUTRAL, which will be used later when inferring the generated results. Here's the code you'll need to do this:

```
ENTAILS_LABEL = "_0"
NEUTRAL_LABEL = "_1"
CONTRADICTS_LABEL = "_2"
label_inds = tokenizer.convert_tokens_to_ids([
    ENTAILS_LABEL,
    NEUTRAL_LABEL,
    CONTRADICTS_LABEL])
```

4. As you'll recall, the T5 model uses prefixes to know which task it is supposed to perform. The following function provides the XNLI prefix, along with the premise and hypothesis in the proper format:

```
def process_nli(premise, hypothesis):
    return f'xnli: premise: {premise} hypothesis: {hypothesis}'
```

5. In the next step, for each label, a sentence will be generated, as illustrated in the following code snippet:

```
pairs =[(sequence_to_classify,\
    hypothesis_template.format(label)) for label in
    candidate_labels]
seqs = [process_nli(premise=premise,
    hypothesis=hypothesis)
    for premise, hypothesis in pairs]
```

6. You can see the resulting sequences by printing them as follows:

```
print(seqs)
['xnli: premise: Wen werden Sie bei der nächsten Wahl
wählen?  hypothesis: Dieses Beispiel ist spor.',
'xnli: premise: Wen werden Sie bei der nächsten Wahl
wählen?  hypothesis: Dieses Beispiel ist ekonomi.',
'xnli: premise: Wen werden Sie bei der nächsten Wahl
wählen?  hypothesis: Dieses Beispiel ist politika.']
```

These sequences simply say that the task is XNLI-coded by xnli: ; the premise sentence is "Who will you vote for in the next election?" (in German), and the hypothesis is "this example is politics", "this example is a sport" or "this example is economy".

7. In the next step, you can tokenize the sequences and give them to the model to generate the text according to it:

```
inputs = tokenizer.batch_encode_plus(seqs,
    return_tensors="pt", padding=True)
out = model.generate(**inputs, output_scores=True,
    return_dict_in_generate=True,num_beams=1)
```

8. The generated text actually gives scores for each token in the vocabulary, and what we are looking for is the entailment, contradiction, and neutral scores. You can get their score using their token IDs as follows:

```
scores = out.scores[0]
scores = scores[:, label_inds]
```

9. You can see these scores by printing them:

```
print(scores)
tensor([[-0.9851,  2.2550, -0.0783],
        [-5.1690, -0.7202, -2.5855],
        [ 2.7442,  3.6727,  0.7169]])
```

10. The neutral score is not required for our purpose, and we only need contradiction compared to entailment. So, you can use the following code to obtain these scores only:

```
entailment_ind = 0
contradiction_ind = 2
entail_vs_contra_scores = scores[:, [entailment_ind,
    contradiction_ind]]
```

11. Now that you have these scores for each sequence of the samples, you can apply a softmax layer to them to obtain the probabilities as follows:

```
entail_vs_contra_probas = softmax(entail_vs_contra_scores,
    dim=1)
```

12. To see these probabilities, you can use print:

```
print(entail_vs_contra_probas)
tensor([[0.2877, 0.7123],
        [0.0702, 0.9298],
        [0.8836, 0.1164]])
```

13. Now, you can compare the entailment probability of these three samples by selecting them and applying a softmax layer over them as follows:

```
entail_scores = scores[:, entailment_ind]
entail_probas = softmax(entail_scores, dim=0)
```

14. To see the values, use print as follows:

```
print(entail_probas)
tensor([2.3438e-02, 3.5716e-04, 9.7620e-01])
```

15. The result means the highest probability belongs to the third sequence. In order to see it in a better shape, use the following code:

```
print(dict(zip(candidate_labels, entail_probas.tolist())))
{'ekonomi': 0.0003571564157027751,
'politika': 0.9762046933174133,
'spor': 0.023438096046447754}
```

The whole process can be summarized as follows: each label is given to the model with the premise, and the model generates scores for each token in the vocabulary. We use these scores to find out how much the entailment token scores over the contradiction.

Massive multilingual translation

Speaking of cross-lingual models and models that can perform well on multilingual data, one of the tasks that are always important in this field is translation. However, different approaches exist for translation; some approaches suggest having different models for each language pair, whereas others suggest using a single model for all of the language pairs.

M2M100 is one of the most important models in this field and can translate 9,900 directions in 100 languages. Using this model and performing translation tasks is simple with the help of the `transformers` library:

```
from transformers import (
    M2M100ForConditionalGeneration, M2M100Tokenizer)
model = M2M100ForConditionalGeneration.from_pretrained(
    "facebook/m2m100_1.2B")
tokenizer = M2M100Tokenizer.from_pretrained(
    "facebook/m2m100_1.2B")
```

Before giving the input to the model, you need to also set the source language of the text as well:

```
text_turkish = "ne olacak bu insanlik hali?"
tokenizer.src_lang = "tr"
encoded_text = tokenizer(text_turkish, return_tensors="pt")
```

To generate the output, the language id of the target language should be passed to the model as well, as it will guide the model to generate the translation in the given language:

```
generated_tokens = model.generate(**encoded_text,
    forced_bos_token_id=tokenizer.get_lang_id("fa"))
text_persian = tokenizer.batch_decode(generated_tokens,
    skip_special_tokens=True)
```

Now, you can see the translated output in Persian:

```
print(text_persian[0])
این وضعیت بشری چه خواهد بود؟
```

You can use any of the 101 languages supported by the model for source and destination text. So far, you have learned how massive machine translation models work. In the next part, you will learn more about the fundamental limitations of multilingual models.

Despite the potential of multilingual and cross-lingual models in NLP, their effectiveness is limited in smaller language models such as BERT. This has been the focus of many studies. For example, the mBERT model doesn't perform as well as monolingual models in various tasks, which is why the latter is still commonly used, particularly in classification tasks. On the other hand, recent developments in larger language models have largely addressed the limitations of multilingual models and improved their capacity. This improvement is attributed to an increase in parameters, expanded training datasets, and advanced training methods, such as reinforcement learning. Consequently, there's been a rise in successful multilingual models such as **mT5**, **Bloomz**, **Bactrian-x**, and **Aya-101**. These models are open source and, thus, accessible.

Studies in the field indicate that multilingual models suffer from the so-called **curse of multilingualism** as they seek to appropriately represent all languages. Adding new languages to a multilingual model improves its performance up to a certain point. However, it is also seen that adding more after this point degrades performance, which may be due to shared vocabulary. Compared to monolingual models, multilingual models are significantly more limited in terms of the parameter budget. They need to allocate their vocabulary to each of more than 100 languages.

The existing performance differences between mono- and multilingual models can be attributed to the capability of the designated tokenizer. The study *How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models (2021) by Rust et al.* (`https://arxiv.org/abs/2012.15613`) showed that when a dedicated language-specific tokenizer (rather than a general-purpose one, e.g., a shared multilingual tokenizer) is attached to a multilingual model, it improves in performance for that language.

Some other findings indicate that it is not currently possible to represent all the world's languages in a single model due to an imbalance in the distribution of resources for different languages. As a solution, low-resource languages can be oversampled, while high-resource languages can be undersampled. Another observation is that knowledge transfer between two languages can be more efficient if those languages are close. If they are distant languages, this transfer may have little effect. This observation may explain why we got worse results for Afrikaans and Tamil languages in the previous cross-lingual sentence-pair experiment part.

However, there is a lot of work on this subject, and these limitations may be overcome at any time. As of writing this article, the team of XML-R recently proposed two new models—namely, XLM-R XL and XLM-R XXL—that outperform the original XLM-R model by 1.8% and 2.4% average accuracies, respectively, on XNLI.

Fine-tuning the performance of multilingual models

Now, let's check whether the fine-tuned performance of the multilingual models is actually worse than that of the monolingual models. As an example, let's recall the example of Turkish text classification with seven classes in *Chapter 5*. In that experiment, we fine-tuned a Turkish-specific monolingual model and achieved a good result. We will repeat the same experiment, keeping everything as-is but replacing the Turkish monolingual model with the mBERT and XLM-R models, respectively. Here's how we'll do this:

1. Let's recall the codes in that example again. We fine-tuned the "dbmdz/bert-base-turkish-uncased" model as follows:

```
from transformers import BertTokenizerFast
tokenizer = BertTokenizerFast.from_pretrained(
    "dbmdz/bert-base-turkish-uncased")
from transformers import BertForSequenceClassification
model = BertForSequenceClassification.from_pretrained(\
```

```
    "dbmdz/bert-base-turkish-uncased",
    num_labels=NUM_LABELS,
    id2label=id2label,
    label2id=label2id)
```

With the monolingual model, we got the following performance values:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.091844	0.975510	0.97546	0.975942	0.975535
val	0.280120	0.924898	0.92381	0.924427	0.924510
test	0.280038	0.926531	0.92542	0.927410	0.925425

Figure 13.15 – Monolingual text classification performance (from *Chapter 5*)

2. To fine-tune using mBERT, we need to replace the preceding model instantiation lines only. Now, we will use the "bert-base-multilingual-uncased" multilingual model. We instantiate it like this:

```
from transformers import(
    BertForSequenceClassification, AutoTokenizer)
tokenizer = AutoTokenizer.from_pretrained(
    "bert-base-multilingual-uncased")
model = BertForSequenceClassification.from_pretrained(
    "bert-base-multilingual-uncased",
    num_labels=NUM_LABELS,
    id2label=id2label,
    label2id=label2id)
```

3. There is not much difference in coding. When we run the experiment keeping all other parameters and settings the same, we get the following performance values:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.093405	0.978367	0.978373	0.978547	0.978291
val	0.325458	0.911837	0.911586	0.911678	0.911592
test	0.372160	0.904490	0.903152	0.902647	0.904335

Figure 13.16 – mBERT fine-tuned performance

Hmm! The multilingual model underperforms compared with its monolingual counterpart by roughly 2.2% for all metrics.

4. Let's fine-tune the "xlm-roberta-base" XLM-R model for the same problem. We'll execute the XLM-R model initialization code as follows:

```
from transformers import (
    AutoTokenizer, XLMRobertaForSequenceClassification)
tokenizer = AutoTokenizer.from_pretrained(
    "xlm-roberta-base")
model = XLMRobertaForSequenceClassification\
    .from_pretrained("xlm-roberta-base",
        num_labels=NUM_LABELS,
        id2label=id2label,label2id=label2id)
```

5. Again, we keep all other settings exactly the same. We get the following performance values from the XML-R model:

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.122369	0.968571	0.968665	0.968830	0.968862
val	0.339011	0.912653	0.912454	0.913331	0.912042
test	0.334882	0.915918	0.915662	0.918334	0.914893

Figure 13.17 – XLM-R fine-tuned performance

Not bad! The XLM model did give comparable results. The obtained results are quite close to the monolingual model, with a roughly 1.0% difference. Therefore, although monolingual results can be better than multilingual models in certain tasks, we can achieve promising results with multilingual models. Think of it this way: we may not want to train a whole monolingual model for a 1% improvement in performance that lasts 10 days or more. Such small performance differences may be negligible for us.

Summary

In this chapter, you learned about multilingual and cross-lingual language model pretraining and the difference between monolingual and multilingual pretraining. CLM and TLM were also covered, and you gained knowledge about them. You learned how it is possible to use cross-lingual models on various use cases, such as semantic search, plagiarism, and zero-shot text classification. You also learned how it is possible to train using a dataset from a language and test the model on a completely different language using cross-lingual models. Fine-tuning the performance of multilingual models was evaluated, and we concluded that some multilingual models could be a substitute for monolingual models, remarkably keeping performance loss to a minimum. We also took a look at the models that can translate at a massive scale; M2M100, for example, can translate in 9,900 directions in 100 languages, and we learned how to use this model.

In the next chapter, you will learn how to deploy transformer models for real problems and train them for production at an industrial scale.

References

- Conneau, A., Lample, G., Rinott, R., Williams, A., Bowman, S. R., Schwenk, H. and Stoyanov, V. (2018). *XNLI: Evaluating cross-lingual sentence representations*. arXiv preprint arXiv:1809.05053.
- Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A. and Raffel, C. (2020). *mT5: A massively multilingual pre-trained text-to-text transformer*. arXiv preprint arXiv:2010.11934.
- Lample, G. and Conneau, A. (2019). *Cross-lingual language model pretraining*. arXiv preprint arXiv:1901.07291.
- Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F. and Stoyanov, V. (2019). *Unsupervised cross-lingual representation learning at scale*. arXiv preprint arXiv:1911.02116.
- Feng, F., Yang, Y., Cer, D., Arivazhagan, N. and Wang, W. (2020). *Language-agnostic bert sentence embedding*. arXiv preprint arXiv:2007.01852.
- Rust, P., Pfeiffer, J., Vulić, I., Ruder, S. and Gurevych, I. (2020). *How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models*. arXiv preprint arXiv:2012.15613.
- Goyal, N., Du, J., Ott, M., Anantharaman, G. and Conneau, A. (2021). *Larger-Scale Transformers for Multilingual Masked Language Modeling*. arXiv preprint arXiv:2105.00572.
- Fan, A., Bhosale, S., Schwenk, H., Ma, Z., El-Kishky, A., Goyal, S., ... & Joulin, A. (2021). *Beyond english-centric multilingual machine translation*. The Journal of Machine Learning Research, 22(1), 4839-4886.

14
Serving Transformer Models

So far, we've explored many aspects surrounding Transformers, and you've learned how to train and use a Transformer model from scratch. You also learned how to fine-tune them for many tasks. However, we still don't know how to serve these models in production. Like any other real-life and modern solution, **natural language processing** (**NLP**)-based solutions must be able to be served in a production environment. However, metrics such as response time must be taken into consideration while developing such solutions.

This chapter will explain how to serve a Transformer-based NLP solution in environments where a CPU/GPU is available. **TensorFlow Extended** (**TFX**) as a solution for machine learning deployment will be described here. Also, other solutions for serving Transformers as **application programming interfaces** (**APIs**) such as FastAPI will be illustrated. You will also learn about the basics of Docker, as well as how to Dockerize your service and make it deployable. Lastly, you will learn how to perform speed and load tests on Transformer-based solutions using Locust.

We will cover the following topics in this chapter:

- FastAPI Transformer model serving
- Dockerizing APIs
- Faster Transformer model serving using TFX
- Load testing using Locust
- Faster inference using ONNX
- SageMaker inference

Technical requirements

We will be using Jupyter Notebook, Python, and Docker to run our coding exercises, which will require Python 3.6.0. The following packages need to be installed:

- `tensorflow`
- `pytorch`
- `transformer >=4.00`
- `fastapi`
- `docker`
- `locust`
- `optimum`
- `onnxruntime`

Now, let's get started!

All the notebooks for the coding exercises in this chapter will be available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

FastAPI Transformer model serving

There are many web frameworks we can use for serving. **Sanic**, **Flask**, and **FastAPI** are just some examples. However, FastAPI has recently gained a lot of attention because of its speed and reliability. In this section, we will use FastAPI and learn how to build a service according to its documentation. We will also use `pydantic` to define our data classes. Let's begin!

1. Before we start, we must install `pydantic` and FastAPI:

```
$ pip install pydantic
$ pip install fastapi
```

2. The next step is to make the data model for decorating the input of the API using `pydantic`. But before forming the data model, we must know what our model is and identify its input.

   We are going to use a **question-answering** (**QA**) model for this. As you know from *Chapter 6*, the input is in the form of a question and a context.

3. By using the following data model, you can make the QA data model:

```
from pydantic import BaseModel
class QADataModel(BaseModel):
    question: str
    context: str
```

4. We must load the model once and not load it for each request; instead, we will preload it once and reuse it. Because the endpoint function is called each time we send a request to the server, this will result in the model being loaded each time:

```
from transformers import pipeline
model_name = 'distilbert-base-cased-distilled-squad'
model = pipeline(model=model_name, tokenizer=model_name,
    task='question-answering')
```

5. The next step is to make a `FastAPI` instance for moderating the application:

```
from fastapi import FastAPI
app = FastAPI()
```

6. Afterward, you must make a `FastAPI` endpoint using the following code:

```
@app.post("/question_answering")
async def qa(input_data: QADataModel):
    result = model(question = input_data.question,
context=input_data.context)
    return {"result": result["answer"]}
```

7. It is important to use `async` for the function to make this function run in asynchronous mode; this will be parallelized for requests. You can also use the `workers` parameter to increase the number of workers for the API and make it answer different and independent API calls at once.

8. Using `uvicorn`, you can run your application and serve it as an API. **Uvicorn** is a lightning-fast server implementation for Python-based APIs that makes them run as fast as possible. Use the following code for this:

```
if __name__ == '__main__':
    uvicorn.run('main:app', workers=1)
```

9. It is important to remember that the preceding code must be saved in a `.py` file (`main.py`, for example). You can run it by using the following command:

```
$ python main.py
```

As a result, you will see the following output in your terminal:

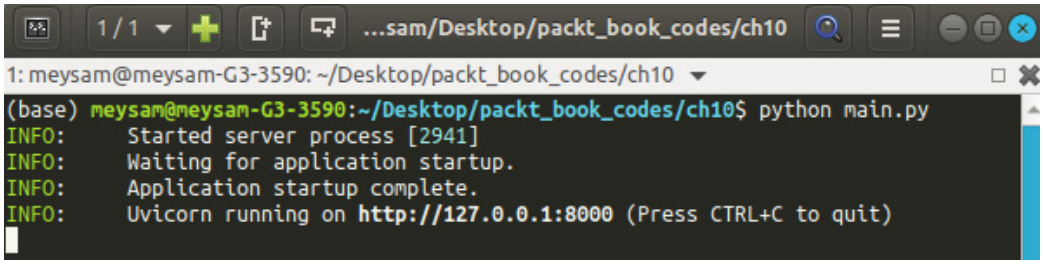


Figure 14.1 – FastAPI in action

10. The next step is to use and test it. There are many tools we can use for this but Postman is one of the best. Before we learn how to use Postman, use the following code:

```
$ curl --location --request POST 'http://127.0.0.1:8000/
question_answering' \
--header 'Content-Type: application/json' \
--data-raw '{
    "question":"What is extractive question answering?",
    "context":"Extractive Question Answering is the task of
extracting an answer from a text given a question. An example
of a question answering dataset is the SQuAD dataset, which is
entirely based on that task. If you would like to fine-tune a
model on a SQuAD task, you may leverage the `run_squad.py`."
}'
```

As a result, you will get the following output:

```
{"answer":"the task of extracting an answer from a text given a
question"}
```

11. cURL is a useful tool but not as handy as Postman. Postman comes with a GUI and is easier to use compared to curl, which is a **command-line interface** (**CLI**) tool. To use Postman, install it from `https://www.postman.com/downloads/`.

12. After installing Postman, you can easily use it, as shown in the following screenshot:

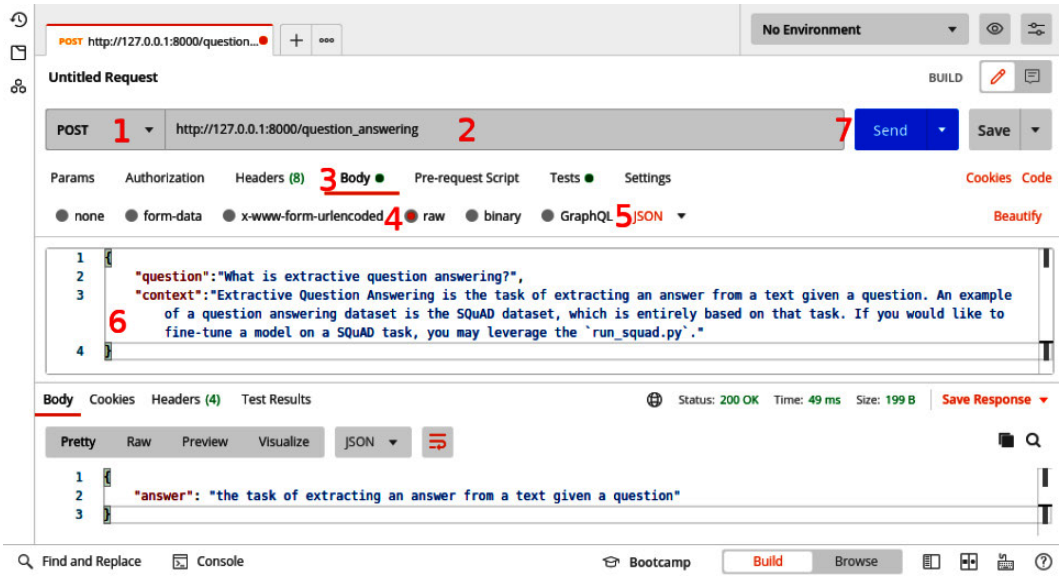


Figure 14.2 – Postman usage

Each step for setting up Postman for your service is numbered in the preceding screenshot. Let's take a look at them:

1. Select **POST** as your method.
2. Enter your full endpoint URL.
3. Select **Body**.
4. Set **Body** to **raw**.
5. Select the **JSON** data type.
6. Enter your input data in JSON format.
7. Click **Send**.

You will see the result in the bottom section of Postman.

In the next section, you will learn how to Dockerize your FastAPI-based API. It is essential to learn about Docker basics to make your APIs packageable and easier for deployment.

Dockerizing APIs

To save time during production and ease the deployment process, it is essential to use Docker. It is very important to isolate your service and application. Also, note that the same code can be run anywhere, regardless of the underlying **operating system** (**OS**). To achieve this, Docker provides great functionality and packaging. Before using it, you must install it using the steps recommended in the Docker documentation (`https://docs.docker.com/get-docker/`):

1. First, put the `main.py` file in the `app` directory.
2. Next, you must eliminate the last part of your code by specifying the following:

```
if __name__ == '__main__':
    uvicorn.run('main:app', workers=1)
```

3. The next step is to make a Dockerfile for your `FastAPI` service; you made this previously. To do so, you must create a Dockerfile that contains the following content:

```
FROM python:3.7
RUN pip install torch
RUN pip install fastapi uvicorn transformers
EXPOSE 80
COPY ./app /app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port",
"8000"]
```

4. Afterward, you can build your Docker container:

```
$ docker build -t qaapi .
```

And easily start it using the following command:

```
$ docker run -p 8000:8000 qaapi
```

As a result, you can now access your API using port `8000`. However, you can still use Postman, as described in the previous section, *FastAPI Transformer model serving*.

So far, you have learned how to make your own API based on a Transformer model and serve it using FastAPI. You then learned how to Dockerize it. It is important to know that there are many options and setups you must learn about regarding Docker; we only covered the basics of Docker here.

In the next section, you will learn how to improve your model serving using TFX.

Faster Transformer model serving using TFX

TFX provides a faster and more efficient way to serve deep learning-based models but it has some important key points you must understand before you use it. The model must be a saved model type from TensorFlow so that it can be used by TFX Docker or the CLI. Let's take a look:

1. You can perform TFX model serving by using a saved model format from TensorFlow. For more information about TensorFlow saved models, you can read the official documentation at `https://www.tensorflow.org/guide/saved_model`. To make a saved model from Transformers, you can simply use the following code:

```
from transformers import TFBertForSequenceClassification
model = \
    TFBertForSequenceClassification.from_pretrained(
        "nateraw/bert-base-uncased-imdb",
        from_pt=True)
model.save_pretrained("tfx_model", saved_model=True)
```

2. Before we understand how to use it to serve Transformers, we need to pull the Docker image for TFX:

```
$ docker pull tensorflow/serving
```

3. This will pull the Docker container of the **TFX** model being served. The next step is to run the Docker container and copy the saved model into it:

```
$ docker run -d --name serving_base tensorflow/serving
```

4. You can copy the saved file into the Docker container using the following code:

```
$ docker cp tfx_model/saved_model tfx:/models/bert
```

5. This will copy the saved model files into the container. However, you must commit the changes:

```
$ docker commit --change "ENV MODEL_NAME bert" tfx my_bert_model
```

6. Now that everything is ready, you can kill the Docker container:

```
$ docker kill tfx
```

   This will stop the container from running.

   Now that the model is ready and can be served by the TFX model Docker container, you can simply use it with another service. The reason we need another service to call TFX is that the Transformer-based models have a special input format provided by tokenizers.

7. To do so, you must make a FastAPI service that will model the API that was served by the TensorFlow serving container. Before you code your service, you should start the Docker container by giving it parameters to run the BERT-based model. This will help you fix bugs in case there are any errors:

```
$ docker run -p 8501:8501 -p 8500:8500 --name bert my_bert_model
```

8. The following code contains the content of the `main.py` file:

```
import uvicorn
from fastapi import FastAPI
from pydantic import BaseModel
from transformers import BertTokenizerFast, BertConfig
import requests
import json
import numpy as np
tokenizer =\
    BertTokenizerFast.from_pretrained(
        "nateraw/bert-base-uncased-imdb")
config = BertConfig.from_pretrained(
    "nateraw/bert-base-uncased-imdb")
class DataModel(BaseModel):
    text: str
app = FastAPI()
@app.post("/sentiment")
async def sentiment_analysis(input_data: DataModel):
    print(input_data.text)
    tokenized_sentence = [dict(tokenizer(input_data.text))]
    data_send = {"instances": tokenized_sentence}
    response = \
    requests.post(
        "http://localhost:8501/v1/models/bert:predict",
        data=json.dumps(data_send))
    result = np.abs(json.loads(response.text)["predictions"][0])
    return {"sentiment": config.id2label[np.argmax(result)]}
if __name__ == '__main__':
    uvicorn.run('main:app', workers=1)
```

9. We have loaded the `config` file because the labels are stored in it, and we need them to return it in the result. You can simply run this file using Python:

```
$ python main.py
```

Now, your service is up and ready to use. You can access it using Postman, as shown in the following screenshot:

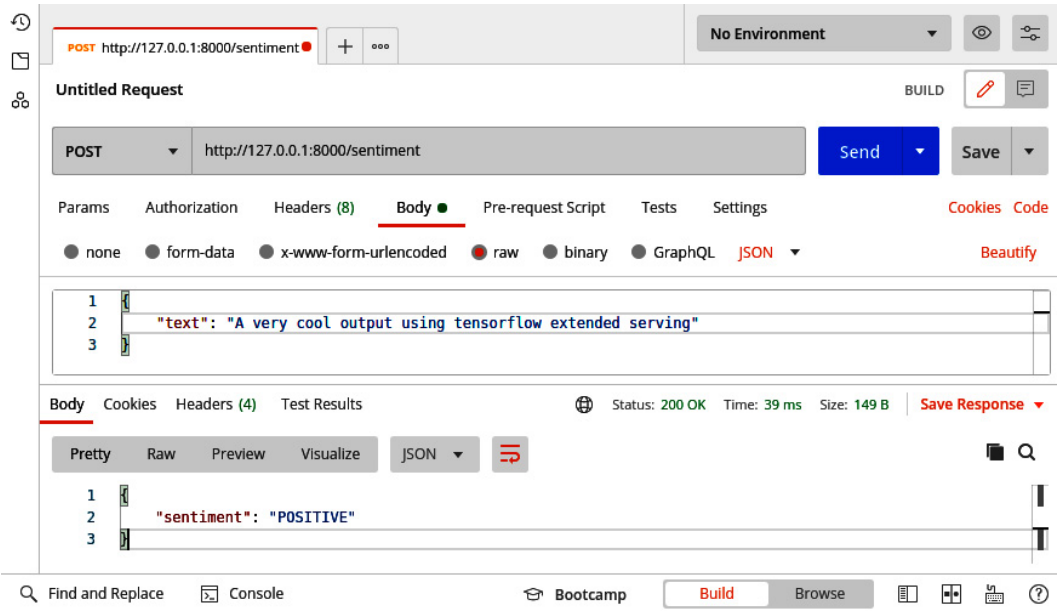


Figure 14.3 – Postman output of a TFX-based service

The overall architecture of the new service within TFX Docker is shown in the following diagram:

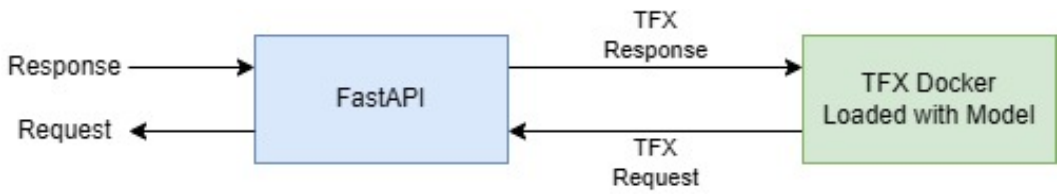


Figure 14.4 – TFX-based service architecture

So far, you have learned how to serve a model using TFX. However, you need to learn how to load test your service using Locust. It is important to know the limits of your service and when to optimize it by using quantization or pruning. In the next section, we will describe how to test model performance under heavy load using **Locust**.

Load testing using Locust

There are many applications we can use to load test services. Most of these applications and libraries provide useful information about the response time and delay of the service. They also provide information about the failure rate. Locust is one of the best tools for this purpose. We will use it to load test three methods for serving a Transformer-based model: using FastAPI only, using Dockerized FastAPI, and TFX-based serving using FastAPI. Let's get started:

1. First, we must install `locust`:

```
$ pip install locust
```

This command will install Locust. The next step is to make all the services serving an identical task use the same model. Fixing two of the most important parameters of this test will ensure that all the services have been designed identically to serve a single purpose. Using the same model will help us freeze anything else and focus on the deployment performance of the methods.

2. Once everything is ready, you can start load testing your APIs. You must prepare an instance `locustfile` to define your user and its behavior. The following code is of a simple `locustfile`:

```
from locust import HttpUser, task
from random import choice
from string import ascii_uppercase
class User(HttpUser):
    @task
    def predict(self):
        payload = {"text": ''.join(choice(ascii_uppercase) for i
in range(20))}
        self.client.post("/sentiment", json=payload)
```

By using `HttpUser` and creating the `User` class that's inherited from it, we can define an `HttpUser` class. The `@task` decorator is essential for defining the task that the user must perform after spawning. The `predict` function is the actual task that the user will perform repeatedly after spawning. It will generate a random string that is `20` characters in length and send it to your API.

3. To start the test, you must start your service. Once you've started your service, run the following code to start the Locust load test:

```
$ locust -f locust_file.py
```

Locust will start with the settings you provided in your `locustfile`. You will see the following in your Terminal:

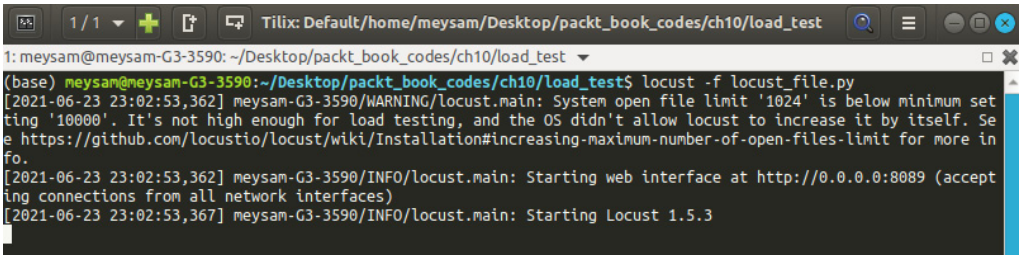


Figure 14.5 – Terminal after starting a Locust load test

As you can see, you can open the URL where the load web interface is located – that is, `http://0.0.0.0:8089`.

4. After opening the URL, you will see an interface, as shown in the following screenshot:

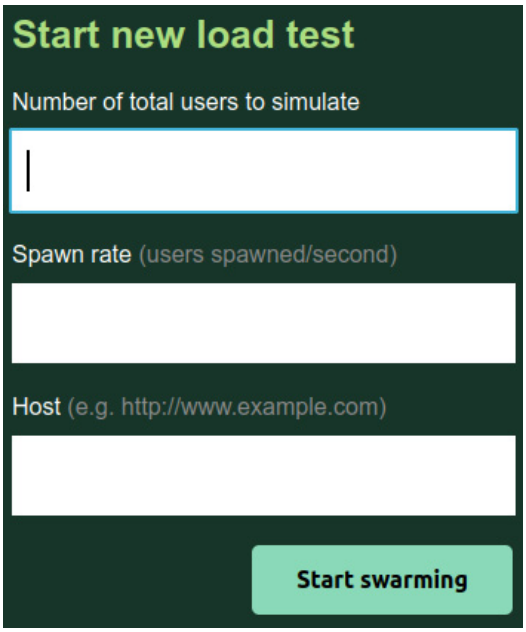


Figure 14.6 – Locust web interface

5. As seen in *Figure 14.6*, we are going to set **Number of total users to simulate** to `10`, **Spawn rate** to `1`, and **Host** to `http://127.0.0.1:8000`, which is where our service is running. After setting these parameters, click **Start swarming**.
6. At this point, the UI will change and the test will begin. To stop the test at any time, click the **Stop** button.

7. You can also click the **Charts** tab to see a visualization of the results:

Figure 14.7 – An overview of Locust test results from the Charts tab

8. Now that the test is ready for the API, let's test all three versions and compare the results to see which one performs better. Remember that the services must be tested independently on the machine where you want to serve them. In other words, you must run one service at a time: run and test the first one, then close it, run and test the second one, then close it, and so on.

   The results are shown in the following table:

	TFX-based FastAPI	FastAPI	Dockerized FastAPI
RPS	38.5	33	34
Average RT (ms)	237	275	270

Figure 14.8 – Comparing the results of different implementations

   In the preceding table, **requests per second** (**RPS**) means the number of requests per second that the API answers, while the average **response time** (**RT**) means the milliseconds that the service takes to respond to a given call. These results show that the TFX-based FastAPI is the fastest. It has a higher RPS and a lower average RT. All these tests were performed on a machine with an Intel(R) Core(TM) i7-9750H CPU with 32 GB RAM, and GPU disabled.

In this section, you learned how to test your API and measure its performance in terms of important parameters such as RPS and RT. However, there are many other stress tests a real-world API can perform, such as increasing the number of users to make them behave like real users. To perform such tests and report their results more realistically, it is important to read Locust's documentation and learn how to perform more advanced tests.

Inference always plays a crucial role when we want to deploy the model. In the next section, we will try to make the model faster using ONNX.

Faster inference using ONNX

Open Neural Network Exchange (**ONNX**) provides faster inference for models that are trained. The `optimum` library, on the other hand, provides easier ONNX exports for even pipelines of Hugging Face-based models. The usage and implementation are straightforward, as you will see:

1. The first thing to do is install the `optimum` and `onnxruntime` libraries:

```
$ pip install optimum[onnxruntime]
```

2. The next step is to load the pipeline using the `optimum` pipeline:

```
from optimum.pipelines import pipeline
pipe = pipeline("text-classification",
    "cardiffnlp/twitter-xlm-roberta-base-sentiment",
    accelerator="ort")
```

3. Two types of accelerators exist: **ONNX runtime** (**ORT**) and **BETTERTRANSFORMER**. ORT is used for the ONNX export of the model. For this specific example, we just picked a multilingual sentiment analysis model based on XLM-Roberta. Now that it has been converted, you can easily run the pipeline:

```
pipe("It was a great movie!")
[{'label': 'positive', 'score': 0.9436209201812744}]
```

This simple change in the loading model and converting it to ONNX improves the latency of the model and provides a faster model.

Hosting models is a challenge but SageMaker provides nice functionality to do it more easily. The next section will describe it.

SageMaker inference

Amazon, in collaboration with Hugging Face, created a nice way of deploying models using only a few lines of code. To deploy any model using SageMaker, you can simply visit the model page at Hugging Face and click the **Amazon SageMaker** button. Please also note that other methods such as Azure are also available:

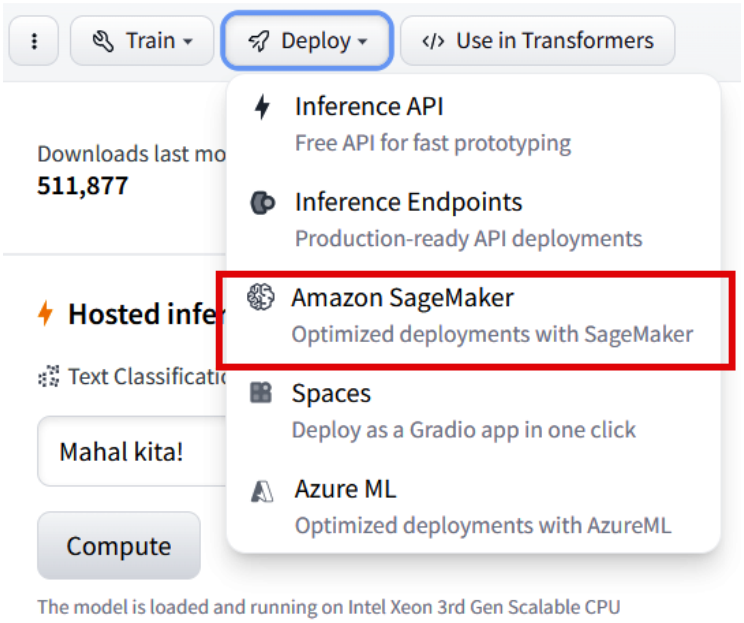


Figure 14.9 – Amazon SageMaker button

This will give the related code to use SageMaker for inference but note that you need to set up the AWS SageMaker environment first. We did not include this part in the book because it is another topic entirely, but you can easily find it in the SageMaker documentation (`https://aws.amazon.com/sagemaker/`).

Let's see how we can use SageMaker for inference:

1. The first step is to install `sagemaker` and then import it:

```
import sagemaker
import boto3
from sagemaker.huggingface import HuggingFaceModel
```

2. The next step is to run this code to get the role:

```
try:
    role = sagemaker.get_execution_role()
except ValueError:
    iam = boto3.client('iam')
    role = \
        iam.get_role(RoleName='sagemaker_execution_role')
['Role']['Arn']
```

3. Afterward, it is required to have the Hugging Face Hub model configuration:

```
hub = {
    'HF_MODEL_ID': \
        'cardiffnlp/twitter-xlm-roberta-base-sentiment',
    'HF_TASK':'text-classification'
}
```

4. It is also necessary to create a SageMaker `huggingface_model` object:

```
huggingface_model = HuggingFaceModel(
    transformers_version='4.26.0',
    pytorch_version='1.13.1',
    py_version='py39',
    env=hub,
    role=role,
)
```

5. The final step is to deploy the model:

```
predictor = huggingface_model.deploy(
    initial_instance_count=1, # number of instances
    instance_type='ml.m5.xlarge' # ec2 instance type
)
```

6. Now it is very easy to use the model with the following example:

```
predictor.predict({
    "inputs": " The movie was not good at all",
})
```

During this section and the entire chapter, you have learned many things about how to serve models and how to use services such as SageMaker.

Summary

In this chapter, you learned about the basics of serving Transformer models using FastAPI. You also learned how to serve models in a more advanced and efficient way, such as by using TFX. You then studied the basics of load testing and creating users. Making these users spawn in groups or one by one, and then reporting the results of stress testing, was another major topic of this chapter. After that, you studied the basics of Docker and learned how to package your application in the form of a Docker container. Finally, you learned how to serve Transformer-based models.

In the next chapter, you will learn about Transformer deconstruction, the model view, and monitoring training using various tools and techniques.

Further reading

To learn more about model serving and how these libraries can be used for very specific use cases, you can follow the following links:

- Locust documentation: `https://docs.locust.io`
- TFX documentation: `https://www.tensorflow.org/tfx/guide`
- FastAPI documentation: `https://fastapi.tiangolo.com`
- Docker documentation: `https://docs.docker.com`
- Hugging Face TFX serving: `https://huggingface.co/blog/tf-serving`

15
Model Tracking and Monitoring

When training deep learning models, it is crucial to take various factors into account during the training process. This approach enables us to conduct highly effective experiments. In this chapter, we will cover **experiment tracking** through sophisticated tools. We will learn how to track experiments by logging and then monitoring them with **TensorBoard** and **Weights & Biases** (**W&B**). These tools enable us to efficiently host and track experimental data such as loss, or other metrics that help us to optimize model training.

In this chapter, we will cover the following topic:

- Tracking model metrics
- Tracking model training with TensorBoard
- Tracking model training live with W&B

Technical requirements

The code for this chapter is found at `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`, which is the GitHub repository for this book. We will be using Jupyter Notebook to run our coding exercises that require Python 3.6.0 or above, and the following packages will need to be installed:

- `tensorflow`
- `pytorch`
- `transformers >=4.00`
- `tensorboard`
- `wandb`

Tracking model metrics

So far, we have trained language models and simply analyzed the final results. In *Chapter 8*, we observed the training process and compared training processes with **Hyperparameter Optimization** (**HPO**). In this section, we will briefly discuss how to visually monitor model training with some external tools. To do this, we will take the example that we developed in *Chapter 5*.

There are several important experiment-tracking frameworks in deep learning, such as MLflow, Neptune, TensorBoard, W&B, CodeCarbon, and ClearML. To keep it simple, we will use TensorBoard and W&B. With the former, we save the training results to a local drive and visualize them at the end of the experiment. With the latter, we monitor the model-training progress live in a cloud platform.

This section will be a short introduction to these tools without going into much detail about them, as this is beyond the scope of this chapter.

Next, let's start with TensorBoard.

Tracking model training with TensorBoard

TensorBoard is a visualization tool specifically for deep learning experiments. It has many features, such as tracking, training, projecting embeddings to a lower space, and visualizing model graphs. We mostly use it for tracking and visualizing metrics such as loss. Tracking a metric with TensorBoard is so easy for Transformers that adding a couple of lines to model-training code will be enough. Everything is kept almost the same.

Now, we will repeat the **Internet Movie Database** (**IMDb**) sentiment fine-tuning experiment from *Chapter 5*, and we will track the metrics. In that chapter, we trained a sentiment model with an IMDb dataset consisting of 4,000 training dataset, 1,000 validation set, and 1,000 test set. Now, we will adapt it to TensorBoard. For more details about TensorBoard, please visit `https://www.tensorflow.org/tensorboard`.

Let's begin:

1. First, we install TensorBoard if it is not already installed, like this:

```
!pip install tensorboard
```

2. Keeping the other code lines of IMDb sentiment analysis as-is from *Chapter 5*, we set the training argument as follows:

```
from Transformers import TrainingArguments, Trainer
training_args = TrainingArguments(
    output_dir='./MyIMDBModel',
    do_train=True,
```

```
    do_eval=True,
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    logging_strategy='steps',
    logging_dir='./logs',
    logging_steps=50,
    evaluation_strategy="steps",
    save_strategy="epoch",
    fp16=True,
    load_best_model_at_end=True
)
```

In the preceding code snippet, the value of `logging_dir` will soon be passed to TensorBoard as a parameter. As the training dataset size is 4,000 and the training batch size is 16, we have 250 steps (4,000/16) for each epoch, which means 750 steps for three epochs.

3. We set `logging_steps` to `50`, which is a sampling interval. As the interval has been reduced, more details about where model performance rises or falls are recorded. We'll do another experiment later on, reducing this sampling interval in step 7 to get a higher resolution for deeper analysis.
4. Now, every 50 steps, the model's performance is measured in terms of the metrics that we define in `compute_metrics()`. The metrics to measure are accuracy, F1, precision, and recall. As a result, we will have 15 (750/50) performance measurements to record. When we run `trainer.train()`, this starts the training process and records the logs in the `logging_dir='./logs'` directory.
5. We set `load_best_model_at_end` to `True` so that the pipeline loads whichever checkpoint has the best performance in terms of loss. Once the training is complete, you will notice that the best model is loaded from `checkpoint-250` with a loss score of `0.263`.
6. Now, the only thing we need to do is to call the following code to launch TensorBoard:

```
%reload_ext tensorboard
%tensorboard --logdir logs
```

This is the output:

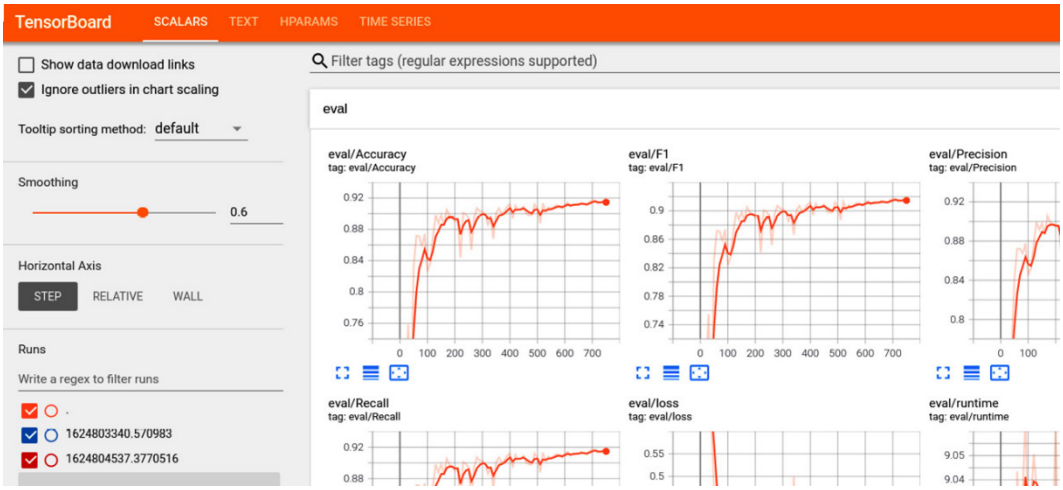


Figure 15.1 – An overview of TensorBoard visualization for training history

As you may have noticed, we can trace the metrics that we defined earlier. The horizontal axis goes from 0 to 750 steps, which is what we calculated before. We will not discuss TensorBoard in detail here. Let's just look at the **eval/loss** chart. When you click on the maximization icon in the bottom left-hand corner, you will see the following chart:

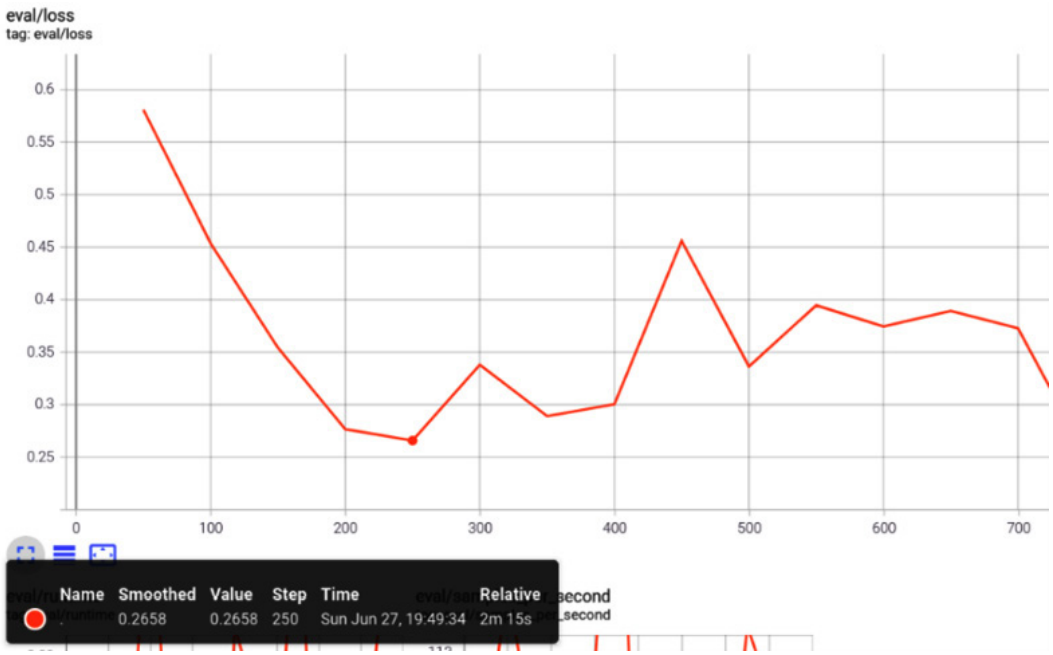


Figure 15.2 – TensorBoard eval/loss chart for logging steps of 50

In the preceding screenshot, we set the smoothing to `0` with the slider control on the left of the TensorBoard dashboard to see scores more precisely and to focus on the global minimum. If your experiment has very high volatility, the smoothing feature can work well to see overall trends. It functions as a **Moving Average** (**MA**). This chart supports our previous observation, in which the best loss measurement is **0.2658** at step **250**.

7. As `logging_steps` is set to `10`, we get a high resolution, as in the following screenshot. As a result, 75 (750 steps/10 steps) performance measurements will be recorded. When we rerun the entire flow with this resolution, we get the best model at step 220, with a loss score of 0.238, which is better than the previous experiment. The result can be seen in the following screenshot. We naturally observe more fluctuations due to the higher resolution:

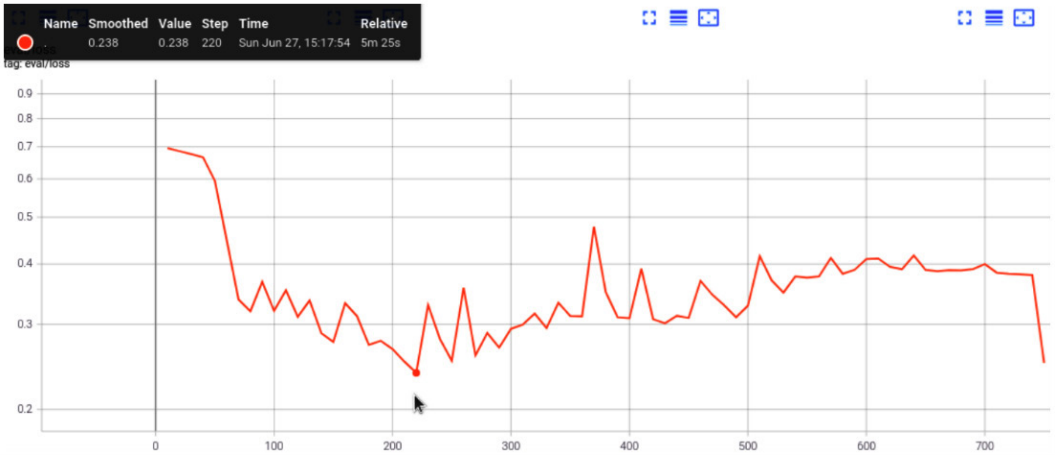


Figure 15.3 – Higher-resolution eval/loss chart for logging steps of 10

We are done with TensorBoard for now. Let's work with W&B!

Tracking model training live with W&B

W&B, unlike TensorBoard, provides a dashboard in a cloud platform, and we can trace and back up all experiments in a single hub. It also allows us to work with a team for development and sharing. The training code is run on our local machine, while the logs are kept in the W&B cloud. Most importantly, we can follow the training process live and share the results immediately with the community or team.

We can enable W&B for our experiments by making very small changes to our existing code:

1. First of all, we need to create an account in `wandb.ai`, then install the Python library, as follows:

```
!pip install wandb
```

2. Again, we will take the IMDb sentiment analysis code and make minor changes to it. First, let's import the library and log in to `wandB`, as follows:

```
import wandb
!wandb login
```

`wandb` requests an API key that you can easily find at the following link: `https://wandb.ai/authorize`.

3. Alternatively, you can set the `WANDB_API_KEY` environment variable to your API key, as follows:

```
!export WANDB_API_KEY=e7d*********
```

4. Again, keeping the entire IMDb sentiment code as-is, we only add two parameters, `report_to="wandb"` and `run_name="..."`, to `TrainingArguments`, which enables logging in to W&B, as shown in the following code block:

```
training_args = TrainingArguments(
    ...  the rest is same ...
    run_name="IMDB-batch-32-lr-5e-5",
    report_to="wandb"
)
```

5. Then, as soon as you call `trainer.train()`, logging starts on the cloud. After the call, please check the cloud dashboard and see how it changes. Once the `trainer.train()` call has been completed successfully, we execute the following line to tell `wandB` we are done:

```
wandb.finish()
```

The execution also outputs run history locally, as follows:

Figure 15.4 – The local output of W&B

When you connect to the link provided by W&B, you will get to an interface that looks something like this:

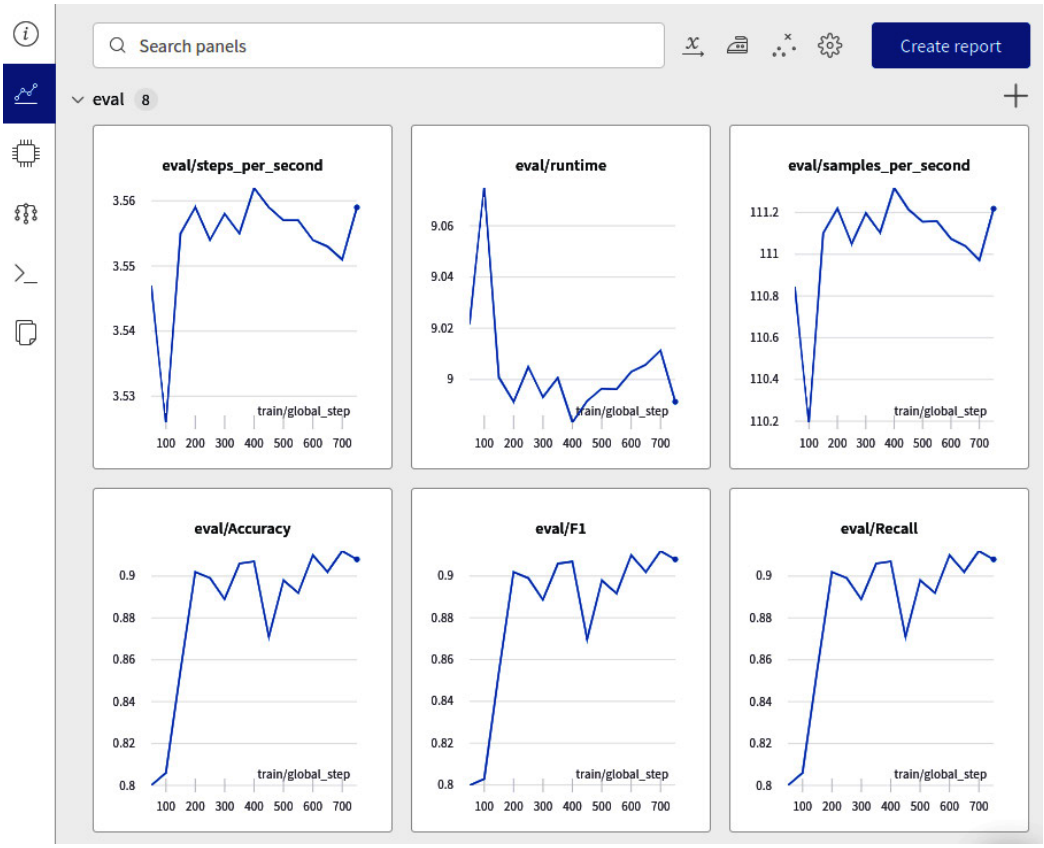


Figure 15.5 – The online visualization of a single run on the W&B dashboard

This visualization gives us a summarized performance result for a single run. As you can see, we can trace the metrics that we defined in the `compute_metric()` function.

Now, let's take a look at the evaluation loss. The following screenshot shows exactly the same plot that TensorBoard provided, where the minimum loss is around **0.2658**, occurring at step **250**:

Figure 15.6 – The eval/loss plot of IMDb experiment on the W&B dashboard

We have only visualized a single run so far. W&B allows us to explore the results dynamically across lots of runs at once—for example, we can visualize the results of models using different hyperparameters such as learning rate or batch size. To do so, we instantiate a `TrainingArguments` object with a different hyperparameter setting and change `run_name="..."` accordingly for each run.

The following screenshot shows our several IMDb sentiment-analysis runs using different hyperparameters. We can also see the batch size and learning rate that we changed:

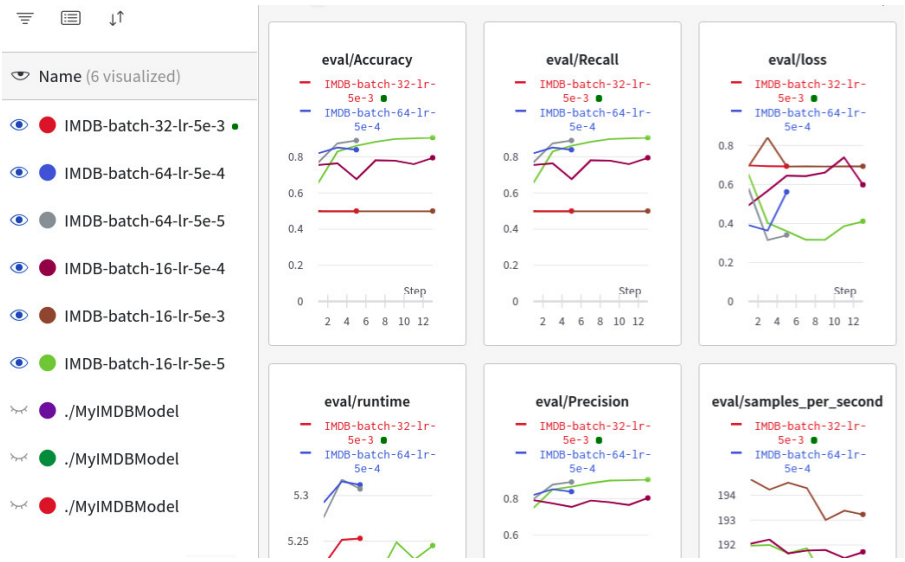


Figure 15.7 – Exploring the results across several runs on the W&B dashboard

W&B provides useful functionality—for instance, it automates hyperparameter optimization and searching the space of possible models, called W&B sweeps. Other than that, it also provides system logs relating to **Graphics Processing Unit** (**GPU**) consumption, **Central Processing Unit** (**CPU**) utilization, and so on. For more detailed information, please check the following website: `https://wandb.ai/home`.

Well done! it's crucial to use such utility tools to develop better models.

Summary

In this chapter, we introduced model training tracking tools. We learned how to track our experiments to obtain higher-quality models and do error analysis. We utilized two tools to monitor training: TensorBoard and W&B. These tools were used to effectively track experiments and to optimize model training.

In the next chapter, we will discuss how to apply self-attention mechanisms to process images.

Further reading

If you're interested in learning more about these topics, you may find the following resources helpful:

- *Biewald, L., Experiment tracking with weights and biases, 2020.* Software available from `wandb.com`, *2(5)*.
- *W&B*: `https://wandb.ai`
- *TensorBoard*: `https://www.tensorflow.org/tensorboard`

Part 4: Transformers beyond NLP

Chapter 16 elaborates on the concept of vision transformers, a revolutionary approach that applies transformer-based models to computer vision tasks, providing a different perspective from traditional convolutional neural networks.

In *Chapter 17*, the focus shifts to Multimodal Generative Transformers, a sophisticated model that can generate complex output spanning multiple modalities, demonstrating the versatility and power of transformer models.

Lastly, *Chapter 18* delves into time series modeling with Transformers, exploring how these models can be adeptly used to analyze and predict sequential data, thereby broadening their application beyond natural language processing.

This part has the following chapters:

- *Chapter 16, Vision Transformers*
- *Chapter 17, Multimodal Generative Transformers*
- *Chapter 18, Revisiting Transformers Architecture for Time Series*

16
Vision Transformers

Transformers have been used to achieve significant goals in the field of **natural language processing** (**NLP**) and, as you have seen in the previous chapters, they can accomplish a lot in many different tasks. In this chapter, we will explore **Vision Transformer** (**ViT**) models. Just as a wide range of models has been created for NLP, various models have also been created for vision, and each one has created a new view of the computer vision perspective. By reading this chapter, you will learn how you can use models such as ViT for computer vision tasks, how pretrained computer vision models based on transformers work, and how you can fine-tune them for specific tasks. We will also look at models that are prompt-based and can provide results based on the visual prompts given to them.

The following topics will be covered in this chapter:

- Vision transformers
- Image classification using transformers
- Semantic segmentation and object detection using transformers
- Visual prompt models

Technical requirements

The following libraries/packages are required to successfully complete this chapter:

- `Anaconda`
- `transformers 4.0.0`
- `pytorch 1.0.2`
- `tensorflow 2.4.0`

All notebooks with coding exercises will be available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

Vision transformers

Transformers solved many problems in the NLP domain and provided good results. However, some researchers started to apply them to other domains rather than text-only ones. Computer vision is one of the fields in which transformers are actively used. In this section, you will learn how it is possible to apply transformers to computer vision. This was one of the essential problems in the early days of adaptation. NLP is composed of text-only data that is mostly seen as a time series character-based input with a respective output. However, computer vision problems come with an image as input followed by the desired output, which can be in the form of numeric values or a matrix. A gray-scale image is represented as a matrix with respective width and height values. The easiest way to see it is as a black-and-white image:

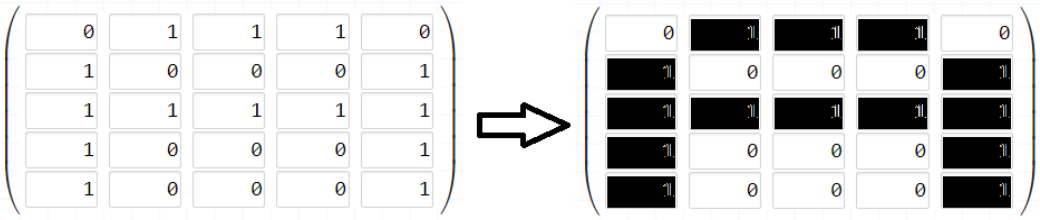


Figure 16.1 – A black and white image of character A

As you can see in *Figure 16.1*, each cell in the matrix resembles a binary value, `1` or `0`. If the value is `0`, it will activate an LED (full light on), and if it's `1`, then the LED will be off (or vice versa). This will show the character **A** on the screen of a very simple monitor.

In the case of gray-scale images, these values are not just `0` and `1` but, instead, they are a spectrum of `0` to `255` (with 256 colors from full black to full white). In the case of an RGB image, it will not be only a single matrix but, instead, three matrices, with each one representing each color (red, green, or blue). This combination will be shown by the pixels on the monitor (with three LEDs for each color). The intensity of the image will also be shown by these numbers in the matrix.

Now that we know what the most essential part of computer vision is (the image), we can move forward to see how this type of data can be processed, learned, and used by a transformer.

Transformers are very good with time series data, but images can also be seen as time series somehow. Imagine converting an image into smaller patches of a *16x16* size (the patch size here is given as an example). These patches also have a sequential format if we keep giving all of the image patches to the model in the same order. It is a very similar approach to convolutional neural networks where we also see the image as patches and apply filters on the patch (or create a filter and move the filter for the patches), but it is very useful if another dense layer-based embedding is followed. This will ensure that each patch is not *16x16* but, instead, is a dense representation of that specific part of the image. Also, don't forget that positional embedding must be added as well.

Just like an encoder-only-based model, these models can also be encoder-only. For example, another bulk token can be added to the first of each operation to create a representation for the entire image (what we already knew as a `[CLS]` token). During the classification, we can use this token to classify the entire image into given classes.

The ViT architecture is shown in *Figure 16.2*:

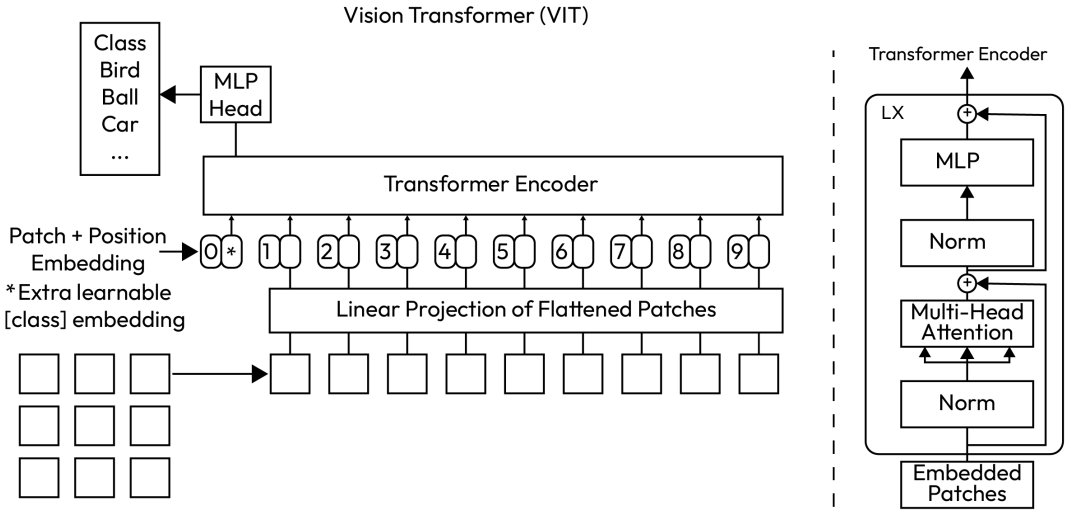


Figure 16.2 – ViT architecture

The rest of the architecture is identical to the transformer encoder block. The main idea in architectures such as ViT is in patching and applying the positional embeddings to the vectors.

Other approaches have been proposed that are very similar to ViT. For example, the **Data-Efficient Image Transformer** (**DeiT**) model uses another token for getting knowledge from other models such as **ResNet**. The main difference is that two loss functions are used in the training process: one for classification cross-entropy loss on the `[CLS]` token and the other for the distillation token for teacher model prediction. This further helps the model to grasp the knowledge from the teacher model and speed up the pretraining phase even further.

Figure 16.3 shows the architecture of the DeiT model:

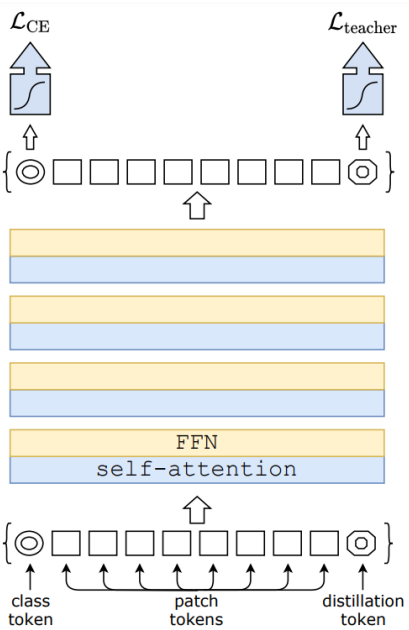


Figure 16.3 – The DeiT model

ImageGPT (**iGPT**) is another approach used with a very similar method. This model is trained to predict the next pixel value, very similar to GPT-based NLP models. This helps the model to predict the rest of the image with given initial pixels. *Figure 16.4* shows a demonstration from the OpenAI website:

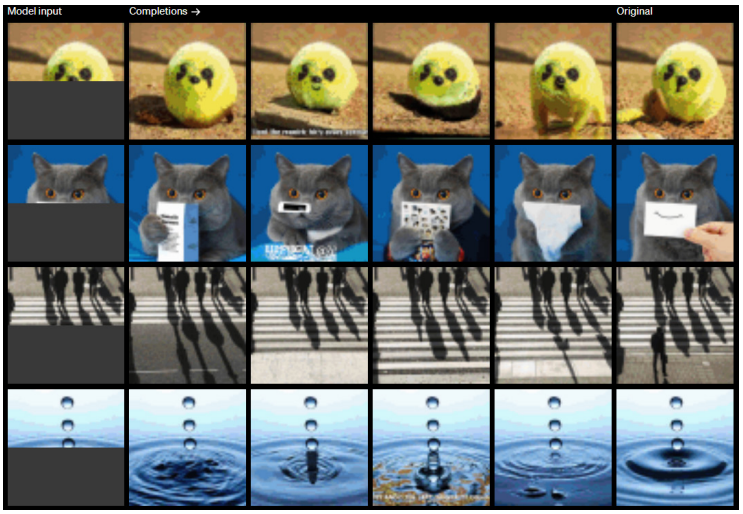


Figure 16.4 – ImageGPT

The **Detection Transformer** (**DETR**) model is used for tasks such as object detection. This architecture uses convolution in combination with transformer blocks. A backbone is used to create the 2D representation of the images. These 2D vector representations of images are passed to a transformer encoder block, which is later followed by a decoder block. The decoder block here is used to create the respective object detection outputs. For each object, a query vector is also given to the decoder block to help it predict which objects are present in the image:

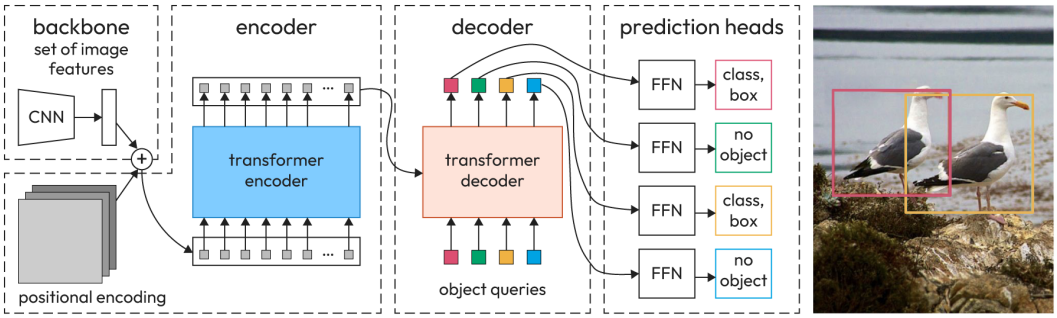


Figure 16.5 – Detection Transformer

Transformer-based models for vision perform more or less the same as CNN-based models. The most important point is the representation scheme used for these kinds of models. Models that are capable of representing text and images in the same form (token-based and patch-based) can unleash many use cases compared to CNN-based models, which see images as pixels and text as tokens. Multimodal learning, which is covered in *Chapter 17*, focuses on multimodal models, and you will learn more about multimodal models that are capable of understanding or generating more than one modality in that chapter. On the other hand, compared to CNN-based models, these models can learn more than one task at the same time more efficiently and they can be used to create foundation models for this scope. Foundation models for vision are those that are created by pretraining models on large amounts of data, very similar to language models.

So far, you have learned about the basics of vision-based transformers. You have also gained knowledge about different approaches in this field.

In the next section, we will take a deeper look at how to use them for image classification.

Image classification using transformers

ViT is a good option for classifying images using transformers. Pretrained models for this transformer model already exist and it is very easy and convenient to use them. Follow these steps:

1. To use ViT, you can simply import it from the `transformers` library and load the preprocessor and the model itself:

```
from transformers import (
    ViTForImageClassification, ViTImageProcessor)
model = ViTForImageClassification.from_pretrained(
    'google/vit-base-patch16-224')
processor = ViTImageProcessor.from_pretrained(
    'google/vit-base-patch16-224')
```

2. You also need to load the image. In our case, we will download and load a sample image:

```
from PIL import Image
import requests
url = 'http://images.cocodataset.org/val2017/000000439715.jpg'
image = Image.open(requests.get(url, stream=True).raw)
```

   This will load a sample image from the `coco` dataset.

3. Using a preprocessor, you need to convert the format of the image to the desired format that is suitable for ViT:

```
inputs = processor(images=image, return_tensors="pt")
```

4. Now, you can easily pass the image to the model and get the output as probabilities of classes:

```
outputs = model(inputs.pixel_values)
```

5. Just like most of the classification approaches based on deep learning and softmax, you need to use `argmax` to get the class ID of the highest probable class:

```
prediction = outputs.logits.argmax(-1)
```

6. Now, you can even print out the class name using the predefined function of this model:

```
print("Class label: ", model.config.id2label[prediction.item()])
```

7. The output will be as follows:

```
Class label: bearskin, busby, shako
```

Up to this point, you have learned how to use ViT, excluding the training phase on your own dataset, which we will cover here as well. If you want to train this classification model on your own dataset, it is important to note that you need to convert the image dataset using the same preprocessor so it is in the right format for the model to be processed.

Semantic segmentation and object detection are other tasks for computer vision, which you will learn about in the next section.

Semantic segmentation and object detection using transformers

In this section, we will focus on two important computer vision tasks: **semantic segmentation** and **object detection**. You will learn how to perform these using transformers.

Models such as **DETR** are capable of detecting objects and semantic segmentation. These tasks are simple and convenient using a pretrained model from transformers.

Using DETR, you can also detect the objects present in an image. Follow these steps:

1. You need to load the model and preprocessor as follows:

```
from transformers import (
    DetrImageProcessor, DetrForObjectDetection)
import torch
from PIL import Image
import requests
processor = DetrImageProcessor.from_pretrained(
    "facebook/detr-resnet-50")
model = DetrForObjectDetection.from_pretrained(
    "facebook/detr-resnet-50")
```

2. The next step is to download the example image and convert it:

```
url = "http://images.cocodataset.org/val2017/000000439715.jpg"
image = Image.open(requests.get(url, stream=True).raw)
```

3. Giving the downloaded image to the model will create the respective output, known as a bounding box:

```
inputs = processor(images=image, return_tensors="pt")
outputs = model(**inputs)
```

4. You can decide which threshold to apply for each bounding box. In this example, we will use `0.9`, which means any detected object above this threshold will be kept:

```
target_sizes = torch.tensor([image.size[::-1]])
results = processor.post_process_object_detection(
    outputs, target_sizes=target_sizes, threshold=0.9)[0]
```

5. You can print the results using this code:

```
for score, label, box in zip(results["scores"],
    results["labels"], results["boxes"]):
    box = [round(i, 2) for i in box.tolist()]
    print(
        f"Detected {model.config.id2label[label.item()]} \
            with confidence "
        f"{round(score.item(), 3)} at location {box}"
    )
```

6. The results will be shown as follows:

```
Detected person with confidence 0.979 at location [514.99,
278.84, 562.75, 351.12]
Detected person with confidence 0.987 at location [387.75,
271.46, 414.89, 305.16]
Detected person with confidence 0.983 at location [561.75,
273.8, 597.74, 368.74]
Detected umbrella with confidence 0.938 at location [327.71,
238.32, 414.96, 276.72]
Detected person with confidence 0.982 at location [48.85, 275.8,
79.67, 345.96]
Detected person with confidence 0.997 at location [250.06,
162.47, 336.12, 398.63]
Detected person with confidence 0.995 at location [114.22,
268.76, 147.84, 399.37]
Detected horse with confidence 0.999 at location [130.74,
237.92, 467.25, 478.41]
Detected person with confidence 0.994 at location [402.1, 273.0,
461.29, 345.74]
Detected umbrella with confidence 0.963 at location [326.78,
230.8, 397.81, 261.1]
Detected person with confidence 0.984 at location [354.0,
267.42, 388.52, 298.78]
Detected umbrella with confidence 0.992 at location [508.45,
266.96, 572.54, 295.99]
```

7. However, these results are very inconvenient and not easy to understand. A better way is to use this function and visualize them:

```
COLORS = [
    [0.0, 0.5, 0.8],
    [0.9, 0.3, 0.1],
    [0.9, 0.6, 0.1],
    [0.4, 0.1, 0.5],
    [0.4, 0.6, 0.1],
    [0.3, 0.7, 0.9]
]
def visualize_prediction(pil_img, output_dict, threshold):
    keep = output_dict["scores"] > threshold
    boxes = output_dict["boxes"][keep].tolist()
    scores = output_dict["scores"][keep].tolist()
    labels = output_dict["labels"][keep].tolist()
    labels = [model.config.id2label[x] for x in labels]


    plt.figure(figsize=(8, 5))
    plt.imshow(pil_img)
    ax = plt.gca()
    colors = COLORS * 100
    for score, (xmin, ymin, xmax, ymax), label,
        color in zip(scores, boxes, labels, colors):
        ax.add_patch(plt.Rectangle((xmin, ymin), xmax - xmin,
            ymax - ymin, fill=False, color=color, linewidth=3))
        ax.text(xmin, ymin, label, fontsize=8,
            bbox=dict(facecolor="yellow", alpha=0.5))
    plt.axis("off")
    plt.show()
```

8. You can easily call this function:

```
visualize_prediction(image, results, 0.9)
```

As a result, you will see the following image:

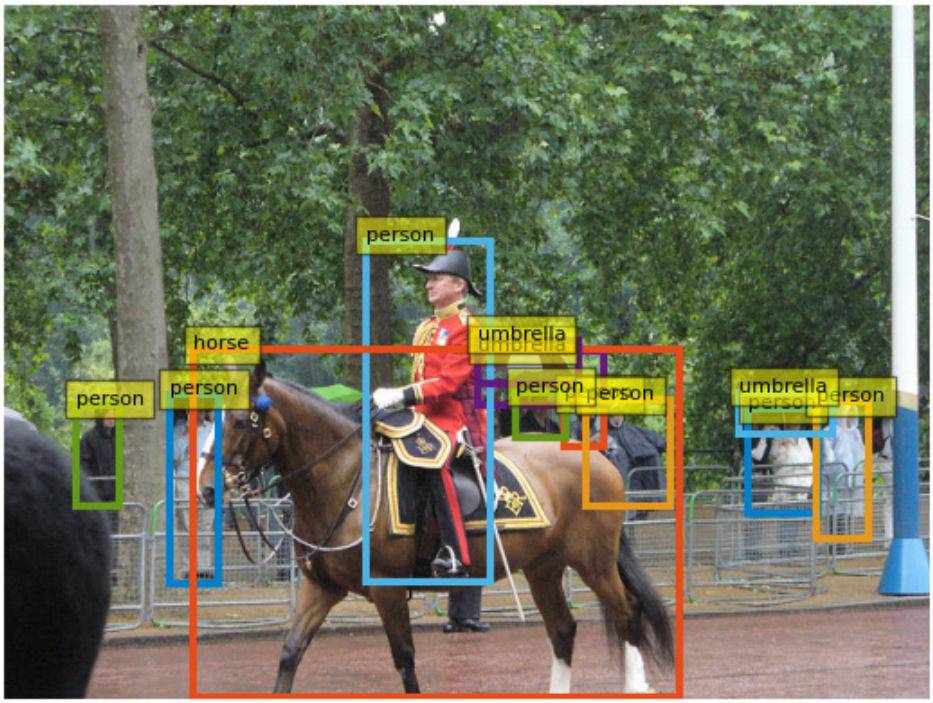


Figure 16.6 – DETR object detection

As you can see from the image, objects are detected and annotated by the model.

Similarly, you can use this model for semantic segmentation as well. Semantic segmentation is the process of labeling each pixel and assigning them to the classes. While object detection deals with objects in bounding boxes, semantic segmentation creates a selection of the objects in a pixel-wise manner. Follow these steps:

1. The first thing you need to do is to load your model:

```
from transformers import (DetrFeatureExtractor,
    DetrForSegmentation)
processor = DetrFeatureExtractor.from_pretrained(
    "facebook/detr-resnet-50-panoptic")
model = DetrForSegmentation.from_pretrained(
    "facebook/detr-resnet-50-panoptic")
```

2. The next step is to download the image that we want to perform segmentation on:

```
import io
import requests
from PIL import Image
import torch
import numpy
url = "https://farm4.staticflickr.
com/3487/3925656789_1b64654c91_z.jpg"
image = Image.open(requests.get(url, stream=True).raw)
```

Now that we have the image, we can use processor and model to get the output:

```
inputs = processor(images=image, return_tensors="pt")
outputs = model(**inputs)
```

Afterwards, you need to use the following functions to extract the result:

```
processed_sizes = torch.as_tensor(
    inputs["pixel_values"].shape[-2:]).unsqueeze(0)
result = feature_extractor.post_process_panoptic(
    outputs, processed_sizes)[0]
```

However, in order to see the image, you can execute the following code:

```
import itertools
import io
import seaborn as sns
import numpy
from transformers.image_transforms import rgb_to_id, id_to_rgb
from matplotlib import pyplot as plt
palette = itertools.cycle(sns.color_palette())
panoptic_seg = Image.open(io.BytesIO(result['png_string']))
panoptic_seg = numpy.array(panoptic_seg,
    dtype=numpy.uint8).copy()
panoptic_seg_id = rgb_to_id(panoptic_seg)
panoptic_seg[:, :, :] = 0
for id in range(panoptic_seg_id.max() + 1):
    panoptic_seg[panoptic_seg_id == id] = \
        numpy.asarray(next(palette)) * 255
plt.figure(figsize=(15,15))
plt.imshow(panoptic_seg)
plt.axis('off')
plt.show()
```

3. This code will convert the output of the model into the proper format to be visualized. Finally, you can see the image as shown in *Figure 16.7*, which is the identical semantically segmented image of the original image:

Figure 16.7 – Semantic segmentation result (left) and original image (right)

Up to this point, you have learned how to use ViT models for image classification, object detection, and semantic segmentation. In the next section, you will learn about visual prompt models and how to use them.

Visual prompt models

Prompt-based models have been an attractive part of artificial intelligence in many aspects. These kinds of models can take guidance in the form of a pattern and create the respective output by understanding it. The prompt can be in many forms or data formats. Textual prompt-based models or visual prompt-based models are also available. A textual prompt is a free text that indicates what the model should do or provide as output. Similarly, a visual prompt is a visual guidance that helps the model understand the task or the instruction itself.

Models such as **CLIP** are capable of understanding images and text at the same time and mapping them to a single vector space. In this vector space, text with similar semantic meaning to images (that visually present the same described objects or scenes in the text) are closer together. A simple approach to use more capabilities of the models is to ground them by using some external data. For example, imagine that you are not trying to search for an image but, instead, for a specific object inside an image. This can be easily done using semantic segmentation or even object detection but, in our case, the text is in free format. This means that a specific user is free to write anything and is not limited by some existing objects that the model already knows or has been trained on. In this case, approaches such as visual prompting or textual prompting are very useful.

CLIPSeg is one of these methods that presents an approach using text and visual prompts. *Figure 16.8* shows how this model works:

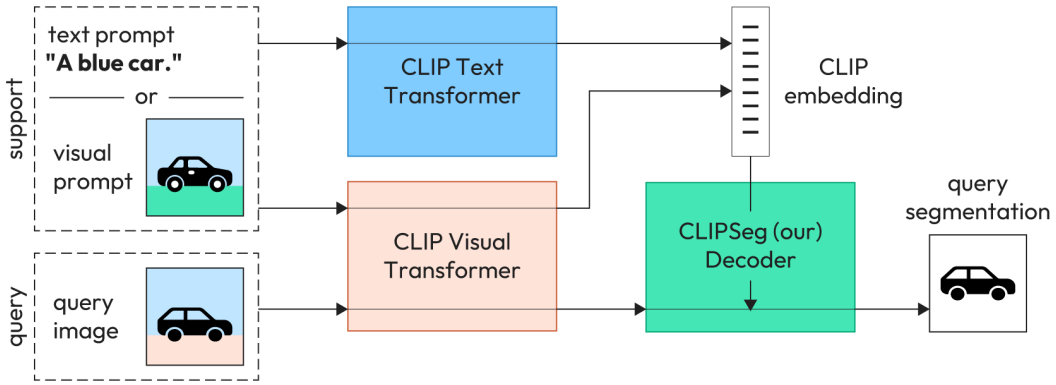


Figure 16.8 – CLIPSeg

As you can see in *Figure 16.8*, CLIPSeg is a decoder that is trained on CLIP visual and text transformers. This decoder takes two different outputs: one as the original image and the second as support, which can be text or another image. Follow these steps:

1. To use this approach, you need to download a sample image. In our case, we will use the previous image from *Figure 16.7*:

```
from PIL import Image
import requests
url = "https://farm4.staticflickr.
com/3487/3925656789_1b64654c91_z.jpg"
image = Image.open(requests.get(url, stream=True).raw)
```

2. Now that we have the image, we can load the model:

```
from transformers import (CLIPSegProcessor,
    CLIPSegForImageSegmentation)
processor = CLIPSegProcessor.from_pretrained(
    "CIDAS/clipseg-rd64-refined")
model = CLIPSegForImageSegmentation.from_pretrained(
    "CIDAS/clipseg-rd64-refined")
```

3. For text prompting, we can use a set of textual descriptions of the objects that we are aiming to find:

```
prompts = ["hat","ball","player", "red shirt"]
inputs = processor(text=prompts, images=[image] * len(prompts),
    padding="max_length", return_tensors="pt")
```

4. Now that we have the prompts and respective inputs ready for the model, we can create the outputs:

```
outputs = model(**inputs)
preds = outputs.logits.unsqueeze(1)
```

5. We can also easily show the predicted segmentation for each of them:

```
import matplotlib.pyplot as plt
_, ax = plt.subplots(1, 5, figsize=(15, 4))
[a.axis('off') for a in ax.flatten()]
ax[0].imshow(image)
[ax[i+1].imshow(torch.sigmoid(preds[i][0])) for i in range(4)];
[ax[i+1].text(0, -15, prompts[i]) for i in range(4)];
```

You will see the output in *Figure 16.9*:

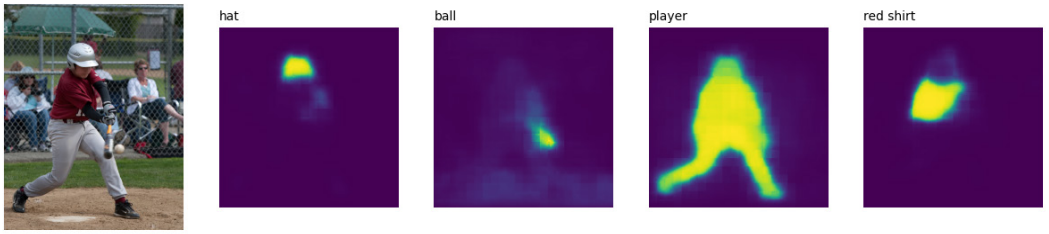


Figure 16.9 – CLIPSeg text prompt outputs

This is just one way of using this model, textual prompting. We can use visual prompts as input as well. Let's do that next:

1. Imagine that we have an image of a baseball ball and we want to search for the ball:

```
url = "https://upload.wikimedia.org/wikipedia/en/1/1e/
Baseball_%28crop%29.jpg"
prompt = Image.open(requests.get(url, stream=True).raw)
```

2. Now that we have the image of the ball, we can encode the image and prompt and then condition the model according to the prompt:

```
encoded_image = processor(images=[image], return_tensors="pt")
encoded_prompt = processor(images=[prompt], return_tensors="pt")
outputs = model(**encoded_image,
    conditional_pixel_values = encoded_prompt.pixel_values)
```

3. Now that we have the outputs, we can visualize them:

```
preds = outputs.logits.unsqueeze(1)
preds = torch.transpose(preds, 0, 1)
_, ax = plt.subplots(1, 2, figsize=(6, 4))
[a.axis('off') for a in ax.flatten()]
ax[0].imshow(image)
ax[1].imshow(torch.sigmoid(preds[0]))
```

4. The result is shown in *Figure 16.10*:

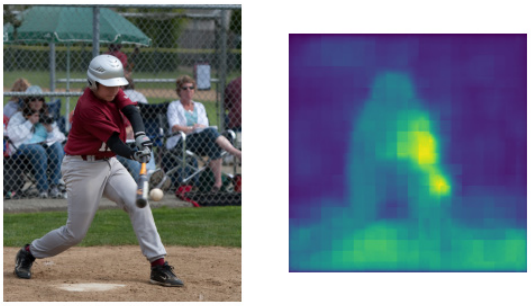


Figure 16.10 – CLIPSeg

As seen in the segmented image, a ball and some other related parts such as a bat are highlighted.

You have now learned how to use different methods such as CLIPSeg, semantic segmentation, and object detection using different visual transformer models.

Summary

In this chapter, you have learned about vision transformers and how to use them for various tasks. Semantic segmentation, object detection, and classification were a few of these tasks. You also gained knowledge about text and visual prompt models, which can take a prompt as input and provide the outputs. Models such as DETR, ViT, and CLIPSeg were described in this chapter.

In the next chapter, you will learn about multimodal models, such as **Stable Diffusion**, and how they work.

17
Multimodal Generative Transformers

Models that can understand more than one type of input are called multimodal models. Multimodal learning has been one of the important fields of **artificial intelligence** (**AI**) and has attracted many researchers for a long time. Even today, these multimodal models are popular in various forms and shapes. In this chapter, we will learn about generative AI using multimodal models, especially text-to-image and text-to-music ones. You will learn about Stable Diffusion and how it works. Also, you will gain knowledge about MusicGen and AudioGen models.

The following topics will be covered in this chapter:

- Multimodal learning
- Stable Diffusion for text-to-image generation
- Music generation using MusicGen
- Text-to-speech generation using transformers

Technical requirements

The following libraries/packages are required to successfully complete this chapter:

- `Anaconda`
- `transformers 4.0.0`
- `pytorch 1.0.2`
- `tensorflow 2.4.0`

All notebooks containing the coding exercises are available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

Multimodal learning

In a broad sense, multimodal learning is the learning process that takes place using different modalities in the context of machine learning. A modality in machine learning is the type of data that we put into the model. Typical types of modalities include textual, visual (image and video), and auditory (sound, voice, and music) data.

A good example of such models is **contrastive language-image pretraining** (**CLIP**), which can represent textual and visual data in the same space. We can create different applications using this representation. For example, we can create vector representations of the images and text, both obtained from the same dataset and create a classifier on top. In *Figure 17.1*, you can see a multimodal approach to predicting phone prices using features such as the camera, RAM, battery, and an image of the device.

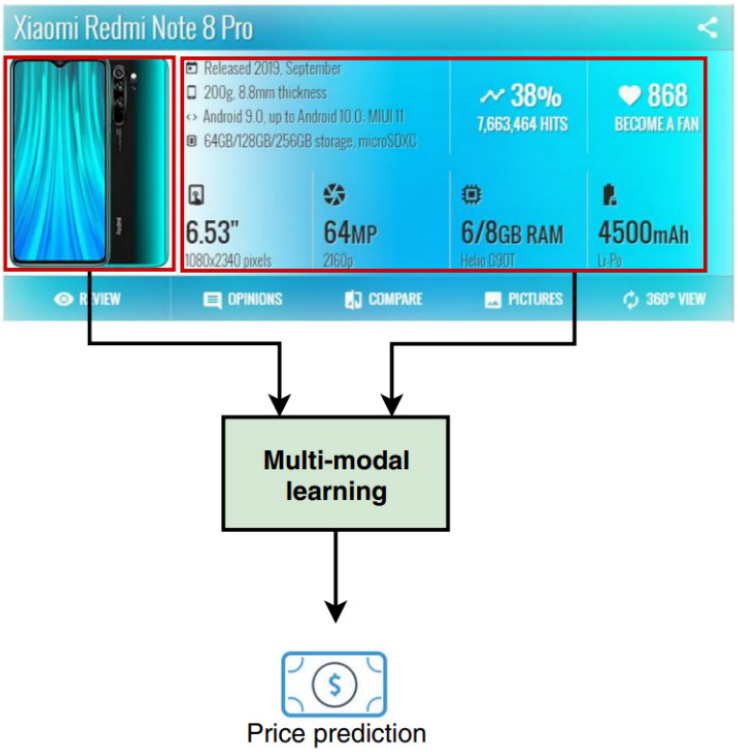


Figure 17.1 – Multimodal price prediction (image courtesy of https://link.springer.com/article/10.1007/s40745-021-00326-z)

Another example is price prediction using an image of the product, its features, and a textual description of it. In this example, we have more than two modalities. Fusing these modalities such that the model can understand them and create the best possible output is always a significant issue in this field. As you can see in *Figure 17.1*, a multimodal approach is proposed to predict the price of mobile phones.

Multimodal learning does not always focus on understanding the data in different modalities; it also involves getting the input from one modality and generating the output in another one. In this sense, image captioning can be considered a form of multimodal learning because the input is visual data, while the output is textual.

Previous approaches for fusing different features from different modalities included a process of weighting manually or training the fusion weights. However, newer approaches tackle this issue by converting all the features into the same or a very similar feature space. For example, a text-vision multimodal model creates patches for images that are seen as tokens similar to text tokens. These visual and text tokens are then represented in a similar way to each other, and instead of giving different modalities to the model, the model sees them in the same way.

In the next subsection, you will learn more about generative multimodal AI, which is capable of getting input in one modality and generating output in another.

Generative multimodal AI

Generative AI or GenAI, which has recently drawn attention from across the world, involves AI models that are capable of generating output. This does not mean that other models do not generate output (something that applies to all AI models). Generative AI emphasizes models that can understand the underlying patterns of training data and generate new data accordingly.

Generative AI has existed for many years, but it has not attracted much public attention because we have little control over its generated outputs. Even if we had control over it, we could not specify exactly what we wanted it to generate. Even if we could have had control over it and had been able to specify what we wanted, this specification process was not easily understood by humans.

More recent models get the description of what we want to generate in text form, and there are even other types of models that accept input in the form of images or voice as well. This enables new technologies to be used more easily by the vast majority of people rather than only a handpicked set of researchers.

In the next section, we will provide a detailed explanation of how Stable Diffusion works with text-to-image generation and then see how it can be used for image generation in action.

Stable Diffusion for text-to-image generation

Text-to-image generation is a widely adopted use case of generative AI. Generating images from text, especially high-quality images, has lots of use cases from game design to marketing. However, in order to understand how this specific kind of model works, we need to first understand a few preliminaries about diffusion models in machine learning.

Diffusion in AI is a term borrowed from physics. The notion of physical reactions that include dissolved materials such as ink in water also applies here to AI. For example, take an ordinary image as our starting point. Forward diffusion is the process of adding noise to the image. As seen in the following figure, this process will turn any image into noise (with a level of noise added to the image) that gradually makes the image indistinguishable from the original one.

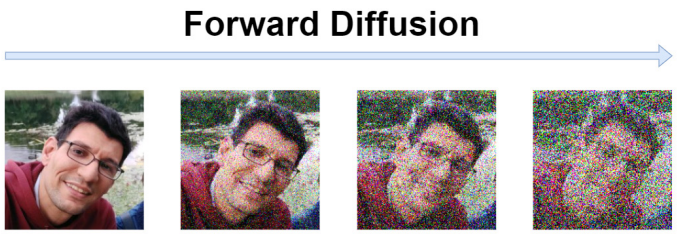


Figure 17.2 – Forward diffusion

This forward process will give us a different version of the same image that we can use to train a model to reverse the process by removing the noise. This is a very efficient technique to create very realistic images. However, there is a major consideration that comes with this approach: It is very slow. You must remove the noise in an image step by step. This process is also called sampling: in each step, the noisy image is given to a model, the model predicts noisy pixels, and the noise is subtracted from the image. This iterative process continues and at the end, you get the image starting from pure noise to a realistic one.

A scheduler, on the other hand, controls the amount of noise added to the image. The scheduler follows a pattern, adding a certain amount of noise to the image at each step. This ensures that a certain pattern of noise volume is followed during the entire process.

Another faster approach that Stable Diffusion uses is **Variational Auto-Encoders** (**VAEs**). A VAE is a type of model that first compresses the input and then tries to decompress it. *Figure 17.3* shows a typical VAE:

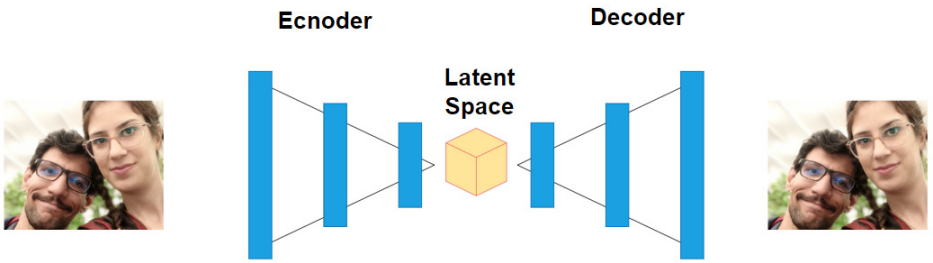


Figure 17.3 – VAE

VAEs work much faster because instead of directly applying noise to the image, they apply noise to the latent-space representation of the image. Latent space is much smaller in terms of size compared to the original image. The process can be reformulated as generating noise with a specific scheduler constraint in a latent-space representation of the same size as the image and adding it to the image.

The process just explained is a way of generating images. But as mentioned previously, if you do not have any control over the process, it could only be used to randomly generate images. To resolve this, a text encoder (**CLIP** as an example) is used to encode text into a dense vector. This dense vector is then used to guide **UNet** (a convolutional neural network) during the denoising and decoding process.

So far, you have learned about Stable Diffusion and how it works. In the next section, you will learn how you can use it to generate images.

Stable Diffusion in action

In this section, you will learn how to use Stable Diffusion in action. This is done via the utilization of diffusers and transformers and is very easy and convenient.

In order to use Stable Diffusion in the easiest way, you first need to install the following libraries:

```
pip install diffusers transformers accelerate safetensors
```

`diffusers` is the library we will use to load Stable Diffusion models:

```
import torch
from diffusers import (
    StableDiffusionPipeline, DPMSolverMultistepScheduler)
model_id = "stabilityai/stable-diffusion-2-1"
pipe = StableDiffusionPipeline.from_pretrained(model_id,
    torch_dtype=torch.float16)
pipe.scheduler = DPMSolverMultistepScheduler.from_config(
    pipe.scheduler.config)
pipe = pipe.to("cuda")
```

Now your pipeline is ready and you can easily use it:

```
prompt = " hyperrealistic portrait of unicorn"
image = pipe(prompt).images[0]
```

The generated image will be something like *Figure 17.4*:

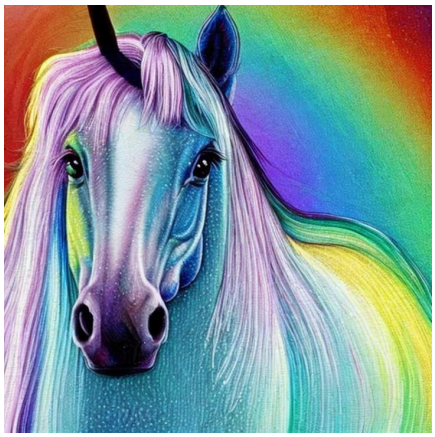


Figure 17.4 – Stable Diffusion output

The unicorn in the preceding image is not as realistic as expected, but nonetheless, the quality of the generated image is not bad at all. Maybe a bit of prompt engineering will help:

```
"hyperrealistic portrait of unicorn, cosmic survival, Hyper quality,
photography, digital art"
```

The next output shows how prompt engineering can improve the result.

Figure 17.5 – Stable Diffusion output with a new prompt

The preceding output is the best I could get with a bit of prompt engineering, but I am sure you will be able to generate better ones. Maybe this is why the prompt-engineering hype came about.

Stable Diffusion is not only controlled using textual prompts but can also be controlled using another image as a source of generation. Newer techniques applied on top of Stable Diffusion, such as ControlNet, are used to provide this capability.

In simpler words, ControlNet controls Stable Diffusion by conditioning it to a specific input. Examples of this functionality are scribbles, edge maps, pose key points, depth maps, segmentation maps, and normal maps.

Music generation using MusicGen

Large language models have been very successful in many fields. The idea of using a large model (typically a transformer) for other tasks is also very interesting. For example, we can create a large music model that can take text prompts as input and generate music as output. **MusicGen** is one of the models in this field.

Using MusicGen with the help of **audiocraft** is very easy. You need to follow these steps:

1. First, install the `audiocraft` library:

```
pip install -U audiocraft
```

2. The next step is to load the model and set the generation duration for the music:

```
from audiocraft.models import musicgen
import torch
model = musicgen.MusicGen.get_pretrained('medium',
    device='cuda')
model.set_generation_params(duration=8)
```

3. Finally, you can pass the prompts to it and it will generate the music. You can also listen to it in your Colab environment:

```
generated_music = model.generate([
    '80s rock music with drums and electric guitar',
    '90s retro action game music'],
    progress=True)
```

4. You can easily listen to it using the following code:

```
from audiocraft.utils.notebook import display_audio
display_audio(generated_music, 32000)
```

Generating music with different prompts can be really fun! Imagine how you could use this generated music for your creative work. You could make lots of music for a game you are developing, or you could even create an application where users can generate music and share it with each other. MusicGen also supports audio prompts as well. This means you can give it a sample of a specific music track and change its style or even use it as a base on which to create different versions.

In the next step, you will learn how you can use speech generation models.

Text-to-speech generation using transformers

Text-to-speech generation has been always an important part of AI agents, which usually need to talk to the user at some point. Using transformers for this specific task is also helpful. They can learn how to replicate different voices as well. This specific topic is not a new one, but the advances in this field are ongoing.

BARK (as in, a dog's bark!) is one of the most successful models in this field. This model can generate realistic human voices along with background noise, music, and sound effects. It is multilingual and also supports multiple speakers. Its usage is very simple with `transformers`:

1. First, you need to import `transformers`:

```
from transformers import AutoProcessor, AutoModel
```

2. The processor and model should be loaded next. The processor is required for processing input:

```
processor = AutoProcessor.from_pretrained("suno/bark")
model = AutoModel.from_pretrained("suno/bark")
```

3. Now, you can use the processor and model to create the speech. Be aware that you can use special tokens such as `[clears throat]` or `[laughs]` to create effects during the speech:

```
inputs = processor(
    text=["Hi! My name is Meysam. I like TRANSFORMERS [laughs]
I mean, I really like it [clears throat] I have been working
in field on NLP (Natural Language Processing) for almost a
decade"],
    return_tensors="pt",
)
speech = model.generate(**inputs, do_sample=True)
```

4. You can easily listen to the generated audio from your Colab:

```
from IPython.display import Audio
Audio(speech.cpu().numpy().squeeze(),
    rate=model.generation_config.sample_rate)
```

So far, you have learned how to generate images, music, and speech. These models are very useful for content creators. Any content creator can use them to generate eye-catching content for their commercials, games, or other digital content.

In the next chapter, you will learn how transformers can be utilized for time-series data.

Summary

In this chapter, you learned about multimodal generative transformers. These transformers are able to generate images, music, and speech, and new generative AI tools are being created from similar models every day. Having read this chapter, you have learned how to use these models and how to integrate them into your day-to-day content creation tasks.

In the next chapter, you will learn about the use of transformers in numerical and time-series data.

18

Revisiting Transformers Architecture for Time Series

Transformers are well known for NLP-specific tasks. Their main power comes from their capabilities to model time series data. This data can be textual or non-textual. In this chapter, you will learn how to use transformers for time series data modeling and predicting. You will also learn the basics of time series concepts and utilize a very simple model on top of that. This will give you a glance at time series data that can be used for various predictions. Conversely, you will also learn how transformers differ and how you apply transformer models for this scope.

The following topics will be covered in this chapter:

- Understanding time series concepts
- Transformers and time series data modeling

Technical requirements

The following libraries/packages are required to successfully complete this chapter:

- `Anaconda`
- `transformers 4.0.0`
- `pytorch 1.0.2`
- `tensorflow 2.4.0`

All notebooks with coding exercises will be available at the following GitHub link: `https://github.com/PacktPublishing/Mastering-Transformers-Second-Edition`.

Understanding time series concepts

Time series data is typically a collection of observations measured by specific methods that are related in terms of measurement intervals. This data can come from physical or non-physical measurements. For example, room temperature can be measured using a physical device and specific time intervals. *Figure 18.1* shows an example measurement.

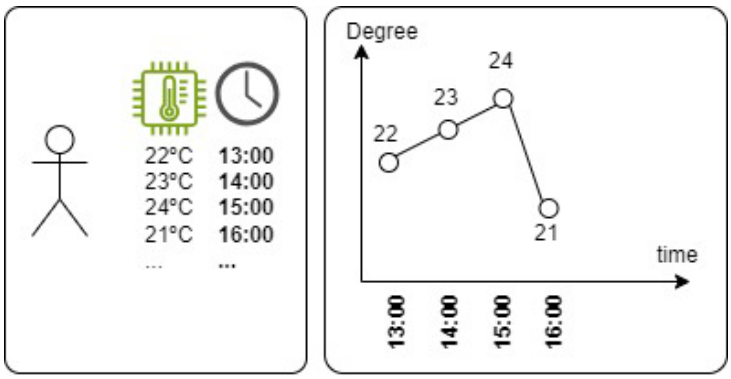


Figure 18.1 – A room temperature measurement and time series representation

Given the data measured from a specific source, different ML tasks can be applied based on the task itself. For example, given a specific time frame, future predictions can be seen as a task, and even with a given time series, classification can be seen as a task.

Time series prediction always has been one of the most important tasks in this field. In this specific task, given the period, future values must be predicted. One of the most well-known examples is using predictive analytics to predict the values of specific market assets, Bitcoin or cryptocurrency price prediction being such an example.

Predicting the future from time series data is also known as time series regression, which learns a statistical model from time series data. This statistical or probabilistic model is used to predict the future with a given specific past or historical data.

Another famous task in this field is time series classification. Given a signal, this type of classifier gets past or historical data and classifies it as a given set of classes. Take the following example – when given a sensor measurement during a specific time of a car engine noise, a model can predict whether the engine has issues or not. The data is also classified in the same way. From this simple example, you can see that the measurement also affects the quality of the data. Time series data, as stated before, is commonly measurements from physical or non-physical phenomena. Speaking in terms of physics and measurement, this can come with a noise or not. Noise during measurement is a common issue, and it can be a result of failure from sensors or different elements as well. For a good classifier or regression model, it is important to overcome this issue.

Simpler approaches, such as a moving-average model, have been proposed for time series data. However, these types of models are not capable of reflecting information passed through time series data. Better and more sophisticated approaches such as **AutoRegressive Integrated Moving Average** (**ARIMA**) perform better, but compared to transformers, each of them has specific shortcomings.

More innovative approaches in this field are also proposed by different types of data modeling. For example, to predict the future price of Bitcoin (or any other stock), you can see it as an environment and the model as an agent. With this paradigm, reinforcement learning approaches can be applied as well. The agent can buy, sell, or keep the assets, and at each time stamp, it has to decide what to do. Training will start with an exploration of the environment for the agent to see what the consequences of the actions are. After learning the consequences, a policy will be shaped for the agent, which will help them to understand what the best action to take is. The next step would be the exploitation of what has been learned so far. The agent will use whatever it has learned to achieve more rewards. This approach might not apply to all types of time series problems, but it is important to know what the best type of modeling is.

Transformers, conversely, as described in the entire book so far, are one of the best tools to understand sequential data, as they have shown their superiority in NLP. Different approaches have been proposed for time series prediction using transformers, including `Transformer`, `Informer`, `LogFormer`, `Reformer`, and `AutoFormer`, each having advantages and disadvantages.

In the next section, you will learn how to use Transformers for time series data modeling further, using them either for classification or prediction. You will also learn the basics of the idea behind this application and how it can be applied.

Transformers and time series modeling

Before jumping into transformers for time series modeling, it is helpful to understand basic approaches such as the moving average. This approach takes a window of historical data and applies an average to find the future value. Then, it continues to iterate over the data and find the next value using the next steps:

1. In order to illustrate this approach, we first need to get the data using `yfinance`, and then install the package:

   ```
   pip install yfinance
   ```

2. Then, you can easily fetch the data:

   ```
   import yfinance as yf
   btc_ticker = yf.Ticker("BTC-USD")
   btc_data = btc_ticker.history(period="max")
   ```

   The data will be fetched, and you can easily see it by printing out the variable:

   ```
   btc_data
   ```

It will show you a nice pandas DataFrame, as shown in *Figure 18.2*.

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2014-09-17 00:00:00+00:00	465.864014	468.174011	452.421997	457.334015	21056800	0.0	0.0
2014-09-18 00:00:00+00:00	456.859985	456.859985	413.104004	424.440002	34483200	0.0	0.0
2014-09-19 00:00:00+00:00	424.102997	427.834991	384.532013	394.795990	37919700	0.0	0.0
2014-09-20 00:00:00+00:00	394.673004	423.295990	389.882996	408.903992	36863600	0.0	0.0
2014-09-21 00:00:00+00:00	408.084991	412.425995	393.181000	398.821014	26580100	0.0	0.0
...	...	...	...	...	...	...	...
2023-12-29 00:00:00+00:00	42614.644531	43124.324219	41424.062500	42099.402344	26000021055	0.0	0.0
2023-12-30 00:00:00+00:00	42091.753906	42584.125000	41556.226562	42156.902344	16013925945	0.0	0.0
2023-12-31 00:00:00+00:00	42152.097656	42860.937500	41998.253906	42265.187500	16397498810	0.0	0.0
2024-01-01 00:00:00+00:00	42280.234375	44175.437500	42214.976562	44167.332031	18426978443	0.0	0.0
2024-01-02 00:00:00+00:00	44187.140625	45877.378906	44187.140625	45774.160156	34518597632	0.0	0.0

Figure 18.2 – Historical Bitcoin data

3. You can also easily visualize it with the following code:

```
from matplotlib import pyplot as plt
plt.plot(btc_data.Close)
plt.show()
```

The results are shown in *Figure 18.3*.

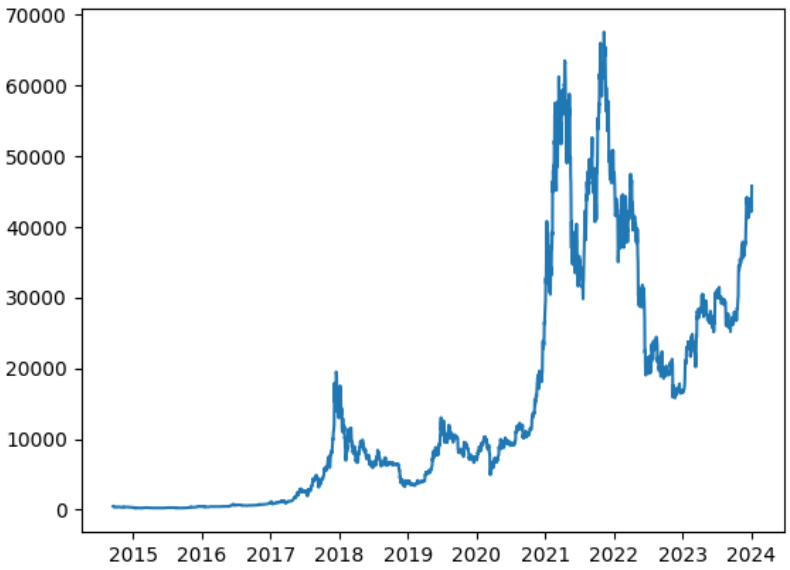


Figure 18.3 – A historical Bitcoin data visualization

4. Now, you can easily apply the moving average to the data:

```
btc_data["moving_average"] = \
    btc_data["Close"].rolling(10).mean()
```

5. You can also use the following command to visualize it:

```
btc_data[['Close','moving_average']].plot()
```

The results are shown in *Figure 18.4*.

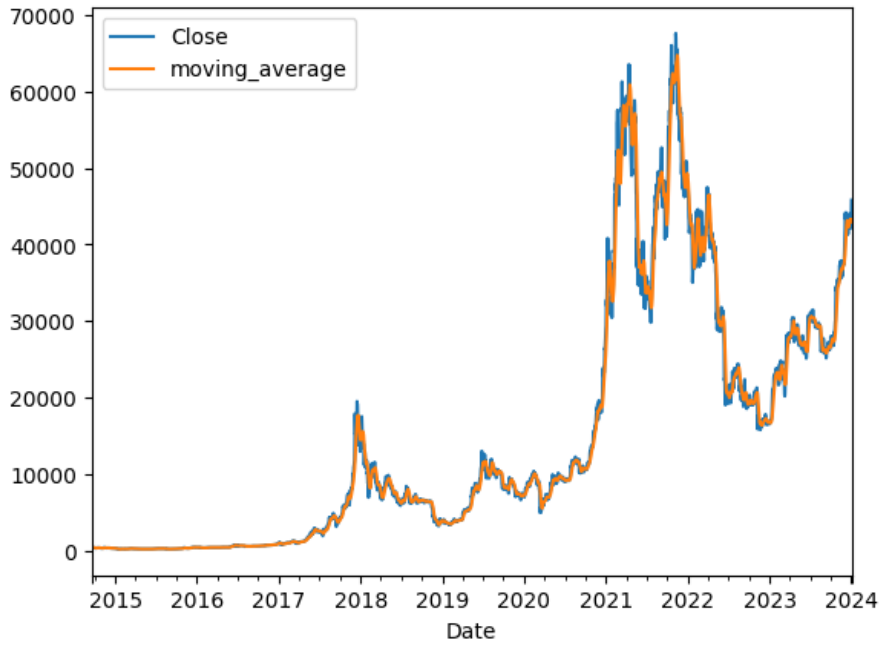


Figure 18.4 – A historical Bitcoin data and moving average visualization

As you can see from the visualization, the moving average looks at the future, but it is not always the best method. In order to see how far its predictions are from the actual values, you can use error metrics such as **Mean Squared Error** (**MSE**) and see the difference. The difference is high because it is only concerned with the values of the past window, which, in our case, in size of 10. This means it just takes the values of 10 steps (timestamps) before and finds the average value. This value is accepted as a future value. However, this specific phenomenon, or similar time series ones, are not as simple as they seem. Many external and internal market parameters are involved, and they sometimes follow specific patterns. A simple model such as the moving average is not capable of capturing such patterns. Conversely, it is not capable of moving too much into the future because if any error is introduced in the past, it will be propagated to the future, sometimes with a higher magnitude and sometimes with a lower magnitude:

1. We can assume that our goal is to just predict the next value; with the given predictions and actual values, we can calculate the MSE:

```
from sklearn.metrics import mean_squared_error
mean_squared_error(btc_data["Close"][9:],
    btc_data["moving_average"][9:])
```

2. The result, as you can see, is an extremely high number:

```
1783634.925773145
```

Also, there is another drawback to methods such as moving average, which occurs when missing values are present. Such methods are incapable of handling missing values, and certain methods to deal with them were introduced, such as replacing them with previous values, the mean value of the window, or even zero. But each of these approaches has issues as well.

Transformers can handle missing values similar to how BERT handles mask tokens. Let's start with an easy example, where we will aim to classify the next-day price of Bitcoin as higher or lower than the previous day. You can think of this example as a simple approach to identifying buying or selling orders. In this example, we will also implement our own transformer model for time series classification and see how it performs:

1. First, we need to start by transforming a dataset into the desired format. Initial Bitcoin prices are in the form of daily close values (in US dollars); for each sample, we will get the past 30 days' data, and the prediction needs to be for the next day after this period.
2. The easiest way is to apply a rolling window over the entire data with a 31-day period:

```
dataset_x = []
dataset_y = []
for j in range(0,30):
    for i in range(j,btc_data.shape[0], 31):
        if btc_data[i:i+31].Close.shape[0] == 31:
            dataset_y.append(
                1 if btc_data[i:i+30].Close.mean() <
                btc_data.iloc[i+30].Close else 0)
        mean_ = btc_data[i:i+30].Close.mean()
        std_ = btc_data[i:i+30].Close.std()
        dataset_x.append(
            list((btc_data[i:i+30].Close - mean_ ) / std_))
```

3. This code will create a series of windows, each starting from 0–30 indices, for each series of windows. It also creates rolling windows and finds the ground truth value for prediction using a simple fact – if the mean value of the previous 30 days is lower than the predicted value, the model should predict 0; otherwise, it should predict 1. Also, the values of the entire window are normalized to make training the model easier.

4. Now, you can convert your dataset to NumPy:

```
import numpy as np
dataset_x = np.array(dataset_x)
dataset_y = np.array(dataset_y)
And respectively, create train and test sets:
train_x, test_x = dataset_x[:3000], dataset_x[3000:]
train_y, test_y = dataset_y[:3000], dataset_y[3000:]
```

5. Now that everything is ready, we can create our model, but remember that it should be a transformer-based one, so we need to first implement the transformer encoder:

```
def encoder(inputs, head_size, num_heads,
    feed_forward_dimension, dropout=0):
    # Attention layer
    x = layers.MultiHeadAttention(
        key_dim=head_size, num_heads=num_heads, dropout=dropout
    )(inputs, inputs)

    x = layers.Dropout(dropout)(x)
    x = layers.LayerNormalization(epsilon=1e-6)(x)
    attention_result = x + inputs

    # Convlution as feedforward
    x = layers.Conv1D(filters=feed_forward_dimension,
        kernel_size=1, activation="relu")(attention_result)
    x = layers.Dropout(dropout)(x)
    x = layers.Conv1D(filters=inputs.shape[-1],
        kernel_size=1)(x)
    x = layers.LayerNormalization(epsilon=1e-6)(x)
    return x + attention_result
```

This function creates the encoder of the transformer.

6. Next, we create the actual model using this block:

```
def build_model(
    input_size,
    head_size,
    num_heads,
    ff_dim,
    num_transformer_blocks,
    dense_units,
    dropout=0,
    dense_dropout=0,
):
```

```
    inputs = keras.Input(shape=input_size)
    x = inputs
    for _ in range(num_transformer_blocks):
        x = encoder(x, head_size, num_heads, ff_dim, dropout)

    x = layers.GlobalAveragePooling1D(
        data_format="channels_first")(x)
    for d in dense_units:
        x = layers.Dense(d, activation="relu")(x)
        x = layers.Dropout(dense_dropout)(x)
    outputs = layers.Dense(n_classes, activation="softmax")(x)
    return keras.Model(inputs, outputs)
```

7. Based on our data, we can create a model using this function:

```
model = build_model(
    x_train.shape[1:],
    head_size=256,
    num_heads=4,
    ff_dim=4,
    dropout=0.25,
    num_transformer_blocks=4,
    dense_units=[128],
    dense_dropout=0.4
)
```

8. Then, we can compile the model and see the summary:

```
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    metrics=["sparse_categorical_accuracy"],
)
model.summary()
```

9. As a result, you see the following summary at the end:

```
Total params: 93130 (363.79 KB)
Trainable params: 93130 (363.79 KB)
Non-trainable params: 0 (0.00 Byte)
```

10. Now that we have the model ready and the data is in proper format, we can train it:

```
callbacks = [keras.callbacks.EarlyStopping(
    patience=5, restore_best_weights=True)]
```

```
model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    callbacks=callbacks,
)
```

Specific fallbacks are important to have – for example, *early stopping* is very useful to apply; it will stop the model from training at earlier stages if loss increases.

11. We can also evaluate the model:

```
model.evaluate(x_test, y_test, verbose=1)
```

As you can see, after only a few epochs and with just a few data samples, we can achieve good results.

Apart from classification, you can also change the same model to predict the next value with very small modifications. Instead of applying classification, you can use the direct value with sigmoid activation. On the other side, you can change the loss function to MSE. Also, another slight change needs to be applied to ground truth values that need to be predicted.

Summary

In this chapter, you have learned the basics of time series data and how to apply transformers to them. Transformers can be applied to time series data, ranging from modeling the data to predicting the next value in time. However, it is important to also note that, based on the data, transformers can be a great choice because they can deal with unknown values in time series that most models are not capable of doing.

Through this book, you learned how to apply transformers to different tasks, including NLP, computer vision, times series, and multimodal tasks. You also learned how to deploy them, how to create inference services, how large language models work, and how you can get the best out of them. Transformers have dominated the AI industry so far, better than any other architecture. **Generative AI** (**GenAI**) is also one of the fascinating, fast-growing AI features that have been adopted by the industry and community so quickly. You also learned many aspects of it and grasped a deeper understanding of how it works.

Index

A

AdapterFusion 234
adapter tuning
  used, for fine-tuning BERT checkpoint 237-241
adaptive learning
  benefits 227, 228
additive methods 235
A Lite BERT (ALBERT) 81-84
Amazon Polarity 200
Amazon SageMaker
  reference link 372
Anaconda 30
  installing, on Linux 31
  installing, on macOS 34
  installing, on Windows 32-34
  transformer, installing with 30
Anaconda for Linux
  download link 31
Anaconda for macOS
  download link 34
Anaconda for Windows
  download link 32
anaphoric relations 282
application programming interfaces (APIs) 359
  dockerizing 364
  used, for accessing datasets 47-50
AR models
  used, for NLG 120-122
artificial intelligence (AI) 3, 39, 403
attention heads
  interpreting 272, 273
  visualizing, with exBERT 273-278
attention head view 279-282
attention mechanism 14-16
audiocraft 409
augmentation
  used, for boosting IMDB text classification 216-221
Autoencoder (AE) 100
autoencoding language model 65, 81
  training, for language 70-80
Auto Regressive (AR) 100
AutoRegressive Integrated Moving Average (ARIMA) 415
average word embeddings 196
Aya-101 353

B

backpropagation 135, 136
back-translation 216
Bactrian-x 353
Bag-of-Words (BoW) 7
BARK 410
batch 56
Benchmark Data for Turkish Text Categorization
reference link 139
benchmarking examples
references 60
benchmarks 44
GLUE 45
SQuAD 47
SuperGLUE 46
working with 44
XGLUE 46
XTREME 46
BERT language model pretraining tasks 66
BERTopic
used, for topic modeling 203-205
BertViz
used, for multiscale visualization of attention heads 278, 279
Bidirectional and Auto-regressive Transformer (BART) 112-114
using, for zero-shot learning 190-193
Bidirectional Encoder Representations from Transformers (BERT) model 5, 23, 29, 66, 100, 106, 233
fine-tuning, for multi-class classification with custom datasets 139-145
fine-tuning, for sentence-pair regression 145-150
fine-tuning, for single-sentence binary classification 127-134
BERT checkpoint 67, 68, 70
fine-tuning, with adapter tuning 237-241
bi-encoder
architecture 186
versus cross-encoder 185, 186
BigBird 297, 310, 317-320
bigram models 9
BIO 161
BitFit 236
bitsandbytes 297, 326
for easier quantization 326, 327
BLEU 131
Blockwise Pattern 310
BLOOM (175B) 257
Bloomz 353
BM25 information retrieval models 5
Boolean model 205
bounding box 393
byte-level BPE (BBPE) 91
Byte-Pair Encoding (BPE) 10, 67, 90, 91, 117, 166, 329
training 93-96

C

catastrophic forgetting 227, 234
causal language modeling (CLM) 330
CC BY 4.0 DEED
reference link 26
central processing unit (CPU) 29, 384
centroids 201
character-level augmentation 214
character-level subwords 91
Chatbots 3
ChatGPT 101, 258
Classification (CLS) 5, 272
classification model
training, with native PyTorch 134-139

CLIPSeg 399
cloze test 66
combined pattern 309
command-line interface (CLI) 35, 362
comma-separated values (CSV) 54
CommonCrawl 334
URL 70
community
trained models, sharing with 80, 81
community-provided models
working with 39-42
context fragmentation 105
contextual word embeddings 6, 13
context vector 6
Contrastive Language-Image Pre-training (CLIP) 25, 398, 404, 407
contrastive learning 214
convolutional neural networks (CNNs) 4
corpus 70
Corpus of Linguistic Acceptability (CoLA) 45
cross-encoder
disadvantages 186
versus bi-encoder 185, 186
cross-encoding 186
cross-lingual classification 344-348
cross-lingual language model (XLM) 329, 332-336
Cross-Lingual Natural Language Inference (XNLI) 329
cross-lingual relation
example 331, 332
cross-lingual similarity tasks 337-339
cross-lingual textual similarity
visualizing 340-344
Cross-lingual TRansfer Evaluation of Multilingual Encoders (XTREME) 44
cross-lingual zero-shot learning 349-352
curse of multilingualism 354

D

data
processing, with map function 53, 54
data augmentation 213
used, for improving performance 214
Data-Efficient Image Transformer (DeiT) 389
data manipulation, with datasets library 51
caching 52
dataset filter 53
indexing 51
local files, working with 54, 55
map function 53
reusability 52
shuffling 51
sorting 51
datasets
accessing, with application programming interface 47-50
preparing, for model training 55, 56
working with 44
decision tree pruning 299
Decoding-Enhanced BERT with Disentangled Attention (DeBERTa) 87, 88
deep learning (DL) 4, 29, 272
leveraging 9
deep neural network (DNN) 10
Detection Transformer (DETR) 391
dilated attention 309
Direct Preference Optimization (DPO) 264
DistilBERT 91, 297
working with 300-302

distributional semantics 7
Doc2vec 5
Document Frequency (DF) 7
document pooling 196
domain
model, adapting to 221-228
domain adaptation 213
dynamic masking 85

E

elbow method 201
ELECTRA model 87
Embeddings from Language Models (ELMo) 6
End-of-Sentence (EOS) 117
exBERT
used, for visualizing attention heads 273-278
experiment tracking 375
explainable artificial intelligence (XAI) 271

F

Falcon(180B) 257
FastAPI Transformer model serving 360-363
faster inference
with Open Neural Network Exchange (ONNX) 371
Faster Transformer model
serving, with TFX 365-367
FastText 9
feed-forward neural networks (FFNNs) 4
few-shot learning 183
fine-tuning 233
first-order Markov processes 9
FISH Mask 236
FLAIR
used, for semantic similarity experiment 193-195
FLAIRAPI 196
FLAN-T5
fine-tuning, for NLI task with Lora 241-251
Flask 360
forward propagation 135
frequently asked questions (FAQs) 205
full fine-tuning 233
full model fine-tuning 233
full parameter fine-tuning 233
full self-attention model 307
Fundamental AI Research (FAIR) 191

G

gated recurrent units (GRUs) 12, 13, 185
General Language Understanding Evaluation (GLUE) 44, 187
Generated Pre-trained Transformer 2 (GPT-2) 29, 99, 104
reference link 41
Generative AI/GenAI 405
Generative Language Models (GLMs) 99-101
training 115-119
working with 102
generative multimodal AI 405
Get Docker
reference link 364
global attention 308
global explanation 288
GloVe 5, 196
GLUE benchmark 45
GLUE macro scores 302
Google Colaboratory (Google Colab) 29
installing 36, 37
using 36, 37

GPT-3 101
GPT-3 (175B) 257
GPT model family 102, 103
graphical user interface (GUI) 31
graphics processing unit (GPU) 29, 299, 384
grid search 228

H

head view 279
hierarchical density-based spatial clustering of applications with noise (HDBSCAN) 203
Hugging Face 39
hyperparameter optimization (HPO) 213, 234, 376
used, for optimizing parameters 228-231
hyperparameters 213

I

image classification
with transformers 392, 393
ImageGPT (iGPT) 390
IMDB text classification
boosting, with augmentation 216-221
inference tasks 45
information technology (IT) 39
InstructGPT 102, 104
instruction fine-tuned embedding models 209-211
Internet Movie Database (IMDb) 344, 376
Inverse Document Frequency (IDF) 7

J

JavaScript Object Notation (JSON) 54, 122

K

kernel 326
kernel attention 326
kernelization 326
key 17
Khmer 344
knowledge distillation 299, 300

L

LaMDA 137B 101
language-agnostic bert sentence embedding (LaBSE) 339
language generation 9
Language-Independent Named Entity Recognition (II)
reference link 162
language modeling 9, 22
large language models (LLMs) 4, 99, 233, 257, 258, 271
fine-tuning 263
fine-tuning, for NER 164-172
instruction 262, 263
reward function 259, 261
working with 37-39
large language models (LLMs), techniques
model quantization 264
Parameter Efficient Fine-Tuning (PEFT) 264
TRL library 264-269
latent-Dirichlet allocation (LDA) 203
latent semantic analysis (LSA) 8
learning-based patterns 321-325
lexico-syntactic pattern 205
LIME 271
interpreting, Transformers' decision 289-293
Linear Transformers 326

Linformer 297, 326
Linux
Anaconda, installing on 31
LLaMA 258
LLaMA(65B) 257
local explanation 288
local files
working with 54, 55
Locality Sensitive Hashing (LSH) function 321
Local Self Attention (LSA) 321
Locust
used, for load testing 368-370
Longformer 309-316
Long Short-Term Memory (LSTM) 6, 12, 13, 185, 331
Low-rank Adaptation of Large Language Models (LoRA) 235, 250
low-rank factorization 326
low-rank fine-tuning 236

M

M2M100 329, 353
machine learning (ML) 8
machine translation (MT) 99, 332
macOS
Anaconda, installing on 34
many-to-many mode 11
many-to-one mode 11
map function 53
used, for processing data 53, 54
Markov processes 9
Masked Language Modeling (MLM) 105, 222, 330
massive multilingual translation 329, 352-354
Mean Absolute Error (MAE) 148
Mean Squared Error (MSE) 148, 417
memory-efficient backpropagation 299
memory performance
benchmarking for 56-60
Microsoft Research Paraphrase Corpus (MRPC) 45
Mistral (7B) 257
model card 41
model decision 288, 289
model metrics
tracking 376
model quantization 264
models
adapting, to domain 221-228
fine-tuning, with run_glue.py 156, 157
model size reduction 299
implementation 300
knowledge distillation 300-302
pruning 302-304
quantization 305, 306
model training
dataset, preparing for 55, 56
tracking, with TensorBoard 376-379
model training live
tracking, with W&B 379-384
model view 279, 283-285
movement pruning
reference link 306
Moving Average (MA) 379
multi-class classification 126
multi-class classification, with custom datasets
BERT, fine-tuning for 139-145
Multi-Genre Natural Language Inference (MultiNLI) 45, 190
multi-head attention mechanism 15, 17-22, 274

multilabel text classification 126, 150-156
multilingual bidirectional and auto-regressive transformer (mBART) 329
multilingual bidirectional encoder representations from transformers (mBERT) 46, 329, 332, 333
Multilingual Common Crawl (mC4) 349
multilingual models
performance, fine-tuning 354-356
multilingual text-to-text transfer transformer (mT5) 329, 349, 353
multimodal learning 24-26, 404, 405
multimodal models 403
multimodal transformers
working with 42-44
multiscale visualization, of attention heads
with BertViz 278, 279
multi-task learning 44, 102, 234
with Text-to-Text Transfer Transformer (T5) 106, 107
multi-task training
with Text-to-Text Transfer Transformer (T5) 107-111
MusicGen
used, for music generation 409

N

named entity recognition (NER) 11, 100, 159, 161, 162, 228, 234
language models, fine-tuning for 164-172
native PyTorch
used, for training classification model 134-139
Natural Language Generation (NLG) 99
with AR models 120-122
natural language inference (NLI) 3, 29, 183, 192, 234, 272, 329, 335, 359, 387
Natural Language Processing (NLP) 160, 214, 297
neural machine translation (NMT) 14
neuron view 279, 285-287
Next Sentence Prediction (NSP) 23, 67, 284, 330
n-gram language models 5, 9
NLI task with Lora
FLAN-T5, fine-tuning for 241-251
NLP approaches
evolution 4-7
NLP libraries 108
non-negative matrix factorization (NMF) 203
N-order Markov processes 9

O

object detection
with transformers 393-396
one-hot encoded document-term matrices 5
one-shot learning 183
one-to-many mode 11
ONNX runtime (ORT) 371
Open Book Question Answering (OBQA) 101, 260
Open Neural Network Exchange (ONNX)
used, for faster inference 371
operating system (OS) 364
Optuna 228
features 230
OPUS (Open Source Parallel Corpus) 335
out-of-memory (OOM) error 249

P

PaLM (540B) 257
Paragraph-ID 185

parallel adapters 235
Parameter-Efficient Fine-Tuning (PEFT) 233, 234, 264
advantages 234
parameters
optimizing, with HPO 228-231
parameter sharing 299
paraphrasing 216
Part-of-Speech (POS) tagging 100, 159, 162, 163
Passage Ranking 209
Pathways Language Model (PaLM) 233
PEFT experiments
working with 236
PEFT, types 235
additive methods 235
low-rank fine-tuning 236
selective methods 236
PEGASUS 106
Penn Treebank POS 162
performance
improving, with data augmentation 214
Performer 297, 326
Permuted Language Modeling (PLM) 105
pipeline parallelism algorithms 299
policy optimization 259
positional encoding 70
Postman
download link 362
pre-training language models (PTLMs) 258
principal component analysis (PCA) 202
probing-classifier approach 287
inner parts of BERT with 287, 288
processing 331
prompt 101
prompt engineering 258
proof-of-concept (PoC) 107
Proximal Policy Optimization (PPO) 264
pruning 299, 302-304
structured pruning 302
unstructured pruning 302
PyTorch 29
installing 35, 36

Q

quantization 299, 305, 306
quantized LoRA (QLoRA)
tuning with 251, 252
query 17
Question Answering (QA) 39, 159, 163, 164
applying, for NLP tasks 180, 181
with token classification 172-180
question-answering (QA) model 360
Question Answering (QA) problem 227
Question Natural Language Inference (QNLI) 45
Quora Question Pairs (QQP) 45

R

random-access memory (RAM) 56
random attention 309
random search 228
Recognizing Textual Entailment (RTE) 45
Recurrent neural networks (RNNs) 4, 185
Reformer 297, 310
Reformer model 321
reinforcement learning from human feedback (RLHF) 104
reinforcement learning (RL) 258, 259, 261
requests per second (RPS) 370
reset gate 13
ResNet 389
response time (RT) 370

Reversible Residual Network (RevNet) 321
reward function 259-261
Reward Modeling (RM) 264
reward model (RM) 104
RLHF paradigm
human feedback 258
self-supervised 258
supervised 258
RNN architecture
advantages 11
disadvantages 11
RNN-based document embeddings 197
RNN models
used, for considering word order 10, 11
Robustly Optimized BERT Pretraining Approach (RoBERTa) 85-87, 188
Root Mean Square Error (RMSE) 131, 148
run_glue.py
utilizing, to fine-tune models 156, 157

S

SageMaker inference 371-373
sampling 406
Sanic 360
SavedModel format
reference link 365
SBERT embeddings 197-200
seed value 52
selective methods 236
self-attention 17, 306
self-attention sparsification 299
self-explainers 288
self-supervision 214
semantic search 183
semantic segmentation
with transformers 393, 396-398
semantic similarity experiment
with FLAIR 193-195
semantic text clustering 183
Semantic Textual Similarity Benchmark (STSB) 45, 146, 188
Semantic Textual Similarity (STS) 194
reference link 187
used, for text clustering 202
Sentence-BERT (SBERT) 183, 339
used, for semantic search 205-209
used, for text clustering 200-203
sentence embeddings 184, 185
used, for solving applications 185
sentence-level augmentation 216
examples 216
sentence order prediction (SOP) 82, 330
sentence-pair regression
BERT model, fine-tuning for 145-150
sentence piece tokenization 10, 67, 91, 92
sentence similarity models
benchmarking 187-190
sentiment analysis 127
separator (SEP) 272
sequence-to-sequence (seq2seq) 6
sequential transfer learning (STL) 44
serial adapters 235
shallow 22
SHapley Additive exPlanations (SHAP) model 271
interpreting, Transformers' decision 293-295
Siamese network structure 186
silhouette method 201
similarity and paraphrase tasks 45
single-sentence binary classification
BERT model, fine-tuning for 127-134
single-sentence tasks 45
Singular Value Decomposition (SVD) 236

SpanBERT 113
SparseAdapter 236
sparse attention
with fixed patterns 307-320
speed performance
benchmarking for 56-60
SQuAD1.1 47
SQuAD2.0 47
SQuAD benchmark 47
SQuAD dataset
reference link 260
Stable Diffusion
reference link 26
used, for text-to-image generation 405-407
working 407-409
Stable Diffusion pipeline 25
Stanford Natural Language Inference (SNLI) Corpus
reference link 241
Stanford Question Answering Dataset (SquAD) 44, 163, 227
Stanford Sentiment Treebank (SST-2) 45
Stanford Sentiment Treebank v2 (sst2) 137
static masking 85
structured pruning 302
SuperGLUE benchmark 46, 87
supervised fine-tuning (SFT) 102, 264
support vector machines (SVMs) 8
surrogate model 289
Synthesizer 326

T

T0 107
Zero-Shot Text Generalization with 111, 112
TAble PArSing (TAPAS) 29
TAPAS base model fine-tuned on WikiTable Questions (WTQ)
reference link 40
Tatoeba 340
t-distributed Stochastic Neighbor Embedding (t-SNE) 202
TensorBoard 375
reference link 376
used, for tracking model training 376-379
TensorFlow 29
installing 35, 36
TensorFlow Extended (TFX) 359
used, for Faster Transformer model serving 365-367
TensorFlow Hub 339
Tensor Processing Units (TPUs) 299
Term Frequency - Inverse Document Frequency (TF-IDF) 201
Term Frequency (TF) 7
text categorization 126
text classification 126, 127
text clustering
with Sentence-BERT 200-203
text representation 183
text-to-image generation
Stable Diffusion for 405-407
text-to-speech generation
with transformers 410
text-to-text models
working with 106
Text-to-Text Transfer Transformer (T5) 99, 106, 349
multi-task learning with 106, 107
multi-task training with 107-111
text (TXT) 54
TF-IDF-based models 5

TF-IDF BoW model
advantages and disadvantages 8
thought vector 6
time series data
concepts 414, 415
time series modeling 415-421
token classification 160, 161
used, for Question Answering (QA) 172-180
token embeddings 10
tokenization algorithms
working with 88-90
tokenizers
working with 37-39
tokenizers library 92
tokens 10
topic modeling 183
with BERTopic 203-205
traditional NLP approaches 7, 8
trained models
sharing, with community 80, 81
training argument definitions 129
transfer learning (TL) 4, 13, 44
using 22-24
transformer-based BERT embeddings 197
transformer models 14
transformer models directory
reference link 39
transformers 415-421
installing 35, 36
installing, with Anaconda 30
used, for image classification 392, 393
used, for object detection 393-396
used, for semantic segmentation 393, 396-398
using 22-24
Transformers' decision
interpreting, with LIME 289-293
interpreting, with SHAP model 293-295
Transformer-XL 102, 105
translation language modeling (TLM) 330
Tree-structured Parzen Estimator (TPE) 228
Triplet network structure 186
TRL library 264-269

U

Ubuntu 31, 335
UNet 407
unidirectionality 102
Uniform Manifold Approximation and Projection (UMAP) 202, 341
unigram models 9
Universal Language Model Fine-tuning (ULMFiT) 6
Universal Sentence Encoder (USE) 183, 186
unstructured pruning 302
update gate 13
uvicorn 361

V

value 17
Variational Auto-Encoders (VAEs) 406
vector space models (VSMs) 7, 184
Vision Transformer (ViT) 24, 387-391
Visual Geometry Group (VGG) 22
visual prompt models 398-401
Visual Question Answering (VQA) 172
vocabulary symbols 91

W

W&B sweeps 384
WebText 104

Weights & Biases (W&B) 375
used, for tracking model training live 379-384
Wikipedia corpus 334
Windows
Anaconda, installing on 32-34
Winograd Natural Language Inference (WNLI) 45
word2Vec 5, 185
word embeddings 5
word-level augmentation 215
word order
considering, with RNN models 10, 11
WordPiece model
training 96-98
WordPiece tokenization 10, 67, 91
World Wide Fund for Nature (WWF)
URL 205

X

XGLUE benchmark 46
reference link 46
XLM-R 46
XLM-RoBERTa 199
XLM-R XL 354
XLM-R XXL 354
XLNet 102, 105
XNLI dataset 333-335
XTREME benchmark 46
reference link 46

Z

zero-shot cross-lingual transfer capability 46
zero-shot learning 101, 183
BART, using for 190-193
Zero Shot Learning for Text Classification
reference link 190
Zero-Shot Text Generalization
with T0 111, 112

packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

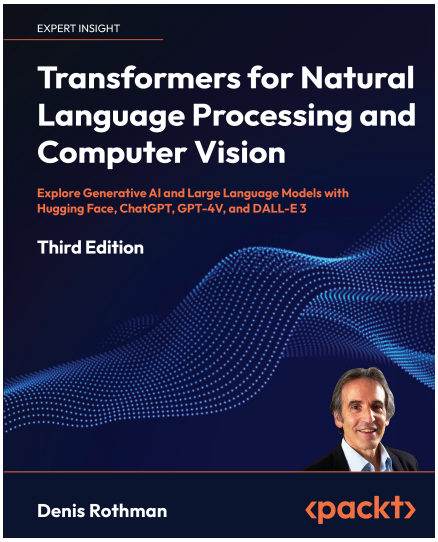
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

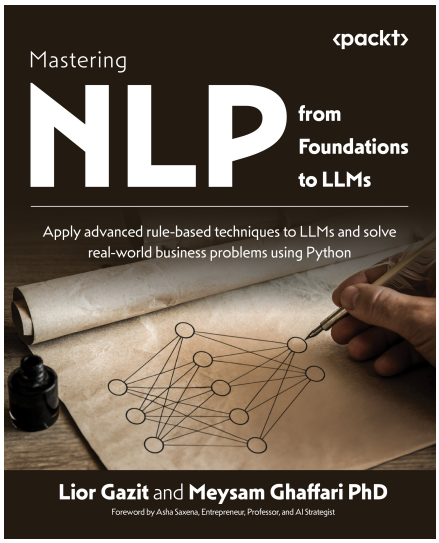


Transformers for Natural Language Processing and Computer Vision

Denis Rothman

ISBN: 978-1-80512-872-4

- Learn how to pretrain and fine-tune LLMs
- Learn how to work with multiple platforms, such as Hugging Face, OpenAI, and Google Vertex AI
- Learn about different tokenizers and the best practices for preprocessing language data
- Implement Retrieval Augmented Generation and rules bases to mitigate hallucinations
- Visualize transformer model activity for deeper insights using BertViz, LIME, and SHAP
- Create and implement cross-platform chained models, such as HuggingGPT
- Go in-depth into vision transformers with CLIP, DALL-E 2, DALL-E 3, and GPT-4V



Mastering NLP from Foundations to LLMs

Lior Gazit, Meysam Ghaffari

ISBN: 978-1-80461-918-6

- Master the mathematical foundations of machine learning and NLP
- Implement advanced techniques for preprocessing text data and analysis Design ML-NLP systems in Python
- Model and classify text using traditional machine learning and deep learning methods
- Understand the theory and design of LLMs and their implementation for various applications in AI
- Explore NLP insights, trends, and expert opinions on its future direction and potential

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Mastering Transformers*, we'd love to hear your thoughts! If you purchased the book from Amazon, please click here to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below

`https://packt.link/free-ebook/9781837633784`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly